sqltoy-orm 中文文档

sqltoy-orm · 全面技术文档（docsify 站点导出）

目录

简介

- sqltoy-orm 是什么
- sqltoy 主要特性
- 支持的数据库
- 换库迁移适配指引
- 技术架构
- 6.0 升级指南
- 从 MyBatis 迁移

环境准备

- 开发环境与依赖
- quickvo 代码生成工具
- quickvo-maven-plugin

快速上手

- 01 helloworld 项目
- 02 helloworld-solon 案例
- 03 增加公共字段处理

核心配置

- sqltoy 配置参数
- 多数据源与事务
- 传统 Spring 项目配置
- 纯 Java 项目如何使用
- Solon 集成参考
- XML Schema 配置指引

对象操作

- 对象化 CRUD
- 树形表节点路径构造

SQL 查询

- 动态 SQL 编写规范
- 常规查询 API
- Link 链式操作
- Lambda 条件查询与更新
- 分页优化
- 复杂查询与数据分析
- 并行查询
- 存储过程调用
- 常见 SQL 案例

缓存翻译

- 缓存翻译使用
- 超大规模主数据缓存(FIFO)
- 动态控制缓存

跨库与方言

- 方言自适配与函数扩展
- 多种数据库适配验证

主键与 DDL

- 主键策略介绍
- POJO 生成表结构 DDL

企业级特性

多租户支持
数据权限传参和越权校验
SQL 拦截加工
数据版本控制
数据脱敏与加解密
分库分表
类型扩展 TypeHandler
慢 SQL 处理

NoSQL

MongoDB 支持
Elasticsearch 支持

工具与扩展

工具类:DTO 与 POJO 互转
表元数据操作 TableApi
配置化服务底座(报表/API服务)
扩展集成(报表/低代码)
GraalVM Native Image 指引
其他特性

参考手册

标签与表达式
注解速查
API 速查
版本更新日志

必杀集锦

必杀集锦

常见问题

常见问题解答 FAQ

项目介绍

Sqltoy——融合 JPA 式对象操作与 SQL 优先查询的 Java ORM 框架

Sqltoy 以直观的动态sql编写模式作为起点，提供缓存翻译、分页优化、快速分页、sql函数不同数据库自适配等特性，并支持 GraalVM 原生镜像（Native Image）编译，为云原生与信创场景提供极速启动、低内存占用的部署形态，对项目实践过程中的各类复杂场景给出了系统的解决方案。

> 通过开源分享已经凝聚了更广泛用户的场景和智慧,欢迎您的加入共同打造一个中国式智慧的ORM框架!

与其他 ORM 的核心区别

SqlToy 的定位是**弥补 JPA 的查询短板、解决 MyBatis 的高频痛点**：既不是"所有 SQL 都要写 XML"的映射框架，也不是"复杂查询乏力"的纯对象框架，而是把两者的长处融合并针对性增强。

对比维度	MyBatis / MyBatis-Plus	Hibernate / JPA	SqlToy
定位	SQL 映射框架，查询强、对象操作偏弱	对象操作强，复杂查询偏弱	**两者融合**：JPA 式对象 CRUD + 超强原生 SQL 查询
动态 SQL	`<if>/<where>` 标签把 SQL 割裂，无法直接在客户端执行	不提供原生 SQL 编写模式	`#[]` 条件片段：SQL 保持原生结构，**可直接复制到数据库客户端运行调试**
update 修改	手写 SQL 或先查后改	先 load 再改，**两次数据库交互**	**弹性更新**：一次交互完成，null 字段自动跳过
高并发台账类场景	需自行实现锁与多次交互	无对应能力	**updateFetch / updateSaveFetch**：一次交互完成锁查询→校验→修改→返回
码值→名称显示	手写 JOIN 或多次查询	关联映射繁琐	**缓存翻译**：免 join，本地缓存零网络开销
分页	通常是简单包裹 `select count(1) from (原sql)`	有分页 API，无 count 优化	**智能 count 改写** + 缓存分页 + `@fast` 快速分页 + 并行分页
跨数据库	SQL 绑定特定数据库方言	对象操作可跨库，查询能力弱	**25 种方言** + 函数自动替换 + 方言 sqlId + 多库重放验证
数据分析	手写复杂透视/汇总 SQL	不支持	pivot/unpivot、分组汇总、同比环比、树形排序汇总 内置内存算法
查询结果封装	ResultMap 手动映射	对象映射	自动映射 + **主子表层次封装**（一条 join SQL 封装成父子对象）
条件式查询	`QueryWrapper` / `LambdaQueryWrapper`（多表 join 需额外插件）	Criteria API，写法冗长	内置 `LambdaQuery/LambdaUpdate/LambdaDelete`：单表与 join 一套构建器，支持 CTE / UNION / `@fast` 下推

> 上表只列关键差异，完整能力见下方功能特性与功能清单；更多"为什么这样设计"的对比示例（sqltoy vs MyBatis 写法对比等）见必杀集锦。

关于 jOOQ / Fluent-Mybatis 等"代码拟合 SQL"框架

这类框架的出发点是用 Java 对象化拟合 SQL、数据库改字段时代码直接报错。sqltoy 的看法是：**单表简单查询它们确有优势，但多表复杂查询属于过度包装。**判断的核心标准是：**编写调试 → 项目融入 → 后期维护能否高度一致**：

- **双向转化成本**：SQL 先在客户端调试，再"翻译"成 Java 代码融入项目；需求变更后改 Java，还要再"翻译"回客户端验证——sql 复杂一点、变更频繁一点，这种双向转化的消耗会持续放大。而 sqltoy 的 SQL 保持原生形态，**copy 到客户端就能跑、改完 copy 回来就能用**；
- **表达能力**：SQL 的函数体系与 `/*+ hint */` 优化器提示非常丰富，Java API 拟合能覆盖全吗？多表关联、多层嵌套的复杂统计用 Java 拼装，可读性远低于原生 SQL；
- **收益与成本**："改字段编译报错"的收益有限——见过谁做项目天天改表名、改字段名？而不完全覆盖的拟合 API，最终依然要写字符串兜底。

功能特性

sqltoy提供高效的ORM操作，包括对象化CRUD、级联加载、自动DDL生成等。在数据修改方面，提供弹性字段更新、强事务处理能力，支持分库分表、多种主键策略和数据加密。查询方面支持直观的SQL编写、缓存翻译优化、跨数据库自适配，提供四层优化的分页机制（自动count优化、缓存分页、快速分页、并行分

页）。此外还支持数据分析（行列转换、同比环比、树形处理）、层次化数据结构、多租户隔离、数据加解密与脱敏等企业级特性。

类别	核心能力	文档
对象操作	JPA 风格 CRUD、弹性更新、updateFetch / updateSaveFetch、级联、查询层次封装、树形表路由	CRUD
SQL 查询	动态 SQL（`#[]` + filters）、Lambda 条件（单表/join 同一构建器）、缓存翻译、四层分页优化（count 优化 / 缓存 / 快速 / 并行）、并行查询、存储过程、流式查询	动态 SQL / Lambda / 分页
数据分析	行转列 / 列转行、分组汇总、同比环比、树形排序汇总、分组拼接、日期数字格式化	数据分析
跨数据库	25 种方言（含 SAP HANA、瀚高 HighGo）、函数自动替换、多方言 sqlId、多库适配验证	方言 / 数据库清单
企业级	分库分表、多租户、数据权限与越权校验、脱敏加解密、数据版本控制、SQL 拦截、慢 SQL 处理	分库分表 / 安全
NoSQL	Elasticsearch（sql / json 双模式）、MongoDB（查询 / 聚合 + 缓存翻译）	Mongo / ES
工程化	quickvo 代码生成、autoDDL 自动建表、debug 热加载、GraalVM AOT、Spring Boot / Spring / Solon / 纯 Java	配置 / quickvo / GraalVM

> 完整功能清单见 sqltoy 功能清单

- 快速开始
- 开发文档

sqltoy 的查询长这样——SQL 保持原生，`#[]` 内条件为 null 自动裁剪，copy 到数据库客户端即可调试，结果直接映射为类型化对象：

```
List<OrderInfoVO> orders = lightDao.find("""
        select * from SQLTOY_ORDER_INFO t
        where 1=1
        #[and t.status in (:status)]
        #[and t.create_time >= :beginTime]
        """,
        MapKit.keys("status", "beginTime").values(new Integer[] { 1 }, LocalDateTime.now().minusDays(30)),
        OrderInfoVO.class);
```

支持的数据库

- 常规的mysql、oracle、db2、postgresql、 sqlserver、dm、kingbase、hana、sqlite、h2、 oceanBase、polardb、gaussdb、tidb、oscar(神通)、瀚高、mogdb、vastbase、stardb
- 支持分布式olap数据库: clickhouse、doris、StarRocks、greenplum、impala(kudu)、TDengine
- 支持elasticsearch、mongodb
- 所有基于sql和jdbc 各类数据库查询

sqltoy-orm是什么?

- 1、sqltoy-orm是一个基于Java语言实现，融合JPA(对象化CRUD)和原生sql查询的ORM框架
- 2、sqltoy-orm是覆盖:简单业务、大型SaaS化多租户复杂业务(如:ERP、MES、CRM)、大数据等多种类型项目实践过程的总结和分享

其发展历程:

- 2007~2008年，做农行的一个管理类项目，因查询统计较多，且查询条件多而且经常要增加条件，就不停想如何快速适应这种变化，一个比较偶然的灵感发现了一套比 mybatis 更简洁的动态sql写法，并作为hibernate jpa 查询方面的补充，显著提升了开发效率。可以看写于2009年的一篇博文: https://blog.csdn.net/iteye_2252/article/details/81683940
- 2008~2012年，因一直做金融类企业项目，所面对的数据规模基本上是千万级别的，因此sqltoy一直围绕jpa进行sql查询增强，期间已经集成了缓存翻译、快速分页、行列旋转等比其他框架更具特色的查询特性。
- 2013~2014年，因为了避免让开发者在项目中同时使用两种技术，因此在sqltoy中实现了基于对象的crud功能，形成了完整的sqltoy-orm框架。
- 2014~2017年, 在负责拉卡拉大数据平台过程中，对sqltoy进行了大幅重构，实现了底层结构的合理化，在拉卡拉CRM和日均千万累计达百亿级别的大数据平台上得到了强化和检验。
- 2018~至今, 在建设SaaS化多租户ERP复杂场景下和flink-cdc结合MPP数据库的实时数仓过程中得到了充分锤炼，sqltoy已日趋成熟稳定，持续开源跟大家一起分享和共建!

几个标志性能力诞生的节点：

- 2008年，一个偶然的灵感获得了一套简洁的动态SQL编写模式
- 2010年，一个性能问题的思考加入了缓存翻译
- 2011年，一个高要求的分页性能场景，催生了@fast快速分页
- 2012年，针对数据统计，融入了行列转换、同比环比等特性
- 2016年，sqltoy在阿里Code平台首次开源（平台后来关停，2018年迁至 GitHub/Gitee 持续维护），2020年起加大开源推广力度

适用场景与领域扩展

sqltoy 来自真实项目的锤炼，特别适合以下场景：

- **后台管理类系统**：查询条件多变、复杂查询和统计多的管理类项目，是 sqltoy 动态 SQL 的典型适用场景
- **大型 SaaS 化多租户业务**：ERP、MES、CRM 等复杂业务，配合多租户、数据权限、分库分表等企业级特性
- **报表与数据分析**：分组汇总、行列转换、同比环比、树形排序汇总等内存算法自然集成，sqltoy 可以**作为报表平台的扩展**，让灵活报表的开发变得非常容易
- **查询服务底座**：sqlId + 命名参数的模式天然适合把查询做成配置，**作为配置化查询服务**快速开发数据接口
- **大数据/实时数仓**：clickhouse、doris、StarRocks 等 OLAP 数据库完善支持，并有与 flink-cdc 结合的实践检验

sqltoy 功能清单

sqltoy 以直观的动态 sql 编写模式为起点，融合 JPA 对象化操作与原生 sql 查询能力。以下按类别列出完整功能清单，每项均可跳转到对应章节查看详情。

一、对象操作（JPA 风格）

特性	说明	详见
对象化 CRUD	save / saveAll / update / delete / load 等完整接口（LightDao 约 110 个方法），POJO 由 quickvo 生成并带映射注解	对象化 CRUD
弹性更新	一次数据库交互完成修改（区别于 hibernate 先 load 再改），null 字段不参与更新，保障高并发下数据准确	对象化 CRUD
updateFetch	锁查询 → 逻辑校验 → 修改 → 返回最新结果，一次交互完成；秒杀、库存台账、资金台账场景	对象化 CRUD
updateSaveFetch	锁查询、记录不存在则插入、存在则修改，并返回修改后结果	对象化 CRUD
updateByQuery	基于单表条件批量更新，支持 `set 字段=字段+?` 依据字段值计算的自更新模式	对象化 CRUD
级联操作	`@OneToOne` / `@OneToMany` 级联保存、加载（支持过滤/排序）、修改、删除，批量级联底层 in 查询优化	对象化 CRUD
查询层次封装	一条 join SQL 按注解自动把主子表结果封装为父子对象（1..n 层级）	常规查询 API
树形表路由	`wrapTreeTableRoute` 构造节点路径/层级/叶子标记，非递归查询整棵子树	树形表路由
主键策略	22 位/26 位有序数字、雪花、UUIDv7、ULID、sequence/identity、redis 规则业务单据号，支持自定义扩展	主键策略
数据版本控制	`@DataVersion` 乐观锁，防止并发修改覆盖	数据版本控制
公共字段赋值	创建人/修改人/创建时间/修改时间/租户统一补漏赋值（可暂停/恢复）	公共字段处理

二、SQL 查询

特性	说明	详见
动态 SQL	`#[]` 条件片段 + `@if/@loop/@include/@joinFields` 宏 + filters 参数规整；带动态片段的 SQL 可直接在数据库客户端执行调试	动态 SQL
Lambda 条件体系	以属性方法引用（`StaffInfo::getStatus`）组织查询/更新/删除，单表与多表 join 共用一套构建器；`getOne` 严格语义、`matchAll` 守卫、`setExpr` 表达式更新、CTE/ `@fast` 下推/UNION 组合。6.0.8 起内置（原 sqltoy-plus 并入核心）	Lambda 条件查询与更新
Link 链式操作	`query()/save()/update()/delete()/load()/unique()/execute()/batch()/store()/treeTable()/tableApi()/elastic()/mongo()` 等链式入口，可指定数据源、锁模式、批量大小、强制更新字段	Link 链式操作
filters 过滤器	eq/between/to-date/to-number/split/primary（首要参数）/cache-arg（缓存反向匹配替代 like）等 20+ 种	动态 SQL
多形态查询 API	findOne / find / findPage / findTop / findRandom / getValue / getCount / isUnique / loadByQuery / findEntity（单表 EntityQuery 快捷查询）	常规查询 API
缓存翻译	码值→名称免 join；API 直取缓存供下拉框等组件；反向匹配 key；支持条件翻译、多值拆分、租户隔离、i18n；超大规模数据按需动态加载只保留常用数据	缓存翻译使用
分页	自动 count 优化 + page-optimize 缓存分页 + `@fast` 快速分页（先分页后关联）+ 并行分页 + 自定义 count-sql	分页优化
层次化查询	findByQuery + hiberarchy 按注解自动分层封装	常规查询 API
并行查询	parallelQuery 同时执行多个无依赖 SQL（可混合分页），提升页面整体响应	并行查询
存储过程	executeStore / executeMoreResultStore，支持 out 参数与多结果集	存储过程
流式查询	fetchStream 超大数据量逐行回调，不撑爆内存	常规查询 API
SQL 执行	executeSql / batchUpdate（可并行）/ insertReturnPrimaryKey / truncate	Link 链式操作

三、数据分析（内存算法，跨库一致）

特性	说明	详见
行转列 / 列转行	pivot / unpivot，支持动态列，几行配置完成	复杂查询与数据分析
分组汇总	summary 任意层级小计/总计/求平均	复杂查询与数据分析
同比环比	cols-chain-relative（列环比）/ rows-chain-relative（行环比），自动百分比输出	复杂查询与数据分析
树形排序汇总	tree-sort 树形排序 + 逐层向上汇总	复杂查询与数据分析
分组拼接	link 标签代替 group_concat / listagg / WM_CONCAT，跨库一致	复杂查询与数据分析
结果格式化	日期/数字格式化，支持中文大写金额（capital-rmb）等财务场景	动态 SQL 规范

四、跨数据库

特性	说明	详见
25 种方言	MySQL、Oracle、PostgreSQL、SQL Server、DB2、SAP HANA、达梦、金仓、瀚高、OceanBase、TiDB、ClickHouse、Doris、StarRocks、TDengine 等	支持的数据库
函数自动替换	`functionConverts=default` 开启，nvl/concat/to_char/datediff/uuid/date_add 等 28 个函数按数据库自动适配，支持自定义扩展	方言与函数
多方言 sqlId	`sqlId_dialect` / `dialect_sqlId` 命名，框架按当前库自动匹配专属 SQL	方言与函数
多库适配验证	redoDataSources 让任意查询在其他库重放执行，便于开展跨库回归测试	多库适配验证
保留字兼容	reservedWords 声明后对象操作与跨库 SQL 自动适配	方言与函数

五、企业级特性

特性	说明	详见
分库分表	注解 + xml 双模式声明策略，按条件自动路由库/表	分库分表
多租户	`@Tenant` + TenantFilterInterceptor 统一过滤与赋值	多租户支持
数据权限	统一权限参数注入 + 越权访问校验	数据权限
数据安全	RSA 字段加解密存储与查询解密；手机号/姓名/卡号等脱敏（支持离散脱敏）	脱敏与加解密
SQL 拦截	SqlInterceptor 集中加工 SQL（如改写 schema、注入过滤条件）	SQL 拦截
慢 SQL	超时阈值打印 + 慢 SQL 收集处理器（可扩展上报监控）	慢 SQL 处理

六、NoSQL

特性	说明	详见
Elasticsearch	eql 支持 sql 模式与原生 json 模式查询，分页/Top，可挂缓存翻译与脱敏	Elasticsearch
MongoDB	mql 查询与聚合管道，fields 别名映射 + 缓存翻译 + 脱敏	MongoDB

七、工程化与集成

特性	说明	详见
quickvo 代码生成	连接数据库按表生成带注解 POJO/VO（独立工具 + maven 插件），支持 lombok、swagger	quickvo / 插件
自动 DDL	autoDDL + packagesToScan 由 POJO 直接创建表结构	POJO 生成 DDL
debug 模式	打印执行 SQL/参数/耗时，sql 文件变更自动检测热加载	配置参数
GraalVM AOT	spring-starter/solon-plugin 内置 AOT 支持，6.0.8 起按配置动态注册 native 元数据	GraalVM Native Image 指引
对象转换	convertType / MapperUtils 实现 DTO 与 POJO 快速互转（支持 @SqlToyFieldAlias 别名）	工具类
表元数据	TableApi 获取表/列信息、truncate/drop	TableApi
多框架集成	Spring Boot starter / 传统 Spring / Solon / 纯 Java 四种接入方式	配置
类型扩展	TypeHandler 处理 JSON 等非标准类型，geometry 空间类型（JTS）支持	TypeHandler / 其他特性

支持的数据库

sqltoy 内置 25 种数据库方言（`dialect`，6.0 新增 SAP HANA，6.0.5 新增瀚高 HighGo），并通过 `DialectFactory` 自动识别数据库类型、适配分页与函数。只要数据库提供 JDBC 驱动，sqltoy 即可对其进行查询。

主流关系型数据库

MySQL、Oracle、PostgreSQL、SQL Server、DB2、SAP HANA、H2、SQLite、TiDB、OceanBase、PolarDB

国产数据库

达梦（DM）、人大金仓（Kingbase）、神通（Oscar）、瀚高（HighGo）、MogDB、Vastbase、StarDB、GaussDB、OpenGauss

分布式 OLAP 数据库

ClickHouse、Doris、StarRocks、Greenplum、Impala（Kudu）、TDengine

NoSQL

Elasticsearch、MongoDB

其他

所有基于 SQL 和 JDBC 的各类数据库均可进行查询。对于未单独列出方言的数据库（尤其 PostgreSQL 系的衍生库），可通过配置 `dialect` / `dialectMap` 指定按某种方言处理，详见常见 SQL 案例。

> 跨数据库适配（函数自动替换、方言 sqlId、多库适配验证）见跨库与方言章节。

换库迁移适配指引

本章面向**数据库换库迁移**场景（从 Oracle、MySQL 等迁移到达梦、金仓、瀚高、神通、MogDB、Vastbase、GaussDB、OpenGauss 等国产数据库），把 sqltoy 的跨数据库适配能力整理为「评估 → 配置 → 改造 → 验证 → 性能 → 切换」六步可执行清单；每步的原理与完整参数说明见对应章节链接。选型论证与整体介绍见源码仓库 `docs/multi-db-adaptation.md`（README「文档与资源」表有入口）。

一、评估先行：摸清适配面

1. **SQL 来源分类**：对象操作（save/update/delete/load/updateFetch 等）的分页、锁、merge、主键回填由框架按方言生成，**无需改造**；真正需要评估的是自定义 sql（xml 配置、`@Sql` 注解、代码 sql 三种形态，见动态 SQL 编写规范）；
2. **确认目标库覆盖**：方言清单见支持的数据库；未单独列出方言的国产库（尤其 PostgreSQL 系衍生库）先以 `dialect` / `dialectMap` 别名到最接近的方言上线：

```
spring.sqltoy.dialectMap.kingbase=mysql
```

3. **主键策略评估**：`identity` / `sequence` 在各库存在方言与大小写差异，换库迁移建议改用程序化主键（22 位/26 位/雪花/UUID）从源头消除，见主键策略；存量 sequence 表可继续使用，框架按方言处理回填。

二、配置层：开启跨库适配能力

```
# 跨库函数自动适配（内置 28 个函数）
spring.sqltoy.functionConverts=default
# 项目中使用到的保留字，对象操作与跨库 SQL 自动适配
spring.sqltoy.reservedWords=maxvalue,minvalue
# 代理库/配置方言与真实方言不一致场景（如 OceanBase 按 mysql 配置），变体查找跟随真实方言
spring.sqltoy.realDialectFirst=true
```

- 函数适配覆盖 `nvl/ifnull/coalesce`、`to_char/date_format`、`decode`、`group_concat/listagg/string_agg`、`sysdate/now/current_timestamp`、`trunc/date_trunc`、`uuid`、`date_add` 等常见差异；不能安全转换的形态**响亮保留**（原样交目标库报错），绝不静默改写，见方言自适配与函数扩展；
- `<sql dialect="...">` 按指定方言固化解析、`realDialectFirst` 真实方言优先匹配变体，同见该章 2.1、2.2 节。

三、SQL 改造层：按差异类型分类处理

差异类型	典型表现	处理方式
函数差异	`nvl`、`decode`、`to_char`、`sysdate`、`group_concat`	`functionConverts=default` 自动适配，无需人工
伪表	oracle 系必须 `from dual`，pg/mysql/sqlserver 写了反而报错	6.0.6 起用 `@virtualTable()` 宏，解析期按方言展开或剔除
结构性差异	oracle `connect by` 与递归 CTE、库专属 hint 等	方言变体 `sqlId_dialect` / `dialect_sqlId`，框架按当前库自动选取
主键返回大小写	PG 折叠家族 `getGeneratedKeys` 列提示大小写敏感	框架默认对 PG 折叠家族转小写；可以 `dialectReturnPrimaryColumnCase` 按方言覆盖
建表 DDL	各库类型、分区语法差异	POJO 注解 + quickvo 按库生成 DDL，见POJO 生成表结构 DDL

伪表宏写法（展开规则详见方言自适配与函数扩展 第 3 节）：

```
select '0' id,'全局公共' name @virtualTable()
union
select id,name from sys_tenant_info
```

结构性差异写法：为 sqlId 挂方言变体，业务代码只调基础 sqlId，查找顺序 `dialect_sqlId → sqlId_dialect → sqlId`：

```
<!-- 基础版（递归 CTE）：pg / mysql8+ / 达梦 / 金仓等执行本条 -->
<sql id="dept_sub_tree">…递归 CTE…</sql>
<!-- oracle 环境自动命中本条：connect by 层次查询 -->
<sql id="dept_sub_tree_oracle">…connect by…</sql>
```

四、验证层：上线前把所有目标库跑一遍

```
spring.sqltoy.redoDataSources[0]=dmdb
spring.sqltoy.redoDataSources[1]=kingbasedb
```

配置后任意查询在测试阶段**同时在上述数据源重放执行一次**，不兼容写法当场暴露，而不是等客户现场换库时才报错，见多种数据库适配验证。与之配套的两层框架保障：

- 各方言的分页/锁/merge/sequence 经真库探针逐项验证，并以反射穷举断言锁定覆盖完整性（新增数据库类型漏登记适配时测试直接失败）；
- 函数转换的"响亮保留"原则保证验证期报错即适配面清单，不存在静默错值。

五、性能层：迁移后性能不降反升

- **分页**：`findPage` 默认启用智能 count 优化（剔除 order by、裁剪 select 列）；频繁翻页场景加 `<page-optimize parallel="true" alive-max="100" alive-seconds="120"/>` 缓存 count 并并行执行；大表 join 分页用 `@fast()` 先分页后关联，见分页优化；
- **缓存翻译**：码值转名称在内存完成，消灭字典表 JOIN，支持增量更新与 FIFO 超大规模主数据缓存，见缓存翻译使用；
- **高并发台账**：`updateFetch` / `updateSaveFetch` 一次数据库交互完成锁查询→校验→修改→返回，见对象化 CRUD；
- **慢 SQL 治理**：慢 SQL 自动识别处理，见慢 SQL 处理；SQL 签名（`/*-id-*/`）便于数据库日志秒级定位。

六、切换与长期运维

- 并行期结合多数据源与事务按数据源灰度切换；
- 新国产库出现时先以 `dialectMap` 别名接入，官方方言支持后移除别名即获原生适配；
- 将 `redoDataSources` 保留在回归环境，作为跨库适配的长期门禁。

七、相关章节

- 支持的数据库 · 方言自适配与函数扩展 · 多种数据库适配验证
- 分页优化 · 缓存翻译使用 · 主键策略 · POJO 生成表结构 DDL

技术架构

sqltoy 是一个**以动态 SQL 为核心、面向企业级复杂场景**的 Java ORM 框架。它既可以与 Spring/Spring Boot/Solon 集成，也支持纯 Java 环境使用。理解其内部结构，有助于在复杂场景下灵活扩展。

一、整体设计

sqltoy 的核心思路：

- **SQL 与代码分离**：SQL 写在 `*.sql.xml` 中（支持 debug 模式热加载），通过 `sqlId` 调用，也可直接传 SQL。
- **对象与表映射**：POJO/VO 通过注解（`@SqlToyEntity` / `@Entity` / `@Id` / `@Column`）与数据库表建立映射，由 `EntityManager` 统一管理。
- **方言隔离**：所有数据库差异收敛到 `dialect` 包，通过 `DialectFactory` 自动识别数据库类型并路由到对应实现，业务 SQL 保持跨库一致。
- **内存算法增强**：分页优化、行列转换、汇总、同比环比、树形排序、缓存翻译等，大量在应用内存中以算法完成，既减轻数据库压力又实现跨库一致。
- **三种查询组织方式**：SQL 模式（sqlId + `#[...]`，适合复杂动态与配置化）、`EntityQuery` / `EntityUpdate`（单表字符串条件）、Lambda 条件（属性方法引用，单表与 join 共用一套构建器）——三者共用同一条执行链路，可按场景混用。

二、核心包结构

框架核心位于 `org.sagacity.sqltoy` 包下（模块 `sagacity-sqltoy`）：

包 / 类	职责
`SqlToyContext`	**整个框架的根上下文**，承载全部配置（数据源、方言、SQL 资源、缓存翻译、主键策略、拦截器等）
`SqlToyConstants`	框架常量，可通过 `sqltoy-default.properties` 加载，亦可经 `SqlToyContext` 的 dialectConfig 覆盖
`dao`	对外 DAO 接口，统一以 `LightDao` 为操作入口（约 110 个方法）及 `impl` 实现（`DefaultLightDaoImpl` / `LightDaoImpl`）；`SqlToyDao` / `SqlToyLazyDao` 为早期历史接口，新项目请一律使用 `LightDao`
`support.SqlToyDaoSupport`	所有数据库交互操作的**基类实现**（框架真正的"干活"类）
`service.SqlToyCRUDService`	通用 CRUD Service，简单增删改查无需自己写 Service
`link`	**链式 API**：`Query` / `Save` / `Update` / `Delete` / `Load` / `Store` / `Unique` / `Execute` / `Batch` / `Elastic` / `Mongo` / `TreeTable` / `TableApi`，均继承 `BaseLink`
`lambda`	**Lambda 条件体系**（6.0.8 起）：`LambdaQuery` / `LambdaUpdate` / `LambdaDelete` 与 `JoinCol` / `ExcludeCols` / `LambdaFn` / `OuterRef`；`engine`（条件段合并、参数取号、延迟组装）、`join`（join 上下文、表别名、可移植 EXISTS 改写）、`condition`（比较/聚合/逻辑条件）、`toolkit`（属性元数据提取、序列化镜像）
`model`	数据模型：`Page`、`QueryExecutor`、`EntityQuery` / `EntityUpdate`、`ParallelQuery`、`QueryResult`、`StoreResult`、`TreeTableModel`、`Summary`、`ColsChainRatio` / `RowsChainRatio`、`CacheArg`、`TableMeta` / `ColumnMeta`、`LockMode` / `SaveMode` / `MaskType` 等
`config`	SQL/实体加载与解析：`EntityManager`、`EntityScanner`、`SqlScriptLoader`（解析 `*.sql.xml`）、`SqlXMLConfigParse`、`SqlFileModifyWatcher`（debug 热加载）；`config.annotation`（各类注解）、`config.model`（解析后的配置模型）
`dialect`	数据库方言：`Dialect` 接口 + `DialectFactory`；`impl` 下 25 种方言实现（6.0 新增 SAP HANA，6.0.5 新增瀚高 HighGo）；`executor`（并行查询执行器、`ParallelUtils`）；根级 `PageOptimizeUtils`（分页/count 优化引擎）、`QueryExecutorBuilder`、`CrossDbAdapter`
`plugins`	扩展点：`calculator`（汇总/同环比/树排序/列转行等内存算法）、`datasource`（动态数据源选择）、`ddl`（POJO→DDL）、`function`（28 个跨库函数适配）、`id`（主键策略；`id.macro` 为 `@if/@loop/@include/@case/@df/@substr` 等宏指令实现）、`interceptors`（多租户过滤）、`json`（`JsonProvider` 可插拔 JSON 库）、`nosql`（ES）、`overtime`（慢 SQL）、`secure`（加解密/脱敏）、`sharding`（分库分表）；根级还有 `SqlInterceptor`、`TypeHandler`、`CrossDbAdapter`、`IUnifyFieldsHandler`
`translate`	缓存翻译子系统：`TranslateManager`、`TranslateFactory`、`TranslateConfigParse`（解析 `sqltoy-translate.xml`）、`CacheUpdateWatcher`、`DynamicCacheFetch`；`cache`（ehcache/caffeine/FIFO 动态缓存实现）
`callback`	22 个 SPI 回调接口：`RowCallbackHandler`、`StreamResultHandler`、`TransactionHandler`、`UpdateRowHandler`、`EntityUpdateCallback`、`DecryptHandler` 等
`integration`	框架无关 SPI：`AppContext`、`ConnectionFactory`、`DistributeIdGenerator`、`MongoQuery`
`utils`	工具类：`SqlUtil`、`BeanUtil`、`CollectionUtil`、`DateUtil`、`ResultUtils`、`DBTransUtils`、几何类型工具等（`ParallelUtils` / `TranslateUtils` / `ParamFilterProcessor` 等已按职责迁移至 `dialect` / `translate` / `config` 包）

三、集成模块

除核心模块外，sqltoy 提供多种框架集成：

模块	artifactId	说明
核心	`sagacity-sqltoy`	ORM 核心，可纯 Java 使用
传统 Spring	`sagacity-sqltoy-spring`	Spring 事务感知的 DAO 实现、`SpringAppContext` / `SpringConnectionFactory`
Spring Boot	`sagacity-sqltoy-spring-starter`	自动配置（`spring.sqltoy.*`）、任务线程池、GraalVM AOT 原生支持
Solon	`sagacity-sqltoy-solon-plugin`	Solon 框架适配（`sqltoy.*` 配置前缀、`@Db` 注入）

四、一次查询的运行流程

以 `lightDao.findPage(page, "sqlId", params, VO.class)` 为例：

1. `LightDao` 接口 → `SqlToyDaoSupport` 基类实现；

2. 从 `SqlToyContext` 取出该 `sqlId` 对应的 `SqlToyConfig`（已由 `SqlScriptLoader` 预解析）；
3. 经 `filters` 规整参数、`#[]` 动态片段裁剪，生成最终 SQL 与参数数组；
4. `DialectFactory` 识别数据库方言，生成分页 SQL 与优化后的 count SQL（`PageOptimizeUtils`）；
5. 若配置了 `sharding` / `tenant` 拦截器、`translate` 缓存翻译、`secure` 脱敏，则依次加工；
6. 执行查询，结果集经内存算法（summary/pivot/chain-ratio/tree-sort 等）二次计算；
7. 封装为 `Page` 返回。

整体流程示意：

```
flowchart LR
    A[LightDao 接口] --> B[SqlToyDaoSupport<br/>操作实现]
    B --> C[SqlToyConfig<br/>sqlId 预解析]
    C --> D[filters 参数规整<br/>动态 SQL 片段裁剪]
    D --> E[DialectFactory<br/>方言路由]
    E --> F[分页 SQL + count 优化<br/>PageOptimizeUtils]
    F --> G[拦截与增强<br/>sharding · tenant · translate · secure]
    G --> H[内存算法<br/>summary · pivot · tree-sort]
    H --> I[封装 Page 返回]
```

> 想深入源码，建议从 `SqlToyDaoSupport`（操作实现）、`LightDao`（API 入口）、`DialectFactory`（方言路由）、`SqlToyContext`（配置中枢）、`TranslateManager`（缓存翻译）、`PageOptimizeUtils`（分页优化）这几个类入手。

6.0 升级指南

从 5.6.x 升级到 6.0.9（6.0 线）的变更点与注意事项。变更内容均对照 6.0 源码核实。

一、环境要求

项	要求
JDK	**17+**（面向 Spring Boot 3/4）；JDK 8 项目请使用 `5.6.96.jre8`（最终 jre8 版本，支持 quickvo-maven-plugin 插件）
依赖坐标	不变：`com.sagframe:sagacity-sqltoy`（及 spring / spring-starter / solon-plugin），仅版本号升级为 6.0.9
配置参数	`spring.sqltoy.*` 在 6.0.2 的 58 个参数基础上新增 2 个：`realDialectFirst`（6.0.3 新增）、`ddlLowerOrUpper`（6.0.3 起暴露为 starter 属性），其余参数无增减（后续版本新增参数见下文各版本章节，均为可选项），存量配置无需调整

二、API 变更（需要改代码）

变更点	5.6.x	6.0.x
并行查询	`lightDao.parallQuery(List<ParallQuery>, ...)`	`lightDao.parallelQuery(List<ParallelQuery>, ...)`（模型类同步更名 `ParallQuery` → `ParallelQuery`）
连接反调	`doConnection(Connection conn, Integer dbType, String dialect)`	`doConnection(Connection conn, DBProfile profile)` —— `DBProfile` 提供完整的数据库执行档案

`DBProfile`（`org.sagacity.sqltoy.model.DBProfile`）提供：`getDbType()` / `getDialect()` / `getUrl()` / `getProductName()` / `getMajorVersion()`，以及 `isMysqlFamily()` / `isOracleFamily()` / `isOceanBase()` / `isBackslashEscape()` 等便捷判断，详见其他特性。

> 其余高频 API（LightDao 其余 71 个方法名、共 109 个声明，Link 链式 API、QueryExecutor/EntityQuery/EntityUpdate、既有配置参数）**均无变化**，升级成本主要集中在上面两处。

三、包结构调整（引用了框架内部类时需调整 import）

类	5.6.x 位置	6.0.x 位置
`PageOptimizeUtils`	`dialect.utils`	`dialect`
`ParallelUtils`	`utils`	`dialect.executor`
`QueryExecutorBuilder`	`utils`	`dialect`
`CrossDbAdapter`	`plugins`	`dialect`
`MacroIfLogic`	`utils`	`config`
`ParamFilterUtils`	`utils`	`config`（更名为 `ParamFilterProcessor`）
`TranslateUtils`	`utils`	`translate`
`HttpClientUtils`	`utils`	`plugins.nosql`
`MongoElasticUtils`	`utils`	`plugins.nosql`（更名为 `MongoElasticOperations`）
`SavePKStrategy`	`dialect.model`	`model`

四、新增能力

- **SAP HANA 方言**：方言总数达到 24 种，见支持的数据库；
- 内部模型补强：`ExecuteParams`、`ResultColumnMeta` 等（框架内部使用，不影响既有 API）。

> 更完整的版本演进细节可参阅源码仓库的 `sqltoy-orm-core/changelog.md`。

五、6.0.3 更新说明

6.0.3 在 6.0.2 基础上进一步增强，均为**增量能力，无需改动既有代码**：

SQL 方言体系

- `<sql>` 元素新增 `dialect` 属性：声明本条 sql 按指定方言形态解析固化，加载时按该方言做函数/保留字转换；执行期当前库方言与声明一致时跳过转换，不一致时惰性转换适配（`mql` / `eql` 不支持），详见方言自适配与函数扩展；
- 新增 `spring.sqltoy.realDialectFirst` 配置（默认 `false`）：开启后 sqlId 方言变体查找优先按**连接探测的真实方言**（`DBProfile.realDialect`，典型如 OceanBase 按 mysql 方言配置、代理库场景），真实方言变体不存在时回退原配置方言查找链。

POJO 生成 DDL（详见POJO生成表结构DDL）

- 新增 `@Partition` / `@PartitionDef` / `@MppTable` 实体注解：描述表分区策略与 MPP 分析库（ClickHouse/Doris/StarRocks）表引擎元数据，配合 quickvo 依据数据库分区/表定义自动产生；
- 建表语句自动渲染 `PARTITION BY` 分区子句（RANGE/LIST/HASH/RANGE COLUMNS/LIST COLUMNS 及分区明细）；
- 新增 Doris、StarRocks、ClickHouse、DB2、HANA、SQLite 专用 DDL 生成器（此前 Doris/StarRocks 复用 MySQL 生成器，其余落入默认生成器）；
- `@Foreign` 外键的 `deleteRestict` / `updateRestict` 支持完整参照动作（`ReferentialAction`：CASCADE/RESTRICT/SET_NULL/NO_ACTION/SET_DEFAULT），并按数据库兼容性自动裁剪；
- JSON 列类型按数据库降级承载（HANA→NCLOB、SQL Server→NVARCHAR(MAX)、DB2/Oracle11→CLOB）。

修复与优化

- 分库分表并行执行：`ShardingResult` 新增 `cause` 字段保留原始异常，全局回滚抛出的异常携带根因堆栈；`@Sharding.maxConcurrents` 语义修正（0 表示不限制、1 表示串行）；
- `executeSql` / `batchUpdateByJdbc` 手动提交场景失败时强制回滚，`autoCommit` 恢复移入 finally，避免异常连接带着被改写的提交方式归还连接池；
- 查询结果映射：`number-format` 格式化后的带千分位逗号数值（如 `8,888.0000`）再映射到数值属性时自动剥离分组逗号；
- sqlserver 分页 WITH 部分剥离置前、union 派生表包装拼接顺序修复。

六、6.0.4 更新说明

6.0.4 在 6.0.3 基础上进一步增强，同样为**增量能力，无需改动既有代码**：

缓存翻译 rest 模式认证增强（详见缓存翻译使用）

- `rest-translate` / `rest-checker` / `rest-increment-checker` 新增 `auth-type` 认证模式：`basic`（默认，兼容既有 `username` / `password`）与 `jwt`；
- jwt 获取 token 三种方式三选一（同配按优先级生效）：`token-url` 登录接口表单提交 username/password 获取 token（响应 json 按 `token-path` 提取，支持 `data.token` 点路径）并缓存、`token-secure-key` 本地自签名 jwt（支持 HS256/384/512 共享密钥与 RS256/384/512 的 PKCS#8 私钥，无需登录接口）、`authorization` 直接配置静态长效 token；
- 配套 `token-header`（默认 `Authorization`）、`token-prefix`（默认 `Bearer`，设为 `none` 发裸 token）、`token-expire`（默认 1800 秒，响应含 `expires_in` 时以响应为准）；token 缓存复用、临近过期自动刷新、请求遇 401 自动失效重登（或重新签名）并重试一次。

跨数据库函数适配完善（详见方言自适配与函数扩展）

- 新增源函数识别：`nvl2(a,b,c)`（非 oracle 系转 case when 判空）、单参 `isnull(x)`（mysql 返回 0/1 语义）、`sys_timestamp`（oracle 系透传、mysql 系保留小数秒精度、sqlserver 用 sysdatetime）、`strpos(str,sub)`（pg 系/clickhouse 透传，其余转 instr）、`substr(s,-n)` 负起点、`curdate()` / `CURRENT_DATE` 新函数类（oracle 系转 `TRUNC(CURRENT_DATE)`，db2 用 CURRENT DATE 关键字体，sqlite 取本地时钟）；
- `group_concat` / `listagg` / `string_agg` 支持 `within group (order by ...)` 尾部子句、参数内嵌 order by 与 DISTINCT 前缀识别；
- `datediff(month,...)` 月差口径统一为年月分量差（不再按 oracle MONTHS_BETWEEN 小数口径注册）；`to_char` 转 sqlserver 改以 FORMAT 承担，修复大写格式 token（如 `YYYY-MM-DD HH24:MI:SS`）的错值；sqlite 目标 `now()` / `CURRENT_DATE` / `CURRENT_TIMESTAMP` 统一取本地时钟。

修复

- 修复 SubStr 负起点与 mysql 契约偏离的静默错值（pg 系/sqlserver/h2/db2/hana 四处：起点算出小于 1 时统一返回空串）；
- 修复无分钟格式（如 `yyyy-MM-dd HH`）的独立 HH 时间 token 在 pg/oracle 系按 12 小时制解释导致分桶边界错乱的问题。

配套工具

- 配套的 quickvo-maven-plugin 升级至 **2.0.2**。

七、6.0.5 更新说明

6.0.5 在 6.0.4 基础上进一步增强，同样为**增量能力，无需改动既有代码**：

新增数据库支持（方言总数 24 → 25，详见支持的数据库）

- **瀚高 HighGo**：PG12 内核 + oracle 兼容层，SQL 生成层复用 PostgreSQL 系（`HighgoDialect extends PostgreSqlDialect`），差异集中在函数面（无 `uuid()` / `gen_random_uuid()` 但有 `sys_guid()`、`last_day` / `add_months` / `trunc` 原生、无 `listagg` 而 `string_agg` 原生），由函数适配器 HIGHGO 分支承担；同时支持官方驱动 URL `jdbc:highgo://` 与 vanilla pg 驱动（经 `version()` 内核探测区分）。

跨数据库函数适配完善（详见方言自适配与函数扩展）

- 新增 11 个函数类（均默认注册进 default）：`find_in_set`、`uuid`（唯一 id 生成）、`year/month/day` 分量提取、`last_day/eomonth`、`add_months`、`date_add/date_sub/dateadd` 日期区间加减、`left/right`、`lpad/rpad`、`extract/date_part`、`trunc/date_trunc` 日期截断、`mod` 取模；
- 既有函数源拼写扩充：`instr` 补 `locate/posstr`、`length` 补 `character_length/octet_length`、`now` 补 `current_timestamp`、`if` 补 `iif`、`substr` 补 `mid`、`to_date` 补 `str_to_date`；
- 均经多数据库真库探针验证，无法安全转换的形态保持"响亮保留"（原样交目标库报错），避免静默错值。

主键插入返回值转小写（PostgreSQL 系）

- 单行插入返回主键值时的主键列名大小写转换（`getReturnPrimaryKeyColumnCase`）由原仅 PostgreSQL/PostgreSQL14 扩展到 **PG 折叠家族**全量：OpenGauss、GaussDB、MogDB、Vastbase、StarDB、Oscar、Kingbase、HighGo 一并转小写。原因：`getGeneratedKeys` 列提示在 PG 折叠内核上大小写敏感（如 opengauss/vastbase/highgo 的 "ID"/"Id" 报 column does not exist、kingbase 混合大小写报错），转小写对全家族安全；mysql 系/tidb/oceanbase/sqlserver/db2/dm/oracle 维持 DEFAULT（驱动忽略提示或折叠一致）。仍可通过 `dialectReturnPrimaryColumnCase` 配置按方言覆盖。

其他优化与修复

- 修复串首裸函数永不转换的缺陷（整条 sql 即函数表达式时适配正则 `\W(函数名)\(` 恒不命中，入口加空格哨兵）；
- 补齐 HIGHGO/H2/SQLITE 的 TO_DATE/TO_TIMESTAMP cast 族处理。

八、6.0.6 更新说明

6.0.6 在 6.0.5 基础上进一步增强，同样为**增量能力，无需改动既有代码**：

虚表宏 @virtualTable()（详见方言自适配与函数扩展）

- 解决"select 常量行 union 真实查询"类 sql（字典下拉兜底行、报表固定合计行等）跨数据库无法统一编写的问题：oracle/db2/hana 必须挂伪表（`from dual` / `from sysibm.sysdummy1` / `from dummy`），而 mysql/pg/sqlserver 等原生支持无 from 的 select（写 `from dual` 反而语法错）；
- 统一写法：`select '0' id,'全局公共' name @virtualTable() union select id,name from t`，sql 解析期按方言自动展开——oracle 系（oracle/oracle11/dm/oceanbase）→ `from dual`、db2 → `from sysibm.sysdummy1`、hana → `from dummy`、其余方言直接剔除；
- 兼容别名 `@dual()` / `@dummyTable()`，三名等价、大小写不敏感、括号内空白容错；同一条 sql 可出现多次（每个常量 select 各写一个）；

- xml 配置 sql、`@Sql` 注解 sql、代码 sql（`findBySql` 等）同样有效（统一解析管线）；不含宏的 sql 零影响（含 `@` 的普通 sql 如 email 不受影响）；
- 展开发生在解析期并随 SqlToyConfig 固化（与函数/保留字转换同一时机），运行期零开销；`realDialectFirst` 场景宏按解析方言展开，运行期惰性层不再重复展开。

配套工具

- 配套的 quickvo-maven-plugin 升级至 **2.0.3**。

九、6.0.7 更新说明

6.0.7 在 6.0.6 基础上进一步增强，同样为**增量能力，无需改动既有代码**：

JSON 库可插拔（JsonProvider SPI）

- sqltoy 不再硬绑 fastjson2：JSON/JSONB 列映射、缓存翻译的 rest/service 路径、以及 Elasticsearch 全栈（含 jwt 认证）统一改为委托 `org.sagacity.sqltoy.plugins.json.JsonProvider`，内置 fastjson2 / jackson3（`tools.jackson`）/ jackson2（`com.fasterxml`）三个实现；
- 解析顺序为 `spring.sqltoy.jsonProvider` 显式配置 > classpath 自动探测 fastjson2 → jackson3 → jackson2。fastjson2 居首是为了让此前就跑通 JSON 列的存量项目行为完全不变；
- 配置取值支持内置别名 `fastjson2` / `jackson3` / `jackson2`（大小写、连字符、首尾空格不敏感），或自定义实现的全限定类名；
- 探测到的候选须先通过一次冒烟解析才被采纳——库在 classpath 上并不等于可用（例如 jackson3 的 databind 需要 `jackson-annotations` 2.20+，与 jackson2 共存时旧版本会在冲突仲裁中胜出并抛 `NoSuchFieldError: POJO`），不可用时记 WARN 并降级到下一个候选，marker 类不存在时静默跳过；
- 启动期即尝试初始化：显式配置非法或不可用时**直接失败并保留根因**（不静默回落，否则配置形同虚设）；仅缺可用 JSON 库时只记 WARN、不阻断启动——**不用 JSON 列与 ES 特性的项目现在无需引入任何 JSON 库**，触及 JSON 功能时才抛出带明确指引的异常，替代原先的 `NoClassDefFoundError`；
- core 的 jackson2 / jackson3 依赖均为 `provided + optional`，不传递给使用方；
- 详见配置说明与 Elasticsearch。

字段名列表拼接宏 @joinFields()（详见补充说明：标签与表达式参考）

- 面向页面配置化的报表与接口服务：由配置决定 `select` / `group by` / `order by` 的列。写法 `@joinFields(:fields,default)`，也可在 sql 里直写位置参数形态 `@joinFields(?,default)`——它与 `@value` 处于同一处理位置，命名参数转成 `?` 后两种写法完全等价，被消费的参数会从参数集合中剔除、不影响其余 `?` 的对位；
- `fields` 为 null、空集合或全空白时输出 `default`；`default` 取自 sql 文本（开发者编写，可信）原样输出、不做标识符校验，因此可写 `t.*` 或 `sum(t.amount)`，含逗号或括号时用 `{}`、`''` 或 `""` 包裹；未提供 `default` 则输出空串；
- 入参归一：单个字符串或长度为 1 的数组按逗号切割（页面多选常序列化成 `["a,b,c"]`），长度大于 1 时元素即字段、不做二次切割，因此 `["a,b","c"]` 会因 `"a,b"` 非法而报错，而不是被静默拆成三个字段；空白段跳过以容忍尾逗号；
- 安全：字段名走字符白名单（字母含中文、数字、下划线，最多三段点号限定 `schema.table.column`），显式排除 `$` 与 `#`（与自身的 `#[]` 动态段和 `${}` / `#{}` 参数占位冲突）以及空白、引号、括号、分号、逗号、注释符、反斜杠；非法值直接抛异常并带上具体值，不做静默清洗（经 DAO 层会被包装成 `DataAccessException`，捕获时取根因）；
- 合法字段按方言仅对**保留字**加引用符（mysql 系反引号、oracle/pg 系双引号、sqlserver 方括号），非保留字保持裸标识符，以规避 oracle/pg 上引号引入的大小写敏感；点号限定名逐段处理，`t.order` 的列段同样会被引用；
- **前提**：保留字引用依赖 `spring.sqltoy.reservedWords` 配置，而该配置**无默认值**；未配置时保留字集合为空、`convertWord` 原样返回不加引用符，字段列表含保留字（`order` / `desc` / `level` / `group` 等）会产出裸标识符并被数据库拒绝——PostgreSQL、MySQL 9、SQL Server 2025 三库实测均报语法错误；
- 只受理纯标识符，`sum(col)`、`col as alias` 这类表达式仍用 `@value`（安全自负）；运算符写法（`a % b`、`a || b`）不是函数调用，同样不在受理范围；
- 已在 PostgreSQL、MySQL 9、SQL Server 2025 三个真实数据库上完成端到端验证（普通列、default 回退、group by 聚合、保留字列、注入拦截）。

`#[]` 剔除后自动清理括号组内悬挂的 and/or

- 形如 `#[and ( #[t.a=:a] #[or t.b=:b] )]` 的写法，在 `a` 为 null 时既往会展开成 `and ( or t.b=:b )` 的残缺 SQL；现剔除完成后统一清理括号内 trim 后的、以 `and` / `or` 开头或结尾的悬挂连接符，使括号组各成员可独立可选（`and|or` 后紧跟 `\s+` 或 `)` 才命中，`( order by ...)` 这类以 `or` 开头的关键字不受影响），详见动态 SQL 编写规范。

十、6.0.8 更新说明

6.0.8 是 6.0 线里**变更最重的一个版本**：lambda 条件体系并入核心、GraalVM AOT 重写。**纯 JVM 运行的项目无需改动代码**；使用 sqltoy-plus 的项目按第 2 条迁移；构建 native 镜像的项目需按第 4 条检查构建环境。

1. lambda 条件体系并入核心（详见Lambda 条件查询与更新）

- 新增包 `org.sagacity.sqltoy.lambda`：`LambdaQuery` / `LambdaUpdate` / `LambdaDelete` + `JoinCol` / `ExcludeCols` / `LambdaFn` / `OuterRef` 等，以实体属性方法引用（`User::getStatus`）表达条件；
- `LightDao` 新增三个 default 工厂 `lambdaQuery/lambdaUpdate/lambdaDelete(Class)`，执行全部由构建器终端完成（`find/getOne/findPage/count/exists/update/delete`），也可 `LambdaQuery.create(entityClass).bind(lightDao)` 先构建后执行；

- **单表只是 join 的退化形态**：不调用 join 步骤即走单表通道（条件转 `EntityQuery` / `EntityUpdate`，复用 `findEntity` / `findPageEntity` / `getCount` / `deleteByQuery` / `updateByQuery` 链路，分片/缓存翻译/加解密脱敏/租户过滤全部继承）；调用 `leftJoin/rightJoin/innerJoin` + `on` 即切换 join 通道（生成完整 SQL 走 `find` / `findPage` / `getCount`，支持 `select/selectAll(ExcludeCols...)/JoinCol.of|avg|count|sum|min|max|distinct().as()`、CTE/ `@fast` 下推/UNION）；
- 行为差异（**需要留意**）：`getOne()` 严格语义——0 条返回 null、多于 1 条抛 `DataAccessException`（不再静默取第一条）；按条件 `update/delete` 无任何 where 时默认抛异常，需显式 `matchAll()` 放行；
- 条件参数名由 sqltoy-plus 的 `type.valuePairs.MPG1` 形式改为全构建器唯一且跨嵌套共享序号的 `:p1…:pN` ——**断言 SQL 文本的单元测试需同步**；`having` / `inSql` 的外部参数请避开 `p` + 纯数字形式；
- 值过滤由 sqltoy-plus 的全局静态策略改为**构建器实例级** `valueFilter(...)`（默认滤 null，须在添加条件之前设置）；
- `LambdaUpdate.setExpr(字段, 表达式[, value])` 支持基于字段当前值的表达式更新（如 `setExpr(User::getLoginCount, "loginCount + 1")`），复用 `updateByQuery` 既有的"字段=表达式"形态。

2. 自 sqltoy-plus 迁移清单

迁移点	动作
依赖	移除 `sqltoy-plus` 依赖（能力已内置 core），不再需要 `@EnableSqlToyPlus` 与映射策略 Bean
import	`com.sagframe.sagacity.sqltoy.plus.*` → `org.sagacity.sqltoy.lambda.*`
类名	`LambdaQueryWrapper` / `LambdaUpdateWrapper` → `LambdaQuery` / `LambdaUpdate`；`LambdaColumn` → `JoinCol`；`SFunction` → `LambdaFn`；`FiledNature` → `record FieldNature(entityClass, fieldName)`
DAO	`SqlToyPlusDao.findList/findOne/findPage/count/exists/delete/update(wrapper)` → 构建器终端；`SqlToyHelperDao` 已删除
入口	`Wrappers.lambdaQuery(cls)` → `LambdaQuery.create(cls)` 或 `lightDao.lambdaQuery(cls)`；`lambdaQueryChain/lambdaUpdateChain` → `lightDao.lambdaQuery/lambdaUpdate`
多表	`Wrappers.lambdaMultiWrapper(toClass).select(...).from(...)` → `lightDao.lambdaQuery(toClass)[.select(JoinCol...)].from(...).leftJoin(...).on(...)`；`lambdaUpdateMultiWrapper/lambdaDeleteMultiWrapper` → `lightDao.lambdaUpdate/lambdaDelete` + join 步骤
参数名	`type.valuePairs.MPG1` → `:p1…`
字符串 wrapper	`QueryWrapper` 等不再提供：单表字符串条件用 `EntityQuery`，复杂动态用 SQL 模式

> 更完整的字段级对照见 Lambda 条件查询与更新 · 九、自 sqltoy-plus 迁移对照。

3. 多表 join 更新与删除

join 更新/删除并入 `lambdaUpdate()/lambdaDelete()`（与查询同款"不调 join 即单表"判定）：join 更新生成 `UPDATE 主表 别名 JOIN ... SET 别名.字段=:p WHERE ...`、join 删除生成 `DELETE 别名[,...] FROM 主表 别名 JOIN ... WHERE ...`（`deleteTables` 缺省主表），均经 `executeSql` 执行。关系型库采用可移植的 EXISTS-over-derived 改写，已在 mysql/postgresql/sqlserver/oracle/db2 等真库验证；clickhouse/doris/starrocks 等无事务性行级 DML 能力的库不支持。**整对象并入更新不支持 join 通道**（用 `lightDao.update(entity)`）。

4. GraalVM native / AOT 重写（详见GraalVM Native Image 指引）

- 修复 5.6 以来 **AOT 实体反射 hints 恒为空**的缺陷：AOT 期 Spring 不注册 `BeanPostProcessor`、不预实例化单例，原先从 `getBean` 拿到的 `SqlToyContextProperties` 是未绑定的空实例，导致实体 hints 全丢；现改为经 `Binder` 从 `Environment` 直接绑定 `spring.sqltoy` 配置，按 `packagesToScan` / `annotatedClasses` 用 `EntityScanner` 扫描实体。**native 场景必须配置** `packagesToScan`；
- 按配置**动态注册**：资源 pattern（`sqlResourcesDir` 多目录/ `sqlResources` / `translateConfig` /密钥文件）、15 类扩展点反射（`unifyFieldsHandler`、`typeHandler`、`sqlInterceptors`、`sqlFormater`、`translateCacheManager`、`dataSourceSelector`、`connectionFactory`、`fieldsSecureProvider`、`desensitizeProvider`、`customFilterHandler`、`overTimeSqlHandler`、`dialectDDLGenerator`、`firstBizCodeTrace`、`dynamicCacheFetch`、`dynamicFecthCacheManager`）、经 `FunctionUtils.resolveFunctionClassNames` 解析后的 `functionConverts` 函数类、`lambda` 的 `SerializedLambda` /镜像类序列化，以及 XML 解析反射赋值的 8 个模型类（unpivot/环比/tree-sort 等标签属性，缺失时会**静默丢失**）；
- 按 **classpath 条件注册**（native 镜像构建期冻结，运行期无法再切换）：`cacheType` 对应的 caffeine/ehcache 管理器、mongo/redis 集成类、JSON 三个 provider、PG 系 `PGobject` 的 5 个驱动同构类（postgresql/opengauss/vastbase/gaussdb/kingbase）、mssql `Geometry` / `Vector`、`oracle.jdbc.OracleTypes`；
- 新增配置 `native-reflection-classes` / `native-reflection-packages`（登记非实体 DTO/VO）与 `native-entity-proxies`（构建期预生成 `updateSaveFetch` 代理，需应用自行引入 `net.bytebuddy:byte-buddy`）；
- starter 的 `sqlToyLazyDao` / `sqlToyDao` / `lightDao` / `sqlToyCRUDService` 四个 `@Bean` 返回类型由接口改为**具体实现类**并显式 `@ConditionalOnMissingBean(接口.class)`：Spring AOT 按声明类型预判 bean 类型，接口声明会使事务 CGLIB 代理无法在构建期预生成，native 运行期报 "CGLIB runtime enhancement not supported on native image"；应用侧如有"接口返回类型 + `initMethod`"的 `@Bean`，同样应改为具体类型声明；

- **构建环境要求 GraalVM 25+**（实测矩阵：17 与 23 不识别新版 reachability-metadata schema，被 native-maven-plugin 直接拒绝；21 不支持 `typeReached` 条件元数据，条件恒真导致 spring-aop 的 AspectJ 序列化条目被误注册；25.0.4 双框架全矩阵验证）；
- 已知限制：`updateSaveFetch` 的 `EntityUpdateCallback` 路径依赖 byte-buddy 运行期生成实体子类，native 下需配置 `native-entity-proxies` 预生成代理，未列出的实体不可用（insert 分支不受影响）。

5. 配套验证工程

新增 sqltoy-aot-verify（spring + solon 双栈共用矩阵，PostgreSQL pgvector + postgis）：覆盖 CRUD × json/vector/geometry 三形态、`functionConverts` 方言转换、ehcache/caffeine 双缓存翻译、`updateSaveFetch` 代理路径、lambda 单表/join/setExpr/matchAll 守卫/删除；JVM 双框架 17/17 PASS，配置 `native-entity-proxies` 后 native 冒烟 17 PASS + 0 KNOWN-LIMITATION。

十一、6.0.9 更新说明

6.0.9 以跨库实证修补为主，无破坏性变更，升级只需改版本号。

1. CTE 物化 hint 按目标库保留/剔除

`WITH xx AS [NOT] MATERIALIZED (..)` 仅 PostgreSQL 系（含 openGauss/金仓/Vastbase/GAUSSDB/瀚高）原生支持；其余库（mysql/sqlserver/oracle/db2 等）执行时框架在解析期自动剔除 hint——只失去物化优化、不改查询语义，无需再按方言各写一份 sqlId。lambda 侧 `.with(name, q, Boolean.TRUE/FALSE)` 的渲染门禁与之同源（详见Lambda 条件查询与更新）。

2. 函数适配真库实证修正

- `to_char` / `to_date` / `str_to_date` 的裸 `HH` 全库统一为 **24 小时制**口径（小写 `hh` 为 12 小时、`HH24` 恒为 24）——DB2/H2 真库实测原先按 12 小时输出致静默错值；
- `to_date` 目标为 pg 系/HANA/ClickHouse 且格式含时间 token 时自动改写 `to_timestamp` / `toDateTime`，规避时间部分被静默截断（真库实证 pg 的 `to_date` 只返回 date）；
- `trim(from x)`（省略剔除字符形态）在 mysql 9.3 报 1064、oracle 23ai 报 ORA-30000，自动改写为等价的 `trim(x)` / `ltrim` / `rtrim`；
- `substr` 起点 0 统一按"从开头截取"；sqlite 目标下 `mid` 自动转 `substr`（sqlite 原生无 mid）；mysql 的 `left/right(str,0)` 与负数长度按"返回空串"对齐；
- `instr` 支持四参形态 `instr(str,sub,pos,occurrence)`；
- `find_in_set` 比较符扩展为任意位置口径：`=1/=2/>1/<3` 按整数位置表达式改写（原 `>0/=0` 布尔口径族不变）；
- `datediff` 的 oceanbase 并入 oracle 分支（OB oracle 模式租户无 datediff；mysql 模式按约定配 `dialect=mysql`）。

3. 分页 count 缓存按方言隔离

`pageOptimize` 的总数缓存键由 sqlId 升级为 `sqlId@dialect`：同一 sql 经 `redoDataSources` /多数据源在不同方言库上执行时，count 结果分别缓存、不再互相串用；`PageOptimizeUtils.remove(sqlId)` 连带清除该 sql 全部方言缓存。配套修复跨库适配自定义 countSql 时直接改写共享 `SqlToyConfig` 实例、污染同 sqlId 后续查询的缺陷。

4. lambda 条件体系细化

- 新增 `likeExact/likeExactLeft/likeExactRight`：值中的 `%` 先转义为字面量再按方向补 `%`——用户输入含 `%` 时按普通字符匹配而非被当通配符放大命中范围（`like/likeLeft/likeRight` 语义不变）；
- `setExpr` 表达式引号内含字面量 `?` 时快速失败（更新通道做全量 `?` 文本替换，引号内 `?` 会被误当参数位）；
- 空 groupBy 列与空 having 条件跳过组装（原会生成 `GROUP BY` / `HAVING` 非法片段）。

5. 其他

- SqlScriptLoader 初始化/销毁线程安全加固（标志 volatile 化、方法 synchronized）。

十二、升级步骤建议

1. 依赖版本号统一升级为 6.0.9（starter / spring / solon-plugin 与 core 同版本）；
2. 全局搜索 `parallQuery` / `ParallQuery`，改为 `parallelQuery` / `ParallelQuery`（IDE 重命名即可）；
3. 如有自定义 `DataSourceCallbackHandler` 实现，把 `doConnection` 签名调整为 `DBProfile` 参数；
4. 如引用了上表所列框架内部工具类，按新包位置调整 import；
5. **使用 sqltoy-plus 的项目**：按第十节第 2 条清单迁移到 `org.sagacity.sqltoy.lambda`，移除 sqltoy-plus 依赖与 `@EnableSqlToyPlus`，并回归 `getOne()` 多处匹配、无条件 update/delete 两类行为变化；
6. **构建 native 镜像的项目**：确认 GraalVM 25+、配置 `packagesToScan`、按需登记 `native-reflection-classes/packages`，用到 `updateSaveFetch` 的实体加入 `native-entity-proxies`；
7. 启动验证：观察 debug 模式下 SQL 输出与业务回归（缓存翻译、分页、并行查询、lambda 条件优先）；断言 SQL 文本的单元测试需同步 `:p1…` 参数名。

从 MyBatis / MyBatis-Plus 迁移到 sqltoy

sqltoy 与 MyBatis 的**动态 SQL 心智完全一致**（SQL 写在 xml、条件按需拼），差别在于"条件怎么表达"和"对象操作有多强"。迁移通常是**渐进式**的：先让新写的查询走 sqltoy，再逐步改造旧 Mapper。

一、概念对照

MyBatis / MyBatis-Plus	sqltoy 对应	说明
Mapper XML 的 `<select id="findXxx">`	`*.sql.xml` 的 `<sql id="findXxx">`	文件按模块放，支持 debug 热加载
`<if test="x!=null"> and t.a=#{x}</if>`	`#[and t.a=:x]`	参数为 null 自动剔除，**SQL 能直接粘到客户端执行**
`<where>` / `<trim prefixOverrides>`	`where 1=1` + `#[]`	不需要 `<where>` 标签兜底
`#{}`	`:name` 命名参数	最终都转成 `?` 走 `PreparedStatement`
`${}`（字符串拼接）	`@value(:name)` / `@joinFields(:cols,default)`	标识符场景有白名单校验，见标签与表达式
`<foreach collection="ids">`	`t.id in (:ids)`	数组/集合自动展开，超 1000 自动切分
ResultMap + `<result column="A" property="a">`	结果类型（VO/POJO）+ 列别名 `AS a`	列名与属性名不一致用 `@SqlToyFieldAlias`
`<collection>` 嵌套映射	层次封装（hiberarchy）或按 VO 平铺	一条 join SQL 封装父子对象，见常规查询 API
分页插件（PageHelper 等）	`findPage`	自动 count 优化 + `@fast` + 并行分页
`@Select` 注解	`@Sql` 注解 / 代码内 SQL	复杂 SQL 仍建议放 xml
MP `QueryWrapper` / `LambdaQueryWrapper`	`EntityQuery` / `LambdaQuery`	前者字符串条件，后者属性引用
MP `BaseMapper` / `IService`	`LightDao` / `SqlToyCRUDService`	`LightDao` 约 110 个方法，见 API 速查
MP `@TableLogic` 逻辑删除	统一 SQL 片段 / 拦截器	sqltoy 不内置逻辑删除，用 `<sql>` 公共片段或 `SqlInterceptor` 统一加条件
MP `@Version` 乐观锁	`@DataVersion`	见数据版本控制
MP `@TableField(fill=...)`	`unifyFieldsHandler`	统一补漏赋值，见增加公共字段处理

二、落地顺序（建议按此推进）

1. **依赖与配置**：引入 `sagacity-sqltoy-spring-starter`，配 `sqlResourcesDir` + `debug=true`，见开发环境与依赖；
2. **建 SQL 目录**：按模块建 `*.sql.xml`（如 `com/acme/crm/modules/staff.sql.xml`），先把**新需求**写成 sqltoy 形式，不动旧 Mapper；
3. **生成 POJO**：用 quickvo 按现有表生成实体（注解齐全，别手写）；
4. **改造查询**（收益最大、风险最低）：从"条件最多、最难维护"的查询开始，按第三节对照改写；
5. **改造写入**：把"先 load 再改再 update"换成弹性更新；高并发台账类改为 `updateSaveFetch`（查增改一体）；
6. **灰度与校验**：新旧实现并行跑一段时间，对比结果集（sqltoy 的 `redoDataSources` 可帮你在另一套库上重放验证，见多库适配验证）。

三、典型改写

3.1 多条件动态查询

```
<!-- MyBatis -->
<select id="findOrder">
    select * from sqltoy_order_info t
    <where>
        <if test="beginDate!=null"> and t.trans_date &gt;= #{beginDate} </if>
        <if test="endDate!=null"> and t.trans_date &lt;= #{endDate} </if>
        <if test="status!=null and status!=''"> and t.status=#{status} </if>
        <if test="orderId!=null and orderId!=''"> and t.order_id=#{orderId} </if>
    </where>
</select>
```

```
<!-- sqltoy：条件与 SQL 不割裂，可直接粘到客户端跑 -->
<sql id="findOrder">
    <value><![CDATA[
        select * from sqltoy_order_info t
        where 1=1
        #[and t.trans_date>=:beginDate]
        #[and t.trans_date<=:endDate]
        #[and t.status=:status]
        #[and t.order_id=:orderId]
    ]]></value>
</sql>
```

`status!=''` 这类"空串也当没传"的诉求用 filter 统一处理，不必在每处判断：

```
<filters>
    <!-- 空白串转 null，从而剔除对应的 #[] 片段 -->
    <blank params="status,orderId" />
</filters>
```

3.2 like 查询

```
<!-- MyBatis: concat('%',#{name},'%') -->
<!-- sqltoy: 通配符由框架补 -->
#[and t.staff_name like :staffName]
<filters>
    <r-like params="staffName" />   <!-- 值后补 %：以 name 开头 -->
</filters>
```

3.3 集合 in 与空集合

```
#[and t.organ_id in (:organIds)]
```

- 传 `List`/数组即自动展开成多个 `?`；
- **空集合自动跳过该条件**（不会生成 `in ()` 语法错误）；
- 超过 1000 个值自动切分（Oracle 限制）。

3.4 分页

```
// MyBatis-Plus
Page<OrderInfo> page = orderMapper.selectPage(new Page<>(pageNo, pageSize), wrapper);
```

```
// sqltoy
Page<OrderInfoVO> page = lightDao.findPage(new Page(pageSize, pageNo),
        "crm_order_findlist", paramsMap, OrderInfoVO.class);
```

分页 SQL 里的 count 由框架改写（剔除 order by、裁剪 select 列）；大表 join 分页用 `@fast()`，见分页优化。

3.5 单表条件更新

```
// MyBatis-Plus
new LambdaUpdateWrapper<StaffInfo>()
        .set(StaffInfo::getStatus, 2)
        .eq(StaffInfo::getStaffId, id)
```

```
// sqltoy：单表用 lambda（复用实体链路，分片/翻译/租户都生效）
lightDao.lambdaUpdate(StaffInfo.class)
        .set(StaffInfo::getStatus, 2)
        .eq(StaffInfo::getStaffId, id)
        .update();

// 或字符串条件
lightDao.updateByQuery(StaffInfo.class,
        EntityUpdate.create().set("status", 2).where("staff_id=?").values(id));
```

3.6 先查后改 → 弹性更新 / 查增改一体（updateSaveFetch）

```
// MyBatis 常见写法：load → 判断 → update（两次交互，且可能覆盖并发修改）
OrderInfo o = mapper.selectById(id);
o.setStatus(2);
mapper.updateById(o);
```

```
// sqltoy 弹性更新：update 本身就是弹性的——一次交互，null 字段不参与
// （不会误清空别人改过的字段，也不用手拼 update set 片段）
OrderInfo orderInfo = new OrderInfo();
orderInfo.setOrderId(id);
orderInfo.setStatus(2);
lightDao.update(orderInfo);

// 高并发"查、增、改"：一次交互完成 锁查询 → 存在则修改、不存在则插入 → 返回结果
// uniqueProps 指定业务唯一键属性（不传则按主键检索）；回调中 entity 为代理对象，
// get 读数据库当前值、set 写回当前行，抛异常则整体回滚
OrderInfo entity = new OrderInfo();
entity.setOrderId(id);
entity.setStatus(2);
OrderInfo order = lightDao.updateSaveFetch(
        entity,
        (loaded, rowIndex) -> {
            if (!"1".equals(loaded.getStatus())) {
                throw new IllegalStateException("状态已变更，请刷新后重试");
            }
            loaded.setStatus(2);
        },
        "orderId");
```

> 若记录**必然存在**、且需自由编写查询条件与锁策略，则用 `updateFetch`（sql + `UpdateRowHandler`，见对象化 CRUD 第 10 节）；台账类"存在则改、不存在则插"优先 `updateSaveFetch`。

3.7 批量插入

```
// MyBatis: foreach 拼 values（SQL 长度随数据量爆炸）
// sqltoy: 批量接口，自动分批（batchSize 默认 200）
lightDao.saveAll(list);
lightDao.saveAllIgnoreExist(list);   // 已存在则跳过
```

四、常见坑

现象	原因与处理
SQL 里参数没生效	sqltoy 命名参数**大小写敏感**，且 `#[]` 内的参数名必须与传入 Map 的 key 完全一致
结果属性全为 null	列名与属性名不一致：加 `AS` 别名，或配置 `humpMapResultTypeLabel` / 用 `@SqlToyFieldAlias`
`in` 报语法错	用了字符串拼接而不是 `:ids`；空集合交给框架跳过即可
分页总数比数据少	用了 `@fast` + 外层 join 过滤，count 走内层 SQL；用 `<count-sql>` 显式指定，见 FAQ 十
动态 SQL 想拼标识符	不要用 `@value` 直拼用户输入；用 `@joinFields`（白名单）或 `<valid-sqlInjection>`
逻辑删除条件到处写	sqltoy 无内置逻辑删除，用公共 `<sql>` 片段 `@include` 或 `SqlInterceptor` 统一注入
事务行为不一样	对象操作沿用 Spring `@Transactional`；跨库/多数据源见多数据源与事务

五、迁移后能拿到什么

- **查询**：SQL 保持原生形态（可粘客户端调试）、智能 count、`@fast`、并行分页、缓存翻译免 join；
- **写入**：弹性更新 / `updateFetch` / `updateSaveFetch` 把"多次交互 + 手工加锁"变成一个方法调用；
- **跨库**：25 种方言 + 函数自动替换，同一份 SQL 跑多个库；
- **数据分析**：pivot/unpivot、同比环比、树形汇总在内存完成，不必为每个数据库写不同 SQL。

> 相关：必杀集锦（sqltoy 与 MyBatis 写法对比）、功能清单、API 速查

开发环境与依赖

一、环境要求

项	要求
JDK	**17+**（6.0 线要求 JDK 17，面向 Spring Boot 3/4）；JDK 8 项目请使用 `5.6.96.jre8`（最终 jre8 版本，支持 quickvo-maven-plugin 插件）
构建工具	Maven 3.6+ 或 Gradle
数据库	任意 JDBC 数据库（MySQL/Oracle/PostgreSQL/达梦/高斯/SQL Server/DB2 等，详见支持的数据库）
连接池	Druid、HikariCP 等任意连接池（sqltoy 不绑定连接池）

> **注意**
>
> 版本号请以 Maven Central 最新版为准，本文以 `6.0.9` 为例。

二、引入依赖

根据所用框架选择对应 artifact（groupId 均为 `com.sagframe`）：

Spring Boot（推荐，自动配置）

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-spring-starter</artifactId>
    <version>6.0.9</version>
</dependency>
```

传统 Spring（XML/Java 配置）

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-spring</artifactId>
    <version>6.0.9</version>
</dependency>
```

Solon

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-solon-plugin</artifactId>
    <version>6.0.9</version>
</dependency>
```

纯 Java：直接引入核心包 `sagacity-sqltoy` 即可，参见纯 Java 项目如何使用。

三、连接池与数据源

- 演示项目使用 **HikariCP** 连接池（跑在 H2 内存库上，见快速开始），但 sqltoy 并不限定连接池，按实际情况选配即可。
- Spring Boot 默认连接池是 **HikariCP**，需引入 `spring-boot-starter-jdbc` 才会自动初始化数据源。
- 务必**先配置好 dataSource 及连接池**，再配置 sqltoy。

四、缓存翻译依赖（可选）

如使用缓存翻译功能，需引入缓存实现。sqltoy 默认支持 **ehcache**，也可选配 **caffeine**：

```
<dependency>
    <groupId>org.ehcache</groupId>
    <artifactId>ehcache</artifactId>
    <version>3.12.0</version>
    <exclusions>
        <exclusion>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

> 为什么默认不用 Redis 做翻译缓存？因为缓存翻译追求**本地零网络开销**的高频码值转换，本地缓存（ehcache/caffeine）性能更佳；超大规模主数据场景可用 FIFO 动态缓存。

五、下一步

- 完整的建项目→建表→生成 POJO→写查询流程，见 helloworld 快速上手。
- POJO/VO 由 **quickvo** 工具根据数据库表自动生成，见 quickvo 代码生成工具。

quickvo 代码生成工具

sqltoy 的 POJO/VO **不靠运行时反射猜测**，而是由 **quickvo** 工具连接数据库、根据表结构生成带注解的实体类。这样既保证对象与表精确对应，又让 SQL 编写时有完整的属性提示。

- 下载地址（releases）：https://gitee.com/sagacity/sagacity-quickvo/releases
- 完整配置范例参见 `sqltoy-quickstart` 项目 `tools` 目录下的 quickvo 配置。

一、工作原理

quickvo 是一个可执行 jar（`sagacity-quickvo.jar`），通过 `quickvo.bat`（Windows）或 `quickvo.sh`（Linux/Mac）启动：

```
:: quickvo.bat 核心语句
java -cp ./libs/* org.sagacity.quickvo.QuickVOStart quickvo.xml
```

- 将数据库驱动 jar 放入 `libs` 目录，quickvo 会自动加载驱动；
- 读取 `quickvo.xml` 连接数据库，利用 freemarker 模板生成 VO/POJO 类；
- Linux/Mac 上编写 `quickvo.sh` 时注意路径分隔符（反斜杠→正斜杠）。

二、quickvo.xml 配置要点

> `quickvo.xml` 的 schema 指向与 IDE 补全配置见 XML Schema 配置指引。

`quickvo.xml` 主要由 `property`（参数）、`datasource`（数据库）、`tasks/task`（生成任务）等部分组成。下面为独立运行时的结构示意（Maven 插件方式的完整配置见 quickvo-maven-plugin）：

```
<?xml version="1.0" encoding="UTF-8"?>
<quickvo xmlns="http://www.sagframe.com/schema/quickvo">
    <!-- property: 全局参数或参数文件，之后可用 ${} 引用 -->
    <property file="src/main/resources/application.yml" />
    <property name="project.package" value="com.demo" />

    <!-- datasource: 数据库连接信息（可直接写值，也可用 ${} 引用 property） -->
    <datasource name="demo" url="jdbc:mysql://localhost:3306/demo"
                driver="com.mysql.cj.jdbc.Driver" schema="demo"
                username="root" password="***" />

    <!-- tasks: dist 生成路径; task 用 include 正则匹配表; entity 生成 pojo, vo 生成 dto/vo -->
    <tasks dist="src/main/java" encoding="UTF-8">
        <task datasource="demo" author="dev" include="^SQLTOY_\w+" active="true">
            <entity package="${project.package}.entity" substr="Sqltoy" name="#{subName}" lombok-chain="true" />
            <vo package="${project.package}.dto" substr="Sqltoy" name="#{subName}VO" lombok-chain="true" />
        </task>
    </tasks>

    <!-- 还可配置: primary-key(主键策略)、cascade(主子表级联)、type-mapping(类型映射)、swagger(v3/v2/false) -->
</quickvo>
```

关键配置说明：

配置	说明
`property`	全局参数或参数文件（`file` 引入外部配置，`name` / `value` 定义参数）
`datasource`	数据库连接；**多实例库务必正确配置 `schema` 或 `catalog`**（见下方注意事项）
`tasks` / `task`	`dist` 生成路径、`encoding` 编码；`task` 用 `include` 正则匹配表名、`active` 是否启用，可配多个 task 生成到不同包
`entity` / `vo`	分别生成 POJO 与 DTO/VO；`package` 目标包、`substr` 去表名前缀、`name` 类名模板（`#{subName}`）、`lombok-chain` 链式 set
`swagger`	生成 swagger 注解：`v3` / `v2` / `false`（不生成）
`field.support.linked.set`	开关：生成的 VO 中 set 方法是否返回自身（链式）
`primary-key`	表主键策略；identity 类型无需配置。quickvo 会按主键类型/长度自动设置：`bigint`→snowflake，22 位字符→DefaultIdGenerator，26 位→NanoTimeIdGenerator
`cascade`	级联操作配置：load 级联（如只加载有效子表）、update 级联（delete 无需配置，按主外键直接删子表）、`orderBy`（加载子表排序，如 `showIndex desc`）
`type-mapping`	字段类型映射；`import-types` 可额外指定 VO 的 import 类型。若类型匹配不上，quickvo 会在日志中打印提示，据此补充 type-mapping

> quickvo 5.0.1 起 entity 支持 **lombok**，并支持自定义 api-doc 框架。

三、生成结果与注解

一张表会生成 **AbstractVO + VO** 两个类，VO 继承 AbstractVO。sqltoy 通过注解解析 VO：

- `@Entity(tableName="sqltoy_dict_detail", pk_constraint="PRIMARY")`：对应具体表（`org.sagacity.sqltoy.config.annotation.Entity`）；
- `@SqlToyEntity`：标注 VO 检索时的精确定位——找到 `@SqlToyEntity` 后再找其父类，从而定位到真正与表对应的类，并检索 `@Id`、`@Column` 形成对象与表的完整映射；
- `@Id`、`@Column`：主键与字段映射；
- `@PartitionKey`：分区字段标记（MPP 库建表与 DML 的分区键提示）；分区表还会在实体上生成 `@Partition` / `@PartitionDef`（分区策略与分区明细），ClickHouse/Doris/StarRocks 表生成 `@MppTable`（表引擎元数据）——这三者配合 sqltoy 6.0.4 的 POJO 生成 DDL 功能（见POJO生成表结构DDL），旧版 quickvo 可手工标注。

quickvo 会把数据库表/字段注释一并写入 VO，便于阅读维护。

> **警告**
>
> 请勿修改 VO 中"标记区间"内的内容（重新生成会覆盖）。如需扩展属性或 get/set 方法，请在该区间**之外**添加。

四、关键注意事项

多实例数据库下字段重复/错乱：当多个实例库中存在同名表（如 `SYS_STAFF_INFO`）时，生成的 VO 可能出现字段重复或混入其他库的字段。

解决办法：**正确配置数据源的 `schema` 或 `catalog`**（一般配置 `schema` 即可）。底层取表信息、取列信息都依赖该配置，配置错误会取到不属于当前库的表/字段。

五、严格 VO(DTO) 与 POJO(entity) 分层

中大型项目建议将对外交互的 VO/DTO 与持久层 POJO 严格分层，通过 quickvo 配置生成到不同包，并在业务中做 VO↔POJO 转换（参见工具类：DTO 与 POJO 互转）。

- 分层示例项目：https://github.com/sagframe/sqltoy-strict

六、相关

- 把 quickvo 集成进 Maven 构建，见 quickvo-maven-plugin。
- 主键策略详情见主键策略介绍。

quickvo-maven-plugin

除了独立运行 quickvo（见 quickvo 代码生成工具），更推荐用 **quickvo-maven-plugin** 把代码生成集成进 Maven 构建，一条命令即可根据数据库表生成/更新 POJO 与 VO。

> **注意**
>
> 当前版本 **2.0.4**；自 2.0.0 起对之前的版本进行了大幅重构，并针对 19 种数据库（oracle、db2、sqlserver、mysql、postgresql、hana、kingbase、dm、opengauss、oceanbase、doris、tidb、starrocks、clickhouse 等）进行了测试验证。完整功能清单与配置说明见插件仓库 README。

- 项目地址：https://gitee.com/sagacity/maven-quickvo-plugin （完整配置参见其 README）

一、在 pom.xml 中配置插件

```
<plugin>
    <groupId>com.sagframe</groupId>
    <artifactId>quickvo-maven-plugin</artifactId>
    <version>2.0.4</version>
    <configuration>
        <configFile>./src/main/resources/quickvo.xml</configFile>
        <baseDir>${project.basedir}</baseDir>
    </configuration>
    <dependencies>
        <!-- 放入对应数据库驱动 -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>${mysql.version}</version>
        </dependency>
    </dependencies>
</plugin>
```

二、编写 quickvo.xml

在 `src/main/resources` 下创建 `quickvo.xml`。它可以直接复用项目的 `application.yml` 中的数据源配置（通过 `<property file>` 引入 + `${}` 引用）：

```
<?xml version="1.0" encoding="UTF-8"?>
<quickvo xmlns="http://www.sagframe.com/schema/quickvo"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.sagframe.com/schema/quickvo https://sagframe.github.io/schema/quickvo.xsd">
    <!-- 引入 db 配置文件，后续可用 ${} 引用其中的值 -->
    <property file="src/main/resources/application.yml" />
    <property name="project.version" value="1.0.0" />
    <property name="project.name" value="sqltoy-helloworld" />
    <!-- 项目默认包路径 -->
    <property name="project.package" value="com.sqltoy.helloworld" />

    <!-- 数据库定义，这里也可直接写具体值 -->
    <datasource name="helloworld" url="${spring.datasource.url}"
        driver="com.mysql.cj.jdbc.Driver" schema="helloworld"
        username="${spring.datasource.username}" password="${spring.datasource.password}" />

    <!-- dist: 生成代码存放路径(相对 pom 中 baseDir); encoding: 文件编码 -->
    <tasks dist="src/main/java" encoding="UTF-8">
        <!-- 可配置多个 task，把 pojo 生成到不同包路径; include 用正则匹配表名 -->
        <task datasource="helloworld" author="zhongxuchen" include="^SQLTOY_\w+" active="true">
            <!-- entity 生成 pojo; substr 去掉表名前缀; name 用 #{subName} 占位; lombok-chain 生成链式 set -->
            <entity package="${project.package}.entity" substr="Sqltoy" name="#{subName}" lombok-chain="true" />
            <!-- vo 生成 dto/vo -->
            <vo package="${project.package}.dto" lombok-chain="true" substr="Sqltoy" name="#{subName}VO" />
        </task>
    </tasks>
</quickvo>
```

> **零安装变体**：演示项目 sqltoy-helloworld 的 datasource 直接指向内存 H2，url 携带 `INIT=RUNSCRIPT` 在连接建立时执行建表脚本，配合插件依赖里的 h2 驱动，`mvn quickvo:quickvo` 无需任何数据库即可体验；对着真实库生成时换回上面的 `${spring.datasource.url}` 引用形态即可。

```
<datasource name="helloworld" url="jdbc:h2:mem:quickvogen;INIT=RUNSCRIPT FROM 'src/main/resources/mock/h2/schema-h2.sql'"
    driver="org.h2.Driver" schema="PUBLIC" catalog="" username="sa" password="" />
```

常用配置说明：

配置	说明
`<property file>`	引入外部配置文件（如 application.yml），之后可用 `${key}` 引用
`<datasource>`	数据库连接；`schema` / `catalog` 在多实例库下务必正确配置
`<tasks dist encoding>`	生成代码的根路径与编码
`<task include active>`	`include` 用正则匹配要生成的表（如 `^SQLTOY_\w+`），`active` 是否启用
`<entity>` / `<vo>`	分别生成 POJO 与 DTO/VO；`package` 目标包，`substr` 去除表名前缀，`name` 类名模板（`#{subName}` 为去前缀后的表名），`lombok-chain` 生成链式 set

> 主键策略（`primary-key`）、级联（`cascade`）、类型映射（`type-mapping`）、swagger 注解等更多配置项，参见 quickvo 代码生成工具 与官方 README。

三、执行生成

在项目根路径下执行：

```
mvn quickvo:quickvo
```

执行后：

- `src/main/java/com/sqltoy/helloworld/dto/` 下生成 `OrderInfoVO.java`；
- `src/main/java/com/sqltoy/helloworld/entity/` 下生成 `OrderInfo.java`，包含 `@Entity`、`@Id`、`@Column` 等描述对象与表关系的注解。

生成的 POJO 形如：

```
@Data
@Accessors(chain = true)
@Entity(tableName="sqltoy_order_info", comment="sqltoy订单信息演示表", pk_constraint="PRIMARY")
public class OrderInfo implements Serializable {
    private static final long serialVersionUID = 7200696852961513069L;
/*---begin-auto-generate-don't-update-this-area--*/
    /** 订单编号 */
    @Id(strategy="generator", generator="org.sagacity.sqltoy.plugins.id.impl.NanoTimeIdGenerator")
    @Column(name="ORDER_ID", comment="订单编号", length=32L, type=java.sql.Types.VARCHAR,
            nativeType="VARCHAR", nullable=false)
    private String orderId;
    // ... 其余字段
/*---end-auto-generate-don't-update-this-area--*/
}
```

> **警告**
>
> `begin-auto-generate` 与 `end-auto-generate` 之间的内容由 quickvo 自动生成，**请勿手工修改**（重新生成会覆盖）；扩展属性/方法请写在区间之外。

完整建项目→生成→查询流程见 helloworld 快速上手。

学习sqltoy-orm的基本原则

- 不要带入mybatis(plus)等开源项目的使用惯性思维
- 用显式逻辑和直截了当的思维来看待和学习sqltoy-orm
- 看到一些跟以往项目模式有些许差异的地方，不要急于否定

快速搭建sqltoy项目的步骤

> 本文与演示项目 sqltoy-helloworld 完全同步：该项目默认运行在 **H2 内存库**上（建表与种子数据自动加载），clone 后 `mvn test` 零安装即可运行（9 个演示用例全绿）。

1、创建一个springboot项目，并配置好数据源

- 参见:sqltoy演示项目sqltoy-helloworld

```
@SpringBootApplication
@ComponentScan(basePackages = { "com.sqltoy.helloworld" })
@EnableTransactionManagement
public class SqlToyApplication {
    /**
     * @param args
     */
    public static void main(String[] args) {
        SpringApplication.run(SqlToyApplication.class, args);
    }
}
```

- 数据库连接池使用hikari(可以根据自己情况自行选择)

```
<!-- spring自带的数据库连接池(版本由 spring-boot-starter-parent 管理) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<!-- 零安装内存库:演示项目默认跑在 H2 上 -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
</dependency>
```

- 配置application.yml(H2 零安装形态,与演示项目一致)

```
spring:
    # 零安装:默认使用 H2 内存库,建表与种子数据由 spring.sql.init 自动加载(mock/h2/)
    datasource:
       name: dataSource
       type: com.zaxxer.hikari.HikariDataSource
       driver-class-name: org.h2.Driver
       username: sa
       password: ""
       isAutoCommit: false
       # 随机库名:每个 Spring 上下文独立,测试互不干扰
       url: jdbc:h2:mem:helloworld-${random.uuid};DB_CLOSE_DELAY=-1
    # 演示表结构 + 种子数据(H2 版 DDL 见第 3 步)
    sql:
       init:
         mode: always
         schema-locations: classpath:mock/h2/schema-h2.sql
         data-locations: classpath:mock/h2/data-h2.sql
    #将sqltoy 放于开始位置，避免很多开发者忽视了spring.sqltoy 开头，变成了sqltoy开头导致无法加载
    sqltoy:
       #非必选项，设置sql.xml文件存放路径(会自动向下扫描,多路径范例:classpath:com/sqltoy/helloworld,classpath:com/sqltoy/system
       sqlResourcesDir: classpath:com/sqltoy/helloworld
       # 默认为false，debug模式将打印执行sql,并自动检测sql文件更新并重新加载
       debug: true
```

- 切换 MySQL 等真实库:替换 datasource 段并把 `sql.init.mode` 改为 `never`(建表参照第 3 步的 MySQL DDL)

```
spring:
    datasource:
       name: dataSource
       type: com.zaxxer.hikari.HikariDataSource
       driver-class-name: com.mysql.cj.jdbc.Driver
       username: helloworld
       password: helloworld
       isAutoCommit: false
       url: jdbc:mysql://127.0.0.1:3306/helloworld?useUnicode=true&characterEncoding=utf-
8&serverTimezone=GMT%2B8&useSSL=false&allowPublicKeyRetrieval=true
```

工程实践：推荐把 `*.sql.xml` 放在代码目录（而非 resources），与 VO 同模块存放。好处：① 开发者在自己模块下就能完成 sql 编写与维护，减少目录切换；② 便于整个模块连同 sql 一起剥离复用，向产品化模块演进。这需要在 pom.xml 中把 java 目录下的 xml 纳入资源打包：

```
<resources>
    <resource>
        <directory>src/main/java</directory>
        <excludes>
            <exclude>**/*.java</exclude>
        </excludes>
        <includes>
            <include>**/*.xml</include>
        </includes>
    </resource>
    <resource>
        <directory>src/main/resources</directory>
        <includes>
            <include>**/*.xml</include>
            <include>**/*.properties</include>
            <include>**/*.yml</include>
            <!-- H2 建表/种子数据脚本 -->
            <include>**/*.sql</include>
        </includes>
    </resource>
</resources>
```

2、在pom.xml中引入sqltoy-orm-spring-starter

- springboot场景

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-spring-starter</artifactId>
    <!-- JDK8 请使用 5.6.96.jre8（jre8 最终版本，同样支持 quickvo-maven-plugin 插件） -->
    <version>6.0.9</version>
</dependency>
<!-- 使用缓存翻译(见《缓存翻译使用》)需要缓存实现,sqltoy 默认支持 ehcache(runtime 作用域,打包进 boot jar) -->
<dependency>
    <groupId>org.ehcache</groupId>
    <artifactId>ehcache</artifactId>
    <scope>runtime</scope>
</dependency>
```

- solon场景

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-solon-plugin</artifactId>
    <!-- JDK8 请使用 5.6.96.jre8（jre8 最终版本，同样支持 quickvo-maven-plugin 插件） -->
    <version>6.0.9</version>
</dependency>
```

3、创建表:sqltoy_order_info

- H2 零安装形态:DDL 放在 `src/main/resources/mock/h2/schema-h2.sql`,由 `spring.sql.init` 启动时自动执行(演示项目还包含缓存翻译用的机构表/字典表及种子数据 `data-h2.sql`):

```
DROP TABLE IF EXISTS SQLTOY_ORDER_INFO;
CREATE TABLE SQLTOY_ORDER_INFO (
    ORDER_ID     VARCHAR(32) NOT NULL,
    ORDER_TYPE   VARCHAR(32),
    PRODUCT_CODE VARCHAR(32),
    UOM          VARCHAR(30),
    PRICE        DECIMAL(24, 6),
    QUANTITY     DECIMAL(24, 6),
    TOTAL_AMT    DECIMAL(24, 6),
    STAFF_CODE   VARCHAR(32),
    ORGAN_ID     VARCHAR(32),
    STATUS       INT,
    CREATE_BY    VARCHAR(32),
    CREATE_TIME  TIMESTAMP,
    UPDATE_BY    VARCHAR(32),
    UPDATE_TIME  TIMESTAMP,
    CONSTRAINT PK_ORDER_INFO PRIMARY KEY (ORDER_ID)
);
COMMENT ON TABLE SQLTOY_ORDER_INFO IS 'sqltoy订单信息演示表';
COMMENT ON COLUMN SQLTOY_ORDER_INFO.ORDER_ID IS '订单编号';
-- ... 其余列注释见演示项目 schema-h2.sql
```

- MySQL 等真实库形态:

```
DROP TABLE IF EXISTS SQLTOY_ORDER_INFO;
CREATE TABLE SQLTOY_ORDER_INFO(
    `ORDER_ID` VARCHAR(32) NOT NULL  COMMENT '订单编号' ,
    `ORDER_TYPE` VARCHAR(32)   COMMENT '订单类别' ,
    `PRODUCT_CODE` VARCHAR(32)   COMMENT '商品代码' ,
    `UOM` VARCHAR(30)   COMMENT '计量单位' ,
    `PRICE` DECIMAL(24,6)   COMMENT '价格' ,
    `QUANTITY` DECIMAL(24,6)   COMMENT '数量' ,
    `TOTAL_AMT` DECIMAL(24,6)   COMMENT '订单总金额' ,
    `STAFF_CODE` VARCHAR(32)   COMMENT '销售员' ,
    `ORGAN_ID` VARCHAR(32)   COMMENT '销售部门' ,
    `STATUS` INT   COMMENT '订单状态' ,
    `CREATE_BY` VARCHAR(32)   COMMENT '创建人' ,
    `CREATE_TIME` DATETIME   COMMENT '创建时间' ,
    `UPDATE_BY` VARCHAR(32)   COMMENT '更新人' ,
    `UPDATE_TIME` DATETIME   COMMENT '更新时间' ,
    PRIMARY KEY (ORDER_ID)
)  COMMENT = 'sqltoy订单信息演示表';
```

4、配置sqltoy生成pojo、dto的maven插件quickvo-maven-plugin

- pom.xml中加入quickvo-maven-plugin

```
<plugin>
    <groupId>com.sagframe</groupId>
        <artifactId>quickvo-maven-plugin</artifactId>
        <version>2.0.4</version>
    <configuration>
        <configFile>./src/main/resources/quickvo.xml</configFile>
        <baseDir>${project.basedir}</baseDir>
    </configuration>
    <dependencies>
        <!-- H2 驱动:quickvo 经 INIT=RUNSCRIPT 自举内存表结构,零安装即可逆向生成 -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <version>2.4.240</version>
        </dependency>
        <!-- 对着真实库生成时补对应驱动,如 mysql -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>${mysql.version}</version>
        </dependency>
    </dependencies>
</plugin>
```

- 在src/main/resources下面创建quickvo.xml(与演示项目一致,H2 自举形态) 有关quickvo.xml的配置，详细请参见:quickvo-maven-plugin

```
<?xml version="1.0" encoding="UTF-8"?>
<quickvo xmlns="http://www.sagframe.com/schema/quickvo"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.sagframe.com/schema/quickvo https://sagframe.github.io/schema/quickvo.xsd">
    <property name="project.version" value="1.0.0" />
    <property name="project.name" value="sqltoy-helloworld" />
    <!-- 定义项目的默认包路径 -->
    <property name="project.package" value="com.sqltoy.helloworld" />

    <!-- 默认指向内存 H2:url 中的 INIT=RUNSCRIPT 在连接建立时用 mock/h2/schema-h2.sql 建表。
         对着自己的真实库生成时:改 url/driver/schema/username/password(也可用 <property file>
         引入 application.yml 后以 ${spring.datasource.url} 引用) -->
    <datasource name="helloworld" url="jdbc:h2:mem:quickvogen;INIT=RUNSCRIPT FROM 'src/main/resources/mock/h2/schema-h2.sql'"
        driver="org.h2.Driver" schema="PUBLIC" catalog="" username="sa" password="" />

    <!-- dist 定义生成的java代码存放路径,相对于pom中baseDir -->
    <tasks dist="src/main/java" encoding="UTF-8">
        <!-- 可以设置多个任务便于将pojo生成到不同包路径下 -->
        <task datasource="helloworld" author="zhongxuchen"  include="^SQLTOY_\w+" active="true">
            <entity package="${project.package}.entity" substr="Sqltoy" name="#{subName}" lombok-chain="true" />
            <vo package="${project.package}.dto" lombok-chain="true" substr="Sqltoy" name="#{subName}VO" />
        </task>
    </tasks>

    <!-- 类型映射:H2 的 TIMESTAMP 默认生成 java.sql.Timestamp,映射为 LocalDateTime
         与公共字段赋值处理器(SqlToyUnifyFieldsHandler 赋 LocalDateTime 值)保持一致 -->
    <type-mapping>
        <sql-type native-types="TIMESTAMP" jdbc-type="TIMESTAMP" java-type="java.time.LocalDateTime" />
    </type-mapping>
</quickvo>
```

5、执行quickvo，生成pojo和dto

- 在项目根路径下执行:mvn quickvo:quickvo
- 在src/main/java/目录下com.sqltoy.helloworld.dto包下面会生成OrderInfoVO.java
- 在src/main/java/目录下com.sqltoy.helloworld.entity包下面会生成OrderInfo.java
  会包含:@Entity、@Id、@Column 等描述对象跟数据库表关系的注解

```
@Data
@Accessors(chain = true)
@Entity(tableName="sqltoy_order_info",comment="sqltoy订单信息演示表",pk_constraint="PRIMARY")
public class OrderInfo implements Serializable {

    /**
     *
     */
    private static final long serialVersionUID = 7200696852961513069L;
/*---begin-auto-generate-don't-update-this-area--*/

    /**
     * 订单编号
     */
    @Id(strategy="generator",generator="org.sagacity.sqltoy.plugins.id.impl.NanoTimeIdGenerator")
    @Column(name="ORDER_ID",comment="订单编号",length=32L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String orderId;

    /**
     * 订单类别
     */
    @Column(name="ORDER_TYPE",comment="订单类别",length=32L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=true)
    private String orderType;

    // ... 其余字段省略(price/quantity/totalAmt/staffCode/organId/status 及审计字段,
    //     createTime/updateTime 经 type-mapping 生成为 LocalDateTime)
/*---end-auto-generate-don't-update-this-area--*/
}
```

> `begin-auto-generate` 与 `end-auto-generate` 之间的内容由 quickvo 自动生成,重新生成会覆盖;扩展属性/方法(如 VO 上的 `@Translate` 翻译属性)请写在区间之外——演示项目的 OrderInfoVO 正是这样扩展的。

6、创建一个service和单元测试类

- 1、创建OrderInfoService接口

```
package com.sqltoy.helloworld.service;

import java.util.Map;

import org.sagacity.sqltoy.model.Page;
import com.sqltoy.helloworld.dto.OrderInfoVO;

public interface OrderInfoService {
    /**
     * 创建订单
     *
     * @param orderInfoVO
     */
    public void createOrderInfo(OrderInfoVO orderInfoVO);

    /**
     * 分页查询订单信息
     *
     * @param pageModel
     * @param queryMap
     * @return
     */
    public Page<OrderInfoVO> searchOrderInfo(Page pageModel, Map queryMap);
}
```

- 2、编写OrderInfoService实现类OrderInfoServiceImpl

```
/**
 * 订单服务逻辑实现
 *
 * @author zhongxuchen
 * @date 2025/2/5
 */
@Service("orderInfoService")
public class OrderInfoServiceImpl implements OrderInfoService {
    // 注入sqltoy框架自带的LightDao
    @Autowired
    LightDao lightDao;

    // 所有单表操作对象化完成，类似jpa
    @Override
    @Transactional
    public void createOrderInfo(OrderInfoVO orderInfoVO) {
        // 调用框架自带的dto<-->pojo 映射方法创建Entity实例
        OrderInfo orderInfoEntity = lightDao.convertType(orderInfoVO, OrderInfo.class);
        // 调用save完成保存
        lightDao.save(orderInfoEntity);
    }

    // 复杂查询通过sql.xml 定义具体sql内容
    @Override
    public Page<OrderInfoVO> searchOrderInfo(Page pageModel, Map queryMap) {
        String sql = """
                select * from SQLTOY_ORDER_INFO t
                where 1=1
                #[and t.status in (:statusAry)]
                #[and t.create_time>=:beginTime]
                #[and t.create_time<=:endTime]
                """;
        return lightDao.findPage(pageModel, sql, queryMap, OrderInfoVO.class);
    }

}
```

- 3、编写单元测试类OrderInfoServiceTest(与演示项目一致,含断言)

```
@SpringBootTest
public class OrderInfoServiceTest {
    @Autowired
    OrderInfoService orderInfoService;

    @Autowired
    LightDao lightDao;

    @Test
    public void testCreateOrderInfo() {
        String productCode = "P-DEMO-" + System.nanoTime();
        OrderInfoVO orderInfoVO = new OrderInfoVO();
        orderInfoVO.setOrderType("PO");
        orderInfoVO.setOrganId("100007");
        orderInfoVO.setProductCode(productCode);
        orderInfoVO.setPrice(BigDecimal.valueOf(100));
        orderInfoVO.setQuantity(BigDecimal.valueOf(100));
        orderInfoVO.setTotalAmt(BigDecimal.valueOf(10000));
        orderInfoVO.setUom("KG");
        orderInfoVO.setStaffCode("S0001");
        orderInfoVO.setStatus(1);
        // 定义SqlToyUnifyFieldsHandler公共字段赋值后，就无需设置
        // orderInfoVO.setCreateBy("S0001");
        // orderInfoVO.setCreateTime(LocalDateTime.now());
        orderInfoService.createOrderInfo(orderInfoVO);

        // 查回验证:主键由 NanoTimeIdGenerator 生成(26位),审计字段自动填充
        List<OrderInfo> saved = lightDao.findEntity(OrderInfo.class,
                EntityQuery.create().where("productCode=:pc").values(MapKit.map("pc", productCode)));
        assertEquals(1, saved.size());
        assertEquals(26, saved.get(0).getOrderId().length());
        assertEquals("system-auto", saved.get(0).getCreateBy());
    }

    @Test
    public void testUpdateOrderInfo() {
        // 自建自更(不依赖库中既有数据):先保存一张订单
        OrderInfo order = new OrderInfo().setOrderType("PO").setProductCode("P-UPD-" + System.nanoTime())
                .setUom("KG").setPrice(BigDecimal.valueOf(100)).setQuantity(BigDecimal.valueOf(100))
                .setTotalAmt(BigDecimal.valueOf(10000)).setStaffCode("S0001").setOrganId("100007").setStatus(1);
        lightDao.save(order);

        // 弹性 update:实体上为 null 的字段被跳过,只更新已赋值字段
        OrderInfo update = new OrderInfo().setOrderId(order.getOrderId());
        update.setQuantity(BigDecimal.valueOf(120));
        lightDao.update(update);

        OrderInfo reloaded = lightDao.load(new OrderInfo().setOrderId(order.getOrderId()));
        assertEquals(0, reloaded.getQuantity().compareTo(BigDecimal.valueOf(120)));
        // price 未赋值(null)——弹性更新不碰它,原值保留
        assertEquals(0, reloaded.getPrice().compareTo(BigDecimal.valueOf(100)));
    }

    @Test
    public void testSearchOrderInfo() {
        // xml 式动态条件分页:statusAry/beginTime 生效,endTime 为 null 自动忽略
        Page<OrderInfoVO> pageModel = orderInfoService.searchOrderInfo(new Page(10, 1),
                MapKit.keys("statusAry", "beginTime", "endTime").values(new Integer[] { 1 },
                        LocalDateTime.parse("2024-10-17T00:00:01"), null));
        System.out.println(JSON.toJSONString(pageModel.getRows()));
        assertTrue(pageModel.getRecordCount() >= 3); // 种子数据含 3 张 status=1 的订单
    }

}
```

7、运行验证

```
git clone https://gitee.com/sagacity/sqltoy-helloworld.git
cd sqltoy-helloworld
mvn test
```

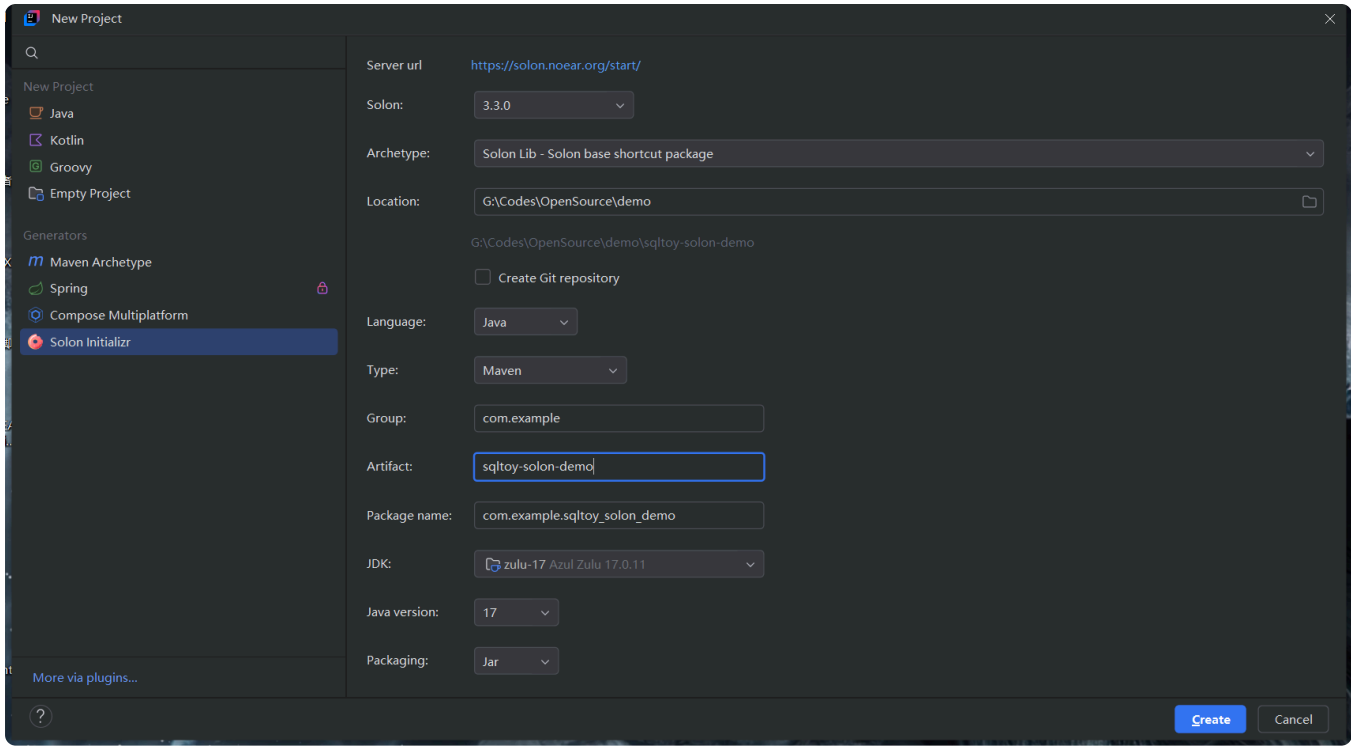
```
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0 — BUILD SUCCESS
```

除本文的 CRUD/分页用例外,演示项目还包含:缓存翻译四种用法的 `TranslateTest`(xml `<translate>` 标签、 `QueryExecutor.translates` 、`EntityQuery.translates` 、VO 属性 `@Translate` 注解,详见缓存翻译使用与公共字段赋值);以及 lambda 条件查询的 `LambdaQueryTest`(实体属性方法引用表达条件 + `setExpr` 表达式更新,详见Lambda 条件查询与更新)。

Sqltoy集成到Solon案例

一、创建项目

- 打开 IDE（如 IntelliJ IDEA 或 Eclipse），选择创建 Maven 项目。这里直接使用idea solon插件创建。
- 案例地址：sqltoy-solon-demo



二、添加依赖

- sqltoy-solon插件

```
<!--sqltoy-solon-->
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-solon-plugin</artifactId>
    <version>6.0.9</version>
</dependency>
```

- MySQL连接器

```
<!-- mysql连接器 -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.7.0</version>
    <scope>runtime</scope>
</dependency>
```

三、配置数据源

- app.yml配置

```
myapp:
  db1:
    schema: demo
    jdbcUrl: jdbc:mysql://127.0.0.1:3306/demo?useUnicode=true&characterEncoding=utf-
8&useSSL=true&serverTimezone=Asia/Shanghai&tinyInt1isBit=false
    driverClassName: com.mysql.cj.jdbc.Driver
    username: root
    password: root
```

- 注入数据源配置(DbConfig.java)

```
@Configuration
public class DbConfig {
    @Bean(name = "db1", typed = true)
    public DataSource db1(@Inject("${myapp.db1}") HikariDataSource ds) {
        return ds;
    }
}
```

连接池这里用了HikariDataSource，因此要多加个依赖：

```
<!-- 连接池 -->
<dependency>
    <groupId>com.zaxxer</groupId>
    <artifactId>HikariCP</artifactId>
</dependency>
```

- 然后是sqltoy的配置

```
sqltoy:
  # xml中的sql扫描路径
  sqlResourcesDir: classpath:sqltoy
  debug: true
```

四、在solon中使用sqltoy的api

1. 先创建个Controller调试（TestController.java）

```
@Controller
public class TestController {
    @Db
    private LightDao lightDao;

    @Get
    @Mapping("/order/findAll")
    public Object query() {
        return lightDao.find("sqltoy_order_info_find", new HashMap<>());
    }
}
```

2. 创建模板sql（sqltoy/sqltoy_order_info.sql.xml）

```
<?xml version="1.0" encoding="utf-8"?>
<sqltoy xmlns="http://www.sagframe.com/schema/sqltoy"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.sagframe.com/schema/sqltoy https://sagframe.github.io/schema/sqltoy.xsd">

    <sql id="sqltoy_order_info_find">
        <value>
            <![CDATA[
            select *
            from sqltoy_order_info
            order by ORDER_ID desc
            ]]>
        </value>
    </sql>

</sqltoy>
```

五、调试

1. 运行程序（App.java）
2. 访问接口：`http://localhost:8080/order/findAll`

公共字段统一赋值背景

- 常规业务逻辑为了审计需要:
  1)在单据创建时需要维护初始的:创建人、创建时间;
  2)每次修改需要维护:修改人、修改时间
  3)修改记录时，修改人和修改时间很多情况下会从前端传递之前的值，所以需要进行强制覆盖性修改
  因此在数据库表设计时，往往需要统一设计4个字段:create_by\create_time\update_by\update_time
  在代码层面每次手工给这些字段赋值就带来大量重复工作，且容易产生遗漏!
  这就需要在框架层面提供统一的处理! 另一个关键价值：修改时自动维护修改人、修改时间（必要时强制覆盖最后修改时间），**为数据留痕和基于更新时间的增量 ETL 同步打好基础**——只要坚持统一赋值，增量抽取程序就可以直接依赖 update_time 做增量识别，无需额外埋点。

sqltoy自定义公共字段赋值接口实现

- 1、sqltoy提供了标准接口:org.sagacity.sqltoy.plugins.IUnifyFieldsHandler
  以创建为例，sqltoy统一字段赋值的逻辑:
  1) 字段不存在，会自动跳过(对象化CRUD很方便判断实际字段名称)
  2) 弹性赋值:比如字段:createBy已经赋值就会自动跳过

```
 //逻辑示意:
 if(entity.getCreateBy()==null){entity.setCreateBy(unifyCreateMap.get("createBy"));}
```

- 2、在application.yml中配置sqltoy的公共字段处理类

```
spring:
    sqltoy:
        unifyFieldsHandler: com.sqltoy.plugins.SqlToyUnifyFieldsHandler
```

- 3、SqlToyUnifyFieldsHandler代码

```java
package com.sqltoy.plugins;

import java.time.LocalDateTime;
import java.util.Map;

import org.sagacity.sqltoy.model.IgnoreCaseSet;
import org.sagacity.sqltoy.model.MapKit;
import org.sagacity.sqltoy.plugins.IUnifyFieldsHandler;

/**
 * @project sqltoy-helloworld
 * @description 统一字段赋值范例
 * @author zhongxuchen <a href="mailto:zhongxuchen@gmail.com">联系作者</a>
 * @version v1.0,Date:2018年1月18日
 */
public class SqlToyUnifyFieldsHandler implements IUnifyFieldsHandler {
    private String defaultUserName = "system-auto";

     /**
      * 设置创建记录时需要赋值的字段和对应的值(弹性模式:即以传递的值优先，为null再填充)
      */
     public Map<String, Object> createUnifyFields() {
        LocalDateTime nowTime = LocalDateTime.now();
        // 获取用户信息
        String userId = getUserId();
        // 字段不存在会自动跳过，如:createBy和createdBy,你可以两种都写上
        return MapKit.keys("createBy", "createTime", "updateBy", "updateTime", "enabled").values(userId, nowTime,
                    userId, nowTime, 1);
     }

    /**
     * 设置修改记录时需要赋值的字段和对应的值(弹性)
     */
    @Override
    public Map<String, Object> updateUnifyFields() {
        return MapKit.keys("updateBy", "updateTime").values(getUserId(), LocalDateTime.now());
    }

    /**
     * 强制修改的字段(一般针对updateTime属性)
     */
    @Override
    public IgnoreCaseSet forceUpdateFields() {
        IgnoreCaseSet forceUpdateFields = new IgnoreCaseSet();
        forceUpdateFields.add("updateTime");
        return forceUpdateFields;
    }

    /**
     * 获取当前用户Id信息
     * @return
     */
    private String getUserId() {
        // 实际项目一般通过ThreadLocal+Filter方式存放和获取当前用户信息
        // 比如spring-security
        // return (SpringSecurityUtils.getCurrentUser() != null) ? SpringSecurityUtils.getCurrentUser().getId() : defaultUserName;
        return defaultUserName;
    }
}
```

单元测试验证公共字段统一赋值

```
@SpringBootTest
public class OrderInfoServiceTest {
    @Autowired
    OrderInfoService orderInfoService;

    @Test
    public void testCreateOrderInfo() {
        OrderInfoVO orderInfoVO = new OrderInfoVO();
        orderInfoVO.setOrderType("PO");
        orderInfoVO.setOrganId("T001");
        orderInfoVO.setProductCode("P0001");
        orderInfoVO.setPrice(BigDecimal.valueOf(100));
        orderInfoVO.setQuantity(BigDecimal.valueOf(100));
        orderInfoVO.setTotalAmt(BigDecimal.valueOf(10000));
        orderInfoVO.setUom("KG");
        orderInfoVO.setStaffCode("S0001");
        orderInfoVO.setStatus(1);
        // 定义SqlToyUnifyFieldsHandler公共字段赋值后，就无需设置
        // orderInfoVO.setCreateBy("S0001");
        // orderInfoVO.setCreateTime(LocalDateTime.now());
        // orderInfoVO.setUpdateBy("S0001");
        // orderInfoVO.setUpdateTime(LocalDateTime.now());
        orderInfoService.createOrderInfo(orderInfoVO);
    }
}
```

sqltoy 配置参数

- Spring 项目以 `spring.sqltoy.` 为前缀；
- Solon 项目以 `solon.sqltoy.` 为前缀；
- 传统 Spring XML 模式则在 `SqlToyContext` bean 上以 `<property>` 设置同名属性。

下面按用途分类列出**全部配置参数**（以 spring-boot-starter 的 `SqlToyContextProperties` 与 `SqlToyContext` 为准，含默认值）。绝大多数项目只需设置其中 3~5 个，见文末常规项目用法。

一、资源加载与调试

```
# sql.xml 资源路径，多个用逗号分隔；支持 classpath*:com/**/*.sql.xml 的 AntPath 模式(5.6.75+)
spring.sqltoy.sqlResourcesDir=classpath:com/nebula/crm/modules
# 直接指定具体的 sql.xml 文件资源(数组)，与 sqlResourcesDir 二选一或并用
spring.sqltoy.sqlResources[0]=classpath:com/nebula/system.sql.xml
# 实体 Entity 扫描包路径(数组)，以 @Entity/@SqlToyEntity 注解为依据；默认无需配置(首次使用时加载)
spring.sqltoy.packagesToScan[0]=com.yourproject.modules
# 额外注解 class 类(数组)，一般已无必要
#spring.sqltoy.annotatedClasses[0]=com.yourproject.SomeEntity
# sql 文件编码
spring.sqltoy.encoding=UTF-8
# 是否开启 debug 模式，默认 false；为 true 时输出执行 sql 日志与耗时，且 sql 文件更新检测间隔为 2 秒(false 时 15 秒)
spring.sqltoy.debug=true
# sql 文件脚本变更检测间隔(秒)；<0 或 >86400(一天) 表示关闭更新检测
spring.sqltoy.scriptCheckIntervalSeconds=2
# 缓存更新、sql 脚本更新 延迟多少秒后开始检测
spring.sqltoy.delayCheckSeconds=300
# 发现重复 sqlId 时是否抛异常终止程序，默认 true；false 则后面的覆盖前面的
spring.sqltoy.breakWhenSqlRepeat=true
# sql 日志输出格式化器：defaultSqlFormater 或 default(使用 druid 的 sql 格式化，需引入 druid)
spring.sqltoy.sqlFormater=defaultSqlFormater
```

二、数据源与方言

```
# 多数据源场景下指定默认数据源，效果同 spring 的 @Primary
spring.sqltoy.defaultDataSource=crmDataSource
# 自定义数据源选择器(实现 DataSourceSelector)，用于动态数据源路由
#spring.sqltoy.dataSourceSelector=com.yourproject.MyDataSourceSelector
# 连接管理实现扩展(实现 ConnectionFactory)：纯 Java 环境常用
#spring.sqltoy.connectionFactory=org.sagacity.sqltoy.integration.impl.SimpleConnectionFactory
# 数据库方言，一般无需设置(框架自动识别)
#spring.sqltoy.dialect=mysql
# sqlId 方言变体查找优先按连接探测的真实方言(6.0.3 新增，默认 false)
# 典型场景：OceanBase 按 mysql 方言配置而真实库为 oceanbase、代理库等；开启后真实方言变体
# 不存在时回退原有配置方言查找链，真实方言与配置方言一致时行为不变
#spring.sqltoy.realDialectFirst=false
# 数据库方言参数配置(Map)，用于覆盖/补充方言行为
#spring.sqltoy.dialectConfig.xxx=yyy
# 不同数据库方言的映射(Map)，如把未内置的库按某方言处理：OSCAR-->oracle
#spring.sqltoy.dialectMap.OSCAR=oracle
# 跨库适配验证：任意查询会在这些数据源上重放一次(数组)，用于产品化多库测试
#spring.sqltoy.redoDataSources[0]=oracleDB
# 数据库保留字，逗号分隔(对象操作自动兼容；自定义 sql 跨库时按此适配)
spring.sqltoy.reservedWords=maxvalue,minvalue,name
# 默认查询超时时长(秒)
#spring.sqltoy.defaultStatementTimeout=30
```

三、批量、分页与执行

```
# 批量操作(saveAll/updateAll 等)每批次数量，默认 200
spring.sqltoy.batchSize=200
# 数据库端默认提取记录量(fetchSize)，默认 -1，一般无需设置
#spring.sqltoy.fetchSize=1000
# 分页最大单页数据量，默认 100000(10万)；<0 表示不限制。防止 pageNo=-1 做全量提取
spring.sqltoy.pageFetchSizeLimit=100000
# 默认每页记录数，默认 10
spring.sqltoy.defaultPageSize=10
# 未匹配的数据库类型，分页采用 limit ? offset ? 还是 limit ?,? 模式
#spring.sqltoy.defaultPageOffset=false
# 翻页超出最大页时是否回到第一页
spring.sqltoy.overPageToFirst=true
# 大批量数据修改时输出日志提醒的记录数阈值，默认 2000
spring.sqltoy.updateTipCount=2000
# 拆分 merge into 为 updateAll + saveAllIgnoreExist 两步(seata 分布式事务不支持 merge)
#spring.sqltoy.splitMergeInto=false
# executeSql 变更操作型 sql 执行时，空白参数是否默认转为 null
#spring.sqltoy.executeSqlBlankToNull=true
```

四、缓存翻译

```
# 缓存翻译配置文件，默认 classpath:sqltoy-translate.xml;classpath:translates
# (classpath:translates 指该路径下以 .trans.xml/.translates.xml/-translate.xml/-translates.xml 结尾的文件)
spring.sqltoy.translateConfig=classpath:sqltoy-translate.xml
# 缓存翻译使用的缓存组件，默认 ehcache，可选 caffeine
spring.sqltoy.cacheType=caffeine
# 自定义缓存管理器(实现 TranslateCacheManager)
#spring.sqltoy.translateCacheManager=com.yourproject.MyTranslateCacheManager
# 动态获取缓存的实现(一般基于 redis)，用于超大规模主数据，详见 FIFO 动态缓存
#spring.sqltoy.dynamicCacheFetch=com.yourproject.MyDynamicCacheFetch
# 动态捞取缓存的缓存管理器，框架提供了默认实现(FIFODynamicFetchCacheManager)
#spring.sqltoy.dynamicFecthCacheManager=...
# 分布式 id key 的缓存天数(一般基于 redis，避免长期占用空间)，多少天失效
#spring.sqltoy.distributeIdCacheExpireDays=30
```

五、函数适配（跨库）

```
# 开启数据库 sql 函数自动适配：也可挂载自定义函数：default,com.yourpackage.Instr
# (类名一致则自定义实现覆盖框架默认；close 表示关闭)
spring.sqltoy.functionConverts=default
```

六、字段处理与类型

```
# 统一字段处理器：对 createBy/createTime/updateBy/updateTime 等做补漏性赋值(为空时才赋)
spring.sqltoy.unifyFieldsHandler=com.sqltoy.plugins.SqlToyUnifyFieldsHandler
# 自定义 TypeHandler(仅特殊场景!)。JSON/JSONB、vector、geometry 列和枚举类型
# 框架均已内置处理、无需配置；此处仅用于真正特殊的值对象映射，如 Money(CNY,88000)<->"CNY:88000"
spring.sqltoy.typeHandler=com.sqltoy.plugins.MoneyTypeHandler
# JSON 列的底层 JSON 库可插拔:缺省按 classpath 自动探测 fastjson2 -> jackson3 -> jackson2。
# 探测到的库还要先通过一次冒烟解析才会被采纳—库在 classpath 上不代表可用(例如 jackson3 的
# databind 需要 jackson-annotations 2.20+,若被仲裁成更旧版本会抛 NoSuchFieldError),不可用时
# 记 WARN 并降级到下一个候选;皆不可用时触及 JSON 列会给出明确指引(而非 NoClassDefFoundError)。
# 需要固定用某个库时显式指定(例如 fastjson2 与 jackson 同时在 classpath 上但只想用 jackson):
# 取值可以是内置别名 fastjson2 / jackson3 / jackson2,也可以是自定义实现的全限定类名
# (实现 org.sagacity.sqltoy.plugins.json.JsonProvider 接口)
#spring.sqltoy.jsonProvider=jackson2
#spring.sqltoy.jsonProvider=com.xxx.MyJsonProvider
# 查询返回 List<Map> 时，map 的 label 是否转驼峰，默认 true
spring.sqltoy.humpMapResultTypeLabel=true
# 获取 ResultSetMetaData 列标题的处理策略：default(不处理)/upper(转大写)/lower(转小写)
spring.sqltoy.columnLabelUpperOrLower=default
# sql 日志输出时 LocalDateTime 的格式，可设 auto 或 yyyy-MM-dd HH:mm:ss.SSSSSS 等
spring.sqltoy.localDateTimeFormat=yyyy-MM-dd HH:mm:ss.SSSSSS
# sql 日志输出时 LocalTime 的格式，可设 auto 或 HH:mm:ss.SSSSSS 等
spring.sqltoy.localTimeFormat=HH:mm:ss.SSSSSS
# 默认区域设置(如 zh_CN、en-US)，影响日期/数字格式化解析的区域符号；为 null 则用 JVM 默认区域
#spring.sqltoy.defaultLocale=zh_CN
# like 查询 ESCAPE 子句是否使用双反斜杠：true=ESCAPE '\\'，false=ESCAPE '\'，null=按方言自动判断
#spring.sqltoy.backslashEscaping=
```

七、安全（加解密与脱敏）

```
# 字段加解密的 RSA 密钥
spring.sqltoy.securePrivateKey=classpath:mock/rsa_private.key
spring.sqltoy.securePublicKey=classpath:mock/rsa_public.key
# 自定义加解密算法实现(默认 RSA，无需设置)，接口 org.sagacity.sqltoy.plugins.secure.FieldsSecureProvider
#spring.sqltoy.fieldsSecureProvider=com.yourproject.FieldsSecureProvider
# 自定义脱敏实现(默认已提供，无需设置)，接口 org.sagacity.sqltoy.plugins.secure.DesensitizeProvider
#spring.sqltoy.desensitizeProvider=com.yourproject.DesensitizeProvider
```

SQL 注入校验正则（`sqlInjectionRegexes`）

```
# 自定义 SQL 注入检测的正则表达式(数组)，替换内置默认的关键词/函数/注释/条件形态 Pattern
# 主要作用于 <filters> 中的 <valid-sqlInjection> 参数校验，也可配合自定义拦截器使用
#spring.sqltoy.sqlInjectionRegexes[0]=^(SELECT|INSERT|UPDATE|DELETE|DROP|ALTER|CREATE|TRUNCATE)\\b
```

> 说明：不配置时使用框架内置默认（函数调用、注释、永真条件、SQL 关键词四类 Pattern）。该配置在 JVM 内**首个初始化的 SqlToyContext 生效**，后续 context 不可削弱（安全攸关）。solon-plugin 同样支持该属性。

八、DDL 自动建表

```
# 是否自动通过 POJO 创建/更新表结构，默认 false，需配合 packagesToScan
spring.sqltoy.autoDDL=true
# 生成 DDL 时表名/字段名转小写还是大写(6.0.3 起可通过 spring.sqltoy 前缀配置)
#spring.sqltoy.ddlLowerOrUpper=lower
# 自定义数据库 DDL 产生器(实现 DialectDDLGenerator)
#spring.sqltoy.dialectDDLGenerator=com.yourproject.MyDDLGenerator
# 单记录保存采用 identity/sequence 主键策略并返回主键值时，字段名称大小写处理(Map: lower/upper)
#spring.sqltoy.dialectReturnPrimaryColumnCase.mysql=lower
```

九、拦截器、慢 SQL 与扩展

```
# sql 执行拦截器(数组)，可改变当前 sql，如统一租户隔离/越权过滤
spring.sqltoy.sqlInterceptors[0]=org.sagacity.sqltoy.plugins.interceptors.TenantFilterInterceptor
# 自定义 filter 处理器(预留备用)
#spring.sqltoy.customFilterHandler=...
# 慢 sql 时长阈值(毫秒)，超过则日志输出并存入慢 sql 队列，默认 8000(8秒)
spring.sqltoy.printSqlTimeoutMillis=3000
# 慢 sql 收集处理器，默认 DefaultOverTimeHandler
# 获取：lightDao.getSqlToyContext().getOverTimeSqlHandler().getSlowest(100, true)
spring.sqltoy.overTimeSqlHandler=org.sagacity.sqltoy.plugins.overtime.DefaultOverTimeHandler
# 业务代码调用点获取(FirstBizCodeTrace)，用于追踪 sql 的业务来源
#spring.sqltoy.firstBizCodeTrace=...
```

线程池（taskExecutor）

并行查询、并行分页、并行批量等使用的线程池，前缀 `spring.sqltoy.taskExecutor.`：

```
# 指定线程池名称，指定后以该线程池为默认；默认 none(使用 sqltoy 内置线程池)
#spring.sqltoy.taskExecutor.targetPoolName=none
# 线程名前缀，默认 sqltoyThreadPool
spring.sqltoy.taskExecutor.threadNamePrefix=sqltoyThreadPool
# 核心线程数，默认 CPU核数/2+1
spring.sqltoy.taskExecutor.corePoolSize=8
# 最大线程数，默认 CPU核数*3
spring.sqltoy.taskExecutor.maxPoolSize=24
# 缓冲队列容量，默认 200
spring.sqltoy.taskExecutor.queueCapacity=200
# 线程空闲存活时间(秒)，默认 60
spring.sqltoy.taskExecutor.keepAliveSeconds=60
# 关闭时是否等待任务完成，默认 true
spring.sqltoy.taskExecutor.waitForTasksToCompleteOnShutdown=true
# 超时中断时间(秒)，默认 -1
spring.sqltoy.taskExecutor.awaitTerminationSeconds=-1
```

十、NoSQL（Elasticsearch）

ES 节点配置前缀 `spring.sqltoy.elastic.`（支持多节点/多集群），完整说明见 Elasticsearch 支持：

```
spring.sqltoy.elastic.defaultId=default
spring.sqltoy.elastic.endpoints[0].id=default
spring.sqltoy.elastic.endpoints[0].url=http://192.168.56.101:9200
spring.sqltoy.elastic.endpoints[0].username=elastic
spring.sqltoy.elastic.endpoints[0].password=skyline
spring.sqltoy.elastic.endpoints[0].sqlPath=_nlpcn/sql
```

> MongoDB 复用 spring-data 的 `spring.data.mongodb.*` 配置，见 MongoDB 支持。

十一、GraalVM native 构建期参数（6.0.8 起）

这三个参数**只在 native 镜像构建期生效**，JVM 运行模式完全忽略它们。它们用于补齐"框架无法从配置推断"的反射与代理需求，完整背景与验证结果见 GraalVM Native Image 指引。

```
# 非实体 DTO/VO 的反射登记（packagesToScan 只识别 @Entity，DTO/VO 扫不到）
# 按包批量登记：递归扫描包下所有 class
spring.sqltoy.native-reflection-packages[0]=com.yourcorp.project.dto
spring.sqltoy.native-reflection-packages[1]=com.yourcorp.project.vo
# 按类登记：少量特殊类、按类名引用的类
spring.sqltoy.native-reflection-classes[0]=com.yourcorp.project.report.TreeModel
# 为使用 updateSaveFetch(EntityUpdateCallback) 的实体在构建期预生成代理子类
# 需应用自行引入 net.bytebuddy:byte-buddy（starter 中为 optional，不传递）
spring.sqltoy.native-entity-proxies[0]=com.yourcorp.project.entity.StockVO
```

参数	取值	作用
`native-reflection-packages`	包路径数组	递归登记包内所有类的反射（普通 DTO/VO、record、enum 均可）
`native-reflection-classes`	全限定类名数组	单独登记个别类（DTO/VO、按类名引用的类）
`native-entity-proxies`	全限定类名数组	为列出的实体在构建期预生成 `updateSaveFetch` 代理（实体不能是 `final`，需可访问无参构造器）

> Solon 侧前缀为 `solon.sqltoy.*`，参数名相同。yaml 形态与双栈完整示例见 GraalVM Native Image 指引 第 3 节。

常规项目用法

配置项看似较多，实际绝大多数项目只需设置 3~5 个参数：

```
# 推荐复杂查询放 xml 中
spring.sqltoy.sqlResourcesDir=classpath:com/nebula/crm/modules
# 输出执行的 sql、参数和耗时，便于定位问题
spring.sqltoy.debug=true
# 公共字段统一赋值处理
spring.sqltoy.unifyFieldsHandler=com.sqltoy.plugins.SqlToyUnifyFieldsHandler
# 慢 sql 的时长标准定义
spring.sqltoy.printSqlTimeoutMillis=3000
# 自定义 TypeHandler(仅特殊场景!)。JSON/vector/geometry/枚举框架均已内置处理；
# 此处仅用于特殊的值对象映射，完整演示见 sqltoy-showcase 的 CustomTypeHandlerTest
spring.sqltoy.typeHandler=com.sqltoy.plugins.MoneyTypeHandler
```

多数据源与事务

sqltoy 本身不绑定数据源，多数据源可直接复用社区成熟的 **dynamic-datasource**（baomidou）方案：`@DS` 注解切换数据源、`@Transactional` 处理单库事务、`@DSTransactional` 把多个数据源的操作纳入同一事务。

> 本文整理自原演示子项目 `trunk/sqltoy-dynamic-datasource`（已随 sqltoy-showcase 单模块重构从仓库移除，本文代码示例自成一体、可直接落地）；sqltoy 自身的命名多数据源演示见 showcase 的 `MultiDataSourceTest`。本文示例以 **sqltoy 6.0.4** 为准

一、引入依赖

```
<!-- dynamic-datasource（spring-boot3 对应 starter，在 Spring Boot 4.0.1 下可用） -->
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>dynamic-datasource-spring-boot3-starter</artifactId>
    <version>4.5.0</version>
</dependency>
<!-- 连接池（以 druid 为例） -->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-3-starter</artifactId>
    <version>1.2.28</version>
</dependency>
```

> **警告**
>
> 使用 druid 时需**排除其自动配置**，否则与 dynamic-datasource 冲突：`@SpringBootApplication(exclude = DruidDataSourceAutoConfigure.class)`

二、配置多数据源

`primary` 指定默认数据源；`datasource` 下声明各个库；sqltoy 通过 `defaultDataSource` 指定默认库：

```
spring:
    datasource:
        dynamic:
            primary: datasourceA          # 默认数据源
            datasource:
                datasourceA:
                    url: jdbc:mysql://127.0.0.1:3306/dyndatasourcea?useUnicode=true&characterEncoding=utf-
8&serverTimezone=GMT%2B8&useSSL=false
                    username: dyndatasourcea
                    password: dyndatasourcea
                    driver-class-name: com.mysql.cj.jdbc.Driver
                datasourceB:
                    url: jdbc:mysql://127.0.0.1:3306/dyndatasourceb?useUnicode=true&characterEncoding=utf-
8&serverTimezone=GMT%2B8&useSSL=false
                    username: dyndatasourceb
                    password: dyndatasourceb
                    driver-class-name: com.mysql.cj.jdbc.Driver
            druid:                        # 连接池公共配置
                initial-size: 5
                min-idle: 5
                maxActive: 20
                validationQuery: SELECT 1 FROM DUAL
                testWhileIdle: true
    sqltoy:
        # 注意前缀是 spring.sqltoy. ，不要写成 sqltoy. 导致无法加载
        sqlResourcesDir: classpath:com/sqltoy/dyndatasource
        debug: true
        # 指定 sqltoy 默认数据源
        defaultDataSource: datasourceA
```

三、使用 @DS 切换数据源

在 Service 方法上用 `@DS("数据源名称")` 切换，sqltoy 的操作会路由到对应库执行：

```
@Service
public class OrderInfoServiceImpl implements OrderInfoService {
    @Autowired
    private LightDao lightDao;

    @Override
    @DS("datasourceA")     // 切换到 A 库
    public long saveLocalAOrders(List<OrderInfoVO> orderInfoVOs) {
        return lightDao.saveAll(orderInfoVOs);
    }

    @Override
    @DS("datasourceB")     // 切换到 B 库
    public long saveLocalBOrders(List<OrderInfoVO> orderInfoVOs) {
        return lightDao.saveAll(orderInfoVOs);
    }
}
```

- `primary` 指定的默认数据源**无需** `@DS` 注解；
- `@DS` 也可标注在类上，对整个类的操作生效。

四、事务处理

单库事务：正常使用 Spring 的 `@Transactional`（primary 数据源无需额外指定）：

```
@Transactional
public void executeSql(String sql) {
    lightDao.executeSql(sql, MapKit.map());
}
```

跨库事务：使用 dynamic-datasource 提供的 `@DSTransactional`，把多个数据源的操作纳入同一事务——任一失败整体回滚：

```
@Service
public class DynamicServiceImpl implements DynamicService {

    @Autowired
    private OrderInfoService orderInfoService;

    /**
     * A、B 两个库的操作纳入同一事务
     */
    @Override
    @DSTransactional
    public void saveOrder(List<OrderInfoVO> orderInfoVOAs, List<OrderInfoVO> orderInfoVOBs) {
        orderInfoService.saveLocalAOrders(orderInfoVOAs);   // 写 A 库
        orderInfoService.saveLocalBOrders(orderInfoVOBs);   // 写 B 库
    }
}
```

五、其他多数据源方式

方式	说明
多个 LightDao	定义多个 `LightDao` bean 分别注入不同 dataSource，Service 中按 bean 名称引入
查询级切换	`QueryExecutor.dataSource(...)` 与链式 `.dataSource(...)`（见 Link 链式操作）
xml 指定	`<sql dataSource="...">` 在单条 SQL 上指定数据源（见动态 SQL 规范）
多库事务(JTA)	结合 JTA 实现（如 Atomikos/Narayana）

> 更多常见问题（如分库分表为什么不支持跨库 join 分页、与其他 ORM 组合使用等）见 FAQ。
>
> 🎬 原完整示例源码（`DynamicServiceImpl.java` / `OrderInfoServiceImpl.java`）已随 `trunk/sqltoy-dynamic-datasource` 子项目移除，本文代码示例自成一体、可直接落地；sqltoy 自身的命名多数据源演示见 sqltoy-showcase 的 `MultiDataSourceTest`。

传统spring项目配置

- 引入sqltoy依赖（传统 Spring 用 sagacity-sqltoy-spring；Spring Boot 项目才用 starter）

```
<!-- 传统 spring（非 boot） -->
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-spring</artifactId>
    <!-- JDK8 请使用 5.6.96.jre8（jre8 最终版本，同样支持 quickvo-maven-plugin 插件） -->
    <version>6.0.9</version>
</dependency>
```

- 基于spring xml的bean定义sqltoy上下文和lightDao

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd"
    default-autowire="byName" default-lazy-init="true">
    <!-- 定义sqltoy上下文 -->
    <bean id="sqlToyContext" name="sqlToyContext" class="org.sagacity.sqltoy.SqlToyContext" init-method="initialize"
        destroy-method="destroy">
        <!-- 指定sql.xml 文件的路径实现目录的递归查找,可以用逗号分隔配置多个路径，非必须属性 -->
        <property name="sqlResourcesDir" value="classpath:com/company/project" />
        <!-- 针对不同数据库函数进行转换,非必须属性 -->
        <property name="functionConverts" value="default" />
        <!-- 统一公共字段赋值,原理:只做补漏，开发者已经设置了值不会被覆盖,当没有被赋值时才会起作用 -->
        <property name="unifyFieldsHandler">
            <!-- 这里是示意 -->
            <bean class="com.company.framework.sqltoy.SqltoyUnifyFieldsHandler" />
        </property>
        <!-- 缓存翻译管理器,非必须属性(仅当不使用缓存翻译) -->
        <property name="translateConfig" value="classpath:sqltoy-translate.xml" />
        <!-- 默认值为:false -->
        <property name="debug" value="${sqltoy.debug}" />
        <!-- 默认值为:200,提供sqltoy批量更新的batch量 -->
        <property name="batchSize" value="${sqltoy.batchSize}" />
        <!-- 默认值为:100000,设置分页查询最大的提取数据记录量,防止恶意提取数据造成系统内存压力以及保障数据安全 -->
        <property name="pageFetchSizeLimit" value="50000" />
        <!-- 默认dataSource -->
        <property name="defaultDataSource" ref="dataSource" />
        <!-- sql执行时长超过多少毫秒则打印该sql -->
        <property name="printSqlTimeoutMillis" value="2000" />
        <!-- 应用启动后多久再进行检测 -->
        <property name="delayCheckSeconds" value="300" />
        <property name="breakWhenSqlRepeat" value="false"/>
        <property name="overPageToFirst" value="true"/>
        <!-- 关键2个配置: appContext 和 connectionFactory -->
        <property name="appContext">
            <bean class="org.sagacity.sqltoy.integration.impl.SpringAppContext"/>
        </property>
        <property name="connectionFactory">
            <bean class="org.sagacity.sqltoy.integration.impl.SpringConnectionFactory"/>
        </property>
    </bean>

    <!-- 定义 lightDao：推荐统一使用 LightDao 作为对象操作入口 -->
    <bean id="lightDao" name="lightDao"
        class="org.sagacity.sqltoy.dao.impl.LightDaoImpl" />
    <bean id="sqlToyCRUDService" name="sqlToyCRUDService"
        class="org.sagacity.sqltoy.service.impl.SqlToyCRUDServiceImpl" />
</beans>
```

- 具体使用

```
@Autowired
private LightDao lightDao;

public void createOrgan(OrganInfo organInfo){
    lightDao.save(organInfo);
}
```

纯java项目如何使用

sqltoy推荐在spring或solon 框架下运行，并未完全构建独立使用的框架体系，可结合AI自行优化扩展，下面提供给一个范例

- 步骤1:引入pom

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy</artifactId>
   <version>6.0.9</version>
</dependency>
```

- java代码，注册DataSource、构建SqlToyContext、lightDao

```
public void doDB() {
    try {
        //构建sqlToyContext
        SqlToyContext sqlToyContext = new SqlToyContext();
        // sqlResourcesDir 指目录;具体到单个 xml 文件用 setSqlResources(List)
        sqlToyContext.setSqlResources(Arrays.asList("classpath:sqltoy/demo.sql.xml"));
        sqlToyContext.initialize();
        Map<String, String> map = new HashMap<>();
        map.put(DruidDataSourceFactory.PROP_URL, "jdbc:mysql://192.168.56.101:3306/java20");
        // 设置驱动Driver
        map.put(DruidDataSourceFactory.PROP_DRIVERCLASSNAME, "com.mysql.cj.jdbc.Driver");
        // 设置用户名
        map.put(DruidDataSourceFactory.PROP_USERNAME, "root");
        // 设置密码
        map.put(DruidDataSourceFactory.PROP_PASSWORD, "123456");
        // 创建数据源
        DataSource dataSource = DruidDataSourceFactory.createDataSource(map);
        sqlToyContext.setDefaultDataSource(dataSource);
        // 框架默认提供了DefaultLightDaoImpl实现
        LightDao lightDao = new DefaultLightDaoImpl(sqlToyContext);
        // 非事务
        // lightDao.find("select * from staff_info where status=:status",
        // MapKit.map("status", "1"), StaffInfo.class);
        // 这里都是示意，请按实际逻辑编写
        Object result = DBTransUtils.doTrans(lightDao.getDataSource(), () -> {
            // 这里可以
            lightDao.updateByQuery(StaffInfo.class,
                    EntityUpdate.create().set("sexType", "F").where("staffId=?").values("S0001"));
            return lightDao.find("select * from staff_info where status=:status", MapKit.map("status", "1"),
                    StaffInfo.class);
        });
    } catch (Exception e) {

    }
}
```

Solon 集成参考

sqltoy 通过独立的 `sagacity-sqltoy-solon-plugin` 模块支持 Solon 框架，API 与 Spring 版完全一致（LightDao / SqlToyLazyDao / SqlToyCRUDService 等），差异只在配置与注入方式。入门实战见 helloworld-solon 案例。

一、依赖与版本要求

```
<dependency>
    <groupId>com.sagframe</groupId>
    <artifactId>sagacity-sqltoy-solon-plugin</artifactId>
    <!-- 与 core 保持同版本号 -->
    <version>6.0.9</version>
</dependency>
```

- 基于 **Solon 4.1.1** 构建，传递依赖 `solon-aot`、`solon-data`；
- 数据源由 Solon 容器管理（HikariCP、Druid 等连接池自行注册为 bean），sqltoy 不重复托管。

二、配置（solon.sqltoy 前缀）

配置前缀为 `solon.sqltoy`（兼容旧前缀 `sqltoy`）；属性名与 Spring 版 `spring.sqltoy.*` 几乎一一对应，参数含义直接参阅sqltoy 配置参数即可：

```
solon.sqltoy:
  # xml 中的 sql 扫描路径(多个用逗号分隔)
  sqlResourcesDir: classpath:sqltoy
  # 默认 classpath:sqltoy-translate.xml
  translateConfig: classpath:sqltoy-translate.xml
  debug: true
  # 函数跨库自适配(默认开启)
  #functionConverts: default
```

与 Spring 版的两点差异：

差异点	说明
`cacheType` 默认值为 `solon`	缓存翻译默认使用 Solon 本地缓存管理器（`SolonTranslateCacheManager`）；Spring 版默认 `ehcache`
debug 联动	Solon 以 debug 模式启动时（`Solon.cfg().isDebugMode()`），强制开启 sqltoy 的 debug 输出

三、@Db 注入

Solon 侧不使用 `@Autowired`，而是通过 `org.sagacity.sqltoy.solon.annotation.Db` 注入（可标注字段或方法参数）：

注入目标	说明
`javax.sql.DataSource`	直接注入数据源
`SqlToyLazyDao`	全功能 Dao
`LightDao`	轻量 Dao（推荐 Service 中直接使用）
`SqlToyCRUDService`	通用 CRUD 服务

`@Db` 的 value 指定数据源 bean 名，缺省为默认数据源：

```
@Controller
public class TestController {

    // 默认数据源
    @Db
    private LightDao lightDao;

    // 指定数据源 bean 名(对应 @Bean(name="db1") 的数据源)
    @Db("db1")
    private SqlToyLazyDao db1Dao;
}
```

多数据源的选择机制（sql 中 `datasource` 属性、lazyDao 链式 `.dataSource(...)`、`defaultDataSource` 兜底等）与 Spring 版一致，参见多数据源与事务。

四、GraalVM Native

solon-plugin 内置 `org.sagacity.sqltoy.solon.integration.aot.SqltoyRuntimeNativeRegistrar`（实现 Solon 的 `RuntimeNativeRegistrar` 接口），构建 native image 时自动注册 sqltoy 配置资源（`sqltoy-translate.xml` 等）与实体类反射元数据，无需手工追加 reflect-config；6.0.8 起同样按配置动态注册资源 pattern、扩展类反射，并支持用 `native-reflection-packages` / `native-reflection-classes` 登记非实体 DTO/VO 的反射。

> Spring Boot 侧的 AOT 说明与完整 native 指引见 GraalVM Native Image 指引。

XML Schema 配置指引

sqltoy 的三类 XML 配置文件均提供官方 XSD schema（托管在 GitHub：sagframe/schema）。配置正确后，在 IDEA 中编辑这些文件可获得**标签补全、属性提示与错误校验。**

三类配置文件与 schema 指向

配置文件	用途	xmlns（命名空间，永远不变）	xsi:schemaLocation 中的文件位置
`*.sql.xml`	sql / mql / eql 查询定义	`http://www.sagframe.com/schema/sqltoy`	`https://sagframe.github.io/schema/sqltoy.xsd`
`sqltoy-translate.xml`	缓存翻译配置	`https://www.sagframe.com/schema/sqltoy-translate`	`https://sagframe.github.io/schema/sqltoy-translate.xsd`
`quickvo.xml`	quickvo 代码生成配置	`http://www.sagframe.com/schema/quickvo`	`https://sagframe.github.io/schema/quickvo.xsd`

重要

`xmlns` 是 schema 的身份标识，**不需要可访问、永远不要修改**；`xsi:schemaLocation` 的第二个地址只是"文件在哪"的提示。框架运行时**不会**联网读取 XSD，它只服务于 IDE 的补全与校验。

旧地址过渡说明

历史版本使用的是 `http://www.sagframe.com/schema/sqltoy/` 下的地址：

旧地址（仍可访问）	新地址（推荐）
`http://www.sagframe.com/schema/sqltoy/sqltoy.xsd`	`https://sagframe.github.io/schema/sqltoy.xsd`
`http://www.sagframe.com/schema/sqltoy/sqltoy-translate.xsd`	`https://sagframe.github.io/schema/sqltoy-translate.xsd`
`http://www.sagframe.com/schema/sqltoy/quickvo.xsd`	`https://sagframe.github.io/schema/quickvo.xsd`

针对存量文件：

- **不影响运行**：框架运行时不读取 XSD，带着旧地址的文件照常工作；
- **仅影响 IDE 体验**：旧地址为 http 协议、以 `application/octet-stream` 类型返回，部分网络环境或 IDEA 版本可能抓取失败，导致编辑时无法获得补全；
- **建议逐步替换**为新地址即可，无需一次性整改——新地址为 https + 标准 XML 类型，首次 Alt+Enter 抓取即可生效。

标准文件头（copy 即用）

`*.sql.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<sqltoy xmlns="http://www.sagframe.com/schema/sqltoy"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.sagframe.com/schema/sqltoy https://sagframe.github.io/schema/sqltoy.xsd">

</sqltoy>
```

`sqltoy-translate.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<sagacity xmlns="https://www.sagframe.com/schema/sqltoy-translate"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://www.sagframe.com/schema/sqltoy-translate https://sagframe.github.io/schema/sqltoy-translate.xsd">

</sagacity>
```

`quickvo.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<quickvo xmlns="http://www.sagframe.com/schema/quickvo"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.sagframe.com/schema/quickvo https://sagframe.github.io/schema/quickvo.xsd">

</quickvo>
```

IDEA 中启用补全与校验

1. 打开任一配置文件，把光标放到标红的地址上，按 **Alt+Enter** → 选择 **Fetch external resource**（抓取一次即可，IDEA 全局缓存共享，其他项目无需重复）；
2. 之后编辑文件即有标签/属性补全，写错标签名或属性名会即时标红；
3. 也可以在 `Settings | Languages & Frameworks | Schemas and DTDs` 中手动添加 URI 到 XSD 文件的映射。

相关页面

- 动态 SQL 编写规范：`*.sql.xml` 中 `<sql>` 元素与子标签的完整规范
- 缓存翻译使用：translate.xml 的配置与用法
- quickvo 代码生成工具：quickvo.xml 的配置要点

说明

- sqltoy的对象化crud跟JPA比较类似，主要特点:
  1) 通过quickvo类似hibernate-tools工具产生POJO
  2) 支持复合主键，如update操作lightDao.update(new StaffInfo("S0001","HUAWEI_SH").setName("李四"));
  3) 提供显式快捷api和链式api 2种模式,链式API提供更多的参数,如指定数据源、设定并行、设定batchSize等参数,如lightDao.update().dataSource(xxx).forceUpdateProps("status").batchSize(500).many(entities);
  4) 没有逻辑删除的概念,在sqltoy中就是状态更新
  5) 相比较于JPA，sqltoy主要优化了update(提供了弹性修改)，新增了updateSaveFetch和updateFetch（一次数据库交互完成锁查询、修改或插入并返回结果，详见）
  6) sqltoy支持:OneToOne 和 OneToMany 两种形式的级联(有部分改进,如:加载支持过滤等)，但只支持一级级联；查询结果依据注解自动分层封装(hiberarchy)见常规查询 API
  7) sqltoy在级联加载和级联删除等操作上底层做了优化,采用了id in (:ids)形式的查询和删除,用算法组织最终数据，减少数据库IO提升效率
  8) 详细请参见:org.sagacity.sqltoy.dao.LightDao接口,其有详细备注

1、单条记录保存

- 接口规范，涉及显式指定数据源，可用lightDao.save().dataSource(xxx).one(entity)链式操作

```
/**
 * 保存对象,并返回主键值
 * @param entity
 * @return Object 返回主键值
 */
public Object save(Serializable entity);

/**
 * 提供链式操作模式保存操作集合,如:
 *       lightDao.save().dataSource(xxx).one(entity)
 * @return
 */
public Save save();
```

- 简单范例

```
StaffInfo staffInfo = new StaffInfo();
staffInfo.setStaffCode("S2018");
staffInfo.setStaffName("测试员工9");
lightDao.save(staffInfo);
```

- 级联保存范例

```
//主对象中配置级联信息
@Schema(name="ComplexpkHead",description="复合主键级联操作主表")
@Data
@Accessors(chain = true)
@Entity(tableName="sqltoy_complexpk_head",comment="复合主键级联操作主表",pk_constraint="PRIMARY")
public class ComplexpkHead implements Serializable {

    /**
     * 主键关联子表信息
     */
    @OneToMany(fields={"transDate","transCode"},mappedFields={"transDate","transId"},delete=true)
    private List<ComplexpkItem> complexpkItems=new ArrayList<ComplexpkItem>();

    //OneToOne的配置
    //@OneToOne(fields={"transDate","transCode"},mappedFields={"transDate","transId"},delete=true)
    //private ComplexpkItem complexpkItem;
}

@Test
public void testSaveCascade() {
    // 主表记录
    ComplexpkHead head = new ComplexpkHead();
    head.setTransDate(LocalDate.parse("2020-09-08"));
    head.setTransCode("S0001");
    head.setTotalCnt(BigDecimal.valueOf(10));
    head.setTotalAmt(BigDecimal.valueOf(10000));

    // 子表记录1
    ComplexpkItem item1 = new ComplexpkItem();
    // 这里id是为了便于演示手工指定
    item1.setId("S000101");
    item1.setProductId("P01");
    item1.setPrice(BigDecimal.valueOf(1000));
    item1.setAmt(BigDecimal.valueOf(5000));
    item1.setQuantity(BigDecimal.valueOf(5));
    head.getComplexpkItems().add(item1);

    // 子表记录2
    ComplexpkItem item2 = new ComplexpkItem();
    item2.setId("S000102");
    item2.setProductId("P02");
    item2.setPrice(BigDecimal.valueOf(1000));
    item2.setAmt(BigDecimal.valueOf(5000));
    item2.setQuantity(BigDecimal.valueOf(5));
    head.getComplexpkItems().add(item2);

    lightDao.save(head);
}
```

2、批量保存

- 接口规范，涉及显式指定数据源，可用lightDao.save().dataSource(xxx).many(entities)链式操作
- 大批量并行保存:lightDao.save().parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).many(entities)

```
/**
 * 批量保存对象，并返回数据更新记录量
 * @param <T>
 * @param entities
 * @return Long 数据库发生变更的记录量
 */
public <T extends Serializable> Long saveAll(List<T> entities);

/**
 * 批量保存对象并忽视已经存在的记录
 * @param <T>
 * @param entities
 * @return Long 数据库发生变更的记录量
 */
public <T extends Serializable> Long saveAllIgnoreExist(List<T> entities);

/**
 * 提供链式操作模式保存操作集合,如:
 *       lightDao.save().dataSource(xxx).many(entities)
         lightDao.save().parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).many(entities)
 * @return
 */
public Save save();
```

3、单笔记录修改

- 接口规范:sqltoy的更新不同于JPA需要先查询加载实体对象,直接赋予主键和需要修改的属性值即可

```
/**
 * 修改数据并返回数据库记录变更数量(非强制修改属性，当属性值为null不参与修改)
 * @param entity
 * @param forceUpdateProps 强制修改的字段属性
 * @return Long 数据库发生变更的记录量
 */
public Long update(Serializable entity, String... forceUpdateProps);

/**
 * 深度修改,不管是否为null全部字段强制修改
 * @param entity
 * @return Long 数据库发生变更的记录量
 */
public Long updateDeeply(Serializable entity);

/**
 * 提供链式操作模式修改记录
 *       <li>lightDao.update().dataSource(xxx).forceUpdateProps("status").one(entity)</li>
 *       <li>lightDao.update().dataSource(xxx).forceUpdateProps("status").many(entities)</li>
 *       <li>大批量并
行:lightDao.update().parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).forceUpdateProps("status").many(entities)</li>
 * @return
 */
public Update update();

/**
 * 级联修改数据并返回数据库记录变更数量
 * @param entity
 * @param forceUpdateProps
 * @param forceCascadeClasses 级联对象为null或空时，是否表示强制删除级联记录
 * @param subTableForceUpdateProps 设置级联修改对象强制修改的属性
 * @return Long 数据库发生变更的记录量
 */
public Long updateCascade(Serializable entity, String[] forceUpdateProps, Class[] forceCascadeClasses,
        HashMap<Class, String[]> subTableForceUpdateProps);
```

- 简单更新使用范例

```
//对工号S0001的员工更新姓名和状态
lightDao.update(new StaffInfo("S0001").setName("张三").setStatus(1));
//复合主键（工号、租户）
lightDao.update(new StaffInfo("S0001","HUAWEI_SH").setName("张三").setStatus(1));
```

- 级联修改使用范例

```
Map<Class,String[]> subTableForces=new HashMap<>();
subTableForces.put(ComplexpkItem.class,new String[]{"status"});

//单条记录修改一旦涉及cascadeForceUpdate或cascadeClasses 就开启了级联行为，否则update只针对自身
lightDao.update().dataSource(crmDataSource).forceUpdateProps("status").cascadeForceUpdate(subTableForces).one(entity);
```

4、批量修改

- api规范

```
/**
 * 批量修改操作，并可以指定强制修改的属性(非强制修改属性，当属性值为null不参与修改)
 * @param <T>
 * @param entities
 * @param forceUpdateProps
 * @return Long 数据库发生变更的记录量
 */
public <T extends Serializable> Long updateAll(List<T> entities, String... forceUpdateProps);

/**
 * 批量深度修改，即全部字段参与修改(包括为null的属性)
 * @param <T>
 * @param entities
 * @return Long 数据库发生变更的记录量
 */
public <T extends Serializable> Long updateAllDeeply(List<T> entities);
```

- 大批量并行更新范例

```
lightDao.update().dataSource(crmDataSource).forceUpdateProps("status")
      .parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10))
      .many(entities);
```

5、基于查询条件修改（updateByQuery）

- 说明：无需先加载实体对象，直接基于单表查询条件批量修改字段，一次数据库交互即可完成。适用于"把符合条件的记录的某些字段统一更新"的场景（逻辑删除本质也是状态更新，可用它实现）。
- api规范

```
/**
 * 基于单表查询完成数据修改
 * @param entityClass 实体类
 * @param entityUpdate 通过 EntityUpdate 组织 set 修改字段与 where 条件
 * @return Long 数据库发生变更的记录量
 */
public Long updateByQuery(Class entityClass, EntityUpdate entityUpdate);
```

- 使用范例

```
// 将工号 S0001 的员工性别改为 F（set 用对象属性名，where 用 ? 占位、values 按序传值）
lightDao.updateByQuery(StaffInfo.class,
        EntityUpdate.create().set("sexType", "F").where("staffId=?").values("S0001"));

// 多字段更新 + 多条件
lightDao.updateByQuery(OrderInfo.class,
        EntityUpdate.create()
            .set("status", 2)
            .set("updateTime", LocalDateTime.now())
            .where("organId=? and createTime<?")
            .values("T001", LocalDateTime.now().minusMonths(3)));

// 指定数据源：skipNotExistColumn() 表示 set 的字段在表中不存在时自动跳过
lightDao.updateByQuery(StaffInfo.class,
        EntityUpdate.create().dataSource(crmDataSource)
            .set("status", 0).where("staffId=?").values("S0001")
            .skipNotExistColumn());
```

`EntityUpdate` 链式方法：`set(属性，值)`（可多次调用设置多个字段）、`where(条件)`、`values(值...)`、`blankToNull(Boolean)`、`skipNotExistColumn()`、`dataSource(...)`、`showSql(Boolean)`。`where` 中使用 `?` 占位、由 `values(...)` 按顺序传值。与 `deleteByQuery`（见下节）相对应，一个按条件改、一个按条件删。

- 列自增 / 表达式更新（`set 字段=字段+?`）

`set` 的 key 除了写普通字段名，还可以写成 `字段=表达式` 形式，实现基于字段当前值的计算更新（自增、扣减等）。表达式中的 `?` 由 `set` 的 value 绑定；字段名（如 `totalAmt`）会自动转换为数据库列名（`total_amt`）。

```
// 自更新：total_amt = total_amt + 10
// 等价 SQL：update sqltoy_staff_info set total_amt=total_amt+10 where staff_name like '张'
lightDao.updateByQuery(StaffInfo.class,
        EntityUpdate.create()
            .set("totalAmt=totalAmt+?", 10)
            .where("staffName like ?").values("张"));

// 库存扣减：quantity = quantity - 5，可同时叠加普通字段更新
lightDao.updateByQuery(OrderInfo.class,
        EntityUpdate.create()
            .set("quantity=quantity-?", 5)     // 表达式更新（基于字段当前值计算）
            .set("status", 2)                  // 普通字段更新（可并存）
            .where("orderId=?").values("10001"));
```

要点：表达式务必用 `?` 占位、并由 `set` 的 value 传值（如 `set("totalAmt=totalAmt+?", 10)`）；`set` 的 key 左侧是字段名、右侧是 SQL 表达式，可包含 `+ - * /` 等运算。表达式更新可与普通 `set(字段，值)` 混用，框架会自动按"先 set 后 where"的顺序绑定参数。

- 实战：逻辑删除（状态更新）

sqltoy 没有独立的"逻辑删除"概念——逻辑删除本质就是把状态/标记字段更新一下（参见下节"删除操作"说明）。单条用弹性更新即可，按条件批量则用 `updateByQuery`，一次数据库交互完成。

```
// 方式一：单条逻辑删除—弹性更新，只改 status（为 null 的字段不参与更新）
lightDao.update(new OrderInfo("10001").setStatus(0));

// 方式二：按条件批量逻辑删除—updateByQuery
// 把某部门 3 个月前、状态为 1 的订单标记为已删除(0)，并记录操作人/时间
lightDao.updateByQuery(OrderInfo.class,
        EntityUpdate.create()
            .set("status", 0)
            .set("updateBy", "S0001")
            .set("updateTime", LocalDateTime.now())
            .where("organId=? and status=? and createTime<?")
            .values("T001", 1, LocalDateTime.now().minusMonths(3)));
```

查询时排除已逻辑删除的记录（在 sql 中加状态条件即可）：

```
<sql id="find_order">
    <value><![CDATA[
        select * from sqltoy_order_info t
        where t.status<>0          -- 排除已逻辑删除
        #[and t.organ_id=:organId]
    ]]></value>
</sql>
```

> 提示：若配置了统一字段处理器（`unifyFieldsHandler`），对象化 `update` 会自动补漏 `updateBy` / `updateTime`；用 `updateByQuery` 时建议如上显式 `set` 审计字段。

6、删除操作

- sqltoy没有逻辑删除的概念
  (逻辑删除本质就是更新状态或更新标记字段，请用更新操作代替)
- api规范

```
/**
 * 提供链式操作模式删除操作集合 示例:
 *       <li>lightDao.delete().dataSource(xxxx).one(entity);</li>
 *       <li>lightDao.delete().batchSize(1000).autoCommit(true).many(entities);</li>
 *       <li>lightDao.delete().parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).many(entities);</li>
 * @return
 */
public Delete delete();

/**
 * 删除单条对象并返回数据库记录影响的数量
 * @param entity
 * @return Long 数据库发生变更的记录量(删除数据量)
 */
public Long delete(final Serializable entity);

/**
 * 批量删除对象并返回数据库记录影响的数量
 * @param entities
 * @return Long 数据库记录变更量(删除数据量)
 */
public <T extends Serializable> Long deleteAll(final List<T> entities);

/**
 * 根据id集合批量删除
 * @param entityClass
 * @param ids
 * @return
 */
public Long deleteByIds(Class entityClass, Object... ids);

/**
 * 基于单表查询进行删除操作,提供在代码中进行快捷操作
 * @param entityClass
 * @param entityQuery 例如:lightDao.deleteByQuery(DictDetail.class,EntityQuery.create().where("status=?").values(0));
 * @return Long 数据库记录变更量(插入数据量)
 */
public Long deleteByQuery(Class entityClass, EntityQuery entityQuery);
```

- 使用范例

```
//复合主键
lightDao.delete(new StaffInfo("S0001","HUAWEI_SH"));
//单主键，删除多条
lightDao.deleteByIds(OrderInfo.class,"10001","10002");
//删除符合条件的数据
lightDao.deleteByQuery(DictDetail.class,EntityQuery.create().dataSource(xxxx).where("status=?").values(0));
```

7、saveOrUpdate操作

- 说明
  1) saveOrUpdate的逻辑是记录存在就做修改操作，如果不存在则新建(主键值为null则必然是新建)
  2) 类mysql数据库采用先update后insert ignore方式，其他采用merge into 模式
- api规范

```
/**
 * 提供链式操作模式保存操作集合,如:
 *    <li>lightDao.save().dataSource(xxx).saveMode(SaveMode.UPDATE).many(entities)</li>
 * @return
 */
public Save save();

/**
 * 保存或修改数据并返回数据库记录变更数量
 * @param entity
 * @param forceUpdateProps 强制修改的字段
 * @return Long 数据库发生变更的记录量
 */
public Long saveOrUpdate(Serializable entity, String... forceUpdateProps);

/**
 * 批量保存或修改操作(当已经存在就执行修改)
 * @param <T>
 * @param entities
 * @param forceUpdateProps 强制修改的字段
 * @return Long 数据库发生变更的记录量
 */
public <T extends Serializable> Long saveOrUpdateAll(List<T> entities, String... forceUpdateProps);
```

- 使用范例

```
//简单批量保存或修改
lightDao.saveOrUpdateAll(entities,"name","status","quantity");

//SaveMode分:APPEND\UPDATE\IGNORE三种
lightDao.save().dataSource(xxx).saveMode(SaveMode.UPDATE)
    .parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).many(entities);
```

8、对象加载

- api规范

```
/**
 * 提供链式操作模式对象加载操作集合
 *    <li>lightDao.load().dataSource(xxxx).lock(LockMode.UPGRADE_NOWAIT).one(new StaffInfo("S0001"));</li>
 *    <li>lightDao.load().parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).many(entities);</li>
 *    <li>加载主对象，同时级联加载子对象:lightDao.load().cascade(OrderItem.class,OrderDeliveryPlan.class).many(entities);</li>
 *    <li>只根据主对象的主键级联加载子对象:lightDao.load().cascade(OrderItem.class,OrderDeliveryPlan.class).onlyCascade().many(entities);
</li>
 * @return
 */
public Load load();

/**
 * 根据实体对象的主键值获取对象的详细信息
 * @param entity
 * @return entity
 */
public <T extends Serializable> T load(final T entity);

/**
 * 根据主键获取单个实体对象
 * @param <T>
 * @param entityClass
 * @param id
 * @return
 */
public <T extends Serializable> T loadById(final Class<T> entityClass, Object id);

/**
 * 根据主键获取对象,提供读取锁设定
 * @param entity
 * @param lockMode LockMode.UPGRADE 或LockMode.UPGRADE_NOWAIT等
 * @return entity
 */
public <T extends Serializable> T load(final T entity, final LockMode lockMode);

/**
 * 对象加载同时指定加载子类，实现级联加载
 * @param entity
 * @param lockMode
 * @param cascadeTypes
 * @return entity
 */
public <T extends Serializable> T loadCascade(final T entity, final LockMode lockMode, final Class... cascadeTypes);

/**
 * 根据集合中的主键获取实体的详细信息(底层是批量加载优化了性能,同时控制了in 1000个问题)
 * @param entities
 * @return entities
 */
public <T extends Serializable> List<T> loadAll(List<T> entities);

/**
 * 提供带锁记录的批量加载功能
 * @param <T>
 * @param entities
 * @param lockMode
 * @return
 */
public <T extends Serializable> List<T> loadAll(List<T> entities, final LockMode lockMode);

/**
 * 根据id集合批量加载对象
 * @param <T>
 * @param entityClass
 * @param ids
 * @return
 */
public <T extends Serializable> List<T> loadByIds(final Class<T> entityClass, Object... ids);

/**
 * 根据id集合批量加载对象,并加锁
 * @param <T>
 * @param entityClass
```

```
 * @param lockMode
 * @param ids
 * @return
 */
public <T extends Serializable> List<T> loadByIds(final Class<T> entityClass, final LockMode lockMode,
        Object... ids);

/**
 * 选择性的加载子表信息
 * @param entities
 * @param cascadeTypes
 * @return
 */
public <T extends Serializable> List<T> loadAllCascade(List<T> entities, final Class... cascadeTypes);

/**
 * 锁住主表记录并级联加载子表数据
 * @param <T>
 * @param entities
 * @param lockMode
 * @param cascadeTypes
 * @return
 */
public <T extends Serializable> List<T> loadAllCascade(List<T> entities, final LockMode lockMode,
        final Class... cascadeTypes);
```

- 大规模并行加载范例

```
lightDao.load().parallelConfig(ParallelConfig.create().groupSize(5000).maxThreads(10)).many(entities);
```

- 级联加载

```
lightDao.load().cascade(OrderItem.class,OrderDeliveryPlan.class).many(entities);

lightDao.loadCascade(new OrderInfo("S0001"), null, OrderItem.class,OrderDeliveryPlan.class);
```

- 仅级联加载子对象(将子对象构造到主对象的对应属性上)

```
//.onlyCascade()即只会通过entities集合pojo的主键关联查询子对象，主对象不再做查询行为
lightDao.load().cascade(OrderItem.class,OrderDeliveryPlan.class).onlyCascade().many(entities);
```

9、updateSaveFetch操作（锁查询→存在则修改、不存在则插入→返回结果，一次交互）

- 说明
  1) sqltoy 独有（JPA 没有）：**一次数据库交互**完成 锁查询+存在则修改/不存在则插入+返回修改或插入后的记录，四步一气呵成
  2) 适用场景：库存台账、客户资金账等高并发强事务环节——"存在则更新、不存在则初始化插入"的 upsert 语义天然规避并发首查竞争（两个请求同时发现记录不存在，只有一个 insert 成功，另一个走修改）
  3) **同类场景优先选它，而非 updateFetch**：基于实体 + 主键/唯一键检索，无需编写 sql；记录不存在自动补插，调用方不需要先判断存在性。updateFetch（见下节）面向"记录必然存在、且需要自由编写查询条件与锁策略"的场景
- api（三个常用重载，均可省略 uniqueProps 按主键检索）

```
/**
 * 适用于库存台账、客户资金账等高并发强事务场景，一次数据库交互实现：
 * 1、锁查询；2、记录存在则修改；3、记录不存在则执行insert；4、返回修改或插入的记录信息
 */
public <T extends Serializable> T updateSaveFetch(final T entity, final UpdateRowHandler updateRowHandler,
        final String... uniqueProps);

// 实体回调形态（推荐）：回调里只操作实体 get/set，无需触碰 ResultSet
public <T extends Serializable> T updateSaveFetch(final T entity, final EntityUpdateCallback<T> callback,
        final String... uniqueProps);

// 带锁等待时长（单位秒）的形态
public <T extends Serializable> T updateSaveFetch(final T entity, final EntityUpdateCallback<T> callback,
        final int lockWaitTimeout, final String... uniqueProps);
```

- 使用范例

```
//引入依赖类（实体回调形态需要 byte-buddy 支撑代理实体）
<dependency>
    <groupId>net.bytebuddy</groupId>
    <artifactId>byte-buddy</artifactId>
    <version>1.18.15</version>
</dependency>
```

```
@Test
public void testUpdateSaveFetch() {
    StaffInfoVO staffInfo = new StaffInfoVO();
    staffInfo.setStaffId("S0001");
    staffInfo.setBeginDate(LocalDate.parse("2019-01-01"));
    staffInfo.setEndDate(LocalDate.now());
    staffInfo.setStaffName("陈");
    lightDao.updateSaveFetch(staffInfo, (entity, rowIndex) -> {
        // 这里entity取值，实际通过代理，走的是rs.getString("tel_no")获取的值
        String telNo = entity.getTelNo();
        if (telNo != null) {
            // 这里set，实际是rs.updateString("tel_no", telNo.substring(0, 3) + "#**#" +
            // telNo.substring(7));
            entity.setTelNo(telNo.substring(0, 3) + "#**#" + telNo.substring(7));
        }
    });
}
```

注意

`EntityUpdateCallback` 回调里的 `entity` 是**代理对象**：`get` 自动读数据库当前值（`rs.getXxx`）、`set` 自动更新行（`rs.updateXxx`），无需手工同步 ResultSet。回调中抛出异常则整体回滚。

- 注意

  1) 实体主键尽量不要使用 identity、sequence（记录不存在走 insert 时，主键由实体自带最直接）

  2) 业务唯一键场景（如"商品+日期"的台账，以商品编码+日期唯一定位记录而非主键）通过 `uniqueProps` 指定属性名，不设置则按主键检索

  3) 并发竞争激烈的场景建议用 `lockWaitTimeout` 重载显式设定锁等待时长（单位秒），锁等待超时会抛异常回滚

  4) GraalVM native 镜像下 `EntityUpdateCallback` 形态依赖 byte-buddy 生成代理子类，需配置 `native-entity-proxies` 在构建期预生成（未列出的实体不可用，insert 分支不受影响），详见 GraalVM Native 指引

10、updateFetch操作（sql化锁查询→校验修改→返回结果，一次交互）

- 说明

  1) sqltoy 独有（JPA 没有）：**一次数据库交互**完成 查询+锁定+逻辑校验+修改+返回修改后结果

  2) 适用场景：记录**必然存在**、且查询条件需要自由编写（多表关联、复杂条件、指定锁策略）的强事务环节——查询的目的是**锁住记录做逻辑校验**（如扣减后不能小于 0），校验通过才修改

  3) 记录可能不存在、且以主键/唯一键定位的台账场景，优先用上节 `updateSaveFetch`（自动 upsert）

警告

updateFetch 的核心目的是**数据逻辑校验**（如扣减后不能小于 0），**页面展示性查询不要用此方法。**

- api

```
/**
 * 获取并锁定数据并进行修改（一般为简单查询，锁住记录进行逻辑校验后修改）
 * @param queryExecutor    查询执行器，定义查询的sql、条件参数和锁策略
 * @param updateRowHandler 行数据修改回调处理器，校验并修改行数据后提交更新
 * @return 修改后的行记录集合
 */
public List updateFetch(final QueryExecutor queryExecutor, final UpdateRowHandler updateRowHandler);
```

- 使用范例（库存扣减：锁定 → 校验库存充足 → 扣减 → 返回最新数据）

```
List result = lightDao.updateFetch(
        // 简单查询 + 悲观锁策略
        new QueryExecutor("sqltoy_stock_find")
                .names("productId").values("P0001")
                .lock(LockMode.UPGRADE),
        // 注意：UpdateRowHandler 的方法均为 default，不是函数式接口，需用匿名类覆写
        new UpdateRowHandler() {
            @Override
            public void updateRow(ResultSet rs, int index, BiConsumer<String, Object> setVal) throws Exception {
                double quantity = rs.getDouble("quantity");
                if (quantity < 1) {
                    // 逻辑校验不通过直接抛异常，本次修改整体回滚
                    throw new IllegalArgumentException("库存不足!");
                }
                // 通过反调设置修改后的值：<属性名，新值>
                setVal.accept("quantity", quantity - 1);
            }
        });
```

`UpdateRowHandler` 提供三个可覆写方法：`updateRow(rs,index)`、`updateRow(rs,index,BiConsumer<属性,值>)`、`updateRow(rs,index,ThreeBiConsumer<属性,强制更新字段[],值>)`（第三个可指定强制更新字段）。

与上节 `updateSaveFetch` 互补：`updateSaveFetch` 以实体定位、不存在自动插入；`updateFetch` 修改**已存在**的记录，查询条件由 sql 自由定义。

树形表节点路径构造（wrapTreeTableRoute）

组织机构、行政区划、商品分类等**树形表**，通常冗余一个节点路径字段（如 `node_route`），用 `like '路径%'` 即可**非递归**查出整棵子树，避免逐层递归查询带来的数据库压力。`wrapTreeTableRoute` 用于构造与维护这些冗余字段：**节点路径（nodeRoute）、层次等级（nodeLevel）、是否叶子节点（leaf）**。

一、API

```
/**
 * 构造树形表的节点路径、层次等级、是否叶子节点等必要信息
 * @param treeTableModel 树形表模型对象，提供id字段、pid父级字段等层级结构配置
 * @return 包装处理成功返回true，失败返回false
 */
public boolean wrapTreeTableRoute(final TreeTableModel treeTableModel);
```

等价链式写法：`lightDao.treeTable().treeModel(treeTableModel).submit()`

二、TreeTableModel 配置

两种构造模式：**基于实体**（表名取自 `@Entity` 注解）或**基于表名**全参构造 `(tableName, pidValue, idField, pidField, nodeRouteField, nodeLevelField, leafField, isChar, idLength)`。

配置	说明
`idField` / `pidField`	主键字段 / 父级字段
`rootId`（`pidValue`）	根节点的父级值（如 `0`）
`nodeRouteField`	节点路径字段，形如 `,100008,100009,`
`nodeLevelField`	层次等级字段
`isLeafField`（`leafField`）	是否叶子节点字段
`idLength` / `idTypeIsChar`	id 长度 / 是否字符型（层级定长拼接场景）
`setConditions`	附加过滤条件
`appendZero`	是否补零
`splitSign`	路径分隔符

三、使用范例

```
// 方式一：基于实体对象（表名取自 @Entity 注解）
lightDao.wrapTreeTableRoute(new TreeTableModel(organInfo)
        .idField("organCode").pidField("organPid")
        .nodeRouteField("nodeRoute").nodeLevelField("nodeLevel")
        .isLeafField("isLeaf"));

// 方式二：基于表名全参构造
// (tableName, 根父级值pidValue, idField, pidField, nodeRouteField, nodeLevelField, leafField, isChar, idLength)
lightDao.wrapTreeTableRoute(new TreeTableModel("sqltoy_organ_info", "0",
        "organ_id", "organ_pid", "node_route", "node_level", "leaf_field", true, 4));
```

构造完成后，即可用路径字段高效查询整棵子树：

```
<sql id="sqltoy_treeTable_search">
    <value><![CDATA[
        select * from sqltoy_organ_info t
        where t.node_route like :nodeRoute
    ]]></value>
</sql>
```

```
// 查询某节点下全部子级（nodeRoute 形如 ",100008,%"，尾部 % 匹配整棵子树）
List<OrganInfoVO> subOrgans = lightDao.find("sqltoy_treeTable_search",
        MapKit.map("nodeRoute", ",100008,%"), OrganInfoVO.class);
```

四、说明

- 配置了统一字段处理器（`unifyFieldsHandler`）时，路由级联与叶子标记更新的语句会**自动附加** `updateUnifyFields()` 中的公共更新字段（如最后修改人、最后修改时间），按实体属性名匹配列名，实体中不存在的字段自动忽略。
- 树形表的**查询侧排序与逐层汇总**（`<tree-sort>` 标签）见复杂查询与数据分析；二者配合可实现"构造路径 + 排序汇总"的完整树形方案。
- 对象的级联关系声明（`@OneToMany` / `@OneToOne`）见对象化 CRUD。

动态 SQL 编写规范

sqltoy 的动态 SQL 设计目标：**让 SQL 保持整洁、可直接在数据库客户端运行**，同时把 mybatis 中夹杂在 SQL 里的 `<if>` 等动态逻辑剥离出来。

- SQL 可以写在 `*.sql.xml` 中（推荐，便于维护、支持热加载），也可以直接在 Java 代码中传入；条件查询还可用 Lambda 条件体系（6.0.8 起内置，原第三方 sqltoy-plus 已并入核心）。
- 简单场景直接用 `lightDao` 的显式接口；复杂场景用 `findByQuery` / `findPageByQuery` 配合 `QueryExecutor` 或 `EntityQuery` 组织参数。所有显式接口底层都基于 `QueryExecutor`，理解它即可灵活运用。

一、`#[]` 条件片段原理

sqltoy 用 `#[ ... ]` 标记一段**动态条件片段**，核心规则：

1. `#[ ... ]` 表示其中的条件是动态的；**必要条件不用加** `#[]`。
2. 片段内只要**有一个参数为 null（或空白）**，整个片段就被自动剔除。
3. 支持**多层嵌套**：`#[t.type=:type #[and t.amt>:amt and t.quantity>:quantity]]`。 - **剔除规则逐层独立**：每一层 `#[...]` 只看自己这一层的直接参数。内层参数为 null 时仅剔除内层，外层与括号照常保留：

   `sql where t.type=:type #[and (t.amt>:amt #[or t.qty>:qty])] -- amt=100、qty=null → where t.type=:type and (t.amt>:amt) -- amt=100、qty=5 → where t.type=:type and (t.amt>:amt or t.qty>:qty)`

- **外层的直接参数为 null 时，整个外层片段连同内层一起剔除**（内层参数有值也救不回）。因此嵌套天然表达**主从/层级条件**：外层是内层参与运算的前提。三层及以上同理，逐层递归：

  `sql where #[t.a=:a #[and t.b=:b #[and t.c=:c]]] -- b 为 null → where t.a=?`

- **参数用法与单层完全一致**，以下形态在嵌套层内同样适用：命名参数 `:paramName`；数组下标取值，同一数组的元素可分别引用（如 `:starEndTime[0]` 与 `:starEndTime[1]` 组成时间区间）；元组多参数 `(t.tech_group,t.prod_group) in (:techGroups,:prodGroups)`；集合自动展开 `t.id in (:ids)`（数组/集合展开为多占位符）。同名参数可在多层重复出现，各层各自判空、互不影响。
- **括号分组与多成员**：`and ( ... )` 括号组内的多个 `#[...]` 成员各自独立判空剔除（如 `#[and ( #[t.a=:a] or #[t.b=:b] )]`）——任一成员自身参数有值即保留该成员，全部为 null 时整组剔除；成员剔除后括号内残留的悬挂 `and` / `or` 连接符由框架自动清理（6.0.7 起，左括号后、右括号前的悬挂连接符一律剔除，`( order by ... )` 这类以 or 开头的合法关键字不受影响），无需手工规避。 4. 必要条件（未加 `#[]`）当参数为 null 时，会自动转成 `is null` / `is not null`。 5. sqltoy 默认隐藏了 `blank-to-null="true"`：空白字符串自动按 null 处理。 6. `#` 恰好是 MySQL 等数据库的注释符，所以**带 `#[]` 的 SQL 可以直接复制到客户端执行**，便于调试。

二、XML 形态

> **提示**
>
> `*.sql.xml` 的 XSD schema 指向与 IDE 补全配置见 XML Schema 配置指引。

```
<!-- 默认即 blank-to-null="true"，等价于 <sql id="show_case" blank-to-null="true"> -->
<sql id="show_case">
    <filters>
        <!-- 通过 filters 对条件参数做前置规整处理 -->
        <eq params="status" value="-1" />
        <!-- 关闭 blank-to-null 的两种做法:
             1、blank-to-null="false"
             2、用 blank 标签对任意参数(包括不存在的)处理一次，即自动关闭默认设置 -->
        <blank params="anyParamNameIncludeNotExist"/>
    </filters>
    <value><![CDATA[
        select *
        from sqltoy_device_order_info t
        where
              -- orderType 为 null 时自动转成 t.order_type is null
              t.order_type=:orderType
              and t.ORGAN_ID in (:authedOrganIds)
              #[and t.status=:status]
              #[and t.ORDER_ID=:orderId]
              #[and t.TRANS_DATE>=:beginAndEndDate[0]]
              #[and t.TRANS_DATE<:beginAndEndDate[1]]
              #[and (t.TECH_GROUP,t.PROD_GROUP) in (:techGroups,:prodGroups)]
    ]]></value>
</sql>
```

`<sql>` 标签常用属性：`id`（必填）、`type`（`search` / `insert` / `update` / `delete`，`select` 为历史兼容写法）、`blank-to-null`（默认 `true`）、`debug`（是否输出执行日志）、`query-timeout`（查询超时秒数）、`dataSource`（指定数据源名称）、`dialect`（6.0.3 新增，声明本条 sql 按指定方言形态解析固化，详见方言自适配与函数扩展）、`union-all-count`（union 分页场景按分支拆解计算 count，减少不必要执行）。NoSQL 场景使用 `eql`（Elasticsearch）/ `mql`（MongoDB）节点，属性不同，见 Elasticsearch 支持与 MongoDB 支持。

三、Java 形态（XML 能做的代码里也能做）

```
String whereString = """
        orderType=:orderType
        and organId in (:authedOrganIds)
        #[and status=:status]
        #[and orderId=:orderId]
        #[and transDate>=:beginAndEndDate[0]]
        #[and transDate<:beginAndEndDate[1]]
        #[and (techGroup,prodGroup) in (:techGroups,:prodGroups)]
        """;
List result = lightDao.findEntity(DeviceOrderVO.class, EntityQuery.create()
        // .blankNotNull()  // 关闭 blank-to-null
        .where(whereString)
        .values(MapKit.keys("orderType", "authedOrganIds", "status", "orderId", "beginAndEndDate", "techGroups", "prodGroups")
                .values("PO", authedOrganIds, "1", "S0001", beginAndEndTime, techGroups, prodGroups))
        .filters(new ParamsFilter("status").eq("-1")));
```

四、filters 参数过滤器完整参考

`<filters>` 用于在 SQL 执行前对条件参数做规整（转 null、类型转换、格式化、缓存反向匹配等），是 sqltoy 动态 SQL 的核心。Java 中对应 `ParamsFilter`，通过 `.filters(...)` 传入。

> **设计哲学——前置规整**：sqltoy 简洁之道的原理就在这里——通过 filters 在进入 SQL 前把条件参数修整到位，**保持对 SQL 内容体最小的干扰。** `#[]` 只保留"参数有值就拼进去"这一种本征形态，其余条件逻辑（置 null、格式化、拆分合并）全部前置到 filters 完成，SQL 因此能始终维持直观、简洁的原生结构。

4.1 转 null 类（满足条件则把参数置 null，从而剔除对应 `#[]` 片段）

标签	属性	说明
`<blank>`	`params`(默认 `*`)、`excludes`	参数为空白则转 null
`<eq>`	`params`、`value`、`excludes`	参数等于给定值则转 null（数组中任一值匹配即转）
`<neq>`	`params`、`value`	参数不等于给定值则转 null
`<gt>` / `<gte>`	`params`、`value`	参数大于 / 大于等于给定值则转 null
`<lt>` / `<lte>`	`params`、`value`	参数小于 / 小于等于给定值则转 null
`<between>`	`params`、`start-value`、`end-value`、`excludes`	参数在区间内则转 null

4.2 like 模糊查询处理

标签	属性	说明
`<l-like>`	`params`、`append-str`	参数非空时右边补 `%`（即 `xxx%`）
`<r-like>`	`params`、`append-str`	参数非空时左边补 `%`（即 `%xxx`）
`<escapeLike>`	`params`	对 like 参数中的特殊符号转义

4.3 类型转换与格式化

标签	属性	说明
`<to-number>`	`params`、`data-type` (decimal/integer/double/float/long)	参数转数字类型
`<to-array>`	`params`、`data-type` (string/decimal/...)	参数转数组（单值转为长度 1 的数组）
`<to-string>`	`params`、`add-quote` (none/single/double)	参数转字符串，可加引号
`<to-date>`	`params`、`format`、`type`、`increment-time`、`increment-unit`	转日期；`format` 支持 `yyyy-MM-dd`、`FIRST_OF_MONTH`、`LAST_OF_YEAR`、`FIRST_OF_WEEK` 等；`increment-time` 可写 `${incrementDays}` 动态参数
`<split>`	`params`、`split-sign` (默认 `,`)、`data-type`	把拼接字符串切割成数组（常用于 in 条件）
`<default>`	`params`、`value`、`data-type`、`split-sign`、`is-array`	设置参数默认值，`value` 支持 `sysdate()-2d`、`first_of_month` 等表达式
`<replace>`	`params`、`regex`、`value`、`is-first`	对参数做正则替换
`<to-in-arg>`	`params`、`single-quote` (默认true)	将数组拼接成单个字符串作为 `in (:args)` 条件值
`<remove-null>`	`params`、`remove-blank` (默认true)	剔除数组/集合中为 null 或空白的值
`<date-format>`	`params`、`format`	对日期参数格式化（较少用）

4.4 高级过滤器

标签	属性	说明
`<primary>`	`param`、`excludes`	**首要参数**：如订单编号有值时表示精准查询，除 `excludes` 排除的权限条件外，其他条件不再参与过滤
`<exclusive>`	`param`、`set-params`、`compare-type`、`compare-values`、`set-value`	互斥参数处理
`<clone>`	`param`、`as-param`	复制一个参数的值另存为另一个参数
`<cache-arg>`	`param`、`cache-name`、`cache-key-index`、`cache-mapping-indexes`、`cache-mapping-max`、`alias-name`、`unmatched-return-self` 等	**缓存反向匹配**：用名称匹配出对应代码数组（如输入"张明"匹配出包含该姓名的工号数组）再参与查询，替代低效 like。可含 `<filter cache-index compare-param compare-type split-sign>` 子标签
`<custom-handler>`	`params`、`type`	自定义数据处理器
`<valid-sqlInjection>`	`params`、`level` (STRICT_WORD/RELAXED_WORD/SQL_KEYWORD)	SQL 注入参数校验

> **缓存反向匹配（cache-arg）** 是 sqltoy 的特色能力，详见缓存翻译使用与超大规模主数据缓存。

4.5 参数形态完整示例

一个 `<sql>` 浓缩 sqltoy 支持的全部参数形态（单值、集合展开、数组下标区间、元组多参数、层级嵌套、括号组），配合 filters 前置规整，构成"**查询条件与具体 SQL 分离**"的典型业务形态：

```
<!-- 1、条件值处理跟具体 sql 分离 -->
<!-- 2、条件值前置通过 filters 定义的通用方法加工规整（大多数不需要额外处理） -->
<sql id="show_case">
    <filters>
        <!-- 参数 statusAry 只要包含 -1（代表全部）则置 null，不参与条件检索 -->
        <eq params="statusAry" value="-1" />
    </filters>
    <value><![CDATA[
        select t.*
        from sqltoy_device_order_info t
        where #[t.status in (:statusAry)]                          -- 集合自动展开
        #[and t.ORDER_ID=:orderId]                                 -- 单值
        #[and t.ORGAN_ID in (:authedOrganIds)]                     -- 集合（数据权限）
        #[and t.STAFF_ID in (:staffIds)]                           -- 集合（多选）
        #[and t.TRANS_DATE>=:beginAndEndDate[0]]                   -- 数组下标取首
        #[and t.TRANS_DATE<:beginAndEndDate[1]]                    -- 数组下标取尾（时间区间）
        #[and (t.tech_group,t.prod_group) in (:techGroups,:prodGroups)] -- 元组多参数
        #[and t.staff_name like :staffName]                        -- 模糊匹配
        #[and t.creator_id=:xxxVo.creatorId]                       -- 对象属性点路径（单对象传参，见下）
        #[and t.priority=:priority #[and t.level>=:level]]         -- 层级嵌套：外层 null 内层一并剔除
        #[and ( #[t.order_id=:orderId2] or #[t.biz_no=:bizNo] )]   -- 括号组：成员独立判空，悬挂 or 自动清理
    ]]></value>
</sql>
```

参数形态速查：

形态	写法	说明
单值	`:orderId`	绑定单个值
集合 / 数组展开	`t.status in (:statusAry)`	数组 / 集合自动展开为多个 `?` 占位符
数组下标	`:beginAndEndDate[0]` / `:beginAndEndDate[1]`	同一数组的元素分别引用，构造时间区间等
元组多参数	`(t.tech_group,t.prod_group) in (:techGroups,:prodGroups)`	两个集合按位置配对展开为 `(a1,b1),(a2,b2)`
模糊匹配	`t.staff_name like :staffName`	可与 filters 的 `l-like` / `r-like` / `escapeLike` 组合
对象属性点路径	`:xxxVo.creatorId`	**单对象传参**：values 只传一个 VO/Map 对象，SQL 参数名即对象属性名，框架反射取值；支持子对象属性级联 `xxxVo.name` 与 `a.b[index]`（大小写不敏感，对应 `BeanUtil.reflectBeanToAry` 语义，`updateByQuery` 等接口同样适用）
层级嵌套	`#[t.priority=:priority #[and t.level>=:level]]`	每层独立判空，外层 null 时内层一并剔除
括号组	`#[and ( #[t.order_id=:orderId2] or #[t.biz_no=:bizNo] )]`	成员各自判空，全部为 null 时整组剔除，残留悬挂 `and` / `or` 自动清理

Java 侧传参（与 names 顺序一一对应，不需要的参数传 null 即可）：

```
// 方式一：多值传参（names 与 values 一一对应）
List rows = lightDao.findByQuery(
        new QueryExecutor("show_case")
            .names("statusAry", "orderId", "authedOrganIds", "staffIds", "beginAndEndDate",
                   "techGroups", "prodGroups", "staffName", "priority", "level",
                   "orderId2", "bizNo")
            .values(
                    new Integer[] { 1, 2, -1 },        // statusAry：含 -1 → eq 置 null，该段剔除
                    "O0001",                            // orderId：单值
                    new String[] { "A01", "A02" },      // authedOrganIds
                    new String[] { "S001", "S002" },    // staffIds
                    new LocalDateTime[] { start, end }, // beginAndEndDate：区间
                    new String[] { "T001", "T002" },    // techGroups
                    new String[] { "P001", "P002" },    // prodGroups
                    "%张%",                             // staffName
                    1, 2,                               // priority、level（层级嵌套）
                    null, "BZ2026001")                  // orderId2=null → 括号组只保留 or 分支
            .resultType(OrderVO.class))
        .getRows();

// 方式二：单对象传参（参数名 = 对象名.属性名，:xxxVo.creatorId 走属性点路径）
StaffInfoVO xxxVo = new StaffInfoVO();
xxxVo.setCreatorId("U0001");   // 对应 #[and t.creator_id=:xxxVo.creatorId]
List rows2 = lightDao.findByQuery(
        new QueryExecutor("show_case")
            .names("xxxVo")     // 只传一个对象
            .values(xxxVo)
            .resultType(OrderVO.class))
        .getRows();
```

五、完整 `<sql>` 规范速查

下面罗列了一个 `<sql>` 元素**所有可用的子标签**（已对照 `sqltoy.xsd` 校验）。这是一份"功能全景"参考——**实际编写时按需选取即可，绝大多数 SQL 只需要 `<value>` 加少量标签**（见本节末尾的最小示例）。注意部分标签互斥（如 `pivot` 与 `unpivot`）。

```xml
<?xml version="1.0" encoding="utf-8"?>
<sqltoy xmlns="http://www.sagframe.com/schema/sqltoy"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.sagframe.com/schema/sqltoy https://sagframe.github.io/schema/sqltoy.xsd">
<!--
  id: 建议 moduleName+functionName 命名，避免不同模块重复
  type: search/insert/update/delete（select 为历史兼容写法，等同 search）
  debug: 是否输出该 sql 执行日志（默认取全局 debug 值）
  blank-to-null: 空白字符串是否转 null，默认 true；filters 中一旦出现 <blank params="具体参数"/> 即自动关闭默认
  query-timeout: 单条 sql 的 statement 超时时长（秒）
  dataSource: 指定数据源名称（特殊场景）
-->
<sql id="sqltoy_sql_specs" type="search" debug="false" blank-to-null="true">
    <!-- sql 功能说明，便于阅读维护，不参与执行，须配合 <value> 使用 -->
    <remarks>员工信息查询演示</remarks>

    <!-- ===== filters：对参与查询/执行的参数值做规整处理（详见本页第四节）===== -->
    <filters>
        <!-- 常用：eq、to-date；针对性：cache-arg；兜底：primary -->
        <!-- 参数等于给定值则转 null（如下拉框"全部"传 -1 时不过滤该条件） -->
        <eq params="organType" value="-1" />
        <!-- 转日期：format 可为 yyyy-MM-dd 等自定义格式，或
FIRST_OF_MONTH/LAST_OF_MONTH/FIRST_OF_YEAR/LAST_OF_YEAR/FIRST_OF_WEEK/LAST_OF_WEEK；
             increment-time 加减时间，increment-unit 默认 days -->
        <to-date params="beginDate" format="yyyy-MM-dd" increment-time="1" increment-unit="days"/>
        <!-- 复制参数值另存为新参数（常与 to-date 配合，由单日期构造日期范围） -->
        <clone param="beginDate" as-param="endDate"/>
        <to-number params="amt" data-type="decimal" />
        <!-- 转字符串，add-quote: none/single/double -->
        <to-string params="staffName" add-quote="single" />
        <!-- 对 like 参数中的特殊符号转义 -->
        <escapeLike params="staffName"/>
        <!-- 左边/右边补 %（参数已含 % 则不处理） -->
        <l-like params="staffName"/>
        <r-like params="staffName"/>
        <!-- 缓存反向匹配：用名称 like 匹配出对应编码作为条件，实现精准查询 -->
        <cache-arg param="staffName" cache-name="staffCache" alias-name="staffIds" prior-match-equal="false">
            <!-- 对缓存做过滤，如个人授权机构、生效状态等；cache-index 对应缓存第几列 -->
            <filter compare-param="1" cache-index="4"/>
        </cache-arg>
        <!-- 首要参数：如精准输入订单编号时，除 excludes 排除的权限条件外其他条件全部置 null -->
        <primary param="orderId" excludes="organIds" />
        <!-- 将数组拼接成 in 条件字符串并加单引号 -->
        <to-in-arg params="organIds" />
        <!-- 日期格式化为字符串（可结合 to-number 得到 202112 这类数字月份） -->
        <date-format params="bizMonth" format="yyyyMM"/>
        <!-- 空白转 null（默认所有空白自动转 null，一般无需配置） -->
        <blank params="*" excludes="staffName" />
        <!-- 区间/大小比较满足则转 null -->
        <between params="amt" start-value="0" end-value="9999" excludes="" />
        <lte params="" value="" />
        <lt params="" value="" />
        <gte params="" value="" />
        <gt params="" value="" />
        <!-- 剔除数组/集合中的 null 或空白 -->
        <remove-null params="organIds" remove-blank="true"/>
        <!-- 字符替换（默认正则全部替换，is-first=true 只替换首个） -->
        <replace params="" regex="" value="" is-first="false" />
        <!-- 默认值：sysdate()-1d（d天/h小时/w周/m月/y年）；first_of_month-3d/first_of_year/last_of_month... -->
        <default params="beginDate" data-type="localDate" value="sysdate()-1d" />
        <!-- 排他参数：某参数为特定值时，将其他参数设为特定值 -->
        <exclusive param="" compare-type="eq" compare-values="" set-params="" set-value="" />
        <!-- 自定义参数处理器（实现 FilterHandler，经 spring.sqltoy.customFilterHandler 配置） -->
        <custom-handler params="" type=""/>
    </filters>

    <!-- ===== 缓存翻译：码值→名称，支持 A,B 拼接翻译（split-sign/link-sign）===== -->
    <!-- uncached-template 未匹配时的补充显示，${value} 表示原 key 值，如 [${value}未定义] -->
    <translate cache="dictCache" cache-type="POST_TYPE" columns="POST_TYPE" cache-indexs="1" uncached-template="" />

    <!-- ===== 安全：脱敏与解密 ===== -->
    <!-- 脱敏 type: tel/name/id-card/bank-card/address/email/public-account/discrete-rate/other -->
    <secure-mask columns="telNo" type="tel" head-size="3" tail-size="4" mask-code="*" mask-rate="50" />
```

```
    <!-- 对加密存储的字段查询时解密 -->
    <secure-decrypt columns="idCard"/>

    <!-- ===== 分库分表 ===== -->
    <sharding-datasource strategy="multiDataBase" />
    <sharding-table tables="sys_order" strategy="hisRealTable" params="beginDate" />

    <!-- ===== 分页优化：缓存相同条件的 count 结果 ===== -->
    <!-- alive-max: 同一 sql 保留多少组不同条件的 count；alive-seconds: 存活秒数；parallel: 是否并行 count 与数据查询 -->
    <page-optimize alive-max="100" alive-seconds="600" parallel="false" />

    <!-- ===== 结果集格式化 ===== -->
    <date-format columns="createTime" format="yyyy-MM-dd HH:mm:ss" />
    <!-- 数字格式：#,###.00、capital(中文大写)、capital-rmb(大写金额)、capital-en；可配 rounding-mode/locale/currency -->
    <number-format columns="totalAmt" format="capital-rmb" />

    <!-- ===== 树形排序、层级内排序、逐层汇总 ===== -->
    <tree-sort id-column="organ_id" pid-column="organ_pid" sum-columns="staff_cnt" level-order-column="staff_cnt" order-way="desc">
        <!-- 状态为 0 的不参与汇总 -->
        <sum-filter column="status" compare-type="neq" compare-values="0"/>
    </tree-sort>

    <!-- ===== SQL 正文（必填）===== -->
    <value>
    <![CDATA[
    -- sql 中可以直接写注释
    select t1.*,t2.ORGAN_NAME from
    @fast(select * from sys_staff_info t
          where #[t.sexType=:sexType]
            #[and t.JOIN_DATE>:beginDate]
            #[and t.STAFF_NAME like :staffName]
            -- @if() 做逻辑判断
            #[@if(:isVirtual==true||:isVirtual==0) and t.IS_VIRTUAL=1]
          ) t1,sys_organ_info t2
     where t1.ORGAN_ID=t2.ORGAN_ID
    ]]>
    </value>

    <!-- 自定义 count 语句（仅分页有效；sqltoy 已智能优化 count，极端性能场景才需手写） -->
    <count-sql><![CDATA[]]></count-sql>

    <!-- ===== 结果集分析算法 ===== -->
    <!-- 分组汇总/求平均 -->
    <summary sum-columns="staff_cnt" average-columns="amt" average-radix-sizes="2" reverse="false" sum-site="left" average-skip-
null="false">
        <global sum-label="总计" label-column="organ_name" />
        <!-- order-column 分组排序列，order-with-sum 默认 true，order-way desc/asc -->
        <group sum-label="小计" label-column="organ_name" group-column="organ_id" order-column="staff_cnt"/>
    </summary>
    <!-- 分组拼接（代替 group_concat/WM_CONCAT）：id-columns 分组列，columns 被拼接列(必填)，result-type 可为 ARRAY/LIST/SET -->
    <link id-columns="organ_id" columns="staff_name" sign="," distinct="true"/>
    <!-- 行转列（与 unpivot 互斥） -->
    <pivot category-columns="order_month" group-columns="fruit_name" start-column="sale_count" end-column="total_amt" default-value="0" />
    <!-- 列转行：columns-to-rows 格式 "列:名称,列:名称" -->
    <unpivot columns-to-rows="chinese_score:语文,math_score:数学" new-columns-labels="subject_name,score_value" />
    <!-- 列环比（水平同类列比较）：group-size 每组指标数，relative-indexs 参与比较的指标序号，start-column 起始列 -->
    <cols-chain-relative group-size="3" relative-indexs="1,2" start-column="1" format="#.00%" />
    <!-- 行环比（垂直行间比较）：group-column 分组列，relative-columns 参与比较的列 -->
    <rows-chain-relative group-column="sale_date" relative-columns="sales_amt" format="#.00%" reduce-one="true" multiply="100" insert="true"
/>
</sql>
</sqltoy>
```

子标签速查表

子标签	作用	详见
`<remarks>`	sql 功能说明（不执行）	本页
`<filters>`	条件参数规整（转 null / 类型转换 / 缓存反向匹配等）	本页第四节
`<translate>`	缓存翻译（码值→名称）	缓存翻译使用
`<secure-mask>` / `<secure-decrypt>`	结果集脱敏 / 字段解密	数据脱敏与加解密
`<sharding-datasource>` / `<sharding-table>`	分库 / 分表策略	分库分表
`<page-optimize>` / `<count-sql>`	分页 count 缓存优化 / 自定义 count	分页优化
`<date-format>` / `<number-format>`	结果集日期 / 数字格式化	本页
`<summary>`	分组汇总 / 求平均	复杂查询与数据分析
`<pivot>` / `<unpivot>`	行转列 / 列转行	复杂查询与数据分析
`<cols-chain-relative>` / `<rows-chain-relative>`	列 / 行同比环比	复杂查询与数据分析
`<tree-sort>`	树形排序与逐层汇总	复杂查询与数据分析
`<link>`	分组拼接（代替 group_concat）	复杂查询与数据分析
`<reverse>`	结果集顺序反转	复杂查询与数据分析
`<value>`	SQL 正文（必填），支持 `#[]`、`@if`、`@fast` 等	本页 / 标签参考

常规最小示例

日常 SQL 通常只需用到上面清单中的少数几个标签。例如页面有一个性别下拉框，"全部"选项 `value="-1"`，用 `eq` 过滤器把 `-1` 转成 null，该条件即自动不参与查询：

```
<select name="sexType">
    <option value="-1">全部</option>
    <option value="F">女性</option>
    <option value="M">男性</option>
</select>
```

```
<sql id="sys_staff_find">
    <filters>
        <!-- sexType=-1（全部）时转 null，#[and t.sex_type=:sexType] 整段被剔除 -->
        <eq params="sexType" value="-1" />
    </filters>
    <translate cache="dictCache" cache-type="POST_TYPE" columns="postType" />
    <value><![CDATA[
        select * from sys_staff_info t
        where 1=1
        #[and t.sex_type=:sexType]
        #[and t.staff_name like :staffName]
    ]]></value>
</sql>
```

六、相关高级语法

- `@if()` / `@elseif()` / `@else`、`@fast`、`@blank()`、`@value()`、`@loop()` / `@secure-loop()`、`@include(sqlId)`、`@include(:scriptParam)`、`@joinFields()`（字段名列表安全拼接，页面配置化报表用）、`@split()`（把字符串参数拆分成数组，配合 `in` 展开）等宏指令，见补充说明：标签与表达式参考。
- 更多实战写法（动态 `@include`、配置驱动 SQL 集成、慢 SQL 统计等）见常见 SQL 案例。
- 查询 API（findOne/find/findPage/findTop/findRandom/getValue/isUnique 等）见常规查询 API。

SQL查询功能

lightDao API

前提：模板SQL

```
<sql id="sqltoy_staff_findlist">
    <!-- 分页优化器,通过缓存实现查询条件一致的情况下在一定时间周期内缓存总记录数量，从而无需每次查询总记录数量 -->
    <!-- parallel:是否并行查询总记录数和单页数据，当alive-max=1 时关闭缓存优化 -->
    <!-- alive-max:最大存放多少个不同查询条件的总记录量; alive-seconds:查询条件记录量存活时长(比如120秒,超过阀值则重新查询) -->
    <page-optimize parallel="true" alive-max="100" alive-seconds="120" />
    <value><![CDATA[
        select t1.*,t2.ORGAN_NAME
        -- @fast() 实现先分页取10条(具体数量由pageSize确定),然后再关联
        from  @fast(
              select t.*  from sqltoy_staff_info t
               where t.STATUS=1
                 #[and t.STAFF_NAME like :staffName]
               order by t.ENTRY_DATE desc
               ) t1
        left join sqltoy_organ_info t2 on  t1.organ_id=t2.ORGAN_ID
          ]]>
    </value>
    <!-- 这里为极特殊情况下提供了自定义count-sql来实现针对性的性能优化 -->
    <!-- <count-sql></count-sql> -->
</sql>
```

findOne 查找单个记录

```
StaffInfo staffInfo = lightDao.findOne("sqltoy_staff_findlist", new StaffInfo(), StaffInfo.class);
```

find 查找列表

```
List<StaffInfo> list = lightDao.find("sqltoy_staff_findlist", new StaffInfo(), StaffInfo.class);
```

findEntity 实体查询

```
List<SysLog> list = lightDao.findEntity(SysLog.class, EntityQuery.create().names("operator").values("112233"));
```

findPage 分页查询

```
Page<StaffInfo> page = lightDao.findPage(new Page<>(10, 1), "sqltoy_staff_findlist", new StaffInfo(), StaffInfo.class);
```

findPageEntity 分页实体查询

```
Page<SysLog> page = lightDao.findPageEntity(new Page<>(10, 1), SysLog.class, EntityQuery.create().names("operator").values("112233"));
```

findTop 查找N条记录

```
List<StaffInfo> list = lightDao.findTop("sqltoy_staff_findlist", new StaffInfo(), StaffInfo.class, 12);
```

findRandom 随机查找N条记录

```
List<StaffInfo> list = lightDao.findRandom("sqltoy_staff_findlist", new StaffInfo(), StaffInfo.class, 12);
```

getValue、getCount、isUnique 快捷查询

- api

```
// getValue：获取查询结果第一条、第一列的值（一般用于 select max(x)、统计值等）
public Object getValue(final String sqlOrSqlId, final Map<String, Object> paramsMap);
public <T> T getValue(final String sqlOrSqlId, final Map<String, Object> paramsMap, final Class<T> resultType);

// getCount：获取记录总量（sql 与单表实体查询两种形态）
public Long getCount(final String sqlOrSqlId, final Map<String, Object> paramsMap);
public Long getCount(Class entityClass, EntityQuery entityQuery);

// isUnique：校验对象属性值在表中是否唯一（常用于编码唯一性校验）
public boolean isUnique(Serializable entity, String... paramsNamed);
```

- 使用范例

```
// 取统计值（指定返回类型）
BigDecimal maxAmt = lightDao.getValue("sqltoy_order_max_amt", paramsMap, BigDecimal.class);

// 记录总量
Long total = lightDao.getCount("sqltoy_order_find", paramsMap);
Long cnt = lightDao.getCount(StaffInfo.class, EntityQuery.create().where("status=?").values(1));

// 唯一性校验（返回 true 表示唯一；paramsNamed 为对象属性名，非数据库列名）
boolean unique = lightDao.isUnique(staffInfo, "staffCode");
// 链式写法：lightDao.unique().entity(entity).fields("staffCode").submit()，见 Link 链式操作
```

> 单条记录也可用 `loadByQuery(new QueryExecutor(sql).names(...).values(...)...)`（返回 Object）；常规场景建议直接用 `findOne`。Top / 随机查询也有 QueryExecutor 形态：`findTopByQuery(queryExecutor, topSize)` 与 `findRandomByQuery(queryExecutor, randomSize)`，详见上文 findTop / findRandom 章节。

fetchStream 流式数据获取

- 针对超大量数据，无法直接存放List等集合，提供反调方式由开发者自行将数据逐行写入到特定的文件或其他存储中

```
/**
 * 流式获取查询结果
 * @param queryExecutor       可通过queryExecutor.showsql()开关sql输出日志
 * @param streamResultHandler
 */
//public void fetchStream(final QueryExecutor queryExecutor, final StreamResultHandler streamResultHandler);

List result = new ArrayList();
// sql 可以写在xml中，这里是演示
String sql = "select * from sqltoy_staff_info";
lightDao.fetchStream(new QueryExecutor(sql).resultType(StaffInfoVO.class),
    new StreamResultHandler() {
        @Override
        public void consume(Object row, int rowIndex) {
            result.add(row);
        }

        // end 一般用于写文件flush等
        @Override
        public void end() {
            System.err.println("完成执行");
        }
    }
);

for (Object item : result) {
    System.err.println(JSON.toJSONString(item));
}
```

> 🎬 完整可运行示例：演示项目 `sqltoy-showcase` 的 `FetchStreamTest.java`。

findByQuery 层次结构封装（主子表一次查询返回父子对象）

场景：主子表通过 join 一次性查出（如数据字典类型 + 字典明细），框架依据结果 VO 上的 `@OneToMany` / `@OneToOne` 注解，自动把平面结果集**分组分层封装**成父子对象（1..n 层级）——无需二次查询、无需手工组装。

第一步：在结果 VO 上声明父子关系（`notNullField` 是关键）

```
/**
 * 主键关联子表信息
 * notNullField: 封装层次结构时，以子表该字段值是否为 null 来辨别 join 出的记录子表是否有数据（左连接场景必设）
 */
@OneToMany(fields = { "dictType" }, mappedFields = { "dictType" }, delete = true,
        orderBy = "showIndex desc", notNullField = "dictKey")
private List<DictDetailVO> dictDetailVOs = new ArrayList<DictDetailVO>();
```

`@OneToMany` / `@OneToOne` 属性说明：

属性	说明
`fields`	主表关联列（可多个，对应复合主键）
`mappedFields`	子表对象对应的关联属性
`notNullField`	层次封装时用于判断子表数据是否存在的非空字段（for hiberarchy）
`orderBy`	**级联加载**时的子表排序，如 `showIndex desc`；层次封装（hiberarchy）场景的子项顺序由 SQL 的 `ORDER BY` 决定
`load`	自定义加载 sql（如 `enable=1` 或完整 sql：`select * from t where fkField=:fkField`）
`delete`	是否级联删除
`update`	定制级联保存/修改时对子表的操作（`delete` 表示先删，`status=0` 表示置停用）

第二步：findByQuery + hiberarchy 完成分层封装

```
@Test
public void testOneToMany() {
    // 一条 join sql 同时查出字典主表 + 字典明细
    String sql = "select sdt.DICT_TYPE_NAME, sdt.COMMENTS dictTypeComments, "
            + "sdt.status dictTypeStatus, sdd.*, sdd.STATUS dictStatus from sqltoy_dict_type sdt "
            + " left join sqltoy_dict_detail sdd on sdt.DICT_TYPE = sdd.DICT_TYPE ";
    List<DictTypeVO> result = lightDao
            .findByQuery(new QueryExecutor(sql).resultType(DictTypeVO.class)
                    // 开启层次封装（若已设置 hiberarchyFieldsMap 则无需再设置）
                    .hiberarchy(true)
                    // 父子对象存在同名属性(如都有 status)时，将结果集 label 与对象属性对应：
                    // key 为查询结果的 label(别名)，value 为实际列/属性名
                    .hiberarchyFieldsMap(DictTypeVO.class,
                            MapKit.keys("dictTypeComments", "dictTypeStatus").values("COMMENTS", "status"))
                    .hiberarchyFieldsMap(DictDetailVO.class,
                            MapKit.keys("dictStatus").values("status")))
            .getRows();
    // 输出 JSON 即为 1..n 层级结构：每个字典类型下挂 dictDetailVOs 列表
    for (DictTypeVO dictType : result) {
        System.err.println(JSON.toJSONString(dictType));
    }
}
```

封装后的结果形如：

```
{ "comments": "设备类型", "dictDetailVOs": [ { "dictKey": "NET", "dictName": "网络设备", "dictType": "DEVICE_TYPE", "showIndex": 2,
"status": 1 } ] }
```

注意

- 方法名是 `hiberarchy` / `hiberarchyFieldsMap`（框架历史拼写，不是 hierarchy）。
- `hiberarchyFieldsMap` 会**自动开启**层次封装并登记涉及的类，因此设置了它就无需再调 `hiberarchy(true)`。
- `hiberarchyFieldsMap` 只针对**父子对象存在同名属性**的场景；无同名属性时仅需 `hiberarchy(true)`。
- 层次封装是"一条 join sql 直接打包"，与对象化 CRUD 中的级联 load（按主键批量二次查询组装）是两种互补机制。

🎬 完整可运行示例：演示项目 `sqltoy-showcase` 的 `HierarchyPackagingTest.java`（其 `DictTypeVO` 的 @OneToMany 已配置 `notNullField = "dictKey"`；子项顺序由 join SQL 的 `ORDER BY` 决定）。

存储过程调用

存储过程调用（`executeStore` / `executeMoreResultStore`、链式 `store()`、out 参数、**多结果集**）的完整说明与范例见 存储过程调用。

```
// 快速示例：调用存储过程并获取结果集（无 out 参数时 outParamsType 传 null）
List result = lightDao.executeStore("{call storeName(?,?)}",
        new Object[]{ value1, value2 }, null, VO.class).getRows();
```

Link 链式操作

`LightDao` 除了提供 `save` / `update` / `find` / `findPage` 等显式接口外，还提供一组**链式（Link）入口**，把"设置参数 → 提交执行"组织成流畅的链式调用。当需要动态指定**数据源、锁、超时、批量大小、并行配置、强制更新字段**等时，链式写法更清晰灵活。

一、链式入口概览

入口	返回	用途	提交方法
`lightDao.query()`	`Query`	SQL/实体查询	`find()` / `getOne()` / `getValue()` / `count()` / `findPage()` / `findTop()` / `findRandom()`
`lightDao.save()`	`Save`	保存对象	`one(entity)` / `many(entities)`
`lightDao.update()`	`Update`	修改对象	`one(entity)` / `many(entities)`
`lightDao.delete()`	`Delete`	删除对象	`one(entity)` / `many(entities)`
`lightDao.load()`	`Load`	加载对象（可级联/加锁）	`one(entity)` / `many(entities)`
`lightDao.unique()`	`Unique`	唯一性校验	`submit()`
`lightDao.execute()`	`Execute`	执行 insert/update/delete SQL	`submit()` / `insertReturnPrimaryKey()`
`lightDao.batch()`	`Batch`	批量执行	`submit()`
`lightDao.store()`	`Store`	存储过程调用	`submit()`
`lightDao.treeTable()`	`TreeTable`	树形表节点路径构造	`submit()`
`lightDao.elastic()`	`Elastic`	Elasticsearch 查询	`find()` / `getOne()` / `findPage()` / `findTop()`
`lightDao.mongo()`	`Mongo`	MongoDB 查询	`find()` / `getOne()` / `findPage()` / `findTop()`
`lightDao.tableApi()`	`TableApi`	表元数据/truncate/drop	直接调用

> 几乎所有链式对象都支持 `.dataSource(DataSource)` 来**临时切换数据源**，以及 `.autoCommit(Boolean)`、`.batchSize(int)`、`.parallelConfig(ParallelConfig)` 等。

二、查询 query()

```
// 查询列表：sql 可以是 xml 中的 sqlId，也可以是直接 sql
List<StaffInfoVO> list = lightDao.query()
        .sql("sys_staff_find")
        .names("staffName", "status")
        .values("张三", 1)
        .resultType(StaffInfoVO.class)
        .find();

// 通过对象传参（框架按 sql 中参数名映射对象属性取值）
StaffInfoVO one = (StaffInfoVO) lightDao.query()
        .sql("sys_staff_find")
        .entity(staffInfoVO)
        .resultType(StaffInfoVO.class)
        .getOne();

// 获取单值
Long count = lightDao.query().sql("select count(1) from sqltoy_staff_info").getValue(Long.class);

// 分页（Page 构造为 new Page(pageSize, pageNo)）
Page<StaffInfoVO> page = (Page) lightDao.query()
        .sql("sys_staff_find").resultType(StaffInfoVO.class)
        .findPage(new Page(10, 1));

// 取 Top / 随机
List top = lightDao.query().sql("sys_staff_find").resultType(StaffInfoVO.class).findTop(10);

// 加锁、超时、流式抓取大小
lightDao.query().sql(...).lock(LockMode.UPGRADE).queryTimeout(30).fetchSize(1000).find();
```

`Query` 可链式设置：`sql`、`names`、`values`、`entity`、`resultType`、`dataSource`、`fetchSize`、`maxRows`、`lock`、`lockWaitTimeout`、`queryTimeout`、`humpMapLabel`（Map 结果是否驼峰命名）。

三、保存 save()

```
// 保存单个对象，返回主键
Object pk = lightDao.save().one(entity);

// 批量保存，返回成功条数
Long rows = lightDao.save().many(entities);

// 深度级联保存 + 保存模式 + 批量大小 + 并行
lightDao.save()
        .deeply(true)                 // 级联保存关联对象
        .saveMode(SaveMode.IGNORE)    // APPEND(普通)/UPDATE(saveOrUpdate)/IGNORE(存在则忽略)
        .batchSize(500)
        .parallelConfig(parallelConfig)
        .many(entities);
```

四、修改 update()

```
// 弹性更新：默认 null 字段不参与更新，一次数据库交互完成
Long rows = lightDao.update().one(entity);

// 强制更新指定字段（即使值为 null 也更新）
lightDao.update().forceUpdateFields("photo", "remark").one(entity);

// 深度级联更新 / 指定级联类
lightDao.update().deeply(true).one(entity);
lightDao.update().cascadeClasses(OrganInfo.class).one(entity);

// 批量更新
lightDao.update().batchSize(500).many(entities);
```

`Update` 可链式设置：`deeply`、`updateFields`、`forceUpdateProps`、`forceUpdateFields`、`cascadeClasses`、`cascadeForceUpdate`、`batchSize`、`autoCommit`、`dataSource`、`parallelConfig`。

五、删除 delete() 与加载 load()

```
// 删除
lightDao.delete().one(entity);
lightDao.delete().many(entities);

// 加载单个对象，并级联加载关联对象
StaffInfo staff = lightDao.load().cascade(OrganInfo.class).one(entity);
// 加载全部级联
lightDao.load().cascadeAll().one(entity);
// 加锁加载（悲观锁）
lightDao.load().lock(LockMode.UPGRADE).lockWaitTimeout(10).one(entity);
// 批量加载
List list = lightDao.load().many(entities);
```

`LockMode`：`UPGRADE`（普通悲观锁）、`UPGRADE_NOWAIT`（不等待）、`UPGRADE_SKIPLOCK`（跳过已锁记录）。

六、唯一性 unique()

```
// 校验对象指定属性在数据库中是否唯一（属性名，非数据库列名）
boolean unique = lightDao.unique()
        .entity(entity)
        .fields("staffCode", "tenantId")
        .submit();
```

七、执行 SQL execute() 与批量 batch()

```
// 执行 update/insert/delete/merge 等语句，返回影响行数
Long rows = lightDao.execute()
        .sql("update sqltoy_staff_info set status=:status where staff_id=:staffId")
        .names("status", "staffId").values(0, "S0001")
        .submit();

// insert 并返回主键
Object pk = lightDao.execute().sql("sys_staff_insert").entity(entity)
        .insertReturnPrimaryKey("staffId");

// 批量执行（dataSet 为 List<Map> 或 List<VO>）
Long batchRows = lightDao.batch()
        .sql("sys_staff_insert")
        .dataSet(dataList)
        .batchSize(500)
        .submit();
```

八、存储过程 store()

```
// 调用存储过程并获取结果
StoreResult result = lightDao.store()
        .sql("{call storeName(?,?)}")
        .inParams(value1, value2)
        .resultType(StaffInfoVO.class)
        .submit();
List rows = result.getRows();

// 带 out 参数 / 多结果集
StoreResult more = lightDao.store()
        .sql("{call storeName(?,?,?)}")
        .inParams(value1, value2)
        .outTypes(java.sql.Types.INTEGER)
        .moreResult(true)
        .resultTypes(StaffInfoVO.class)
        .submit();
Object[] outValues = more.getOutResult();
List[] allResults = more.getMoreResults();
```

九、树形表 treeTable()

```
// 构造树形表节点路径(node_route)、层级、叶子标识等
boolean ok = lightDao.treeTable()
        .treeModel(new TreeTableModel(OrganInfo.class)
                .idField("organId").pidField("organPid")
                .nodeRouteField("nodeRoute").nodeLevelField("nodeLevel")
                .isLeafField("isLeaf"))
        .submit();
```

十、NoSQL：elastic() 与 mongo()

```
// Elasticsearch（sql 模式或原生 json 模式）
List list = lightDao.elastic().sql("es_sql_id").names(...).values(...).resultType(...).find();
Page page = lightDao.elastic().sql("es_sql_id").findPage(new Page(10, 1));

// MongoDB
List mList = lightDao.mongo().sql("mql_id").entity(queryVO).resultType(...).find();
```

详见 Elasticsearch 支持 与 MongoDB 支持。

十一、表元数据 tableApi()

```
// 获取表/列元数据
List<TableMeta> tables = lightDao.tableApi().getTables(null, null, "sqltoy%");
List<ColumnMeta> columns = lightDao.tableApi().getTableColumns(null, null, "sqltoy_staff_info");
// 清空表 / 删除表（支持表名或实体类）
lightDao.tableApi().truncate("sqltoy_staff_info");
lightDao.tableApi().drop(StaffInfo.class);
```

Lambda 条件查询与更新

> 自 6.0.8 起提供。以**实体属性的方法引用**（如 `StaffInfo::getStatus` ）组织查询、更新、删除条件，编译期即可发现属性名拼写错误；单表与多表 join 使用同一套构建器，是否 join 由是否调用 join 步骤自动判定。

〇、定位与分工

sqltoy 提供三种组织查询的方式，按场景选择：

方式	适用场景	入口
Lambda 条件（本文）	单表/多表 join 的常规条件查询、按条件更新删除，条件以属性引用表达	`lightDao.lambdaQuery/lambdaUpdate/lambdaDelete`
EntityQuery/EntityUpdate	单表字符串条件（where 文本 + 命名参数）	`lightDao.findEntity/updateByQuery` 等
SQL 模式（sqlId + `#[...]` 动态段 + `@if` 宏）	复杂动态 SQL、多分支条件组合、方言差异、配置化报表	`lightDao.find/findPage` + sqlId，见动态 SQL 编写规范

Lambda 层的动态化以 `IfPresent` 语法糖为推荐惯用法（见动态化）；**复杂动态场景不要试图在 lambda 层堆条件，直接走 SQL 模式**——动态段、宏、方言变体与热更新是 SQL 模式的既有能力。

Lambda 查询在**单表通道**把条件转换为 `EntityQuery` / `EntityUpdate`，复用实体链路的全部内核能力（分库分表、缓存翻译、加解密脱敏、租户过滤、统一更新字段）；**join 通道**拼装完整 SQL 走原生 SQL 通道，实体级增强不再生效（多表 SQL 本就无法套用单实体增强）。

一、入口与构建器

```
LambdaQuery<T>  lightDao.lambdaQuery(Class<T> entityClass);   // 查询（单表或 join）
LambdaUpdate<T> lightDao.lambdaUpdate(Class<T> entityClass);  // 更新（单表或 join）
LambdaDelete<T> lightDao.lambdaDelete(Class<T> entityClass);  // 删除（单表或 join）
```

构建器是单一可变对象，所有方法返回自身，**链式与命令式写法可自由混写**：

```
LambdaQuery<StaffInfo> query = lightDao.lambdaQuery(StaffInfo.class);
if (status != null) {
    query.eq(StaffInfo::getStatus, status);
}
List<StaffInfo> list = query.orderByDesc(StaffInfo::getEntryTime).find();
```

> **注**：上例的 `if` 仅为演示「构建器是单一可变对象，命令式控制流（if/for 等）可与链式自由混写」这一能力，**并非推荐拿 `if` 手动判空**。日常「参数为 null 就跳过该条件」的正常写法是 `eqIfPresent`，一行搞定、无需手写 `if`（详见后文「动态化条件」）：

```
List<StaffInfo> list = lightDao.lambdaQuery(StaffInfo.class)
        .eqIfPresent(StaffInfo::getStatus, status)   // status 为 null 时自动跳过该条件
        .orderByDesc(StaffInfo::getEntryTime)
        .find();
```

也可以先构建后执行：`LambdaQuery.create(...)` 创建的构建器未绑定 dao，终端方法不可用，调用 `bind(lightDao)` 绑定后即可执行（适合在 service 层组装条件、跨层传递后统一执行的场景）：

```
LambdaQuery<StaffInfo> query = LambdaQuery.create(StaffInfo.class).eq(StaffInfo::getStatus, 1);
List<StaffInfo> list = query.bind(lightDao).find();
```

> `LightDao` 上 lambda 相关方法只有三个工厂（`lambdaQuery/lambdaUpdate/lambdaDelete`），**执行全部由构建器终端完成**，dao 层不提供执行重载。

查询终端方法一览：

终端	语义
`find()`	全部符合条件的记录
`getOne()`	单条记录：**0 条返回 null，多于 1 条抛** `DataAccessException`（匹配多条说明条件有缺陷，不静默取第一条）
`findPage(Page)`	分页，`new Page(pageSize, pageNo)`
`findTop(n)` / `findTop(0.2)`	前 n 条 / 前 20%（大于 1 为条数，小于 1 为比例），顺序由 `orderBy` 决定
`findRandom(n)` / `findRandom(0.2)`	随机 n 条 / 随机比例
`count()` / `exists()`	记录数 / 是否存在

指定数据源（多数据源 / 读写分离）

构建器提供 `.dataSource(DataSource)`，显式指定本次操作使用的数据源；不调用时使用 dao 的默认数据源。与 sqltoy 既有惯例一致（`EntityQuery` / `QueryExecutor` / `lightDao.update().dataSource(...)` 同款），**单表与 join 两通道的查询 / count / 更新 / 删除全部终端均生效**：

```
// 单表查询走只读从库
List<StaffInfo> list = lightDao.lambdaQuery(StaffInfo.class)
        .dataSource(readOnlyDs).eq(StaffInfo::getStatus, 1).find();

// join 更新落到指定库
lightDao.lambdaUpdate(StaffInfo.class).dataSource(ds2)
        .leftJoin(Depart.class).on(StaffInfo::getDeptId, Depart::getId)
        .set(StaffInfo::getStatus, 2).eq(Depart::getDeptName, "研发").update();
```

> 单表通道经 `EntityQuery/EntityUpdate` 透传数据源；join 通道经 sqltoy 既有的 `query()` / `execute()` 链式构建器透传（查询/count 用 `query().dataSource(ds)`、更新/删除用 `execute().dataSource(ds)`，二者本就支持指定数据源）；多删除目标的 join 删除，其事务也使用所指定的数据源。`dataSource` 为 null 时行为与未指定完全一致。

分页优化（并行 count + keep-alive 缓存）

`findPage` 可通过 `.pageOptimize(PageOptimize)` 复用 sqltoy 既有的分页优化（不设置时用全局 `SqlToyConfig.pageOptimize`）。两个常用旋钮：`parallel(true)` 让 count 与数据查询并行；`aliveSeconds(n)` 把 count 结果缓存 n 秒（keep-alive），翻页时命中缓存即跳过 count 查询：

```
Page<StaffInfo> page = lightDao.lambdaQuery(StaffInfo.class)
        .eq(StaffInfo::getStatus, 1)
        .pageOptimize(new PageOptimize().parallel(true).aliveSeconds(120))
        .findPage(new Page(20, 1));
```

`PageOptimize` 还提供 `aliveMax`（缓存条目上限）、`skipZeroCount`（命中缓存的 count 为 0 时跳过该缓存、重新查询总记录数）、`parallelMaxWaitSeconds`（并行等待上限）。单表与 join 两通道均生效（单表经 `EntityQuery` 透传、join 经 `QueryExecutor` 透传）。

二、单表查询

条件方法

每个条件方法都有 `(boolean condition, ...)` 重载，`condition` 为 false 时该条件不参与 SQL：

方法	SQL 形态
`eq / ne / gt / ge / lt / le`	`字段 = :p1`、`<>`、`>`、`>=`、`<`、`<=`
`between / notBetween`	`字段 BETWEEN :p1 AND :p2`
`like / notLike`	`字段 LIKE :p1` / `NOT LIKE :p1`，通配符 `%值%` 由框架补入参数值（包含匹配）
`likeLeft / likeRight`	`LIKE :p1`，参数值分别为 `%值`（以值结尾）/ `值%`（以值开头）
`likeExact / likeExactLeft / likeExactRight`	同上三个方向，但值中的 `%` 先转义为字面量再补方向 `%`——用户输入含 `%` 时按普通字符匹配而非被当通配符（如搜 `"50%"` 折扣不会命中 `5xx…` 的任意串）
`isNull / isNotNull`	`字段 IS NULL` / `IS NOT NULL`
`in(Collection)` / `in(值...)`	`字段 IN (:p1)`，空集合/空数组自动跳过
`notIn(Collection)` / `notIn(值...)`	`字段 NOT IN (:p1)`
`inSql(字段, 子查询[, 参数Map])`	`字段 IN (子查询)`，子查询的 sqltoy 命名参数经 Map 提供
`inb(字段A, 字段B, 元组集合)`	多字段元组 in：单列退化为 `字段 IN (:p1,:p2…)`；多列展开为 OR-of-ANDs `((a=:p1 AND b=:p2) OR (a=:p3 AND b=:p4)…)`，规避行值 IN 在 SQLServer/DB2 等库的方言差异

```
List<StaffInfo> list = lightDao.lambdaQuery(StaffInfo.class)
        .eq(StaffInfo::getStatus, 1)
        .like(StaffInfo::getStaffName, "张")
        // 值中的%按普通字符匹配:搜索"50%"不会把%当通配符命中"5"开头的任意串
        .likeExactRight(StaffInfo::getStaffCode, "50%")
        .between(StaffInfo::getEntryTime, start, end)
        .in(StaffInfo::getOrganId, organIds)
        .orderByDesc(StaffInfo::getEntryTime)
        .find();
```

exists / notExists 关联子查询

子查询以 lambda 构建，关联外层列用 `OuterRef.of(外层字段)` 表达（类型安全，无需手写表名/别名）：

```
// 单表通道:存在"状态启用且部门id匹配"的部门记录
lightDao.lambdaQuery(StaffInfo.class)
        .exists(OrganInfo.class, sub -> sub
                .eq(OrganInfo::getStatus, 1)
                .eq(OrganInfo::getOrganId, OuterRef.of(StaffInfo::getOrganId)))  // 关联外层列
        .find();
// ... WHERE ... AND EXISTS (SELECT 1 FROM organ_table sq1 WHERE sq1.STATUS = :p2 AND sq1.ORGAN_ID = staff_table.ORGAN_ID)

// join 通道同样可用,外层列渲染为外层别名(如 s1.ORGAN_ID)
lightDao.lambdaQuery(StaffInfoVO.class).selectAll()
        .from(StaffInfo.class).leftJoin(OrganInfo.class).on(StaffInfo::getOrganId, OrganInfo::getOrganId)
        .notExists(BlackList.class, sub -> sub.eq(BlackList::getStaffId, OuterRef.of(StaffInfo::getStaffId)))
        .find();
```

约定与边界：

- `OuterRef` 仅支持 `eq/ne/gt/ge/lt/le` 六个比较方法（between/like/in 接收 OuterRef 会抛异常）；用在子查询之外也抛异常；
- 单表通道外层列渲染为 `表名.列`、join 通道渲染为 `别名.列`——子查询内必须限定，否则与子查询同名列歧义；
- 子查询表别名以 `sq` 前缀生成（sq1、sq2…），与外层单字母别名不碰撞；嵌套 exists 支持多层；
- 子查询参数与外层共享命名参数序号（`:p1、:p2…` 全局唯一）；
- 子查询实体须与子条件字段所属实体一致，否则装配期抛异常。

终端 `exists()`（是否存在符合条件记录）实现为 `findTop(1)` 命中即停，不做全量 count 聚合。

逻辑组合与嵌套

```
// and/or 连接符、嵌套括号、not 取反（一层分组）
lightDao.lambdaQuery(StaffInfo.class)
        .eq(StaffInfo::getStatus, 1)
        .or(n -> n.eq(StaffInfo::getOrganId, "O001").eq(StaffInfo::getStaffName, "张三"))
        .not(n -> n.isNull(StaffInfo::getMobile))
        .find();
// → status = :p1 OR (organ_id = :p2 AND staff_name = :p3) AND NOT (mobile IS NULL)

// 多层分组：nested/and/or/not 的参数 n 是同类型构建器，可继续往下嵌套，层数不限
lightDao.lambdaQuery(StaffInfo.class)
        .nested(outer -> outer
                .nested(inner -> inner.eq(StaffInfo::getOrganId, "O001").or().eq(StaffInfo::getOrganId, "O002"))
                .eq(StaffInfo::getStatus, 1))
        .or().eq(StaffInfo::getStaffId, "S009")
        .find();
// → ((organ_id = :p1 OR organ_id = :p2) AND status = :p3) OR staff_id = :p4

// accept：复用一段条件、并入当前层（不加括号）；nested：独立分组（加括号）
lightDao.lambdaQuery(StaffInfo.class).eq(StaffInfo::getStatus, 1)
        .accept(q -> q.eq(StaffInfo::getOrganId, "O001").eq(StaffInfo::getStaffName, "张三")).find();
// → status = :p1 AND organ_id = :p2 AND staff_name = :p3
lightDao.lambdaQuery(StaffInfo.class).eq(StaffInfo::getStatus, 1)
        .nested(q -> q.eq(StaffInfo::getOrganId, "O001").eq(StaffInfo::getStaffName, "张三")).find();
// → status = :p1 AND (organ_id = :p2 AND staff_name = :p3)

// last 追加原始 sql 片段（有注入风险，谨慎）
lightDao.lambdaQuery(StaffInfo.class).eq(StaffInfo::getStatus, 1).last("limit 10").find();
```

相邻条件自动补 `AND`；`and()/or()` 显式连接符相邻时合并保留最后一个；首尾悬挂连接符自动剔除；每层嵌套整体以括号包裹，**层数不限**（实际受 JVM 栈深度与目标库表达式嵌套上限约束，常见业务 1~3 层即可满足，更复杂的动态分组建议使用 SQL 模式）。**空分组自动丢弃**：当组内条件全被 `condition=false` 或 `IfPresent` 跳过时，整组连同多余连接符一并消失，不会渲染出 `()` 这类非法 SQL——因此动态分组可放心写，参数缺失时该组自然不参与查询。

分组、排序与聚合过滤

```
lightDao.lambdaQuery(StaffInfo.class)
        .groupBy(StaffInfo::getOrganId)
        .having("count(*) > :min", MapKit.map("min", 5))
        .orderByAsc(StaffInfo::getOrganId)
        .orderByDesc(StaffInfo::getEntryTime)   // 多次 orderBy 合并为一个子句
        .find();
```

仅分组/排序而无 where 条件时，自动以 `1 = 1` 补齐 where。

select 指定字段

```
lightDao.lambdaQuery(StaffInfo.class)
        .select(StaffInfo::getStaffId, StaffInfo::getStaffName)   // 属性名形式
        .eq(StaffInfo::getStatus, 1)
        .find();
```

动态化条件

推荐惯用法是 `IfPresent` 语法糖：值为 null（字符串为空白、集合为空）时条件自动跳过：

方法	跳过条件
`eqIfPresent / neIfPresent / gtIfPresent / geIfPresent / ltIfPresent / leIfPresent`	值为 null
`likeIfPresent`	空白串
`inIfPresent(Collection)` / `inIfPresent(值...)` / `notInIfPresent(Collection)`	集合/数组为 null 或空
`betweenIfPresent(v1, v2)` / `betweenIfPresent(数组)`	双 null 跳过；半开区间自动退化：仅下限→ `ge`，仅上限→ `le`

```
// 页面查询条件的典型写法：参数有没有就拼不拼
lightDao.lambdaQuery(StaffInfo.class)
        .eqIfPresent(StaffInfo::getStatus, status)
        .likeIfPresent(StaffInfo::getStaffName, name)
        .inIfPresent(StaffInfo::getOrganId, organIds)
        .betweenIfPresent(StaffInfo::getEntryTime, start, end)
        .find();
```

被跳过的条件在装配期不会解析列名，因此不产生任何副作用。**更复杂的动态组合（多分支、方言差异、配置化）请使用 SQL 模式。**

值过滤策略

默认过滤 null 值条件；可在添加条件之前设置实例级策略：

```
lightDao.lambdaQuery(StaffInfo.class)
        .valueFilter(v -> v != null && !"".equals(v))   // 实例级，非全局静态
        .eq(StaffInfo::getStaffName, name)
        .find();
```

三、单表更新

```
lightDao.lambdaUpdate(StaffInfo.class)
        .set(StaffInfo::getStatus, 2)
        .setExpr(StaffInfo::getLoginCount, "loginCount + 1")   // 基于当前值计算
        .eq(StaffInfo::getStaffId, staffId)
        .update();
```

setExpr 表达式更新

形态	示例	说明
字面量表达式	`.setExpr(U::getLoginCount, "loginCount + 1")`	表达式中的字段以**属性名**书写，执行时自动转数据库字段，可引用同表其他字段
`?` 占位绑定值	`.setExpr(U::getSalary, "salary * ?", rate)`	单个 `?` 绑定第二个参数
`:name` 命名参数	`.setExpr(U::getSalary, "salary * :rate", rate)`	单表通道支持（复用 updateByQuery 的表达式 set 能力）

表达式内**不支持 `=` 字符**（赋值等号由框架组装），传入即抛异常。

默认值过滤会跳过 `set(col, null)`；需要把字段显式置 null 时用 `setNull`：

```
lightDao.lambdaUpdate(StaffInfo.class).setNull(StaffInfo::getMobile).eq(StaffInfo::getStaffId, id).update();
```

安全边界

- **无任何 where 条件的更新/删除默认抛 `DataAccessException`**，需显式 `.matchAll()` 放行——避免误写全表更新；
- where 为字面量（无命名参数）时内部自动补 `1=:DEFAULT_VALUE` 兜底，规避参数计数校验，使用者无感；
- **整对象更新（按主键）不在 lambda 层提供**，直接使用既有链路：`lightDao.update(entity)` / `lightDao.update(entity, forceUpdateProps)`。

四、单表删除

```
lightDao.lambdaDelete(StaffInfo.class)
        .eq(StaffInfo::getStatus, 0)
        .delete();

// 无条件删除须显式放行
lightDao.lambdaDelete(StaffInfo.class).matchAll().delete();
```

删除条件构建器只支持 where 条件（无 select/orderBy），与查询构建器职责分离。

五、多表 join 查询

在同一个 `LambdaQuery` 上调用 join 步骤即切换为 join 通道，**单表只是 join 的退化形态**：

```
public class StaffInfoVO {
    private String staffId;
    private String staffName;
    private String organName;    // 来自关联表
    // getter/setter
}

List<StaffInfoVO> rows = lightDao.lambdaQuery(StaffInfoVO.class)
        .selectAll(ExcludeCols.of(StaffInfo::getStatus))          // 主表全部字段，排除 status
        .from(StaffInfo.class)                                    // 结果类型为 VO 时须显式指定主表
        .leftJoin(OrganInfo.class).on(StaffInfo::getOrganId, OrganInfo::getOrganId)
        .select(JoinCol.of(OrganInfo::getOrganName).as("organName"))  // 额外列 + 结果别名
        .eqIfPresent(StaffInfo::getStatus, status)
        .orderByAsc(StaffInfo::getStaffId)
        .find();
```

生成 SQL（列按所属实体别名限定，`AS` 别名用于结果对象属性映射）：

```
SELECT s1.STAFF_ID AS staffId, s1.STAFF_NAME AS staffName, o2.ORGAN_NAME AS organName
FROM sqltoy_staff_info s1 LEFT JOIN sqltoy_organ_info o2 ON s1.ORGAN_ID = o2.ORGAN_ID
WHERE s1.STATUS = :p1
ORDER BY s1.STAFF_ID ASC
```

join 步骤

方法	说明
`from(entityClass)`	指定主表；**可省略**——查询场景缺省以结果类型作为主表（结果类型为 VO 等非实体时必须显式指定）
`leftJoin / rightJoin / innerJoin(entityClass)`	发起连接，返回 on 步骤；首次调用即切换 join 通道
`on(字段A, 字段B)`	等值关联；同一 join 可传两对字段（自动 AND）；完成后返回查询构建器，可继续叠加 join 或条件

表别名按实体类名首字母小写加序号生成（`s1`、`o2`…）。join 后缺 `on()`、条件引用未参与 join 的实体，均在装配期抛异常。

select 的几种形态

形态	语义
不写 select	结果类型参与 join 时查其全部字段（`列 AS 属性名`），否则 `*`
`selectAll()`	显式声明主表全部字段，可与 `select(JoinCol...)` 额外列并存（主表字段在前）
`selectAll(Entity.class)`	将某个**被 join 实体**的全部字段以 `别名.列 AS 属性` 并入 select（与主表重复时自动跳过）；该实体须参与 join，否则装配期抛异常
`selectAll(ExcludeCols.of(A::getX, A::getY))`	主表全部字段**排除**指定字段；排除字段须属于主表实体；单表通道转 `EntityQuery.unselect` 同样生效
`select(JoinCol...)`	指定列：`JoinCol.of/count/sum/avg/min/max(字段).as(别名)`，存在即走 join 通道；单表分组聚合同样用它（配合 `from(主表)` + `groupBy` + `having`）
`select(JoinCol.raw("..."))`	**原生 SQL 片段列**：类型化 select 表达不了的形态（窗口函数 `ROW_NUMBER() OVER(..)`、数据库特有函数、`CASE WHEN`、算术表达式等）；`JoinCol.raw(片段).as(别名)` 或片段内联 `... AS 别名`，原样拼入 SELECT、触发 join 通道（结果映射到含该别名字段的 VO）
`distinct()`	结果去重：单表通道转 `EntityQuery.distinct()`，join 通道渲染 `SELECT DISTINCT`（DISTINCT 是量词不是函数，故不提供列函数形态）

`JoinCol.raw(...)` 用于补类型化 select 的盲区（如窗口函数），片段内引用的是**真实数据库列名/表别名**（单表可用非限定列名，多表用自动生成的别名如 `j1`）：

```
// 窗口函数：按薪资排名，rn 映射到 VO 的 rankNo 字段
List<StaffRankVO> ranks = lightDao.lambdaQuery(StaffRankVO.class)
        .select(JoinCol.of(StaffInfo::getStaffName).as("staffName"),
                JoinCol.raw("ROW_NUMBER() OVER(ORDER BY SALARY DESC)").as("rankNo"))
        .from(StaffInfo.class)
        .find();
// → SELECT s1.STAFF_NAME AS staffName, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS rankNo FROM sqltoy_staff_info s1
```

> `JoinCol.raw` 与 `last(sql)` 同为**原始 SQL 逃逸口**：框架原样拼接、不做注入筛查，**禁止拼入终端用户输入**；跨库使用时注意片段本身的方言可移植性（如窗口函数各库语法差异）。

分组聚合（sum/avg/min/max/count）

聚合列用 `JoinCol` 的函数工厂表达，配合 `groupBy` / `having`；**单表分组聚合不需要 join**——`JoinCol` 会把查询切到 SQL 通道，此时用 `from(主表)` 指定聚合对象：

```
public class OrganSalaryStatVO {
    private String organId;
    private BigDecimal totalSalary;
    private BigDecimal avgSalary;
    private Long staffCount;
    // getter/setter
}

List<OrganSalaryStatVO> stats = lightDao.lambdaQuery(OrganSalaryStatVO.class)
        .select(JoinCol.of(StaffInfo::getOrganId).as("organId"),
                JoinCol.sum(StaffInfo::getSalary).as("totalSalary"),
                JoinCol.avg(StaffInfo::getSalary).as("avgSalary"),
                JoinCol.count(StaffInfo::getStaffId).as("staffCount"))
        .from(StaffInfo.class)
        .eqIfPresent(StaffInfo::getStatus, status)
        .groupBy(StaffInfo::getOrganId)
        .having("sum(SALARY) > :min", MapKit.map("min", 10000))   // having 为原始sql,命名参数经Map提供
        .find();
```

join 场景的聚合写法相同，只是多一步 `leftJoin(...).on(...)`；`having` 文本为原始 SQL（列写实际列名），分组键为 null 的行同样参与聚合（SQL 标准行为）。聚合结果按聚合列排序用别名排序：

```
.groupBy(StaffInfo::getOrganId)
.orderByDesc("staffCount")   // 按 select 中 as 定义的别名排序,仅 join 通道有效
```

单表通道没有列别名，调用别名排序会在装配期抛异常；别名须为普通标识符（防注入）。

结果映射与 LEFT JOIN

`AS` 别名与结果类型属性同名即映射；结果类型中没有的列自动忽略。LEFT JOIN 下关联表无匹配行时，对应属性为 null。

join 通道的终端与限制

`find/getOne/findPage/findTop/findRandom/count/exists` 全部可用；**join 通道不继承实体级增强**（分片、缓存翻译、脱敏等），这是多表 SQL 的固有边界。

六、多表 join 更新与删除

join 更新/删除采用**可移植的 EXISTS-over-derived 改写**（非 MySQL 多表 DML 语法）：把 join 与 where 条件整体放入派生表，外层以主键做 EXISTS 关联限定，因此完整保留 LEFT JOIN 语义，并覆盖 Oracle/PostgreSQL 等本身没有多表 `UPDATE ... JOIN` / `DELETE ... JOIN` 语法的数据库：

```
UPDATE 目标表 SET 列 = :p1 WHERE EXISTS (
    SELECT 1 FROM (SELECT 别名.主键 AS LPK1 FROM 主表 别名 LEFT JOIN ... WHERE 条件) sqd
    WHERE sqd.LPK1 = 目标表.主键)

DELETE FROM 目标表 WHERE EXISTS (
    SELECT 1 FROM (SELECT 别名.主键 AS LPK1 FROM 主表 别名 LEFT JOIN ... WHERE 条件) sqd
    WHERE sqd.LPK1 = 目标表.主键)
```

```
// join 更新
lightDao.lambdaUpdate(StaffInfo.class)
        .leftJoin(OrganInfo.class).on(StaffInfo::getOrganId, OrganInfo::getOrganId)
        .set(StaffInfo::getStatus, 2)
        .eq(OrganInfo::getOrganName, "研发部")
        .update();
// UPDATE sqltoy_staff_info SET STATUS = :p1 WHERE EXISTS (
//   SELECT 1 FROM (SELECT s1.STAFF_ID AS LPK1 FROM sqltoy_staff_info s1
//                  LEFT JOIN sqltoy_organ_info o2 ON s1.ORGAN_ID = o2.ORGAN_ID
//                  WHERE o2.ORGAN_NAME = :p2) sqd
//   WHERE sqd.LPK1 = sqltoy_staff_info.STAFF_ID)

// join 删除：deleteTables 指定删除目标（缺省主表）
lightDao.lambdaDelete(StaffInfo.class)
        .leftJoin(OrganInfo.class).on(StaffInfo::getOrganId, OrganInfo::getOrganId)
        .deleteTables(StaffInfo.class)
        .eq(OrganInfo::getOrganName, "市场部")
        .delete();
// DELETE FROM sqltoy_staff_info WHERE EXISTS (SELECT 1 FROM (SELECT s1.STAFF_ID AS LPK1 FROM ... ) sqd WHERE sqd.LPK1 =
sqltoy_staff_info.STAFF_ID)
```

- join 更新/删除的**主表隐式为构建器实体**，无需 `from()`；
- **目标实体必须有 `@Id` 主键**（复合主键按 `LPK1…LPKn` 逐一关联）：EXISTS 改写靠主键回连目标表，无主键实体在装配期抛 `DataAccessException`；
- 改写被 mysql/postgresql/sqlserver/oracle/db2 等关系型库真库验证（属发版多库门禁范围）；clickhouse/doris/starrocks 等没有事务性行级 DML 能力的库不支持，需改走 SQL 模式；
- join 更新的 `setExpr` 表达式**原样输出**（单表通道"属性名自动转列名"不适用），字段请写实际列名，单个 `?` 仍可绑定值；
- join 通道同样受无条件守卫约束（`matchAll()` 放行）。

六之二、组合查询：CTE（WITH AS）/ @fast 分页下推 / UNION

lambda 支持把**多个 lambdaQuery 组合**成一条大 SQL：CTE（`WITH ... AS`）、@fast 分页下推派生表、UNION/UNION ALL。组合时各分支**共享参数序号**（先装配内层、外层续号），参数名全局唯一不撞号。

CTE（WITH AS）

```
// 内层 CTE 体：一个独立的 join lambdaQuery（不直接执行，只贡献 SQL）
LambdaQuery<CteDept> inner = lightDao.lambdaQuery(CteDept.class).from(CteDept.class)
        .select(JoinCol.of(CteDept::getDeptId).as("deptId"), JoinCol.of(CteDept::getDeptName).as("deptName"))
        .eq(CteDept::getStatus, 1);

// 外层：with 注册 CTE，fromCte 引用它；外层类型化到 VO（字段映射 CTE 输出列）
List<CteDeptVO> rows = lightDao.lambdaQuery(CteDeptVO.class)
        .with("d", inner)
        .fromCte("d")
        .eq(CteDeptVO::getDeptName, "研发部")
        .find();
// → WITH d AS (SELECT c1.DEPT_ID AS deptId, c1.DEPT_NAME AS deptName FROM cte_dept c1 WHERE c1.STATUS = :p1)
//   SELECT c2.DEPTNAME AS deptName, c2.DEPTID AS deptId FROM d c2 WHERE c2.DEPTNAME = :p2
```

- **多 CTE / CTE 引用 CTE**：链式 `.with("a", qa).with("b", qb)`，`qb` 内可 `fromCte("a")` → `WITH a AS (..), b AS (..)`。
- **递归 CTE**：`.withRecursive(name, q, "id", "pid", "name")` → `WITH RECURSIVE name(id, pid, name) AS (..)`；**列清单建议必给**——H2 等库的递归 CTE 强制要求列清单，PG/MySQL/SQLServer 亦标准支持；Oracle/SQLServer 递归不带 `RECURSIVE` 关键字，框架按执行库类型自动适配。递归分支须用 **innerJoin** 关联 CTE 自身（LEFT JOIN 无子节点也产行会致递归不终止），且关联方向为「子.pid = 当前节点.id」向下遍历。
- **PG 物化 hint**：`.with(name, q, Boolean.TRUE/FALSE)` → PostgreSQL 渲染 `AS MATERIALIZED` / `AS NOT MATERIALIZED`；**其他库自动忽略**（按 dbType 处理，不支持的不生效）。
- **WITH 位置约束**：生成的 `WITH` 恒位于语句位置 0——sqltoy 靠文本检测 WITH（`SqlWithAnalysis` / `hasWith`）以在 count/分页时把 WITH 剥离到包裹之外，前导分号会破坏该检测；SQLServer 单条语句以 WITH 开头本就不需要前置分号（分号要求仅针对 WITH 前尚有未结束语句的批处理）。
- 外层列解析走**结果 VO 的 EntityMeta**（CTE 非真实表、无 EntityMeta），故外层须类型化到字段与 CTE 输出列对齐的 VO；不支持 CTE 的老库由开发者自行改写（框架不做降级）。

@fast 分页下推派生表

`@fast` 语义：**先对 @fast 内层分页取 pageSize 条，再拿这几条去 join 外表**（避免全量 join 再分页）；分页 count 默认用 @fast 内层 SQL（简化形态）。lambda 以派生表形式表达：

```
LambdaQuery<StaffInfo> inner = lightDao.lambdaQuery(StaffInfo.class).from(StaffInfo.class)
        .eq(StaffInfo::getStatus, 1).orderByDesc(StaffInfo::getEntryDate);   // 内层：过滤+排序（被分页下推）

Page<StaffOrganVO> page = lightDao.lambdaQuery(StaffOrganVO.class)
        .fromFast(inner, "t1")                                  // FROM @fast(内层sql) t1
        .leftJoin(OrganInfo.class).on(StaffOrganVO::getOrganId, OrganInfo::getOrganId)
        .findPage(new Page(10, 1));
// → select t1.*, t2.ORGAN_NAME from @fast(select ... from staff t where t.STATUS=:p1 order by t.ENTRY_DATE desc) t1
//   left join organ t2 on t1.organ_id=t2.ORGAN_ID    （limit/offset 由 sqltoy 下推进 @fast 内层）
```

`@fast` 也可嵌进 CTE 体：`.with("tmp", q)` 且 `q` 内部用 `fromFast(...)` → `WITH tmp AS (SELECT * FROM @fast(..) t1 ...)`。

更精简的 count：`.countSql("select count(1) from staff where status=:p1")` 提供比 @fast 内层 count 更精简的手写 count（仅 join/SQL 通道 findPage 生效），对应 sqltoy 的 `count-sql` 配置。

UNION / UNION ALL

```
LambdaQuery<NameVO> q1 = lightDao.lambdaQuery(NameVO.class).from(Staff.class)
        .select(JoinCol.of(Staff::getName).as("name")).eq(Staff::getStatus, 1);
LambdaQuery<NameVO> q2 = lightDao.lambdaQuery(NameVO.class).from(Staff.class)
        .select(JoinCol.of(Staff::getName).as("name")).eq(Staff::getStatus, 2);

q1.unionAll(q2).find();          // (q1) UNION ALL (q2)  不去重、更快
q1.union(q2).count();            // (q1) UNION (q2)     去重
q1.unionAll(q2).findPage(page);  // 包裹子查询后分页,count 同理
```

- 两侧须映射**同一结果类型**且 select 列/别名一致；均为 join/SQL 通道（需 `from` / `select`）。
- **全部查询终端都作用于组合后的 SQL**（`find/getOne/findPage/findTop/findRandom/count/exists` 均经 `getSql()` 取组合 SQL，故 UNION/CTE 上都能用）；UNION ALL 的 count 可复用 sqltoy 的 union-all-count 优化（拆分支求和）。

> 组合查询（CTE/@fast/UNION）均为 join/SQL 通道，**不继承实体级增强**（分片/翻译/脱敏/租户）；CTE/@fast/UNION 的原始 SQL 片段与别名由开发者保证方言可移植性。

七、参数与 SQL 形态约定

- 条件参数统一命名为 `:p1、:p2…`，全构建器唯一（嵌套条件、join 的 set 与 where 共享序号），SQL 日志可读；
- `having`、`inSql` 的外部命名参数请**避免 `p` 加纯数字**的形式，以免与内部参数重名；
- 嵌套条件整体加括号；仅分组/排序时以 `1 = 1` 补齐 where；
- lambda 解析仅支持 **getter 方法引用**（`User::getName`、`User::isActive`），表达式 lambda（`u -> u.getName()`）无法提取属性名，会快速失败。

七之二、安全边界与使用约定

- **原始 SQL 逃逸口不做注入筛查**：`last(sql)`、`having(sql, params)`、`inSql(col, sql, params)`、`setExpr(col, expr)` 四处接收原始 SQL 文本（与 sqlId 文本同级），**禁止拼接终端用户输入**；参数一律走命名参数绑定；
- **参数命名空间**：内部条件参数为 `:p1、:p2…`，`having` / `inSql` 的外部参数名禁用 `p` 加纯数字形式，调用时即校验抛错；
- **select 别名唯一性**：用户显式别名（`JoinCol.as(...)`、属性形式 select）与任何已有别名重复时抛异常（结果映射歧义）；框架 dump（`selectAll()` / `selectAll(Class)`）之间的同名列按声明顺序**主表优先跳过**（如主表与关联表都有 `status` 列时，关联表 dump 跳过该列），需要关联表同名列时用 `JoinCol.as` 显式改名；
- **join 自关联不支持**：同一实体注册两次（含 from 后再 join 同类）抛异常，lambda join 无法为同实体生成两个别名；
- **结果类型为 VO 时**属性形式 `select(VO::getX)` 在 join 通道不可用（VO 未参与 join，无法限定别名），抛异常；VO 场景用 `JoinCol...as()` 或 `selectAll/selectAll(Class)`；
- **`last(sql)` 的通道限制**：join 通道正常追加到 SQL 末尾；单表通道（EntityQuery 的 where 无法承载尾部原始 sql）调用即抛异常，改用 `findTop/findPage` 或 join 通道；

- **外部参数命名空间**：两个 `inSql` / `having` 共用同一命名参数且**值不同**时抛异常（静默覆盖会产生错误结果）；同值共享绑定是允许的；
- **代理 lambda 不支持**：IDEA Evaluate 窗口或 Mockito mock 产生的动态代理 lambda 在 JDK 模块强封装下无法解析元信息，抛异常并提示加 `--add-opens java.base/java.lang.invoke=ALL-UNNAMED` 或改用方法引用；
- **构建器单次使用、非线程安全**：一个构建器实例对应一次查询组装；终端执行（assemble）后再调用 `from/leftJoin/on` 等结构变更方法会抛异常，条件追加亦不会生效；并发场景每请求新建；
- **字段必须是映射属性**：条件/select/set 引用的属性须为实体 `@Column` 映射字段，未映射属性（辅助 getter、`getClass` 等）装配期抛异常，不会生成 `null = :p1` 这类静默错果；
- **更新/删除构建器拒绝分组排序**：`groupBy/having/orderBy` 仅 `LambdaQuery` 支持，在 `LambdaUpdate/LambdaDelete` 上调用装配期抛异常；
- **`not(...)` 与前条件自动补 AND**：`eq(a).not(n -> ...)` 渲染为 `a AND NOT (...)`；空 `not(...)` 的悬挂 NOT 自动剔除；
- **删除通道的无参数 where 兜底**：与更新同口径（`isNull` 等无命名参数条件、`matchAll` 场景自动补 `1=:DEFAULT_VALUE`），不会触发 deleteByQuery 的空值拒绝；
- **主键可被显式 set/setExpr**：属显式意图（自然键修正等场景），框架不拦截；多行更新为同一主键值会被数据库主键约束拒绝；整对象更新的主键安全由 `lightDao.update(entity)` 保证（主键走 where）。

八、错误语义速查

场景	行为
`getOne()` 匹配多条	抛异常（含匹配条数），提示检查条件唯一性或改用 `find()`
更新/删除无任何 where 条件	抛异常，需显式 `matchAll()`
条件/select/set 字段所属实体未参与 join	抛异常：实体未参与当前查询/join
join 后缺 `on()`	装配期抛异常
join 更新/删除缺主表	装配期抛异常（查询场景主表可缺省为结果类型）
`setExpr` 表达式含 `=`	调用时抛异常
`setExpr` 表达式引号内含字面量 `?`	调用时抛异常（更新通道做全量 `?` 文本替换，引号内 `?` 会被误当参数位）
`ExcludeCols` 字段不属于主表实体	装配期抛异常
`selectAll(ExcludeCols...)` 排除全部字段	装配期抛异常
构建器未绑定 dao 调用终端	抛异常并提示创建方式

九、自 sqltoy-plus 迁移对照

sqltoy-plus	lambda 条件体系
`Wrappers.lambdaQuery(entityClass)`	`LambdaQuery.create(entityClass)` 或 `lightDao.lambdaQuery(entityClass)`
`LambdaQueryWrapper / LambdaUpdateWrapper`	`LambdaQuery / LambdaUpdate`
`SqlToyPlusDao.findList/findOne/findPage/count/exists/delete/update(wrapper)`	构建器终端：`lambdaQuery(...).find()/getOne()/findPage()/count()/exists()`、`lambdaUpdate(...).update()`、`lambdaDelete(...).delete()`
`lambdaQueryChain/lambdaUpdateChain`	`lightDao.lambdaQuery/lambdaUpdate`（工厂即绑定 dao，直接终端）
`Wrappers.lambdaMultiWrapper(toClass).select(...).from(...)`	`lightDao.lambdaQuery(toClass)[.selectAll(...)].from(...).leftJoin(...).on(...)`
`LambdaColumn`	`JoinCol`
`lambdaUpdateMultiWrapper / lambdaDeleteMultiWrapper`	`lightDao.lambdaUpdate/lambdaDelete` + join 步骤
`SFunction`	`LambdaFn`
`FiledValueFilterStrategy` 全局静态	构建器实例级 `valueFilter(...)`
`type.valuePairs.MPG1` 形式参数名	`:p1…`
字符串列名 wrapper（QueryWrapper 等）	不再提供：单表字符串条件用 `EntityQuery`，复杂动态用 SQL 模式
`@EnableSqlToyPlus` / 映射策略 Bean	无需任何配置，能力随 `LightDao` 提供

十、能力边界（不做什么）

- **字符串条件 DSL**：lambda 层只接受方法引用；字符串单表条件用 `EntityQuery/EntityUpdate`，复杂动态用 SQL 模式；
- **实体级增强在 join 通道不生效**（分片/翻译/脱敏/租户）；
- **OLAP 库的 join 更新/删除**：clickhouse/doris/starrocks 等无事务性行级 DML 能力的库，join 更新/删除用例跳过（数据库自身能力边界，由开发者控制）；关系型库的 join 更新/删除采用可移植 EXISTS-over-derived 改写，经 mysql/postgresql/sqlserver/oracle/db2 等真实库验证；
- **整对象并入更新**：按主键整对象更新用 `lightDao.update(entity)`，lambda 层只提供 `set/setExpr` 显式字段；
- **拦截器/插件体系**：lambda 条件最终走 sqltoy 既有执行链路，插件能力（SqlInterceptor 等）照常生效，但 lambda 层自身不提供插件点。

分页优化

sqltoy 提供多层优化的分页机制：**智能 count 优化、缓存分页、快速分页、并行分页**，并支持自定义 count SQL 与跳过 count。开发者无需关心各数据库的分页方言差异，sqltoy 自动适配。

整体流程示意：

```
flowchart TD
    A[分页请求 findPage] --> B[智能改写 count SQL<br/>剔除 order by · 简化 select]
    B --> C{配置 page-optimize?}
    C -->|是| D{缓存命中?}
    D -->|命中| F[复用缓存的 count 结果]
    D -->|未命中| G[执行 count 并写入缓存]
    C -->|否| H[直接执行优化后的 count]
    F --> I[并行执行单页数据查询]
    G --> I
    H --> I
    I --> J[组装 Page 返回<br/>rows + recordCount]
```

一、基本分页 API

```
// 1) Map 传参分页
Page<StaffInfo> page = lightDao.findPage(new Page(10, 1), "sqltoy_staff_find", paramsMap, StaffInfo.class);

// 2) QueryExecutor 灵活分页（可指定数据源、翻译、格式化、分页优化等）
QueryResult result = lightDao.findPageByQuery(new Page(10, 1),
        new QueryExecutor("sqltoy_staff_find").names("status").values(1).resultType(StaffInfo.class));

// 3) 单表实体分页
Page<SysLog> page2 = lightDao.findPageEntity(new Page(10, 1), SysLog.class,
        EntityQuery.create().where("status=:status").names("status").values(1));
```

> `Page` 构造为 `new Page(pageSize, pageNo)`，例如 `new Page(10, 1)` 表示每页 10 条、第 1 页。

`Page` 常用方法：

方法	说明
`getRows()`	当前页数据集合
`getRecordCount()`	总记录数
`getPageNo()` / `getPageSize()`	当前页号 / 每页条数
`getTotalPage()` / `getLastPage()`	总页数 / 末页号
`getNextPage()` / `getPriorPage()` / `getFirstPage()`	下一页 / 上一页 / 首页
`getStartIndex()` / `getNextIndex()` / `getPreviousIndex()`	起始/下一/上一索引
`setSkipQueryCount(Boolean)`	是否跳过总数查询（见第六节）
`setOverPageToFirst(Boolean)`	翻页超出最大页时是否回到第一页

二、智能 count 优化（自动）

分页本质是两步查询：**① 查总记录数，② 查第几页数据。**其中 count 查询往往是大表分页的性能瓶颈。

sqltoy 的 count **不是**简单粗暴地 `select count(1) from (你的sql) as t`，而是智能改写：

- 根据 SQL 逻辑**剔除不必要的** `order by`；
- 判断是否可以直接**切除原 SQL 中** `from` **之前的** `select` **列与函数运算**，替换为 `select count(1) from ...`，避免无谓的函数计算；
- 自动适配各数据库的分页方言（limit / rownum / offset fetch / top 等）。

绝大多数场景下，这套自动优化已能显著降低 count 开销，无需任何额外配置。

三、缓存分页（page-optimize）

对于**查询条件相同、被频繁翻页**的场景，每次翻页都重新 count 是浪费。`<page-optimize>` 通过缓存保留"相同查询条件 → count 结果"，在一定周期内翻页时直接复用 count，无需重复查询总数。

```
<sql id="sqltoy_staff_find">
    <!-- parallel:是否并行执行 count 与单页数据查询;
         alive-max:最多缓存多少组不同查询条件的 count; alive-seconds:缓存存活秒数 -->
    <page-optimize parallel="true" alive-max="100" alive-seconds="120" />
    <value><![CDATA[
        select t.* from sqltoy_staff_info t
        where t.STATUS=1
        #[and t.STAFF_NAME like :staffName]
        order by t.ENTRY_DATE desc
    ]]></value>
</sql>
```

`<page-optimize>` 属性：

属性	默认	说明
`parallel`	`false`	是否并行执行 count 与结果查询以提升效率（`alive-max=1` 时关闭缓存优化）
`parallel-maxwait-seconds`	—	并行查询最大等待秒数，避免执行过久
`alive-max`	`100`	最多缓存多少组不同查询条件对应的 count 记录
`alive-seconds`	`90`	每组查询条件及 count 在内存中的存活秒数，超时则重新查询
`skip-zero-count`	`false`	命中缓存的 count 为 0 时是否跳过该缓存、重新查询总记录数

Java 中对应 `QueryExecutor.pageOptimize(PageOptimize)`：

```
lightDao.findPageByQuery(new Page(10, 1),
        new QueryExecutor("sqltoy_staff_find").pageOptimize(new PageOptimize(100, 120)));
```

四、快速分页（@fast / @fastPage）

先分页、后关联：当主查询需要 join 其他表（尤其一对多）时，常规做法是先 join 出大结果集再分页，代价高昂。`@fast()` 用子查询包裹"决定结果集"的部分，**先按 pageSize 取出分页记录，再与其他表关联**，大幅减少关联数据规模。

```
<sql id="sqltoy_staff_findlist">
    <value><![CDATA[
        select t1.*,t2.ORGAN_NAME
        -- @fast() 先分页取 pageSize 条，再关联
        from @fast(
              select t.* from sqltoy_staff_info t
              where t.STATUS=1
                #[and t.STAFF_NAME like :staffName]
              order by t.ENTRY_DATE desc
             ) t1
        left join sqltoy_organ_info t2 on t1.organ_id=t2.ORGAN_ID
    ]]></value>
</sql>
```

要点：

- `@fast` 与 `@fastPage` **等效**。
- `@fast()` 写在 SQL 中即可，**非分页查询时框架会自动剔除** `@fast`，因此不影响 find/findTop 等其他查询。
- 对 `findTop`、`findRandom`（getRandomResult）同样生效。
- 可配合提取 1..n 倍数据：如 `pageSize=10` 时取出 50 条再做行转列等二次计算。

五、并行分页（parallel）

在 `<page-optimize parallel="true">` 下，sqltoy 会**并行执行 count 查询与单页数据查询**，把原本串行的两步压缩为一步的耗时，适合 count 与数据查询都较重的场景。可用 `parallel-maxwait-seconds` 限制并行最大等待时长。

六、自定义 count-sql 与跳过 count

自定义 count SQL：极特殊场景下，可手写最优 count 语句：

```
<sql id="sqltoy_staff_find">
    <value><![CDATA[ select ... 复杂查询 ... ]]></value>
    <count-sql><![CDATA[ select count(1) from sqltoy_staff_info t where t.STATUS=1 ]]></count-sql>
</sql>
```

跳过 count：滚动加载、无限下拉等不需要总数的场景，可跳过 count 查询以提升性能：

```
Page page = new Page(20, 1);
page.setSkipQueryCount(true);   // 不查询总记录数
Page result = lightDao.findPage(page, "sqltoy_staff_find", paramsMap, StaffInfo.class);
```

> 分页相关源码可参考 `org.sagacity.sqltoy.dialect.PageOptimizeUtils`（count 优化与快速分页引擎）与各数据库 `Dialect` 实现。

复杂查询与数据分析

sqltoy 提供一组**在内存中对结果集做二次计算**的分析能力：多维分组汇总、行转列/列转行、同比环比、树形排序逐层汇总、分组拼接等。

这些算法有两个突出价值：

1. **解放数据库压力**：行列转换、逐层汇总等计算如果交给数据库做，会产生巨额 CPU 与内存开销；sqltoy 在应用内存中单次线性处理，效率优势明显。
2. **跨库适配**：算法完全不依赖数据库方言，同一套 SQL 既能跑在 MySQL、Oracle 上，也能平滑迁移到达梦（DM）、高斯（GaussDB）、PostgreSQL 等国产/主流数据库。

所有分析能力既可以在 `*.sql.xml` 中用标签声明，也可以用 `QueryExecutor` 在 Java 代码中链式构建，二者完全等价。

> 下面统一以一张"水果销售记录表" `sqltoy_fruit_order` 为例演示。

一、多维分组汇总（summary）

1.1 应用场景

在财务报表或销售统计中，通常需要在表格底部或各分组末尾加上"小计""合计"或"平均值"。

- **传统痛点**：使用 SQL 的 `GROUP BY ROLLUP` 不仅繁琐，且 Oracle、MySQL、SQL Server 方言差异极大，后期极难维护。
- **sqltoy 方案**：SQL 只管查出基础明细，由 sqltoy 在内存中自动完成多维汇总与平均值计算。

1.2 XML 配置范例

```
<sql id="group_summary_case">
    <value><![CDATA[
        select t.fruit_name,t.order_month,t.sale_count,t.sale_quantity,t.total_amt
        from sqltoy_fruit_order t
        order by t.fruit_name,t.order_month
    ]]></value>
    <!-- sum-columns:求和列; average-columns:求平均列; reverse:true 表示汇总行置顶 -->
    <summary sum-columns="sale_count,sale_quantity,total_amt" reverse="true">
        <!-- global:全局总计; label-column:总计标签放在哪一列; sum-label:总计文字 -->
        <global label-column="fruit_name" sum-label="总计" />
        <!-- group:按列分组小计; group-column:分组依据列 -->
        <group group-column="fruit_name" label-column="fruit_name" sum-label="小计" />
    </summary>
</sql>
```

1.3 属性说明

`<summary>` 标签属性：

属性	说明
`sum-columns`	需求和的列，逗号分隔
`average-columns`	需求平均的列
`average-radix-sizes`	各平均列保留小数位，如 `1,3,2`；单值表示全部
`average-rounding-modes`	舍入模式：`HALF_UP` / `HALF_DOWN` / `ROUND_DOWN` / `ROUND_UP`
`average-skip-null`	求平均时是否排除 null 值，默认 `false`
`sum-site`	汇总行位置：`top` / `bottom` / `left` / `right`，默认 `top`
`reverse`	数据是否反向排列，默认 `false`
`link-sign`	sum 与 average 同行展示时的拼接符，默认 `/`
`skip-single-row`	单行数据是否也参与汇总，默认 `false`
`has-grouped`	数据是否已完成分组组织，默认 `true`

`<global>`（全局总计）：`label-column`（必填）、`sum-label`（必填）、`average-label`、`reverse`。

`<group>`（分组小计，可多个）：`group-column`（必填）、`label-column`（必填）、`sum-label`（必填）、`average-label`、`order-column`（分组排序列）、`order-with-sum`（默认 `true`）、`order-way`（`desc` / `asc`）。

1.4 效果

品类	销售月份	销售笔数	销售数量(吨)	销售金额(万元)
总计		71	12600	14900
小计		36	5900	7500
苹果	2019年5月	12	2000	2400
苹果	2019年4月	11	1900	2600
苹果	2019年3月	13	2000	2500
小计		35	6700	7400
香蕉	2019年5月	10	2000	2000
香蕉	2019年4月	12	2400	2700
香蕉	2019年3月	13	2300	2700

二、动态行转列（pivot）

2.1 应用场景

将数据库中的"窄表"（多条行记录）转换成前端展示的"宽表"（多列）。例如把"月份、产品、销量"的行数据，横向展开为"产品、3月销量、4月销量、5月销量"。

- **传统痛点**：数据库 `PIVOT` 语法无法应对"动态不固定列"，一旦月份或产品类型动态变化，纯 SQL 就会失效。
- **sqltoy 方案**：结合核心算法支持纯动态集合行转列，几行声明即可搞定。

2.2 XML 配置范例

```
<sql id="pivot_case">
    <value><![CDATA[
        select t.fruit_name,t.order_month,t.sale_count,t.sale_quantity,t.total_amt
        from sqltoy_fruit_order t
        order by t.fruit_name,t.order_month
    ]]></value>
    <!-- group-columns:行分组列; category-columns:横向分类列;
         start-column~end-column:参与旋转的指标列区间(从 sale_count 到 total_amt 共3列) -->
    <pivot group-columns="fruit_name" category-columns="order_month"
           start-column="sale_count" end-column="total_amt" />
</sql>
```

`<pivot>` 属性：`group-columns`（必填）、`category-columns`（必填）、`start-column`（必填）、`end-column`（必填）、`category-sql`（用 SQL 动态产生分类列）、`default-value`、`default-type`。

> **提示**
>
> 符合默认规整标准的纯集合行转列，可直接简写 `<pivot/>`，无需冗长属性。

2.3 效果

品类	2019年3月			2019年4月			2019年5月		
	笔数	数量	总金额	笔数	数量	总金额	笔数	数量	总金额
香蕉	13	2300	2700	12	2400	2700	10	2000	2000
苹果	13	2000	2500	11	1900	2600	12	2000	2400

三、列转行（unpivot）

3.1 应用场景

与 `pivot` 相反。导入的 Excel 原始数据常是宽表（如：学号、语文成绩、数学成绩、英语成绩），为便于清洗、入库或细粒度分析，需要转换为窄表（学号、科目、成绩）。

3.2 XML 配置范例

```
<sql id="sys_student_score_unpivot">
    <value><![CDATA[
        select t.student_no,t.chinese_score,t.math_score,t.english_score
        from sys_student_score t
    ]]></value>
    <!-- columns-to-rows: "列名:指标名称" 逗号分隔; new-columns-labels: 新产生的两列标题 -->
    <unpivot columns-to-rows="chinese_score:语文,math_score:数学,english_score:英语"
             new-columns-labels="subject_name,score_value" />
</sql>
```

`<unpivot>` 属性：`columns-to-rows`（必填，格式 `col1:指标名称1,col2:指标名称2`）、`new-columns-labels`（必填，为新产生的 2 列设定标题，便于映射到 VO 属性，如 `indexName,indexValue`）。

四、同比与环比计算

4.1 应用场景

数据分析看板（BI）中，纯粹展示销售额不够，往往需要展示**环比增长率**（本月比上月）或**同比增长率**。sqltoy 提供 `<cols-chain-relative>`（列环比，水平方向同类列比较）与 `<rows-chain-relative>`（行环比，垂直方向行间比较）两个标签，自动计算并输出百分比/千分比。

4.2 列环比（先行转列再环比）

```
<sql id="cols_relative_case">
    <value><![CDATA[
        select t.fruit_name,t.order_month,t.sale_count,t.sale_quantity,t.total_amt
        from sqltoy_fruit_order t
        order by t.fruit_name,t.order_month
    ]]></value>
    <!-- 先行转列：每个月份下有 笔数/数量/总金额 三个指标 -->
    <pivot group-columns="fruit_name" category-columns="order_month"
           start-column="sale_count" end-column="total_amt" />
    <!-- 列与列之间环比：group-size=3 表示每组3个指标；relative-indexs=1,2 表示对第2、3个指标做环比；start-column=1 从第1列开始 -->
    <cols-chain-relative group-size="3" relative-indexs="1,2" start-column="1" format="#.00%" />
</sql>
```

`<cols-chain-relative>` 属性：`group-size`（必填）、`relative-indexs`（必填）、`start-column`（必填）、`end-column`（负数表示倒数第几列）、`reduce-one`（是否减 1，`(B-A)/A` 否则 `B/A`）、`multiply`（`1` 小数 / `100` 百分比 / `1000` 千分比）、`radix-size`（保留小数位，默认 3）、`rounding-mode`、`format`（`#.00%` / `#.00‰`）、`defaultValue`（首组无对比值时的默认值）。

效果（节选）：

品类	2019年3月					2019年4月					2019年5月				
	笔数	数量	比上月	总金额	比上月	笔数	数量	比上月	总金额	比上月	笔数	数量	比上月	总金额	比上月
香蕉	13	2300		2700		12	2400	4.30%	2700	0.00%	10	2000	-16.70%	2000	-25.90%
苹果	13	2000		2500		11	1900	-5.00%	2600	4.00%	12	2000	5.30%	2400	-7.70%

4.3 行环比

```
<sql id="sales_rows_relative">
    <value><![CDATA[
        select t.sale_date,t.sales_amt from sales_table t order by t.sale_date asc
    ]]></value>
    <!-- group-column:分组列(无分组可写 -1); relative-columns:参与比较的列; insert:true 表示环比值独立新增一列 -->
    <rows-chain-relative group-column="sale_date" relative-columns="sales_amt"
                         format="#.00%" reduce-one="true" multiply="100" insert="true" />
</sql>
```

`<rows-chain-relative>` 属性：`group-column`（必填，可写列名或数字索引，单一类型比较无分组可写 `-1`）、`relative-columns`（必填）、`relative-indexs`、`start-row`、`end-row`（负数表示倒数第几行）、`reduce-one`、`reverse`（默认 `true`）、`multiply`、`radix-size`、`rounding-mode`、`format`、`insert`（默认 `true`，环比值是否自动新增列）、`defaultValue`。

五、树形排序与逐层汇总（tree-sort）

5.1 应用场景

面对层级关系的树形表（组织架构、科目分类、行政区划），前端通常要求按树的深度与血缘顺序排列（总公司 → 华东分公司 → 上海办事处 ...），并且**上层节点自动汇总所有子孙节点的数据。**

5.2 XML 配置范例

```
<sql id="treeTable_sort_sum">
    <value><![CDATA[
        select t.area_code,t.pid_area,sale_cnt from sqltoy_area_sales t
    ]]></value>
    <!-- 组织树形上下归属结构，将底层节点值逐层汇总到父节点，并对同层级按 sale_cnt 降序排列 -->
    <tree-sort id-column="area_code" pid-column="pid_area" sum-columns="sale_cnt"
               level-order-column="sale_cnt" order-way="desc" />
</sql>
```

`<tree-sort>` 属性：`id-column`（必填）、`pid-column`（必填）、`sum-columns`（逐层汇总列）、`level-order-column`（每层内部排序依据列）、`order-way`（`desc` / `asc`，默认 `desc`）。还可包含子标签 `<sum-filter column="..." compare-type="eq|neq|gt|gte|lt|lte|between|in|out" compare-values="..."/>` 过滤哪些节点参与汇总。

5.3 效果

地区	归属地区	销售量
上海	中国	300
松江	上海	120
杨浦	上海	116
浦东	上海	64
江苏	中国	270
南京	江苏	110
苏州	江苏	90
无锡	江苏	70

六、分组拼接（link，代替 group_concat）

`group_concat` / `listagg` / `string_agg` 在各数据库写法不一，sqltoy 用 `<link>` 标签在内存中完成分组拼接，实现跨库一致。

```
<sql id="organ_staff_link">
    <value><![CDATA[
        select t.organ_id,t.staff_name from sqltoy_staff_info t order by t.organ_id
    ]]></value>
    <!-- id-columns:分组列; columns:被拼接列; sign:连接符; distinct:是否去重 -->
    <link id-columns="organ_id" columns="staff_name" sign="," distinct="true" />
</sql>
```

`<link>` 属性：`columns`（必填，可多列如 `staff_id,staff_name`）、`id-columns`（分组列，可多列，注意 SQL 做好排序）、`sign`（连接符，默认逗号）、`distinct`（默认 `false`）、`result-type`（`ARRAY` / `LIST` / `SET`，仅适用单字段，将结果以集合而非拼接字符串呈现）。

> 拼接顺序以结果集顺序为准，**请确保 SQL 已按 `id-columns` 分组列排好序**，否则拼接结果不可预期。

多列拼接 + 自定义分隔符（`columns` 多列时逐列依次拼接，`sign` 可任意字符）：

```
<sql id="order_items_link">
    <value><![CDATA[
        select t.order_id, t.product_name, t.qty
        from sqltoy_order_item t
        order by t.order_id
    ]]></value>
    <!-- 按订单分组，把商品名、数量拼成一行 -->
    <link id-columns="order_id" columns="product_name,qty" sign="|" />
</sql>
```

集合结果（`result-type` 仅适用单字段，返回 `ARRAY` / `LIST` / `SET` 集合而非字符串）：

```
<sql id="staff_tags_link">
    <value><![CDATA[
        select t.organ_id, t.staff_tag from sqltoy_staff_tag t order by t.organ_id
    ]]></value>
    <link id-columns="organ_id" columns="staff_tag" result-type="ARRAY" />
</sql>
```

Java 形态（QueryExecutor.groupConcat）：

```
QueryResult result = lightDao.findByQuery(
        new QueryExecutor("organ_staff_link")
            .groupConcat(new GroupConcat()
                .group("organ_id")        // 等价 id-columns
                .concat("staff_name")     // 等价 columns
                .separator(",")           // 等价 sign
                .distinct(true)));        // 等价 distinct
List rows = result.getRows();
```

六之二、结果集反转（reverse）

`<reverse>` 把结果集**就地反转**（默认整体反转，也可只反转指定行区间），典型场景是"SQL 按时间倒序取最近 N 条，但页面要按时间正序显示"——省掉在 SQL 里再套一层子查询。

```
<sql id="recent_orders">
    <value><![CDATA[
        select t.order_id,t.trans_date from sqltoy_order_info t
        order by t.trans_date desc
    ]]></value>
    <!-- 只反转前 10 行(0 基)，实现"取最近10条再按正序展示" -->
    <reverse start-row="0" end-row="9" />
</sql>
```

属性：

属性	说明
`start-row`	起始行，**从 0 开始**，缺省 0
`end-row`	截止行（含），缺省为最后一行；**负数表示倒数第几行**（-1 即最后一行）

> 行数小于 2、`start-row` 超过最后一行、或区间不合法时**原样返回**，不抛异常。

七、Java 代码方式（QueryExecutor）

上述所有 XML 分析标签都有对应的 `QueryExecutor` 链式方法，可在代码中动态构建：`summary()`、`pivot()`、`unpivot()`、`colsChainRatio()`、`rowsChainRatio()`、`treeSort()`、`groupConcat()`，以及格式化 `numFmt()` / `dateFmt()`、脱敏 `secureMask()` 等。

```
@Autowired
private LightDao lightDao;

public void getComplexReport() {
    QueryResult result = lightDao.findByQuery(
        new QueryExecutor("group_summary_case")
            .names("startDate").values(LocalDate.now().minusMonths(3))
            // 分组汇总：等价于 XML 的 <summary>
            .summary(new Summary()
                .sumColumns("sale_count", "sale_quantity", "total_amt")
                .reverse(true)
                .summaryGroups(new SummaryGroup("fruit_name")
                    .labelColumn("fruit_name").sumTitle("小计")))
            // 列环比：等价于 <cols-chain-relative>
            .colsChainRatio(new ColsChainRatio()
                .groupSize(3).relativeIndexs(1, 2).startColumn("1").format("#.00%"))
            // 数字/日期格式化
            .numFmt("#,###.00", RoundingMode.HALF_UP, "total_amt")
            .dateFmt("yyyy-MM-dd", "order_month")
    );

    List rows = result.getRows();          // 规整计算后的最终结果集
    long executeTime = result.getExecuteTime(); // SQL 执行时长
}
```

> 树形汇总、分组拼接同理：`.treeSort(new TreeSort().idColumn("area_code").pidColumn("pid_area").sumColumns("sale_cnt").levelOrderColumn("sale_cnt").orderWay("desc"))`、`.groupConcat(new GroupConcat().group("organ_id").concat("staff_name").separator(",").distinct(true))`。

并行查询（parallelQuery）

一、适用场景

并行查询用于**同时执行几个没有前后依赖关系的 SQL**，把顺序执行变为并行执行，提升整体效率（如一个页面需要同时加载多块互不依赖的统计数据）。

注意事项：

- **不要用于事务操作过程中。**
- 多个 SQL **共用相同的参数条件**（几个查询参数的合集），sqltoy 会自动为每个 SQL 提取出它实际需要的参数。
- 支持分页、非分页、分页+非分页混合：通过 `ParallelQuery` 设置 `Page` 即表示该查询为分页查询。

二、API

```
// 返回每个查询对应的 QueryResult，顺序与传入的 ParallelQuery 列表一致
public <T> List<QueryResult<T>> parallelQuery(List<ParallelQuery> parallelQueryList, Map<String, Object> paramsMap);
```

`ParallelQuery` 链式构建：`create()`、`sql()`、`page()`、`topSize()`、`randomSize()`、`names()`、`values()`、`paramsMap()`、`resultType()`、`dataSource()`、`showSql()`、`contextData()`。

三、使用范例

```
List<ParallelQuery> queries = new ArrayList<>();
// 普通查询
queries.add(ParallelQuery.create().sql("sys_staff_find").resultType(StaffInfoVO.class));
// 分页查询（设置 page 即为分页）
queries.add(ParallelQuery.create().sql("sys_order_find").page(new Page(10, 1)));
// 取 Top
queries.add(ParallelQuery.create().sql("sys_sale_top").topSize(10));

// 共用参数条件，sqltoy 自动为每个 sql 提取所需参数
Map<String, Object> paramsMap = MapKit.keys("status", "organId").values(1, "T001");
List<QueryResult> results = lightDao.parallelQuery(queries, paramsMap);

List<StaffInfoVO> staff = results.get(0).getRows();
Page orderPage = results.get(1).getPageResult();
List saleTop = results.get(2).getRows();
```

每个查询也可以**单独设置自己的参数**（覆盖/补充共用参数）：

```
queries.add(ParallelQuery.create().sql("sys_order_find")
        .names("status").values(1)
        .page(new Page(10, 1)));
```

> `QueryResult` 常用方法：`getRows()`（结果集）、`getPageResult()`（分页结果）、`getExecuteTime()`（执行时长）。
>
> **6.0 起更名**：`parallQuery` / `ParallQuery` 更名为 `parallelQuery` / `ParallelQuery`，5.6.x 版本请使用旧名称。
>
> 🎬 完整可运行示例：演示项目 `sqltoy-showcase` 的 `ParallelQueryTest.java`（基本并行、ParallelConfig 线程配置、混合分页三种形态）。

存储过程调用

sqltoy 支持调用数据库存储过程，并可获取查询结果、out 参数与多结果集。

一、LightDao 接口

```
// 无结果集调用（in 参数按 ? 顺序传入）
StoreResult executeStore(String storeSqlOrKey, Object[] inParamValues);

// 带 out 参数 / 指定结果类型
// outParamsType 可为 null；resultType 可为 VO、Map.class、LinkedHashMap.class、Array.class、null(二维 List)
StoreResult executeStore(String storeSqlOrKey, Object[] inParamValues,
                         Integer[] outParamsType, Class resultType);

// 多结果集存储过程
StoreResult executeMoreResultStore(String storeSqlOrKey, Object[] inParamValues,
                                   Integer[] outParamsType, Class... resultTypes);
```

`storeSqlOrKey` 可以是 xml 中的 sqlId，也可以直接是 `{call storeName(?,?)}`。

二、链式写法 store()

```
StoreResult result = lightDao.store()
        .sql("{call storeName(?,?)}")
        .inParams(value1, value2)
        .outTypes(java.sql.Types.INTEGER)   // out 参数类型，可省略
        .resultType(StaffInfoVO.class)
        .moreResult(false)                  // 是否多结果集
        .timeout(30)
        .submit();
```

三、StoreResult 结果

方法	说明
`getRows()`	主结果集（List）
`getMoreResults()`	全部结果集（`List[]`，多结果集时使用）
`getOutResult()`	out 参数值数组（`Object[]`）
`getUpdateCount()`	影响行数
`getLabelsList()`	各结果集的列名

四、调用示例

```
// 1) 无结果调用
lightDao.executeStore("{call storeName(?,?)}", new Object[]{value1, value2});

// 2) 存储过程查询（无 out 参数时 outParamsType 传 null）
List result = lightDao.executeStore("{call storeName(?,?)}",
        new Object[]{value1, value2}, null, VO.class).getRows();

// 3) 多结果集
StoreResult more = lightDao.executeMoreResultStore("{call storeName(?,?,?)}",
        new Object[]{value1, value2}, new Integer[]{java.sql.Types.INTEGER},
        VO1.class, VO2.class);
List[] allResults = more.getMoreResults();
Object[] outValues = more.getOutResult();
```

五、多结果集存储过程完整范例

当一个存储过程返回**多个结果集**（如主表 + 明细表，或多组统计结果）时，用 `executeMoreResultStore` 或链式 `store().moreResult(true)`，并通过 `getMoreResults()` 获取全部结果集。

假设存储过程 `proc_order_report`：入参 `organId`、`beginDate`，返回两个结果集（订单列表、订单明细），并有一个 out 参数 `totalCount` 返回总记录数。

```
// 调用 {call proc_order_report(?,?,?)}：前两个为 in 参数，最后一个为 out 参数
StoreResult storeResult = lightDao.executeMoreResultStore(
        "{call proc_order_report(?,?,?)}",
        new Object[]{ "T001", LocalDate.parse("2026-01-01") },  // in 参数（按 ? 顺序）
        new Integer[]{ java.sql.Types.INTEGER },                // out 参数类型
        OrderInfoVO.class,        // 第 1 个结果集的映射类型
        OrderItemVO.class);       // 第 2 个结果集的映射类型

// 1) 获取全部结果集：List[]，每个元素对应一个结果集
List[] allResults = storeResult.getMoreResults();
List<OrderInfoVO> orders = allResults[0];        // 第一个结果集（订单）
List<OrderItemVO> orderItems = allResults[1];    // 第二个结果集（明细）

// 2) 获取 out 参数值
Object[] outValues = storeResult.getOutResult();
Integer totalCount = (Integer) outValues[0];

// 3) getRows() 返回主结果集（即第一个结果集）
List<OrderInfoVO> mainRows = storeResult.getRows();

System.out.println("订单数=" + orders.size()
        + "，明细数=" + orderItems.size()
        + "，总记录数=" + totalCount);
```

等价的链式写法：

```
StoreResult result = lightDao.store()
        .sql("{call proc_order_report(?,?,?)}")
        .inParams("T001", LocalDate.parse("2026-01-01"))
        .outTypes(java.sql.Types.INTEGER)
        .moreResult(true)
        .resultTypes(OrderInfoVO.class, OrderItemVO.class)
        .submit();

List[] all = result.getMoreResults();              // all[0]=订单，all[1]=明细
Integer total = (Integer) result.getOutResult()[0]; // out 参数
```

> 说明：`resultTypes` 按结果集出现顺序依次指定各结果集的映射类型（可为 VO、`Map.class`、`LinkedHashMap.class`、`Array.class`，或 `null` 表示二维 List）；`getMoreResults()` 返回 `List[]`（全部结果集），`getRows()` 返回主（第一个）结果集，`getOutResult()` 返回 out 参数值数组。

sqltoy使用场景案例

1.公共sql片段@include(sqlId)

```
<sql id="showcase_1">
    <value><![CDATA[
        select * from table1 t where t.name like :name
    ]]></value>
</sql>

<sql id="showcase_2">
    <value><![CDATA[
        @include("showcase_1")
        #[and t.status=:status]
    ]]></value>
</sql>
```

2.动态片段@include(:scriptParam)

- @include(sqlId)的一种变种，通过动态传参形式传入sql片段

```
String sql="select * from sqltoy_fruit_order where status=:status @include(:sqlScript)";
List result = lightDao.find(sql,MapKit.keys("status", "saleCount", "sqlScript").values(1, 12, "and sale_count>:saleCount"));
```

3. @if()、@elseif()、@else用法

- Sqltoy针对一些特殊业务场景给sql处理预留一个超级用法:

```
@Test
public void testMultiInnerIfElse1() throws Exception {
    String sql = """
        select * from table where 1=1
        #[@if(:flag==1) and name like :name]
        #[@elseif(:flag==2)
            #[@if(:operateType==1) and status=:status]
            #[@elseif(:operateType==2) and saleType is not :saleType]
            #[@else and saleType is :saleType]
        ]
        #[@else and orderType=:orderType]
        #[@if(:tenantId==4) and tenant=1]
        #[@elseif(:tenantId==3) and tenant=3]
        """;

    SqlToyResult result = SqlConfigParseUtils.processSql(SqlUtil.clearMark(sql),
        new String[]{"flag", "status", "name", "orderType", "saleType", "operateType", "tenantId"},
        new Object[]{2, 1, "陈", "SALE", null, 4, 3});
    System.err.println(JSON.toJSONString(result));
}
```

- @if的逻辑表达式包含:
- 单逻辑判断:@if(:paramName>=value)
- 多逻辑判断:@if(:paramName1>=value1 && :paramName2<=value2)
- 比较符号支持:>、>=、==、<、<=、!=、<>;逻辑运算符:&& 和 ||、include、in、out
- 时间比较:@if(:paramName>=now()+x) 或@if(:paramName>=now()-x)
- 表示时间的:now()、nowtime()、${.now}
- 表示日期的:day()、sysdate()、${.day}
- include包含判断：@if(:statusAry include 1),如果statusAry是数组其中包含1返回true，如果statusAry值是一个字符串，返回statusAry. contains（“1”）的结果。
- exclude @if(:statusAry exclude 1) ,跟include相反，不包含的意思
- 取size:@if(size(:statusAry)>1) ,即通过size(:参数名称)模式取得集合数组的长度
- In 用法: @if(:status in ‘1,2,3’) 当status属于1或2或3时返回true
- out 用法:@if(:status out ‘1,2,3’) 当status不属于1或2或3时返回true

4. @loop()、@secure-loop()、@loop-full()、@secure-loop-full()的用法

- @loop()用于sql动态循环拼接字段、条件，一般针对数组或集合，且不适用于in的场景。
- @secure-loop():跟@loop的区别在于参数不会直接拼接到sql中，而是采用?形式入参，防止sql注入 分三种格式

```
@secure-loop(:loopParam,loopContent)
@loop(:loopParam,loopContent) 不推荐
@secure-loop(:loopParam,loopContent,linkSign)
@loop(:loopParam,loopContent,linkSign) 不推荐
@secure-loop(:loopParam,loopContent,linkSign,startIndex,endIndex)
@loop(:loopParam,loopContent,linkSign,startIndex,endIndex) 不推荐
```

- 示例

```
<sql id="qstart_loop_sql">
    <value><![CDATA[
        select ORDER_ID
        @loop(:fields,:fields[i],",")
        from sqltoy_device_order t
        where #[t.ORDER_ID=:orderId]
        -- @blank(:param) 目的是迎合sqltoy #[] 中参数为null剔除的功能，当值不为null时@blank()被替代为空白字符串(当然也可以用@if来代替@blank)
        #[@if(size(:staffIds)>0) and (@secure-loop(:staffIds," t.STAFF_ID=:staffIds[i] "," or ",1,100))]
        #[@blank(:startDates)
            and ( @loop(:startDates,
                " t.TRANS_DATE between STR_TO_DATE(':startDates[i]','%Y-%m-%d')
                and STR_TO_DATE(':endDates[i]','%Y-%m-%d') ",
            " or "))]
    ]]></value>
</sql>
```

5. sqlToy实现配置化集成，如报表、配置化api服务等

- sqltoy 的开发并非纯粹基于写 Java 代码模式，同时也服务于快速页面配置化报表、接口服务的开发，具体集成机制
- 代码中直接使用xml片段

```
//sqlXml内容可以基于数据库配置，或报表配置模型片段
String sqlXml="""
              <sql>
                内容省略
              </sql>
             """;
//新增参数：new XMLBinding(sqlXml).id(id).lastUpdateTime(lastUpdateTime)
//lastUpdateTime 指xml最后修改时间，如果没有变化则从缓存直接获取xml解析后的模型
lightDao.findByQuery(new QueryExecutor(new XMLBinding(sqlXml).id(id).lastUpdateTime(lastUpdateTime))
                   .values(valueDTO).resultType(OrderInfoVO.class));
```

- 解析xml片段注册到sqltoy上下文

```
String sqlXML=sqlElt.element("sql").getTextTrim();
//结合sqltoy内部工具解析xml形成sqltoy模型
SqlToyConfig sqlToyConfig=lightDao.getSqlToyContext().parseSqlSegment(sqlXML);
//这里考虑一张报表里面存在多个sql片段，确保唯一，用reportId+"_"+index 组合
sqlToyConfig.setId(reportId+"_"+sqlScriptIndex);
//将模型注册到sqltoy上下文，后续就可以lightDao.find(id,paramsMap)调用了
lightDao.getSqlToyContext().putSqlToyConfig(sqlToyConfig);
```

> 完整的"配置存库 → 动态解析 → 报表/API 服务"闭环范例（含 `XMLBinding` 缓存机制、`@joinFields` 动态列）见配置化服务底座（报表 / 配置化 API 服务）。

6. 统计慢查询sql

- 开启默认配置

```
# 打印sql执行超时时长(毫秒，默认为8000毫秒)
#spring.sqltoy.printSqlTimeoutMillis=8000
# 设置默认的超时处理器，会将超时sql做一个归集
spring.sqltoy.overTimeSqlHandler= org.sagacity.sqltoy.plugins.overtime.DefaultOverTimeHandler
```

- 通过api获取慢查询

```
//size     提取记录数量
//hasSqlId 是否是xml中定义含id的sql(另外一种就是代码中直接写的sql)
//public List<OverTimeSql> getSlowestSql(int size, boolean hasSqlId);

List<OverTimeSql> slowSqlList=lightDao.getSqlToyContext().getSlowestSql(10,true);
```

7. 数据库不在适配清单中，但数据库是基于postgresql/opengaussdb衍生出的，怎么适配?

- 7.1 如果是单一数据库可以通过dialect指定具体方言

```
#solon.sqltoy.dialect=postgresql
spring.sqltoy.dialect=postgresql
```

- 7.2 多数据库场景下方言映射,通过conn.getMetaData().getDatabaseProductName() 获取数据库名称

```
#solon.sqltoy.dialectMap.oscar=opengauss
spring.sqltoy.dialectMap.oscar=opengauss
```

缓存翻译介绍

缓存翻译的目的在于简化sql逻辑，避免sql过多的关联，比如：查询员工信息表，员工的岗位、籍贯、技能等级等属于数据字典，要显示相应的中文。支持对多值进行翻译。 如:某个字段结构是A,B,C这种格式，翻译结果为:A名称,B名称,C名称

缓存翻译主要使用场景

1. 减少join联表查询,简化sql并显著提升查询效率
2. 通过API直接获取缓存，为业务逻辑直接提供数据,避免每次查询数据库，从而提升效率
   比如:前端select、checkbox、suggest组件、机构树等就可以直接从缓存获取数据
3. 通过API对缓存数据进行过滤,反向匹配key,用来替代sql like查询

缓存翻译三种方式

1. 基于xml的translate标记
2. 通过findByQuery中QueryExecutor::translates(Translate...translates)
3. 通过DTO属性上加@Translate注解

创建数据字典、机构表

SQL示例脚本translate_template.sql

配置缓存翻译

sqltoy增加缓存翻译的配置

```
spring:
    sqltoy:
        #默认classpath:sqltoy-translate.xml;classpath:translates,按默认命名则无需配置
        translateConfig: classpath:sqltoy-translate.xml
```

sqltoy-translate.xml配置

> `sqltoy-translate.xml` 的 schema 指向与 IDE 补全配置见 XML Schema 配置指引。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<sagacity
    xmlns="https://www.sagframe.com/schema/sqltoy-translate"
    xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://www.sagframe.com/schema/sqltoy-translate https://sagframe.github.io/schema/sqltoy-translate.xsd">
    <!-- 缓存有默认失效时间，默认为1小时,因此只有较为频繁的缓存才需要及时检测 -->
    <cache-translates>
        <!-- 基于sql直接查询的方式获取缓存 -->
        <sql-translate cache="dictKeyNameCache">
            <sql>
            <![CDATA[
                select t.DICT_KEY,t.DICT_NAME,t.STATUS
                from SQLTOY_DICT_DETAIL t
                where t.DICT_TYPE=:dictType
                order by t.SHOW_INDEX
            ]]>
            </sql>
        </sql-translate>
        <!-- 机构号和机构名称的缓存 -->
        <sql-translate cache="organIdNameCache">
            <sql>
            <![CDATA[
                select ORGAN_ID,ORGAN_NAME from SQLTOY_ORGAN_INFO order by SHOW_INDEX
            ]]>
            </sql>
        </sql-translate>
    </cache-translates>

    <!-- 缓存刷新检测,可以提供多个基于sql、service、rest服务检测 -->
    <cache-update-checkers cluster-time-deviation="1">
        <!-- 基于sql的缓存更新检测 -->
        <sql-increment-checker cache="organIdNameCache" check-frequency="60">
            <sql><![CDATA[
            --#not_debug#--
            select ORGAN_ID,ORGAN_NAME
            from SQLTOY_ORGAN_INFO
            where UPDATE_TIME >=:lastUpdateTime
            ]]></sql>
        </sql-increment-checker>

        <!-- 增量更新，带有内部分类的查询结果第一列是分类 -->
        <sql-increment-checker cache="dictKeyNameCache" check-frequency="15" has-inside-group="true">
            <sql><![CDATA[
            --#not_debug#--
            select t.DICT_TYPE,t.DICT_KEY,t.DICT_NAME,t.STATUS
            from SQLTOY_DICT_DETAIL t
            where t.UPDATE_TIME >=:lastUpdateTime
            ]]></sql>
        </sql-increment-checker>
    </cache-update-checkers>
</sagacity>
```

缓存定义与更新检测类型总览

`sqltoy-translate.xml` 支持 **4 种缓存定义**（`<cache-translates>` 下）与 **6 种更新检测**（`<cache-update-checkers>` 下），可按数据来源自由组合：

类型	适用场景	关键属性
`<sql-translate>`	最常用：一条 sql 查出 key/name 全量缓存	`cache`、`datasource`、`sql`（属性或子元素）
`<service-translate>`	缓存数据需经 service 加工（如多来源聚合、脱敏、解密后再入缓存）	`service`（spring bean 名）、`method`
`<rest-translate>`	缓存数据来自 rest 服务（配合 basic/jwt 认证，见下节）	`url`、`username`、`password`、`auth-type` 等
`<local-translate>`	本地静态/动态数据源：properties 文件，或配合 FIFO 动态缓存	`properties`、`sid`、`dynamic-cache` 系列（详见 FIFO 缓存）

4 种缓存定义共用一组容量与生命周期属性：`cache`（必填）、`keep-alive`（存活秒数，默认 3600）、`heap`（堆内条数上限，默认 10000）、`off-heap`（堆外 MB，默认 0）、`disk-size`（磁盘 MB，默认 0）、`i18n`（如 `zh_cn:1,en_us:2`）、`desc`。

`service-translate` 的方法签名约定：`service` 先按 spring bean 名查找，找不到且名称含 `.` 时按全限定类名取 bean；方法接收**单个 String 参数**（即缓存分类 cacheType，未配置 cache-type 时传空），返回值须为 `List<List>` / `List<Object[]>`（或 `Map<String,Object[]>`）：

```
<!-- 示例:dictService.getDictByType(String cacheType) -->
<service-translate cache="dictKeyNameCache" service="dictService" method="getDictByType" keep-alive="30" />
```

更新检测分为**全量检测**（返回完整数据，框架比对后刷新）与**增量检测**（只查 `lastUpdateTime` 之后的变化，首选）两类，`check-frequency` 为检测周期（秒）：

检测器	说明
`<sql-increment-checker>`	最常用：增量 sql，`has-inside-group="true"` 表示结果第一列为内部分类
`<sql-checker>`	全量检测：每次拉取完整数据比对
`<service-increment-checker>` / `<service-checker>`	经 service 方法检测更新（签名约定同 service-translate）
`<rest-checker>` / `<rest-increment-checker>`	经 rest 服务检测更新（认证同 rest-translate）

> 全部 6 种检测器均可叠加 `cluster-time-deviation`（容器属性，默认 1 秒）容忍集群间时钟偏差。

rest 模式认证（basic / jwt）

rest 模式的缓存获取（`rest-translate`）与 rest 形式的更新检测（`rest-checker` / `rest-increment-checker`）支持 **basic（默认）与 jwt 两种认证模式（6.0.4 版本起支持 jwt）**，jwt 模式下 token 的获取方式有三种、三选一：

```
<!-- basic 模式(默认,兼容既有配置):用户名密码基础认证 -->
<rest-translate cache="restCache" url="https://xxx/cache/list"
    auth-type="basic" username="admin" password="xxx" />

<!-- jwt 模式一:登录获取 token(表单提交 username/password,响应json按 token-path 提取并缓存) -->
<rest-translate cache="restCache" url="https://xxx/cache/list"
    auth-type="jwt" username="admin" password="xxx"
    token-url="https://xxx/login" token-path="data.token" />

<!-- jwt 模式二:本地自签名(无需登录接口,服务端以同密钥或公钥验签) -->
<rest-translate cache="restCache" url="https://xxx/cache/list"
    auth-type="jwt" username="admin" token-secure-key="xxxx"
    sign-algorithm="HS256" />

<!-- jwt 模式三:直接配置静态长效 token -->
<rest-translate cache="restCache" url="https://xxx/cache/list"
    auth-type="jwt" authorization="xxxxx" />
```

jwt 取 token 的三种方式 **token-url / token-secure-key / authorization 三选一**（同时配置时按此顺序优先）：

属性	说明	默认值
`auth-type`	认证模式：`basic`（默认）/ `jwt`	basic
`token-url`	获取 token 的登录地址（表单提交 username/password）	-
`token-path`	登录响应 json 中 token 字段路径，支持点路径（如 `data.token`）	access_token
`token-prefix`	请求头值前缀；设为 `none` 时仅发送裸 token	Bearer
`token-header`	携带 token 的请求头名称	Authorization
`token-expire`	token 有效期（秒），登录响应含 `expires_in` 时以响应为准	1800
`token-secure-key`	自签名密钥（HS 系列为共享密钥；RS 系列为 PKCS#8 私钥）	-
`sign-algorithm`	自签名算法：HS256 / HS384 / HS512 / RS256 / RS384 / RS512	HS256
`authorization`	直接配置的静态 token（长效 token 场景）	-

运行机制：token 获取后缓存并在临近过期时自动刷新；请求遇 401 时自动失效并重登（或重新签名）重试一次。自签名模式在本地生成 jwt（claims 含 sub=username、iat、exp），HS 系列为共享密钥、RS 系列为 PKCS#8 私钥（可由 `openssl pkcs8 -topk8 -nocrypt` 转换）。

提示

上面的缓存都是**全量加载**模式。如果数据量达到百万级以上（如平台型电商 SKU、号码归属地），请使用超大规模主数据缓存（FIFO）：缓存不做全量加载，翻译时未命中的 key 动态批量获取，本地仅保留最常用的数据。

测试验证

类OrderInfoVO增加2个属性

```
@Data
@Accessors(chain = true)
public class OrderInfoVO implements Serializable {
    /**
     *
     */
    private static final long serialVersionUID = 3853685083880883292L;
/*---begin-auto-generate-don't-update-this-area--*/
// 中间属性这里省略
/*---end-auto-generate-don't-update-this-area--*/

    //新扩展的2个属性
    /**
     * 机构名称
     */
    private String organName;

    /**
     * 订单类别名称
     */
    private String orderTypeName;
}
```

编写测试类

1. Java代码模式

```
package com.sqltoy.helloworld.service;

import java.time.LocalDateTime;
import java.util.List;

import org.junit.jupiter.api.Test;
import org.sagacity.sqltoy.config.model.Translate;
import org.sagacity.sqltoy.dao.LightDao;
import org.sagacity.sqltoy.model.MapKit;
import org.sagacity.sqltoy.model.QueryExecutor;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import com.sqltoy.helloworld.dto.OrderInfoVO;

@SpringBootTest
public class TranslateTest {
    @Autowired
    LightDao lightDao;

    //基于findByQuery/findPageByQuery/findEntity/findPageEntity 中
    //的QueryExecutor或EntityQuery使用translates(Translate...Translates)
    @Test
    public void testTranslate() {
        String sql = """
                select t.*,
                       -- 额外扩展两个查询属性，用于映射到VO的名称属性上
                       t.order_type orderTypeName,
                       t.organ_id organName
                from SQLTOY_ORDER_INFO t
                where 1=1
                #[and t.status in (:statusAry)]
                #[and t.create_time>=:beginTime]
                #[and t.create_time<=:endTime]
                """;
        List<OrderInfoVO> result = lightDao.findByQuery(new QueryExecutor(sql)
                .translates(new Translate("dictKeyNameCache").setCacheType("ORDER_TYPE").setColumn("orderTypeName"),
                        new Translate("organIdNameCache").setColumn("organName"))
                .values(MapKit.keys("statusAry", "beginTime", "endTime").values(new Integer[] { 1 },
                        LocalDateTime.parse("2024-10-17T00:00:01"), null))
                .resultType(OrderInfoVO.class)).getRows();
        // 打印缓存翻译结果:orderTypeName、organName 两列已由缓存翻译填充
        result.forEach(row -> System.err.println(row.getOrderTypeName() + " | " + row.getOrganName()));
    }
    //xml中的
    //<translate cache="dictKeyNameCache" cache-type="ORDER_TYPE" columns="orderTypeName"/>
    //<translate cache="organIdNameCache" columns="organName"/>
    // sql参见com/sqltoy/helloworld/sqltoy/sqltoy-helloworld.sql.xml 文件,sqlId要全局唯一
    @Test
    public void testTranslateWithSqlId() {
        List<OrderInfoVO> result = lightDao.find("helloworld_search_orderInfo",
                MapKit.keys("statusAry", "beginTime", "endTime").values(new Integer[] { 1 },
                        LocalDateTime.parse("2024-10-17T00:00:01"), null),
                OrderInfoVO.class);
        // 打印缓存翻译结果:orderTypeName、organName 两列已由缓存翻译填充
        result.forEach(row -> System.err.println(row.getOrderTypeName() + " | " + row.getOrganName()));
    }

    /**
     * 在OrderInfoVO属性:organName、orderTypeName上加注解模式实现缓存翻译 如:
     * @Translate(cacheName = "organIdNameCache", keyField = "organId")
     * private String organName;
     */
    @Test
    public void testTranslateByAnnotationTrans() {
        List<OrderInfoVO> result = lightDao.findEntity(OrderInfo.class,
                // select * 场景无需写.select()这里仅仅示范一下
                // EntityQuery.create().unselect(排除的字段)
                EntityQuery.create().select()
                        // 沿用sqltoy动态条件的规则
                        .where("#[and status in (:statusAry)]#[and createTime>=:beginTime]#[and createTime<=:endTime]")
                        .values(MapKit.keys("statusAry", "beginTime", "endTime").values(new Integer[] { 1 },
                                LocalDateTime.parse("2024-10-17T00:00:01"), null)),
```

```
                OrderInfoVO.class);
        // 打印缓存翻译结果:orderTypeName、organName 两列已由缓存翻译填充
        result.forEach(row -> System.err.println(row.getOrderTypeName() + " | " + row.getOrganName()));
    }
}
```

2. xml模式

```
<?xml version="1.0" encoding="UTF-8"?>
<sagacity
    xmlns="https://www.sagframe.com/schema/sqltoy-translate"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://www.sagframe.com/schema/sqltoy-translate https://sagframe.github.io/schema/sqltoy-translate.xsd">
    <!-- 缓存有默认失效时间，默认为1小时,因此只有较为频繁的缓存才需要及时检测 -->
    <cache-translates>
        <!-- 基于sql直接查询的方式获取缓存 -->
        <sql-translate cache="dictKeyNameCache">
            <sql>
            <![CDATA[
                select t.DICT_KEY,t.DICT_NAME,t.STATUS
                from SQLTOY_DICT_DETAIL t
                where t.DICT_TYPE=:dictType
                order by t.SHOW_INDEX
            ]]>
            </sql>
        </sql-translate>
        <!-- 机构号和机构名称的缓存 -->
        <sql-translate cache="organIdNameCache">
            <sql>
            <![CDATA[
                select ORGAN_ID,ORGAN_NAME from SQLTOY_ORGAN_INFO order by SHOW_INDEX
            ]]>
            </sql>
        </sql-translate>
    </cache-translates>

    <!-- 缓存刷新检测,可以提供多个基于sql、service、rest服务检测 -->
    <cache-update-checkers cluster-time-deviation="1">
        <!-- 基于sql的缓存更新检测 -->
        <sql-increment-checker cache="organIdNameCache" check-frequency="60">
            <sql><![CDATA[
            --#not_debug#--
            select ORGAN_ID,ORGAN_NAME
            from SQLTOY_ORGAN_INFO
            where UPDATE_TIME >=:lastUpdateTime
            ]]></sql>
        </sql-increment-checker>

        <!-- 增量更新，带有内部分类的查询结果第一列是分类 -->
        <sql-increment-checker cache="dictKeyNameCache" check-frequency="15" has-inside-group="true">
            <sql><![CDATA[
            --#not_debug#--
            select t.DICT_TYPE,t.DICT_KEY,t.DICT_NAME,t.STATUS
            from SQLTOY_DICT_DETAIL t
            where t.UPDATE_TIME >=:lastUpdateTime
            ]]></sql>
        </sql-increment-checker>
    </cache-update-checkers>
</sagacity>
```

缓存的其他用法

- 1、直接获取缓存:

```
lightDao.getTranslateCache("dictKeyNameCache","ORDER_TYPE");
lightDao.getTranslateCache("organIdNameCache",null,OrganInfoVO.class);
```

```
/**
 * 获取sqltoy中用于翻译的缓存,方便用于页面下拉框选项、checkbox选项、suggest组件等
 * @param cacheName
 * @param cacheType 如是数据字典,则传入字典类型否则为null即可
 * @return
 */
public HashMap<String, Object[]> getTranslateCache(String cacheName, String cacheType);

/**
 * 将缓存数据以对象形式获取
 * @param <T>
 * @param cacheName
 * @param cacheType  如是数据字典,则传入字典类型否则为null即可
 * @param reusltType 缓存定义时的sql属性名称或自定义的properties属性要跟resultType的属性对应
 * @return
 */
public <T> List<T> getTranslateCache(String cacheName, String cacheType, Class<T> reusltType);
```

- 2、调用缓存对集合进行翻译

```
/**
 * 对数据集合通过反调函数对具体属性进行翻译
 * @param dataSet        数据集合
 * @param cacheName      缓存名称
 * @param cacheType      例如数据字典存在分类的缓存填写字典分类，其它的如员工、机构等填null
 * @param cacheNameIndex 默认为1，缓存名称在缓存数组的第几列(因为有:名称、别名、简称、全称之说)
 * @param handler
 */
public void translate(Collection dataSet, String cacheName, String cacheType, Integer cacheNameIndex,
        TranslateHandler handler);

//用法示例
lightDao.translate(staffVOs, "staffIdName", null, 1, new TranslateHandler() {
    //告知key值
    public Object getKey(Object row) {
        return ((StaffInfoVO)row).getStaffId();
    }
    // 将翻译后的名称值设置到对应的属性上
    public void setName(Object row, String name) {
        ((StaffInfoVO)row).setStaffName(name);
    }
});
```

- 3、调用缓存进行过滤

```
/**
 * 通过缓存将名称进行模糊匹配取得key的集合
 * @param cacheMatchFilter
 * @param matchRegexes     数组
 * @return
 */
public String[] cacheMatchKeys(CacheMatchFilter cacheMatchFilter, String... matchRegexes);

//示例
String[] keys = lightDao.cacheMatchKeys(CacheMatchFilter.create()
        .cacheName("organIdNameCache")
        //对第一列机构名称进行匹配，可以matchIndexs(1,2) 机构名称、别称
        .matchIndexs(1)
        //设置返回列为第0列
        .cacheKeyIndex(0)
        //优先以equal方式进行匹配(如有：上海新能源\ 中国上海新能源发展公司，传入:上海新能源 就返回第一个的机构代码，否则就以like形式匹配)
        .priorMatchEqual(true)
        //匹配结果数量
        .matchSize(2), "新能源研究院");
```

缓存翻译的附加场景

一个字段多次翻译 / 逻辑条件翻译

同一个字段可以配置多个 `<translate>` 按不同缓存翻译；配合 `where` 属性可做**逻辑条件翻译**（条件成立才翻译），支持四种逻辑：`==`、`!=`、`in`、`out`：

```
<!-- 仅当订单类型为 PO 时才翻译 -->
<translate cache="orderTypeCache" columns="orderTypeName" where="order_type==PO" />
<!-- 不等于 / in / out 同理 -->
<!-- <translate ... where="status!=0" /> -->
<!-- <translate ... where="order_type in (PO,SPO)" /> -->
<!-- <translate ... where="order_type out (SALES)" /> -->
```

多值拆分翻译（split-sign / link-sign）

字段值为 `F,M` 这类**多代码拼接**时，通过 `split-sign` 指定分隔符逐个翻译，再用 `link-sign` 重新拼接：

```
<!-- "F,M" → 翻译后 "女,男" -->
<translate cache="sexCache" columns="sexName" split-sign="," link-sign="、" />
```

租户隔离翻译

多租户场景下，`cache-type` 可结合公共字段扩展动态传入当前用户所属租户 ID（形如 `${user_tenant_id}`），实现**按租户隔离的缓存**（复用数据字典缓存的分类机制）：

```
<translate cache="tenantConfigCache" cache-type="${user_tenant_id}" columns="configName" />
```

i18n 国际化翻译

在 `sqltoy-translate.xml` 的缓存定义上增加 `i18n` 属性（格式 `语言:列序号`，逗号或分号分隔，如 `zh_cn:1,en_us:2`），即可让翻译按当前语言取对应列的值：

```
<!-- 缓存 sql 需同时查出各语言的名称列，i18n 声明语言与列的对应 -->
<sql-translate cache="dictCache" datasource="dataSource" i18n="zh_cn:1,en_us:2">
    <sql><![CDATA[
        select t.DICT_KEY, t.DICT_NAME, t.DICT_NAME_EN
        from SQLTOY_DICT_DETAIL t
        where t.DICT_TYPE=:dictType
    ]]></sql>
</sql-translate>
```

代码中通过 `I18nThreadHolder` 设置当前线程语言（实际项目一般通过 filter 统一注入用户的 locale，请求结束时 `remove()`）：

```
I18nThreadHolder.put("en_us");
try {
    // 此次查询的缓存翻译将取 en_us 对应列(DICT_NAME_EN)的值
    List result = lightDao.find("sqltoy_dict_find", paramsMap, DictInfoVO.class);
} finally {
    I18nThreadHolder.remove();
}
```

超大规模主数据缓存（FIFO 动态缓存）

缓存翻译通常在启动后**一次性全量加载**缓存数据，这对字典、机构这类中小规模数据非常高效。但平台型电商 SKU、行政区划+门牌、号码归属地等场景，数据量达到**百万级甚至千万级**，而真正高频使用的往往只有几万到十几万条——全量加载既拖慢启动又占用大量内存。

sqltoy 针对这类场景提供了**动态缓存**机制：本地只保留一个有界的 `FIFOMap`（如最多 10 万条），翻译时缓存没有的 key 才通过 `DynamicCacheFetch` 接口**按需动态加载**，频繁使用的数据自动保留，不常用的自动被挤出。

> **注意**
>
> 批量获取接口 `getCache(..., String[] keys)` 与默认缓存管理器 `FIFODynamicFetchCacheManager` 由 **5.6.67** 版本提供（详见 5.6.67 发行说明）。

一、工作原理

```
flowchart TD
    A[查询结果集·含待翻译 key] --> B{本地 FIFOMap 中是否存在该 key}
    B -- 命中 --> C[直接返回翻译值]
    B -- 未命中 --> D[收集未命中 key 并去重]
    D --> E[一次性批量调用 DynamicCacheFetch.getCache]
    E --> F[(Redis / 数据库等海量数据源)]
    F --> G[回填 FIFOMap 完成批量翻译]
    G --> H[超出 maxSize 挤出最不常用 key]
```

关键设计点：

1. **本地有界缓存**：`FIFOMap` 按 `accessOrder` 模式工作，每次 `get` 都会把条目移到队尾（相当于续命），容量超出 `maxSize` 时挤掉队首最不常用的数据——与真实业务"常用数据集中"的规律天然匹配；
2. **批量取数**：一次查询结果集中所有未命中的 key 会**去重收集后一次性传入**接口，整批只需一次 IO，避免逐行查询的性能灾难；
3. **多级存储**：`DynamicCacheFetch` 的实现里通常先查 Redis 等分布式缓存，没有再回源数据库，形成"本地 FIFOMap → Redis → DB"的多级结构。

二、使用步骤（三步）

第 1 步：定义动态缓存（sqltoy-translate.xml）

```
<?xml version="1.0" encoding="UTF-8"?>
<sagacity xmlns="https://www.sagframe.com/schema/sqltoy-translate"
    xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://www.sagframe.com/schema/sqltoy-translate
    https://sagframe.github.io/schema/sqltoy-translate.xsd">
    <cache-translates>
        <!-- 动态缓存:不做全量加载,未命中的key通过DynamicCacheFetch按需批量获取 -->
        <local-translate cache="skuDict" sid="SKU_CACHE" properties="id,name,brand"
            keep-alive="3600" dynamic-cache="true" dynamic-cache-maxSize="100000" />
    </cache-translates>
</sagacity>
```

`local-translate` 属性说明：

属性	说明	默认值
`cache`	缓存名称，翻译时通过 `cache="xxx"` 引用	必填
`dynamic-cache`	开启动态获取模式	false
`dynamic-cache-initSize`	FIFOMap 初始容量	10000
`dynamic-cache-maxSize`	FIFOMap 最大容量，超出后挤出最不常用的数据	100000
`dynamic-cache-loadFactor`	Map 加载因子	0.75
`keep-alive`	数据存活时长（秒），**负数表示永久有效**	default-keep-alive（3600）
`sid`	标识符号，原样传给取数接口（如 sqlId），便于通用型实现区分数据来源	无
`properties`	属性信息，原样传给取数接口（如指定返回哪些列）	无

第 2 步：实现 DynamicCacheFetch 取数接口

```
package org.sagacity.sqltoy.translate;

public interface DynamicCacheFetch {

    /**
     * 初始化方法,可通过 appContext.getBean(...) 获取容器中的bean(如redisTemplate)
     */
    public void initialize(AppContext appContext);

    /**
     * 获取单个key的缓存数据(单对象翻译等非批量场景调用)
     * @return Object[]{key, name1, name2, ...} 未取到返回null
     */
    public Object[] getCache(String cacheName, String cacheType, String sid,
            String[] properties, String key);

    /**
     * 批量获取(5.6.67版本实现,采取批量查询模式,减少IO次数)
     * @param keys 结果集中未命中的key,已去除重复
     * @return Map<key, Object[]>{key,[id,name,email]}结构
     */
    public Map<String, Object[]> getCache(String cacheName, String cacheType, String sid,
            String[] properties, String[] keys);
}
```

参数说明：

参数	说明
`cacheName`	缓存名称，即 `local-translate` 的 `cache`
`cacheType`	数据字典类的分组类型（使用处传了 `cache-type` 时有值），非分组场景为 null
`sid` / `properties`	缓存定义中的标识与属性，原样透传，供通用型实现分支处理
`key` / `keys`	待获取的缓存 key（批量模式下已去重）

完整实现范例（演示项目同款，以数据库演示取数；生产中超大规模数据一般先查 Redis 再回源数据库）：

```
public class BigDictDynamicCacheFetch implements DynamicCacheFetch {

    // initialize在SqlToyContext初始化阶段被调用,此时LightDao等bean可能尚未就绪,
    // 直接getBean易形成循环依赖,因此记录appContext采用懒加载
    private AppContext appContext;
    private volatile LightDao lightDao;

    @Override
    public void initialize(AppContext appContext) {
        this.appContext = appContext;
        // 如需对接redis,可在此处获取:appContext.getBean("redisTemplate")
    }

    // 单key场景直接复用批量逻辑,避免维护两套取数代码
    @Override
    public Object[] getCache(String cacheName, String cacheType, String sid,
            String[] properties, String key) {
        Map<String, Object[]> datas = getCache(cacheName, cacheType, sid,
                properties, new String[] { key });
        return (datas == null) ? null : datas.get(key);
    }

    // 批量获取:一次结果集中未命中的key去重后一次性传入
    @SuppressWarnings({ "unchecked", "rawtypes" })
    @Override
    public Map<String, Object[]> getCache(String cacheName, String cacheType, String sid,
            String[] properties, String[] keys) {
        Map<String, Object[]> result = new HashMap<String, Object[]>();
        if (keys == null || keys.length == 0) {
            return result;
        }
        // resultType=Array.class 返回行为Object[]数组,第0列是key
        List rows = getLightDao().find(
                "select DICT_KEY,DICT_NAME,STATUS from sqltoy_dict_detail where DICT_KEY in (:keys)",
                MapKit.map("keys", Arrays.asList(keys)), Array.class);
        for (Object row : rows) {
            Object[] rowData = (Object[]) row;
            result.put(rowData[0].toString(), rowData);
        }
        return result;
    }

    private LightDao getLightDao() {
        if (lightDao == null) {
            synchronized (this) {
                if (lightDao == null) {
                    lightDao = appContext.getBean(LightDao.class);
                }
            }
        }
        return lightDao;
    }
}
```

第 3 步：注册实现类

Spring Boot（sqltoy-spring-boot-starter），值支持**容器 bean 名称**（优先匹配）或**全限定类名**（反射实例化）：

```
spring:
    sqltoy:
        dynamicCacheFetch: com.xxx.cache.BigDictDynamicCacheFetch
```

传统 Spring XML 项目，直接注入 SqlToyContext 属性：

```
<bean id="sqlToyContext" class="org.sagacity.sqltoy.SqlToyContext" init-method="initialize">
    <!-- 其他属性省略 -->
    <property name="dynamicCacheFetch">
        <bean class="com.xxx.cache.BigDictDynamicCacheFetch" />
    </property>
</bean>
```

三、运行机制细节

批量取数与两遍翻译：结果集第一遍翻译时，未命中的 key 只登记到 `DynamicCacheHolder`（去重集合）而不发起查询；第一遍结束后框架调用批量接口一次性取回，**回填 FIFOMap** 后对未匹配行做第二遍批量回写。因此整批未命中 key 只有一次取数 IO。

FIFO 淘汰策略：`FIFOMap` 基于 `LinkedHashMap(accessOrder=true)` 实现，`get` 命中会把条目移到队尾（续命），`put` 导致超出 `maxSize` 时挤出队首最不常用的条目。`get/put/remove` 均为 synchronized，多线程并发翻译下安全。

keep-alive 过期：按"缓存名称 + cacheType"整组生效——从该组数据初始化起计时，超过 `keep-alive` 秒后由后台守护线程（每 3 分钟检测一轮）整组清空，下次使用时重新动态加载；设置为负数则永不过期。

与 cache-update-checker 增量更新协同：动态缓存同样可以配置 缓存更新检测，但更新时**只覆盖本地已存在的 key**——未加载的 key 本地没有旧值，下次动态获取时自然拿到最新数据。

使用侧零感知：翻译的引用方式与普通缓存完全一致，无论是 sql xml 中的 `<translate cache="skuDict" ... />` 还是 VO 上的 `@Translate(cache="skuDict")`，无需任何额外代码。

四、【可选】自定义 DynamicFecthCacheManager

动态缓存的存储管理由 `DynamicFecthCacheManager` 接口负责（接口名中的 `Fecth` 为源码历史拼写），框架默认提供基于 FIFOMap 的 `org.sagacity.sqltoy.translate.cache.impl.FIFODynamicFetchCacheManager`，绝大多数场景无需替换。

```
package org.sagacity.sqltoy.translate.cache;

public interface DynamicFecthCacheManager {
    // 获取(或懒创建)指定缓存+分组对应的动态缓存Map
    public HashMap<String, Object[]> getDynamicCache(TranslateConfigModel cacheModel, String cacheType);
    // 判断缓存是否已被使用过(供更新检测程序判断是否需要检测)
    public boolean hasCache(String cacheName);
    // 清除某个缓存(或其某个分组)的数据
    public void clear(String cacheName, String cacheType);
    // 初始化(默认实现启动keep-alive过期检测线程)
    public void initialize();
    // 销毁(默认实现停止检测线程)
    public void destroy();
}
```

替换为自定义实现：

```
spring:
    sqltoy:
        # 同样支持bean名称或全限定类名(注意源码属性名就是Fecth)
        dynamicFecthCacheManager: com.xxx.cache.MyDynamicFetchCacheManager
```

五、注意事项

- `dynamic-cache="true"` **必须**配套注册 `DynamicCacheFetch` 实现；未注册时该缓存退化为普通全量加载模式，而 `local-translate` 本身没有配置数据来源，将无法加载到数据；
- 取数返回的 `Object[]` 第 0 列必须是 key，翻译列位置与使用处的 `cache-indexs` 对应（如 `Object[]{key, name, status}` 对应 `cache-indexs="1"`）；
- `dynamic-cache-maxSize` 需结合 JVM 内存评估：10 万条 × 每条约 200 字节 ≈ 数十 MB 量级，多组大缓存叠加时留意堆上限；
- `keep-alive` 到期是整组清空，清空后的第一次查询会有一批集中的动态取数，高频场景建议适当调大 `keep-alive` 或配置负数长期有效 + 增量更新检测的组合。

六、完整可运行示例

🎬 完整示例见演示项目 `sqltoy-showcase`：`BigDictDynamicCacheFetch.java`（取数实现）、`DynamicCacheTest.java`（翻译 + FIFOMap 批量取数与淘汰验证）、`sqltoy-translate.xml` 中的 `bigDictCache` 定义与 `application.yml` 中 `spring.sqltoy.dynamicCacheFetch` 的注册方式。

相关页面

- 缓存翻译使用：缓存翻译的基础配置与各种附加用法
- 动态控制缓存：通过 `TranslateManager` 在运行时动态扩展、刷新缓存

动态控制缓存

sqltoy 自 5.2.8 版本起，可以灵活地**动态扩展缓存、动态扩展缓存更新检测程序**，并实时更新缓存数据。这一切通过 `TranslateManager`（缓存翻译管理器）完成。

一、获取缓存管理器

```
TranslateManager translateManager = lightDao.getSqlToyContext().getTranslateManager();
```

> **前提**：动态扩展缓存翻译，必须配置一个 `translate.xml`（即 `spring.sqltoy.translateConfig` 指向的缓存翻译配置文件），其中可以不具体配置任何缓存。

二、常用操作

`TranslateManager` 提供了对缓存的全面操作能力：

方法	说明
`existCache(String cacheName)`	判断缓存是否存在
`getCacheNames()`	获取全部缓存名称
`getCacheData(String cacheName, String cacheType)`	获取某个缓存的数据（`HashMap<String, Object[]>`）
`putCacheData(String cacheName, String cacheType, HashMap<String,Object[]> cacheValue)`	写入/更新缓存数据
`clear(String cacheName, String cacheType)`	清除某个缓存的数据
`getCacheConfig(String cacheName)`	获取缓存配置模型
`putCache(TranslateConfigModel model)`	**动态新增一个缓存**
`removeCache(String cacheName)`	动态移除一个缓存
`putCacheUpdater(CheckerConfigModel model)`	动态新增缓存更新检测器
`removeCacheUpdater(CheckerConfigModel model)`	移除缓存更新检测器
`getAllTranslates()`	获取全部缓存翻译配置

三、两类典型场景

1）只动态控制缓存数据（缓存已在 translate.xml 中定义）：直接用 `getCacheData` / `putCacheData` / `clear` 读写、刷新缓存内容，例如业务变更后主动刷新某条码值缓存。

2）动态扩展缓存（运行时新增缓存定义）：用 `putCache(TranslateConfigModel)` 动态注册一个新缓存，并可用 `putCacheUpdater` 为其挂载更新检测器，实现运行时按需扩展缓存翻译能力。

> 缓存翻译的基本用法见缓存翻译使用；超大规模主数据的 FIFO 动态缓存见超大规模主数据缓存。

sql数据库方言自适配和函数扩展

1. sqltoy提供了数据库函数自动适配功能，即在运行过程中将sql中的函数自动转换为适配当前数据库方言的函数

```
# 开启sqltoy默认的函数自适配转换函数
spring.sqltoy.functionConverts=default
```

- 如果对个别函数实现要进行替换,比如NVL类名称保持一致即实现了自定义实现对default中的替换

```
# 自定义函数来实现替换Nvl
spring.sqltoy.functionConverts=default,com.yourpackage.Nvl
```

- 你也可以关闭default，完全自行定义开启哪些函数或加载自定义的函数

```
# 启用框架自带Nvl、Instr
spring.sqltoy.functionConverts=Nvl,Instr
# 启用自定义Nvl、Instr
# spring.sqltoy.functionConverts=com.yourpackage.Nvl,com.yourpackage.Instr
```

- sqltoy中functionConverts=default包含以下函数（6.0.5 起共 28 个）

```
org.sagacity.sqltoy.plugins.function.impl.SubStr
org.sagacity.sqltoy.plugins.function.impl.Trim
org.sagacity.sqltoy.plugins.function.impl.Instr
org.sagacity.sqltoy.plugins.function.impl.Concat
org.sagacity.sqltoy.plugins.function.impl.ConcatWs
org.sagacity.sqltoy.plugins.function.impl.Nvl
org.sagacity.sqltoy.plugins.function.impl.DateFormat
org.sagacity.sqltoy.plugins.function.impl.Now
org.sagacity.sqltoy.plugins.function.impl.CurrentDate
org.sagacity.sqltoy.plugins.function.impl.Length
org.sagacity.sqltoy.plugins.function.impl.ToChar
org.sagacity.sqltoy.plugins.function.impl.If
org.sagacity.sqltoy.plugins.function.impl.GroupConcat
org.sagacity.sqltoy.plugins.function.impl.ToNumber
org.sagacity.sqltoy.plugins.function.impl.ToDate
org.sagacity.sqltoy.plugins.function.impl.DateDiff
org.sagacity.sqltoy.plugins.function.impl.Decode
org.sagacity.sqltoy.plugins.function.impl.FindInSet
org.sagacity.sqltoy.plugins.function.impl.Uuid
org.sagacity.sqltoy.plugins.function.impl.YearMonthDay
org.sagacity.sqltoy.plugins.function.impl.LastDay
org.sagacity.sqltoy.plugins.function.impl.AddMonths
org.sagacity.sqltoy.plugins.function.impl.DateAdd
org.sagacity.sqltoy.plugins.function.impl.LeftRight
org.sagacity.sqltoy.plugins.function.impl.Pad
org.sagacity.sqltoy.plugins.function.impl.Extract
org.sagacity.sqltoy.plugins.function.impl.DateTrunc
org.sagacity.sqltoy.plugins.function.impl.Mod
```

- 函数实现接口

```
org.sagacity.sqltoy.plugins.function.IFunction
```

- Trim函数实现示例

```
public class Trim extends IFunction {
    private static Pattern regex = Pattern.compile("(?i)\\Wtrim\\(");

    /*
     * (non-Javadoc)
     *
     * @see org.sagacity.sqltoy.config.function.IFunction#dialects()
     */
    @Override
    public String dialects() {
        return ALL;
    }

    /*
     * (non-Javadoc)
     *
     * @see org.sagacity.sqltoy.config.function.IFunction#regex()
     */
    @Override
    public Pattern regex() {
        return regex;
    }

    /*
     * (non-Javadoc)
     *
     * @see org.sagacity.sqltoy.config.function.IFunction#wrap(int,
     * java.lang.String[])
     */
    @Override
    public String wrap(int dialect, String functionName, boolean hasArgs, String... args) {
        if (args == null || args.length == 0) {
            return super.IGNORE;
        }
        if (dialect == DBType.SQLSERVER) {
            return "rtrim(ltrim(" + args[0] + "))";
        }
        if (dialect == DBType.H2) {
            return "trim(both ' ' from " + args[0] + ")";
        }
        // 其他数据库保持trim(field) 模式
        return super.IGNORE;
    }
}
```

1.1 函数适配完善（6.0.4）

新增源函数 / 源形态识别：

- `nvl2(a, b, c)`：oracle 源函数，非 oracle 系数据库自动转换为 `case when a is not null then b else c` 判空逻辑（oracle 系原生透传；参数个数不对时保留响亮报错）；
- `isnull(x)` 单参判空：mysql 的 isnull 返回 0/1 判空语义，此前被错转为要求 ≥2 参数的 nvl/coalesce/ifnull 导致目标库报参数个数错误，现已正确处理；
- `sys_timestamp`：oracle 源当前时间形态（可带精度参数），oracle 系原生透传，其余数据库转换为对应的当前时间函数（mysql 系保留 fsp 精度参数，sqlserver 精度形态用 `sysdatetime()`）；
- `strpos(str, sub)`：pg 系 / clickhouse 原生写法直接透传（参数序与 instr 一致），其余数据库转换为对应的 instr 实现；
- `substr(s, -n)` 负起点：两参负起点字面量转 `RIGHT(s, n)`（兼容 db2 等起点从 1 计的库）；三参负起点走 case 守卫形态（`start = 串长-n+1`，串长不足 n 时取 1，mysql 语义）；表达式起点无法判断时原样保留交目标库判定；hana 的负起点为 pg 语义会静默错值，同样收敛；
- `curdate()` / `CURRENT_DATE`：新增当前日期（零点）函数类——oracle 系统一转 `TRUNC(CURRENT_DATE)`（保留会话时区）、db2 关键字体 `CURRENT DATE`、sqlite 取本地时钟（其原生 CURRENT_DATE 为 UTC 墙钟，与业务时区不一致）。

既有函数转换修正与增强：

- `group_concat` / `listagg` / `string_agg` 转换增强：支持 `within group (order by …)` 尾部子句、参数内嵌 `order by`（如 pg 源 `string_agg(a, '-' order by b)`）、`DISTINCT` 前缀识别（clickhouse 的 groupArray 不支持 DISTINCT 与排序时显式响亮保留）；分隔符字面量内含 "order by" 文本不再被误判为排序子句；
- `datediff(month, …)` 跨库月差口径统一：MONTHS_BETWEEN 的 oracle 小数口径（日差按 1/31 天月折算）与各库语义分歧，不再作为源函数注册转换，统一按年月分量差计算；

- `to_char` → sqlserver 目标：以 FORMAT（2012+）承担，修复大写格式形态（如 `'YYYY-MM-DD HH24:MI:SS'`）与 .NET token 大小写敏感导致的错值，并归一带引号格式串的字面量保护；
- sqlite 目标的 now() / CURRENT_DATE / CURRENT_TIMESTAMP：统一取本地时钟（原生为 UTC 墙钟，东八区业务差 8 小时）。

1.2 函数适配完善（6.0.5）

新增函数类（11 个，均默认注册进 `default`）：

- `find_in_set(str, strlist)`：mysql 系原生透传，其余库以"首尾补逗号 + instr 精确子串定位"改写（避免 `1` 命中 `11` 的误判）；
- `uuid()` 唯一 id 生成：归一 `uuid` / `guid` / `newid` / `sys_guid` / `random_uuid` / `gen_random_uuid` / `uuid_generate_v4` 等多种源拼写，按目标库转换（mysql 用 `uuid()`、oracle 系用 `sys_guid()`、pg 系用 `gen_random_uuid()`、sqlserver 用 `newid()` 等）；
- `year` / `month` / `day`（含 `hour` / `minute` / `second` 等）日期时间分量提取；
- `last_day` / `eomonth` 当月最后一天；
- `add_months` 月份加减（月末钳制）；
- `date_add` / `date_sub` / `dateadd` / `adddate` / `subdate` 日期区间加减族；
- `left(str, n)` / `right(str, n)` 前缀后缀取串；
- `lpad` / `rpad` 定长填充；
- `extract` / `date_part` 分量提取（ANSI 与 pg 写法）；
- `trunc` / `date_trunc` 日期截断族（oracle 的 `trunc(x[, unit])` 与 pg 的 `date_trunc(unit, x)` 双源形态）；
- `mod(a, b)` 取模（18 环境实测仅 sqlserver 无 mod 函数，转 `%` 运算符）。

既有函数源拼写扩充：

- `instr`：补 `locate` / `posstr`（mysql 与 db2 用户写法）；
- `length`：补 `character_length`（ANSI 写法）/ `octet_length`（pg 字节写法）；
- `now`：补 `current_timestamp` 关键字源拼写（标准 SQL 写法）；
- `if`：补 `iif`（sqlserver 2012+/sqlite 3.32+ 原生）；
- `substr`：补 `mid`（mysql 同义词写法）；
- `to_date`：补 `str_to_date`（mysql 用户写法）。

新增数据库的函数适配（配合 6.0.5 新增瀚高 HighGo）：

- 瀚高 HighGo（PG12 内核 + oracle 兼容层）：无 `uuid()` / `gen_random_uuid()` 但有 `sys_guid()`、`last_day` / `add_months` / `trunc` 原生、无 `listagg` 而 `string_agg` 原生，由函数适配器的 HIGHGO 分支承担。

> 上述新增/扩充函数均经多数据库真库探针验证；无法安全转换的形态（如目标库不支持的聚合排序）保持"响亮保留"，原样交由目标库报错，避免静默错值。

1.3 自行扩展函数适配（开发者侧，无需框架改动）

内置的 28 个函数覆盖的是**跨库写法存在差异**的常见函数。项目里遇到框架没适配的函数——数据库私有函数、行业专用函数、某个库新引入的函数——**由项目开发者自行扩展即可，不需要等框架层面支持**：实现 `IFunction` 并通过 `functionConverts` 挂载，与内置函数走的是完全同一套机制。

契约（`org.sagacity.sqltoy.plugins.function.IFunction`）

方法	作用	返回约定
`dialects()`	声明本函数适配哪些方言	`ALL`（即空串）表示全部方言；否则用逗号或分号分隔**精确枚举**方言名
`regex()`	匹配 SQL 中的函数调用	`Pattern`，用于定位待改写的函数
`wrap(dialect, functionName, hasArgs, args...)`	产出目标方言的 SQL	返回改写后的片段；返回 `IGNORE`（即 null）表示**原样透传**
`wrapArgs(functionName, args...)`	受保护的便捷方法	按“函数名不变、仅重新拼接参数”这一最常见形态生成结果

最小示例：只有 SQL Server 需要改写，其余库原生透传（内置 `Mod` 的实现）

```
public class Mod extends IFunction {
    // 开头的 \W 防止把 ltrim( 之类误命中成 trim(，(?i) 保证大小写不敏感
    private static Pattern regex = Pattern.compile("(?i)\\Wmod\\(");

    @Override
    public String dialects() {
        return ALL;
    }

    @Override
    public Pattern regex() {
        return regex;
    }

    @Override
    public String wrap(int dialect, String functionName, boolean hasArgs, String... args) {
        if (args == null || args.length < 2) {
            return super.IGNORE;
        }
        // sqlserver 无 mod 函数(真库实测)，由 % 运算符承担；其余全库原生透传
        if (dialect == DBType.SQLSERVER) {
            return "(" + args[0].trim() + " % " + args[1].trim() + ")";
        }
        return super.IGNORE;
    }
}
```

必须注意的几点

1. `regex()` 要以 `\W` 开头、以 `\(` 结尾，如 `(?i)\Wmod\(` 。开头的 `\W` 防止子串误命中， `(?i)` 保证大小写不敏感。
2. `dialects()` 是**按逗号/分号切分后逐项精确比对**，不是子串包含匹配——子串匹配有双向偏差（声明 `oracle11` 会被当前方言 `oracle` 误命中，声明 `oracle` 又匹配不上方言 `oracle11` ）。所以自定义函数必须显式枚举要适配的方言名。
3. 匹配是在**字面量掩码后的 SQL** 上进行的，因此字符串常量里出现的同名文本不会被误改写。
4. `wrap` 的 `args` 是已切分好的参数表达式原文，可能带空白，拼接前建议 `trim()` 。
5. **只能改写函数名形态。**运算符写法（如 SQL Server 的 `a % b` 、Oracle 的 `a || b` ）不是函数调用，正则框架捕获不到，因此不在适配范围内——跨库 SQL 请统一写函数形态（ `mod(a,b)` 、 `concat(a,b)` ），可移植性最好。
6. 绝大多数情况下 `wrap` 应当对“本来就原生支持该函数的库”返回 `IGNORE` ，只改写真正有差异的方言。无差别改写会破坏原生行为。

覆盖内置实现

`functionConverts` 按函数名去重，且 `default` 展开的内置函数**排在自定义之后加载**，因此自定义的同名实现会覆盖内置的：

```
# 用自己的 Nvl 覆盖内置 Nvl，其余内置函数照常生效
spring.sqltoy.functionConverts=default,com.yourpackage.Nvl
```

类名简写（ `Nvl` 、 `Instr` ）只在框架自带的 impl 包内解析，自定义实现请写全限定类名。

进阶：尾部子句钩子

少数函数的方言差异不在括号内，而在闭括号之后紧邻的附属子句上，例如聚合的 `within group (order by ...)` 、 `find_in_set(x,y) > 0` 这类布尔口径。这类场景用三个可选钩子：

- `suffixPattern()` ：尾部子句的匹配 pattern，须以 `\s*` 开头承担间隔空白、以 `\(*` 结尾；返回 null（默认）表示本函数不消费尾部子句
- `suffixNeedsParen()` ：true（默认）按平衡括号消费整个子句；false 按 pattern 命中长度消费，用于 `>0` / `=0` 这类裸操作符子句
- `wrapWithSuffix(dialect, functionName, suffix, args...)` ：命中后的处理入口， `suffix` 是子句原文（含前导空白）；返回 `IGNORE` 表示连同子句原样保留

什么时候不该扩展函数，而该用方言 sqlId

如果差异不是“同一个函数换个写法”，而是**整条 SQL 的结构不同**（分页语法、递归 CTE 写法、upsert 语义、窗口函数支持度），函数改写解决不了，应当用 `sqlId_dialect` / `dialect_sqlId` 变体或 `<sql dialect="xxx">` 属性，见第 2 节。

2. 通过sqlId+dialect模式

- 可针对特定数据库写sql,sqltoy根据数据库类型获取实际执行sql,顺序为: dialect_sqlId->sqlId_dialect->sqlId， 如数据库为mysql,调用sqlId:sqltoy_showcase,则实际执行:sqltoy_showcase_mysql

```
<sql id="sqltoy_showcase">
    <value>
    <![CDATA[
    select * from sqltoy_user_log t
    where t.user_id=:userId
    ]]>
    </value>
</sql>
<!-- sqlId_数据库方言(小写) -->
<sql id="sqltoy_showcase_mysql">
    <value>
    <![CDATA[
    select * from sqltoy_user_log t
    where t.user_id=:userId
    ]]>
    </value>
</sql>
```

2.1 `<sql>` 元素的 dialect 属性（6.0.3 新增）

单条 sql 可以通过 `dialect` 属性**声明按指定方言形态解析固化**（此前该属性不被解析而静默忽略）：

```
<!-- 声明本条 sql 按 kingbase 方言形态固化：加载时即按 kingbase 做函数/保留字转换 -->
<sql id="sqltoy_showcase_kingbase" dialect="kingbase">
    <value>
    <![CDATA[
    select ifnull(max(t.amount),0) from sqltoy_user_log t where t.user_id=:userId
    ]]>
    </value>
</sql>
```

工作机制：

- **加载时**：按声明方言对 sql 做一次函数/保留字转换并固化解析标签（如上面的 `ifnull` 在 kingbase 固化为 `coalesce` 形态）；`count-sql` 与主 sql 使用同一生效方言；
- **执行时**：当前数据库方言与声明一致时，走"查询方言==解析标签"早退机制，**跳过函数替换**（零开销）；不一致时按既有惰性转换 + 缓存机制反向适配；
- **优先级**：`mql/eql 强制 mongo/es` > `元素 dialect 属性` > `全局方言`；`mql` / `eql` 不支持该属性；
- 属性值会经方言清单校验，未识别的方言 warn 提示并回退全局方言；
- 与 `realDialectFirst` 协同：`<sql id="x_kingbase" dialect="kingbase">` 被变体匹配命中时，加载已固化 kingbase 形态且标签相等，执行时连惰性转换都省略。

2.2 realDialectFirst：按真实方言优先匹配变体（6.0.3 新增）

sqltoy 判断方言依赖 JDBC 连接的 `productName`，部分场景（OceanBase 按 mysql 配置、代理数据库等）**配置方言与真实方言不一致。**开启本开关后，sqlId 方言变体的查找优先按连接探测的真实方言（`DBProfile.realDialect`）进行：

```
# 默认 false：真实方言变体不存在时回退原有配置方言查找链，真实方言与配置方言一致时行为不变
spring.sqltoy.realDialectFirst=true
```

- 例如全局 `dialect=mysql`、真实库为 kingbase：开启后调用 `sqlId` 会优先找 `sqlId_kingbase`，不存在再回退 `sqlId_mysql` → `sqlId`；
- 经真实方言变体匹配选中的 sql，其函数/保留字替换方言也跟随真实方言（未命中变体回退 base 的 sql 保持配置方言行为）；
- 仅作用于 sql 形态转换层（函数与保留字），分页等执行策略的数据库分派不受影响。

3. 虚表宏 @virtualTable()（6.0.6 新增）

"select 常量行 union 真实查询"类 sql（字典下拉兜底行、报表固定合计行等）在各数据库对"select 常量行（不带 from）"的支持不一致：oracle/db2/hana 必须挂伪表，mysql/pg/sqlserver 等原生支持无 from 的 select（写 `from dual` 反而语法错）。6.0.6 起在需要伪表的位置写 `@virtualTable()` 宏，sqltoy 在 sql 解析期按方言自动展开（在函数/保留字转换前处理，随 SqlToyConfig 固化），一条 sql 全库通用。以 xml 配置为例：

```
<sql id="sys_tenant_options">
    <value>
    <![CDATA[
        select '0' tenant_id,'全局公共' tenant_name @virtualTable()
        union
        select s.tenant_id,s.tenant_name
        from sys_tenant_info s
        where s.status=1
    ]]>
    </value>
</sql>
```

不同数据库上实际执行的 sql：

```
-- oracle / oracle11 / dm(达梦) / oceanbase：展开为 from dual
select '0' tenant_id,'全局公共' tenant_name from dual
union
select s.tenant_id,s.tenant_name from sys_tenant_info s where s.status=1

-- db2：展开为 from sysibm.sysdummy1
select '0' tenant_id,'全局公共' tenant_name from sysibm.sysdummy1
union
select s.tenant_id,s.tenant_name from sys_tenant_info s where s.status=1

-- hana：展开为 from dummy
select '0' tenant_id,'全局公共' tenant_name from dummy
union
select s.tenant_id,s.tenant_name from sys_tenant_info s where s.status=1

-- mysql / postgresql / sqlserver 等：宏被剔除（原生支持无 from 的 select）
select '0' tenant_id,'全局公共' tenant_name
union
select s.tenant_id,s.tenant_name from sys_tenant_info s where s.status=1
```

方言展开对照：

数据库	宏展开结果
oracle / oracle11 / dm（达梦） / oceanbase	`from dual`
db2	`from sysibm.sysdummy1`
hana	`from dummy`
mysql 系（mysql/tidb/doris/starrocks）、pg 系（postgresql/gaussdb/mogdb/openGauss/stardb/vastbase/oscar/kingbase/highgo）、sqlserver、sqlite、h2、clickhouse、impala、tdengine	直接剔除（原生支持无 from 的 select）

使用说明：

- 兼容别名 `@dual()` / `@dummyTable()`，三名等价、大小写不敏感、括号内空白容错；
- 同一条 sql 可出现多次，每处独立按方言展开（每个常量 select 各写一个）；
- xml 配置 sql、`@Sql` 注解 sql、代码 sql（`findBySql` 等）同样有效（统一解析管线）；
- 展开发生在解析期并随 SqlToyConfig 固化（与函数/保留字转换同一时机），运行期零开销；不含宏的 sql 零影响（含 `@` 的普通 sql 如 `email like 'a@%'` 不受影响）；
- 伪表映射与框架生成 merge 语句所用伪表同源（各 Dialect 的 VIRTUAL_TABLE 常量），行为一致；
- `realDialectFirst` 场景下宏按"解析方言"展开固化，运行期惰性转换层不再重复展开宏。

4. 如何同时测试多种数据库环境

- sqltoy提供了spring.sqltoy.redoDataSources参数，来设置查询语句重复执行的数据库

```
# 如在mysql场景下同时测试其他类型数据库，验证sql适配不同数据库，主要用于产品化软件
spring.sqltoy.redoDataSources[0]=pgdb
```

5. 常见问题

- 1) 为什么我的kingbase、polardb、oceanbase这些数据库使用mysql或postgresql数据库模式，还有大量错误？ 解答: 因为sqltoy获取数据库方言是通过jdbc connection获取当前productName来判断数据库类型，并不能完全准确获取数据库的方言模式，所以要通过2种方式之一来解决

方式一: 单一数据库场景下直接设置当前数据库方言

```
spring.sqltoy.dialect=mysql
```

方式二: 多数据库场景下，将识别不准确的alias到正确的数据库方言上

- spring.sqltoy.dialectMap 是Map类型,当前dialect可用SELECT version()或类似语句查询(具体是什么问AI)

```
spring.sqltoy.dialectMap.kingbase=mysql
```

- dialectMap 映射原理，帮助你设置正确的Map key,key indexOf productName

```
// 针对框架未支持的数据库，通过dialectMap的key进行匹配映射到响应的方言上
for (Map.Entry<String, String> entry : dialectMap.entrySet()) {
    if (StringUtil.indexOfIgnoreCase(dbDialect, entry.getKey()) != -1) {
        dilectName = entry.getValue().toLowerCase();
        break;
    }
}
```

- 2) 全局 dialect 固化后，为真实数据库写的方言变体 sql 没有生效？ 解答: 方言变体（`sqlId_dialect` / `dialect_sqlId`）默认按**配置方言**查找。若配置方言与连接的真实方言不一致（如 OB 按 mysql 配置而真实库为 oceanbase），6.0.3 起可开启 `spring.sqltoy.realDialectFirst=true`，变体查找与函数替换将优先跟随真实方言，参见 2.2 realDialectFirst。

多种数据库适配验证

sqltoy 自 2008 年立项起就提倡**产品化**：一套软件产品适配多种数据库（如 CRM 同时适配 Oracle、MySQL、DB2、SQL Server、PostgreSQL、达梦、高斯等），便于在不同客户场景下推广。唯有产品化才能形成真正的竞争力。

一、如何让 SQL 适配多种数据库

1. **主键策略**：少用与数据库强相关的 `identity`、`sequence`，优先用 sqltoy 的程序化主键（22 位/26 位/雪花/UUID 等，见主键策略）。
2. **增删改**：使用 sqltoy 对象化操作（save/update/delete/load），由框架处理方言差异。
3. **查询**：开启 `spring.sqltoy.functionConverts=default`，实现 SQL 中函数按数据库自动适配替换（见方言自适配与函数扩展）。
4. **常量行伪表**："select 常量行 union 真实查询"类写法（字典下拉兜底行、报表固定合计行等）用虚表宏 `@virtualTable()`（6.0.6 新增），解析期按方言自动展开为 `from dual` / `from sysibm.sysdummy1` / `from dummy` 或直接剔除，无需再为不同数据库维护 from dual 写法，详见方言自适配与函数扩展。
5. 必要时用 `sqlId_dialect` / `dialect_sqlId` 为特定数据库提供专属 SQL；6.0.3 起还支持在 `<sql>` 元素上声明 `dialect` 属性按指定方言固化解析，以及开启 `spring.sqltoy.realDialectFirst` 让变体查找优先跟随连接探测的真实方言，详见方言自适配与函数扩展。

二、开启跨数据库适配测试（redoDataSources）

为了在测试功能时**同时验证 SQL 在其他数据库下能否正确运行**，sqltoy 提供了 `redoDataSources` 机制：配置本产品需要适配的其他类型数据库后，任意执行一个查询，框架会**同时在 redoDataSources 数据源上也执行一次**，从而验证 SQL 的跨库适配能力。

第 1 步：用 `@Configuration` 配置多个数据源（关注两个针对数据源的 `@Bean` 定义），例如主库 + 一个 PostgreSQL 库 `pgdb`。

第 2 步：配置需要验证的数据库

```
spring.sqltoy.redoDataSources[0]=pgdb
```

配置后，执行任意查询功能时，sqltoy 会在 `pgdb` 上重放（redo）一次该 SQL。如果 SQL 中存在不兼容的方言写法，重放时即会暴露，便于在开发/测试阶段就发现跨库问题，而不是等到客户现场切换数据库时才报错。

> 提示：`redoDataSources` 仅用于**适配验证**，重放查询不影响主查询结果。生产环境可关闭。

三、配套工具与验证工程

工具	用途
sagacity-db-diversity	**库表结构差异对比**：分别连接参考库与目标库，导出表/字段/主键/外键/索引/注释到 xml 后离线比对，支持单边连接（开发环境只能跳板机访问）、也支持对比不同时间点的导出；换库迁移前先跑一遍，把结构差异清干净
sqltoy-aot-verify	**native 构建验证矩阵**（spring + solon 双栈，PostgreSQL pgvector + postgis），同时覆盖方言函数转换与缓存翻译的双实现
sqltoy-showcase	全功能特性演示工程（双 H2 库模拟分片拓扑 + Testcontainers 真 Mongo/ES）；**各数据库实测**见 sqltoy-quickstart 第 9 课：同一批函数转换 SQL 经 docker-compose 在 PostgreSQL/SQL Server/Oracle/MySQL 真库上运行

`redoDataSources` 的注意事项：

- 重放的是**查询**，写操作不会重放；因此它在开发/测试阶段用，生产环境建议关闭；
- 每次查询都会额外打一次库，**测试环境的耗时与连接数是翻倍的**，压测时不要开；
- 重放暴露的是"方言写法不兼容"，**不能替代业务结果校验**（同一 SQL 在不同库上语义可能有细微差异，如大小写、排序、null 排序位置）。

四、相关

- 函数替换与方言 sqlId：方言自适配与函数扩展
- 未列出的 PostgreSQL 系数据库适配（dialect/dialectMap）：常见 SQL 案例
- 换库迁移的完整评估清单：换库迁移适配指引

主键策略介绍

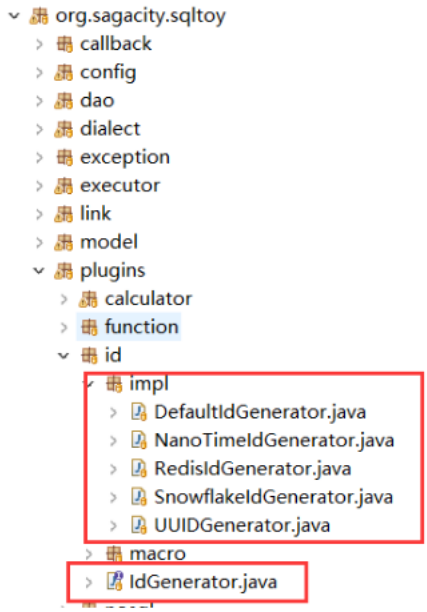
sqltoy 的主键分两类：**程序化主键**（框架或自定义生成器产出，不依赖数据库）与**数据库主键**（`sequence` / `identity`，由数据库产生）。前者跨库无差异，是产品化推荐做法。

一、策略总览

策略关键字（quickvo `generator` / `@Id.generator` 的取值）	实现类（`org.sagacity.sqltoy.plugins.id.impl`）	形态
`default`	`DefaultIdGenerator`	22 位有序数字：13 位当前毫秒 + 6 位纳秒 + 3 位主机 ID，如 `1592214184072221900219`
`nanotime`	`NanoTimeIdGenerator`	26 位有序数字：15 位 `yyMMddHHmmssSSS` + 6 位纳秒 + 2 位（线程 Id + 随机数）+ 3 位主机 ID
`snowflake`	`SnowflakeIdGenerator`	雪花算法，有序数字
`uuid`	`UUIDGenerator`	32 位 UUIDv7（有序版本）
`ulid`	`ULIDGenerator`	26 位有序字符：时间戳 10 位 + 随机熵 16 位（需引入 `ulid-creator`）
`redis`	`RedisIdGenerator`	业务主键：按规则生成，如订单号「年月日 + 流水」
`sequence` / `identity` / `assign`	由数据库或调用方负责	数据库序列 / 自增列 / 手工赋值

> 关键字大小写不敏感，`defaultIdGenerator`、`snowflakeIdGenerator` 等写法同样可用（框架做了命名容错）。

内置生成器实现（`plugins.id.impl`，截图早于 ULID 的加入）：



`ULID` 需要额外依赖：

```
<dependency>
    <groupId>com.github.f4b6a3</groupId>
    <artifactId>ulid-creator</artifactId>
    <!-- 具体版本可调整 -->
    <version>5.2.4</version>
</dependency>
```

二、在 quickvo.xml 中指定策略

```
<!-- 主键策略：identity 列会自动识别，其余按类型/长度自动推断：
     strategy: sequence(需 sequence 名) | assign(手工) |
               generator(default 22位 | nanotime 26位 | snowflake | uuid | ulid | redis | 自定义类) -->
<primary-key>
    <!-- 可用正则批量匹配表名 -->
    <table name="SQLTOY_\w+|LESSON_\w+|SYS_\w+" strategy="generator" generator="default" />
    <!-- 指定数据库序列 -->
    <table name="XXX_TABLE" strategy="sequence" sequence="SEQ_XXXX" />
    <!-- 指定雪花算法 -->
    <table name="sys_staff_info" strategy="generator" generator="snowflake" />
</primary-key>
```

自动推断（不配置 `primary-key` 时）：`bigint` → 雪花算法；22 位字符 → `DefaultIdGenerator`；26 位字符 → `NanoTimeIdGenerator`；`identity` 列 → 由数据库生成，无需配置。

真实的 sequence 配置示例：

```
<primary-key>
    <table name="SQLTOY_SEQ_SHOWCASE" strategy="sequence" sequence="SEQ_SQLTOY_SEQ_CASE"/>
</primary-key>
```

三、生成的注解

quickvo 依据上述配置在 POJO 主键字段上生成 `@Id`：

```
// strategy="generator" + generator 关键字
@Id(strategy = "generator", generator = "org.sagacity.sqltoy.plugins.id.impl.NanoTimeIdGenerator")
@Column(name = "ID", type = java.sql.Types.VARCHAR, length = 26, nullable = false)
private String id;

// strategy="sequence" 时指向序列名
@Id(strategy = "sequence", sequence = "SEQ_XXXX")
@Column(name = "ID", type = java.sql.Types.BIGINT, nullable = false)
private Long id;
```

> `@Id` 只描述生成策略，`@Column` 描述列；两者都要标在主键字段上。属性含义见注解速查 · @Id。

quickvo 实际生成的片段（截图）：

```
/**
 * ID
 */
@Id(strategy="sequence",sequence = "SEQ_SQLTOY_SEQ_CASE")
@Column(name="ID",type=java.sql.Types.INTEGER,nullable=false)
protected String id;
```

四、redis 业务主键

redis 策略属于**业务主键**范畴（订单号、单据号这类有规则的号）。quickvo 用 `business-primary-key` 声明：

```
<!-- 宏：@substr(${field},startIndex,length) @case(value,case1,value1,...) @df(yyMMdd)
     不需要日期时用 @df(''); related-columns 是参与生成的来源字段 -->
<business-primary-key>
    <table name="LESSON_DEVICE_ORDER" column="ORDER_ID"
           generator="redis" signature="PO@df(yyMMdd)" length="16" />
</business-primary-key>
```

一个更完整的真实配置（含宏组合与来源字段）：

```
<!-- 基于redis产生有规则的业务主键 -->
<business-primary-key>
    <!-- 1位购销标志_2位设备分类代码_6位日期_3位流水（如果当天超过3位会自动扩展到4位） -->
    <!-- 业务主键可以充当实际的主键，如果字段是主键会覆盖原来的主键策略 -->
    <!-- 包含的三个宏：
          @substr(${field},startIndex,length)
          @case(value,case1,value1,case2,value2,other)
          @df(yyMMdd) 日期格式化，如果不需要加入日期可以用@df('')
    -->
    <table name="SQLTOY_DEVICE_ORDER_INFO" column="ORDER_ID" generator="redis"
        signature="${psType}@case(${deviceType},PC,PC,NET,NT,OFFICE,OF,SOFTWARE,SF,OT)@day(yyMMdd)"
        related-columns="psType,deviceType" length="12" />
</business-primary-key>
```

> 若业务主键字段本身也是主键（`@Id`），它会**覆盖**原主键策略——即业务号直接作为物理主键；`length` 与流水位数决定号的形态。

使用 redis 生成主键需要应用侧定义 `RedisTemplate`。

@BusinessId 注解与 generateBizId

配置层面**一张表只支持一个**业务主键。当一个表涉及多个业务主键时，用 `@BusinessId` 注解 + `generateBizId` 自行生成后赋值：

```
/** 订单编号（业务主键） */
@BusinessId(signature = "HW@case(orderType,SALE,SC,BUY,PO)@day(yyMMdd)",
        generator = "org.sagacity.sqltoy.plugins.id.impl.RedisIdGenerator",
        length = 20, sequenceSize = 5, relatedColumns = {"orderType"})
private String orderId;
```

`@BusinessId` 属性：

属性	说明
`signature`	识别区分符号表达式，支持 `@case(name,v1,then1,v2,then2)`、`@day(yyMMdd)`、`@substr(name,start,length)` 等宏
`generator`	主键生成策略实现类（如 redis 策略 `RedisIdGenerator`）
`length` / `sequenceSize`	业务主键总长度 / 尾部流水位数（默认 -1 取全局配置）
`relatedColumns`	参与主键生成的关联字段
`start`	流水初始值，默认 1

通过 `LightDao` 生成：

```
// 1) 基于实体对象配置的 @BusinessId 策略，提取属性值生成业务主键
String orderId = lightDao.generateBizId(orderInfo);

// 2) 动态传参生成（业务主键按表维度隔离管理：signature 支持 @case/@day/@substr 宏）
String bizId = lightDao.generateBizId("sqltoy_order_info",
        "HW@case(orderType,SALE,SC,BUY,PO)@day(yyMMdd)",
        MapKit.map("orderType", "SALE"), null, 20, 5);

// 3) 简单形态：唯一标识符号 + 增量
long id = lightDao.generateBizId("ORDER", 1);
```

> 🎬 完整可运行示例：演示项目 `sqltoy-showcase` 的 `BusinessIdAndDataVersionTest.java`（含 @BusinessId 注解用法；运行依赖 redis，已标注 @Disabled 作为参考实现）。

五、使用自定义主键策略

实现 `IdGenerator` 接口（包 `org.sagacity.sqltoy.plugins.id`；旧资料里常写成 `IdGenarator`，属笔误）：

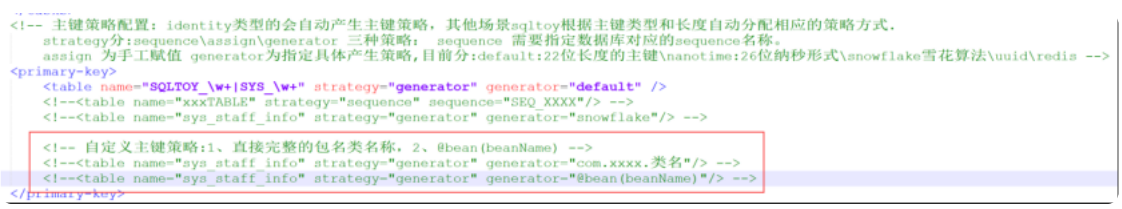
```
public interface IdGenerator {
    /**
     * @param tableName       表名（部分策略需要按表隔离）
     * @param signature       识别区分符号
     * @param relatedColumns  关联字段名
     * @param relatedColValue 关联字段值
     * @param bizDate         当前日期
     * @param idJavaType      主键 Java 类型
     * @param length          主键长度
     * @param sequenceSize    流水号长度（如 20210709[001] 后三位为流水）
     */
    public Object getId(String tableName, String signature, String[] relatedColumns, Object[] relatedColValue,
            Date bizDate, String idJavaType, int length, int sequenceSize);

    /** 可选：初始化（框架启动时回调） */
    public default void initialize(SqlToyContext sqlToyContext) throws Exception { }
}
```



然后在 quickvo.xml 里把 `generator` 指向实现类，支持**两种写法**：全限定类名，或 `@bean(beanName)`（生成器交给 Spring 容器管理，便于注入依赖）：

```
<primary-key>
    <table name="SQLTOY_\w+" strategy="generator"
           generator="com.yourproject.MyIdGenerator" />
    <!-- 或：交给 Spring 容器 -->
    <table name="SQLTOY_\w+" strategy="generator"
           generator="@bean(myIdGenerator)" />
</primary-key>
```



也可以直接在 POJO 上标注，绕过 quickvo：

```
@Id(strategy = "generator", generator = "com.yourproject.MyIdGenerator")
```

> 自定义生成器若依赖 Spring 的 bean（如 redis 模板），用 `initialize(SqlToyContext)` 拿上下文，或在实现类内部自行获取；native 镜像下该实现类需要反射注册，见 GraalVM Native Image 指引。

六、主键重复处理

`default`、`nanotime`、`snowflake` 三种策略依赖**主机 ID** 区分 worker。同一台 IP 的服务器部署多个应用实例时会重复，需显式指定：

```
# default、nanotime：三位数字
java -Dsqltoy.server.id=112

# snowflake：两个参数，数字形式且小于 32
java -Dsqltoy.snowflake.workerId=11
java -Dsqltoy.snowflake.dataCenterId=20
```

> 容器化/云环境（Pod IP 动态变化）建议：显式配置上述参数，或改用 `redis` 策略/自定义生成器集中发号。

七、相关

- `@Id` / `@BusinessId` 属性表：注解速查
- 主键插入返回值的方言差异：方言自适配与函数扩展
- 换库迁移时的主键评估：换库迁移适配指引

POJO生成表结构DDL

- sqltoy支持项目启动时根据POJO实体生成ddl脚本并在数据库中执行

```
spring:
    sqltoy:
        #指定扫描POJO包路径，可以指定一个顶层包会自动递归扫描，也可以指定比较底层的包
        packagesToScan:
            - com.example.modules.commons.entity
            - com.example.modules.system.entity
        autoDDL: true
        #dialectDDLGenerator: com.xxx 自定义扩展实现类
```

- 通过代码生成整个项目POJO对应的数据库脚本

```
@Test
public void testCreateSqlFile() {
    // 指定POJO所在的包路径
    String[] scanPackages = new String[] { "org.sagacity.sqltoy.demo.domain" };
    try {
        /**
         * @param scanPackages
         * @param saveFile            脚本存放文件
         * @param upperOrLower        upper|lower 脚本表名、字段名是否统一转大写或小写
         * @param dbType              数据库类型，用DBType.xx 提供
         * @param schema              针对sqlserver需要提供(其他数据库可为null)，项目启动时会根据connection获取schema
         * @param dialectDDLGenerator 自己指定ddl创建器,如果是：mysql、oracle、pg、sqlserver等数据库无需扩展，传递null即可
         */
        DDLFactory.createSqlFile(scanPackages, "D://sqltoy.sql", "upper", DBType.MYSQL, null, null);
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
```

实体注解与 DDL 元数据

DDL 生成支持以下实体注解（`@Partition` / `@PartitionDef` / `@MppTable` 为 **6.0.3 新增**，由 quickvo 依据数据库的分区定义、MPP 表定义自动产生，也可手工标注）：

注解	说明
`@Foreign(table, field, constraintName)`	**外键标记**（6.0/2023.07 新增）：声明字段关联的表与字段，生成 DDL 时创建外键约束
`@Index(name, columns, ...)`	**索引定义**：声明单列或多列索引（可唯一），生成 DDL 时创建索引，见下方第 4 节
`@Indexes({ @Index, ... })`	**索引集合**：一个实体声明多个索引
`@PartitionKey`	**分区字段标记**：MPP 数据库（如 StarRocks）建表时的分区键
`@Partition`	**分区策略标记**（6.0.3 新增）：描述分区方式与分区键，生成 `PARTITION BY` 子句
`@PartitionDef`	**分区明细**（6.0.3 新增）：配合 `@Partition` 描述每个分区的名称和边界
`@MppTable`	**MPP 表引擎元数据**（6.0.3 新增）：ClickHouse/Doris/StarRocks 建表时的引擎、键模型、分桶等定义

1、外键与参照动作

```
@Accessors(chain = true)
@Entity(tableName = "sqltoy_order_info", pk_constraint = "PRIMARY")
public class OrderInfo implements Serializable {

    @Column(name = "ORGAN_ID")
    // 外键：关联 sqltoy_organ_info 表的 ORGAN_ID 字段
    // deleteRestict/updateRestict 取值见 org.sagacity.sqltoy.config.model.ReferentialAction 常量：
    // 0:CASCADE(级联)、1:RESTRICT(限制，默认)、2:SET_NULL(置空)、3:NO_ACTION(无动作)、4:SET_DEFAULT(置默认值)
    @Foreign(table = "sqltoy_organ_info", field = "ORGAN_ID", constraintName = "FK_ORDER_ORGAN", deleteRestict = 0)
    private String organId;
}
```

生成外键约束时按数据库兼容性自动裁剪：Oracle/DM 不支持 `ON UPDATE` 子句，Oracle 不接受 `NO ACTION` 关键字（ORA-02000），Oracle/DM 与 MySQL 系（InnoDB）不支持 `SET DEFAULT`——不支持的动作自动跳过，退化为默认的 NO ACTION 行为。

2、表分区（@Partition / @PartitionDef / @PartitionKey）

```
@Entity(tableName = "SQLTOY_TRANS_INFO", pk_constraint = "PRIMARY")
// 分区策略：RANGE|LIST|HASH|RANGE_COLUMNS|LIST_COLUMNS:
// columns 为分区键列（缺省时自动收集字段级 @PartitionKey 标记的字段，两者双向联动）;
// expression 为数据库原生分区表达式原文（如 mysql 的 TO_DAYS(createTime)），提供时优先生效
@Partition(strategy = "RANGE", columns = { "BIZ_DATE" }, partitions = {
        @PartitionDef(name = "p202601", value = "2026-02-01"),   // 边界值：日期形态自动加引号
        @PartitionDef(name = "p202602", value = "2026-03-01"),
        @PartitionDef(name = "pmax", value = "MAXVALUE") })      // 兜底分区
public class TransInfo implements Serializable {

    @Column(name = "BIZ_DATE")
    @PartitionKey   // 字段级分区标记，与 @Partition.columns 双向联动
    private LocalDate bizDate;
}
```

生成 MySQL 建表语句时自动渲染：

```
CREATE TABLE SQLTOY_TRANS_INFO (...)
PARTITION BY RANGE (BIZ_DATE) (
   PARTITION p202601 VALUES LESS THAN ('2026-02-01'),
   PARTITION p202602 VALUES LESS THAN ('2026-03-01'),
   PARTITION pmax VALUES LESS THAN (MAXVALUE)
)
```

分区明细渲染规则：RANGE 分区按上界升序排序（MAXVALUE 兜底分区置末），LIST 分区在 Oracle/DM 下渲染 `VALUES (v)`、MySQL 系渲染 `VALUES IN (v)`；HASH/KEY 策略按分区数量渲染 `PARTITIONS n`；pg/oracle 等策略型分区库无明细时保持注释形态，分区由数据库自行管理。

3、MPP 分析库表引擎（@MppTable）

ClickHouse/Doris/StarRocks 建表需要引擎、排序键、分桶等元数据，通过 `@MppTable` 描述：

```
@Entity(tableName = "SQLTOY_ORDER_OLAP")
@MppTable(
    engine = "OLAP",                          // ClickHouse 为 MergeTree/ReplacingMergeTree 等；Doris/StarRocks 为 OLAP
    keyModel = "DUPLICATE",                   // Doris/StarRocks 键模型：UNIQUE/PRIMARY/DUPLICATE/AGGREGATE
    orderBy = { "EVENT_TIME", "ORDER_ID" },   // 排序键：ClickHouse 的 ORDER BY 列；Doris/SR 为 KEY 模型的键列
    distributedBy = { "ORDER_ID" },           // 分桶列：Doris/StarRocks 的 DISTRIBUTED BY HASH 列
    buckets = 10,                             // 分桶数，缺省由数据库自动决定
    properties = { "replication_num=3" })     // 表属性：CK 渲染为 SETTINGS k=v；Doris/SR 渲染为 PROPERTIES("k"="v")
@Partition(strategy = "RANGE", columns = { "EVENT_TIME" })
public class OrderOlap implements Serializable { ... }
```

Doris/StarRocks 生成的建表语句形如：

```
CREATE TABLE SQLTOY_ORDER_OLAP (...)
ENGINE = OLAP
DUPLICATE KEY(EVENT_TIME, ORDER_ID)
COMMENT '订单明细'
PARTITION BY RANGE (EVENT_TIME) (...)
DISTRIBUTED BY HASH(ORDER_ID) BUCKETS 10
PROPERTIES ("replication_num" = "3")
```

ClickHouse 生成的建表语句形如（`engine=ReplacingMergeTree`、`engineArgs` 为版本列等引擎参数、`properties` 渲染为 `SETTINGS`）：

```
CREATE TABLE SQLTOY_ORDER_OLAP (...)
ENGINE = ReplacingMergeTree(ver)
ORDER BY (EVENT_TIME, ORDER_ID)
PARTITION BY EVENT_TIME
SETTINGS index_granularity = 8192
```

4、索引定义（@Index / @Indexes）

在实体类上声明索引，autoDDL 建表时同步创建索引；quickvo 2.x 也会依据数据库已有索引定义自动生成该注解（手工白名单注解不会被覆盖）：

```
@Indexes(indexes = {
    // 常规索引：name(必填)、columns(必填)、isUnique(默认false)、sortTypes(asc/desc，默认升序)
    @Index(name = "idx_staff_name", columns = { "staffName" }, sortTypes = { "asc" }),
    // 唯一索引
    @Index(name = "uniq_staff_code", isUnique = true, columns = { "staffCode" })
})
@Entity(tableName = "SQLTOY_STAFF_INFO")
public class StaffInfo implements Serializable {
    //......
}
```

渲染规则：MySQL 系以表内 `[UNIQUE] KEY indexName (col asc/desc,...)` 形态并入建表语句，其余数据库输出独立的 `CREATE [UNIQUE] INDEX indexName ON tableName (...);` 语句。

5、各数据库 DDL 生成器支持（6.0.3）

数据库	生成器	说明
MySQL / MySQL 5.7	`MySqlDDLGenerator`	支持 `@Partition` 分区子句
Doris / StarRocks	`DorisDDLGenerator` / `StarRocksDDLGenerator`	6.0.3 新增专用生成器（此前复用 MySQL 生成器），支持 `@MppTable` 引擎/键模型/分桶/属性、分区子句，VARCHAR 超 65533 自动降级 STRING
ClickHouse	`ClickHouseDDLGenerator`	6.0.3 新增，支持 MergeTree 系引擎、`ORDER BY`、`SETTINGS`、`PARTITION BY` 表达式
PostgreSQL 系（PG/GaussDB/OpenGauss/MogDB/Vastbase/StarDB/Oscar/Kingbase）	`PostgreSqlDDLGenerator`	含 6.0.3 对 Kingbase 的纳入
Oracle / Oracle11 / 达梦	`OracleDDLGenerator`	
SQL Server	`SqlServerDDLGenerator`	
DB2	`DB2DDLGenerator`	6.0.3 新增专用生成器（含 CLOB、JSON→CLOB 承载）
SAP HANA	`HanaDDLGenerator`	6.0.3 新增（含 NCLOB 大文本、JSON→NCLOB 承载）
SQLite	`SQLiteDDLGenerator`	6.0.3 新增
H2	`H2DDLGenerator`	

其他类型化细节：JSON 列在无原生 JSON 类型的库自动降级承载（HANA→NCLOB、SQL Server→NVARCHAR(MAX)、DB2/Oracle11→CLOB）；pgvector 向量类型仅在维度有效时渲染 `(n)`，无界向量渲染为 `vector`。

> 这些注解只影响 DDL 生成（autoDDL / DDLFactory），不改变运行时行为（`@PartitionKey` 亦作为 MPP 库 DML 的分区键提示）。

SQL拦截加工、多租户过滤

介绍

随着 SaaS 化发展以及一些用户特殊的场景诉求，都需要对项目中的 SQL 做一些集中化的处理，如： * **租户过滤**：自动追加租户 ID 条件，防止数据越权。 * **校验 SQL 注入**：增强安全性。 * **修改 Schema**：根据需要替换实际的 Schema（例如 `select xxx from schema.table`）。

使用说明

1. 配置参数

通过配置 `spring.sqltoy.sqlInterceptors` 数组来注册拦截器。SQLToy 框架默认提供了 `TenantFilterInterceptor`，专门针对对象操作和 `findEntity` 等单表操作进行租户过滤。

配置文件示例 (application.yml)：

```
spring:
  sqltoy:
    # 将 sqltoy 放于开始位置，避免很多开发者忽视了 spring.sqltoy 开头，变成了 sqltoy 开头导致无法加载
    # 多个路径用逗号分隔 (请务必看仔细)
    sqlResourcesDir: classpath:/sqltoy/quickstart
    # 默认为 classpath:sqltoy-translate.xml，一致则可以不用设置
    translateConfig: classpath:sqltoy-translate.xml
    # 针对 json 等特殊类型做处理 (可选配置)
    typeHandler: com.sqltoy.plugins.JsonTypeHandler
    # 可以填 default、defaultFormatter、defaultSqlFormatter 或者具体的类 (含包名)，sqltoy 提供的需要引入 druid jar
    sqlFormatter: defaultSqlFormatter

    # sql 拦截器，可以实现对 sql 的改写，比如多租户、sql 注入校验、改写的 schema 等
    sqlInterceptors:
      # 如果是具体的包路径和类名称，sqltoy 会自动实例化
      - org.sagacity.sqltoy.plugins.interceptors.TenantFilterInterceptor
      # 也可以注册一个 spring 的 bean，这里放入 bean 的名称
      # - yourInterceptorBeanName
```

2. 实体类标注

如果使用 SQLToy 自带的租户过滤，需在 POJO 上增加 `@Tenant` 注解，指定租户字段。

实体类示例 (StaffInfoVO.java)：

```
/**
 * @project sqltoy-mssql
 * @author zhongxuchen
 * @version 1.0.0 Table: SQLTOY_STAFF_INFO
 */
// 指定租户字段为 tenantId
@Tenant(field = "tenantId")
@Entity(tableName = "SQLTOY_STAFF_INFO", comment = "", pk_constraint = "PK_SQLTOY_STAFF_INFO")
public class StaffInfoVO implements Serializable {
    // ... 字段定义
}
```

3. 拦截器逻辑与统一字段处理器

拦截器需要配合 **统一字段处理器 (`IUnifyFieldsHandler`)** 使用，以获取当前用户的授权租户信息。

A. 拦截器实现逻辑 (TenantFilterInterceptor.java)

框架会检查实体类是否包含租户字段，并获取授权租户信息进行过滤。

```
public class TenantFilterInterceptor implements SqlInterceptor {

    @Override
    public SqlToyResult decorate(SqlToyContext sqlToyContext, SqlToyConfig sqlToyConfig,
                                 OperateType operateType, SqlToyResult sqlToyResult, Class entityClass, Integer dbType) {
        // 存在统一字段处理、且是对象实体操作
        if (sqlToyContext.getUnifyFieldsHandler() == null || entityClass == null) {
            return sqlToyResult;
        }

        EntityMeta entityMeta = sqlToyContext.getEntityMeta(entityClass);

        // 不存在租户过滤控制
        if (entityMeta.getTenantField() == null) {
            return sqlToyResult;
        }

        // 【可选】你也可以用这种模式判断是否有租户字段，而非 @Tenant 注解模式
        // if (entityMeta.getColumnName("tenantId") != null) { ... }

        // 授权租户信息为空不做过滤
        // 通过统一字段处理器获取当前用户的授权租户
        String[] tenants = sqlToyContext.getUnifyFieldsHandler().authTenants(entityClass, OperateType.QUERY); // 假设是查询操作

        if (tenants == null || tenants.length == 0) {
            return sqlToyResult;
        }

        // ... 后续进行 SQL 改写，追加 and tenant_id in (...)
        return sqlToyResult;
    }
}
```

B. 统一字段处理器实现 (SqlToyUnifyFieldsHandler.java)

需实现 `IUnifyFieldsHandler` 接口，在 `authTenants` 方法中返回当前登录用户的租户 ID。

```
/**
 * @project sqltoy-showcase
 * @description 统一字段赋值范例
 * @author chenrenfei
 * @version id:SqlToyUnifyFieldsHandler.java, Revision:v1.0, Date:2018年1月18日
 */
public class SqlToyUnifyFieldsHandler implements IUnifyFieldsHandler {

    private String defaultUserName = "system-auto";

    /**
     * 获取当前用户的授权租户信息，用于数据操作过程中进行统一的租户过滤，
     * 例如：update xxxx set xx=xx where id=? and tenant_id='S0002'
     */
    @Override
    public String[] authTenants(Class entityClass, OperateType operType) {
        // 注意这里是模拟参数，实际要通过当前用户信息中动态获取
        // 例如从 SecurityContext 或 ThreadLocal 中获取当前登录用户的租户ID
        return new String[] { "S0002" };
    }
}
```

数据权限传参和越权校验

企业系统里"当前用户能看哪些机构/账套/租户/业务线"这类信息，通常是**每次查询都要带、但业务代码不该操心**的。sqltoy 把它拆成两层能力：

能力	解决的痛点	实现位置
统一传参 + 越权校验	前端没传 → 自动填充授权值；前端传了 → 校验是否越权，越权直接抛异常	`IUnifyFieldsHandler.dataAuthFilters()`
按维度自动过滤	连参数都不想让前端传：直接往 SQL 里注入 `tenant_id='x'` 之类的过滤条件	`SqlInterceptor`（内置 `TenantFilterInterceptor`）

一、统一传参 + 越权校验（dataAuthFilters）

1.1 机制

`IUnifyFieldsHandler` 的 `dataAuthFilters()` 返回一张 **"SQL 参数名 → 授权配置"** 的表。框架在每次查询/执行前，对 SQL 里的**同名参数**做判断：

```
public default IgnoreKeyCaseMap<String, DataAuthFilterConfig> dataAuthFilters() {
    return null;   // 默认不启用
}
```

处理逻辑（每个命名参数独立判断）：

前端传值情况	框架行为
参数为空（null/空白），或等于 `choiceAllValue`（"全选"标记）	**自动把授权范围 `values` 填进该参数**（统一传参，业务代码不用管）
参数有值且 `isForcelimit=true`	把传入的每个值与授权集合比对，**任一值越界即抛** `DataAccessException`
参数有值但 `isForcelimit=false`	不做越权校验，以前端传值为准

越权异常形如：

```
param [organId] value [S004] is beyond the authorized scope
(see the implementation configured by spring.sqltoy.unifyFieldsHandler), please check!
```

1.2 DataAuthFilterConfig 属性

属性	说明
`values`	**授权值**：数组、集合或单值，来自当前登录用户的授权范围（机构 id、账套 id、租户 id...）
`choiceAllValue`	"全选"标记值（如页面下拉的"全部"传 `-1`）；传入该值时视为未指定，改用 `values`
`isForcelimit`	是否强制约束（默认 `true`）；`false` 时不做越权校验
`ignoreType`	越权比对时是否忽略类型（如前端传 `String`、授权值是 `BigInteger`）

1.3 实现示例

```
@Component("unifyFieldsHandler")
public class SqlToyUnifyFieldsHandler implements IUnifyFieldsHandler {

    /** 当前登录用户上下文（由过滤器/拦截器写入 ThreadLocal） */
    @Override
    public IgnoreKeyCaseMap<String, DataAuthFilterConfig> dataAuthFilters() {
        IgnoreKeyCaseMap<String, DataAuthFilterConfig> map = new IgnoreKeyCaseMap<>();
        // ① 机构维度：SQL 里的 :organId 参数
        map.put("organId", new DataAuthFilterConfig()
                .setValues(getAuthOrganIds())      // String[]: 授权机构
                .setChoiceAllValue("-1")           // 前端"全部"传 -1 时，改用授权范围
                .setForcelimit(true));             // 越权即抛异常
        // ② 账套维度：授权值是数字类型，前端传字符串
        map.put("accountSetId", new DataAuthFilterConfig()
                .setValues(getAuthAccountSetIds())
                .setIgnoreType(true));
        return map;
    }
}
```

配置（`unifyFieldsHandler` 同时承担公共字段赋值，本机制复用同一个实现）：

```
spring.sqltoy.unifyFieldsHandler=com.yourproject.SqlToyUnifyFieldsHandler
```

1.4 SQL 侧怎么写

参数名与 `dataAuthFilters()` 的 key **一一对应**即可，无需任何额外标签：

```
<sql id="staff_findlist">
    <value><![CDATA[
        select * from sqltoy_staff_info t
        where t.status=1
        #[and t.organ_id in (:organId)]
        #[and t.tenant_id=:tenantId]
    ]]></value>
</sql>
```

- 前端不传 `organId`（或传 `-1`）→ 框架自动填入授权机构；
- 前端传 `organId=S004` 而授权只有 A/B/C → 抛越权异常；
- 传多个值（数组/逗号串经 `<split>` 过滤器）时逐个比对。

1.5 注意事项

- **参数名必须对齐**：这是最容易踩的坑——SQL 里改名了但 `dataAuthFilters()` 没改，机制会静默失效（参数为空 → 只填授权值反而"看起来正常"，越权校验却不生效）；
- **授权值来源必须可信**：`values` 应由服务端从登录态推导，不能来自前端；
- **只对"有该命名参数"的 SQL 生效**：SQL 里没写这个参数，机制不会凭空注入条件（需要"无条件也过滤"请用第二节的拦截器）；
- **分页 count 缓存**：`values` 是分页 count 缓存 key 的组成部分之一，授权范围变化会自然换 key，不会串数；
- 单表实体链路（`findEntity` / `updateByQuery` / `deleteByQuery`）与非实体查询一致生效。

二、按维度自动过滤（TenantFilterInterceptor）

当过滤条件是"所有查询都要带、且前端根本不用传"时，用 SQL 拦截器更彻底：它在最终 SQL 上注入条件，业务 SQL 与参数都不用出现该字段。

2.1 配置

```
# 内置租户/机构过滤拦截器
spring.sqltoy.sqlInterceptors[0]=org.sagacity.sqltoy.plugins.interceptors.TenantFilterInterceptor
# 拦截器依赖统一字段处理器提供授权值
spring.sqltoy.unifyFieldsHandler=com.yourproject.SqlToyUnifyFieldsHandler
```

2.2 授权值从哪来

拦截器向 `IUnifyFieldsHandler` 取两个值：

```
/** 当前用户授权的租户/机构 id 数组；operType 可区分查询/分页/top/random/count 等场景 */
public default String[] authTenants(Class entityClass, OperateType operType) { ... }

/** 单租户简化形态：只返回当前用户的租户 id */
public default String getUserTenantId() { ... }
```

`authTenants` 优先，返回空时回退 `getUserTenantId`；两者都为空则**不做过滤**（fail-open，注意这是"没配就不拦"的设计，不是"拦住所有"）。

> 分页场景建议实现 `authTenants`：分页 count 缓存需要该值参与 key 组成，否则不同租户会命中同一份 count。

2.3 注入规则

- 租户字段来源：实体上的 `@Tenant(field="...")`，或按约定列名（如 `tenantId`）；
- 仅对**有实体类型**的操作生效（对象 CRUD、`findEntity`、`findPageEntity`、`updateByQuery`、`deleteByQuery` 等）；**纯 sqlId 查询没有实体，不会注入**；
- SQL 里已有该租户条件（`where` 段已出现 `tenant_id=` / `tenant_id in`）时**不重复注入**；
- 没有 `where` 的 SQL 会把条件追加到 `group by` / `order by` 之前；
- `saveOrUpdate`（merge）场景注入到 `on` 子句；
- 授权值以**字面量**拼入（框架对单引号做了转义），值必须来自可信来源。

2.4 与多租户的关系

本节的机制同时是多租户隔离的实现方式之一：`@Tenant` + `TenantFilterInterceptor` 统一过滤与赋值。多租户的完整用法（含写入时自动补租户字段）见多租户支持。

三、两种方式怎么选

场景	选择
页面上有"机构/账套"筛选条件，用户可能选可能不选	`dataAuthFilters()`（统一传参 + 越权校验）
数据隔离必须无条件生效（SaaS 租户、数据权限强隔离）	`TenantFilterInterceptor`（自动注入）
既要隔离、又要在前端越权时**报错提示**	`dataAuthFilters()` + `isForcelimit=true`
过滤维度不是实体字段（跨表、复杂表达式）	自定义 `SqlInterceptor`，见 SQL 拦截加工

> 两者可同时使用：拦截器负责"兜底隔离"，`dataAuthFilters` 负责"前端可见条件的填充与越权提示"。

四、相关

- 多租户完整用法：多租户支持
- 自定义 SQL 加工：SQL 拦截加工
- 统一字段处理器接口（含 `createUnifyFields` / `updateUnifyFields`）：增加公共字段处理
- 配置项总览：sqltoy 配置参数

SQL 拦截加工与多租户过滤

随着 SaaS 化以及一些特殊场景诉求，往往需要对项目中的 SQL 做**集中化处理**，如：统一租户过滤、校验 SQL 注入、修改 schema（`select xxx from schema.table` 按需替换实际 schema）等。sqltoy 通过 **SQL 拦截器（SqlInterceptor）** 机制实现。

一、配置

拦截器通过配置参数 `spring.sqltoy.sqlInterceptors`（数组）注册，可配置多个：

```
spring.sqltoy.sqlInterceptors[0]=org.sagacity.sqltoy.plugins.interceptors.TenantFilterInterceptor
spring.sqltoy.sqlInterceptors[1]=com.yourpkg.YourSqlInterceptor
```

二、SqlInterceptor 接口

自定义拦截器实现 `org.sagacity.sqltoy.plugins.SqlInterceptor` 接口，核心是两个默认方法：

```
public interface SqlInterceptor {
    // 在 SQL 执行前对最终 sql 与参数（SqlToyResult）进行加工，返回加工后的结果
    default SqlToyResult decorate(SqlToyContext sqlToyContext, SqlToyConfig sqlToyConfig,
                                  OperateType operateType, SqlToyResult sqlToyResult,
                                  Class entityClass, Integer dbType) {
        return sqlToyResult;
    }
    // 返回实体对象的租户字段名（用于租户过滤，如 saveOrUpdate / merge into on 条件场景）
    default String[] tenantFieldNames(EntityMeta entityMeta, OperateType operateType) {
        // ...
    }
}
```

在 `decorate` 中即可拿到即将执行的 SQL 与参数，进行改写、追加过滤条件、注入校验、schema 替换等处理。

三、内置租户过滤拦截器

框架默认提供 `TenantFilterInterceptor`，针对**对象操作和 `findEntity` 等单表操作**自动追加租户过滤条件。使用它需要：

1. **POJO 上增加 `@Tenant(field="xxx")` 注解**（可选，标注租户字段）：

```
@Tenant(field = "tenantId")
@Entity(tableName = "sqltoy_order_info")
public class OrderInfo implements Serializable {
    private String tenantId;
    // ...
}
```

2. **通过统一字段处理器（`IUnifyFieldsHandler`）提供当前用户的授权租户**，拦截器据此判断 POJO 中是否含租户字段并追加过滤。

参考实现：`org.sagacity.sqltoy.plugins.interceptors.TenantFilterInterceptor`。

> 多租户的完整配置与 `@Tenant`、统一字段处理器、越权过滤详见多租户支持与数据权限传参和越权校验。

数据版本控制（@DataVersion）

> 自 sqltoy 4.19.62 版本开始支持。

一、目的

防止用户在操作数据期间，数据被其他用户修改而发生变化，从而有效提醒当前用户（乐观锁语义）。

二、逻辑

1. 数据**创建**时自动带入一个版本号；
2. **修改**时页面持有当前数据的版本号；
3. 提交修改时**校验版本是否一致**：一致则修改数据并将版本号 +1；不一致则抛出异常，提示数据已被他人变更。

三、使用说明

在 POJO 类上增加 `@DataVersion` 注解，标记该对象需要版本控制：

```
@DataVersion(field = "dataVersion", startDate = false)
@Entity(tableName = "sqltoy_order_info", pk_constraint = "PRIMARY")
public class OrderInfo implements Serializable {
    @Id(...)
    private String orderId;
    // 版本号字段（field 指向该属性名）
    private Long dataVersion;
    // ... 其他字段
}
```

`@DataVersion` 属性：

属性	默认	说明
`field`	`""`	版本号字段名（对象属性名）
`startDate`	`false`	版本号是否以日期开头，如 `202209231`（`true` 时便于按日期区分版本批次）

修改数据时要带入当前版本号：前端/调用方在提交更新时必须把查询时拿到的 `dataVersion` 一并传回，框架据此做版本校验。

```
// 查询时拿到 dataVersion
OrderInfo order = lightDao.load(queryVO);
// 修改后提交，需保留原 dataVersion 用于校验
order.setStatus(2);
lightDao.update(order);   // 版本一致则更新并 +1，不一致抛异常
```

> 数据版本控制与弹性更新、数据权限等企业级特性可叠加使用。

数据脱敏和加解密

数据脱敏

- sqltoy脱敏包含tel\name\id-card\bank-card\address\email\public-account 几种固定类型
- 针对相对固定长度的字符脱敏可通过head-size、tail-size、mask-code自行定义脱敏规则
- 针对不固定长度可通过脱敏比例mask-rate设置比例，如：mask-rate="50"
- 也可以自行定义脱敏处理函数

```
# 实现org.sagacity.sqltoy.plugins.secure.DesensitizeProvider
# 默认实现类:org.sagacity.sqltoy.plugins.secure.impl.DesensitizeDefaultProvider
spring.sqltoy.desensitizeProvider=your.DesensitizeProvider
```

- xml 用法

```
<sql id="sys_findStaff_info">
    <!-- 安全掩码:tel\name\id-card\bank-card\address\email\public-account\other -->
    <!--最简单用法: <secure-mask columns="mobile_tel" type="tel"/> -->
    <secure-mask type="tel" columns="mobile_tel" head-size="3"
        tail-size="4" mask-code="*****" />
    <value>
        <![CDATA[
        select staff_code,staff_name,address,mobile_tel,id_card,blank_type,blank_code
        from sys_staff_info
        where staff_code=:staffCode
        ]]>
    </value>
</sql>
```

- java代码中用法

```
String sql = """
          select staff_code,staff_name,address,mobile_tel,id_card,blank_type,blank_code
          from sys_staff_info
          where staff_code=:staffCode
                """;
List result = lightDao
        .findByQuery(
                new QueryExecutor(sql).secureMask(MaskType.TEL, "mobile_tel").secureMask(MaskType.ID, "id_card")
                        .values(MapKit.keys("staffCode").values("S0001")).resultType(StaffInfoVO.class))
        .getRows();
```

字段加密解密

- sqltoy支持对字段进行加密存储，查询时可动态解密
- sqltoy 通过一对 RSA 公私钥实现字段加解密（公钥加密、私钥解密）。密钥格式为**去掉 PEM 头尾和换行的单行 Base64**（私钥 PKCS#8、公钥 X.509），用 openssl 一条命令即可生成（Git Bash / Linux / macOS 自带 openssl，Windows 装了 Git 即有）：

```
# 生成 1024 位私钥(PKCS#8)并导出配对公钥(X.509)，去头尾压成单行 Base64
openssl genrsa 1024 2>/dev/null | openssl pkcs8 -topk8 -nocrypt > private.pem
openssl rsa -in private.pem -pubout > public.pem
grep -v '^-' private.pem | tr -d '\n' > rsa_private.key && echo >> rsa_private.key
grep -v '^-' public.pem  | tr -d '\n' > rsa_public.key  && echo >> rsa_public.key
```

> 也可以用纯 Java 生成（JDK 自带 JCE，无需任何依赖）——sqltoy 源码 `src/test` 路径下的 `org.sagacity.sqltoy.utils.SecureUtils` 就是这段逻辑的封装：

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.util.Base64;

public class GenerateRsaKeys {
    public static void main(String[] args) throws Exception {
        KeyPairGenerator generator = KeyPairGenerator.getInstance("RSA");
        generator.initialize(1024); // 2048 for production; widen the encrypted column accordingly
        KeyPair keyPair = generator.generateKeyPair();
        // single-line Base64 without PEM headers — the format sqltoy expects
        // (private key PKCS#8, public key X.509); print two lines and save each to its own file
        System.out.println(Base64.getEncoder().encodeToString(keyPair.getPrivate().getEncoded()));
        System.out.println(Base64.getEncoder().encodeToString(keyPair.getPublic().getEncoded()));
    }
}
```

1024 位密文的单字段密文约 172 字符，加密列建议 `varchar(500)`；生产环境可按安全要求改用更长位数（列宽同步加大）。

```
# 可以自定义加解密算法实现，框架提供了默认实现
# spring.sqltoy.fieldsSecureProvider=org.sagacity.sqltoy.plugins.secure.impl.FieldsRSASecureProvider
spring.sqltoy.securePrivateKey=classpath:mock/rsa_private.key
spring.sqltoy.securePublicKey=classpath:mock/rsa_public.key
```

传统 Spring XML 模式在 `SqlToyContext` bean 上配置同名属性：

```
<property name="securePrivateKey" value="classpath:mock/rsa_private.key" />
<property name="securePublicKey" value="classpath:mock/rsa_public.key" />
```

> 🎬 完整可运行示例：演示项目 `sqltoy-showcase` 的 `SecureEncryptTest.java`（加密存储/自动解密/检索码）与 `SecureMaskTest.java`（先解密再脱敏）；测试资源已内置 `mock/rsa_private.key`、`rsa_public.key` 演示密钥。

- 在pojo对象类上增加注解@SecureConfig,为了便于检索，可以额外增加一个脱敏值，注意sourceField的指向作用

```
@Data
@Accessors(chain = true)
@Entity(tableName="sqltoy_secure_case",comment="安全加解密演示",pk_constraint="PRIMARY")
@SecureConfig(secures = { @Secure(field = "telNoMask", secureType = SecureType.TEL),
        @Secure(field = "telNo", secureType = SecureType.ENCRYPT),
        @Secure(field = "telNoMask", secureType = SecureType.TEL,sourceField="telNo"),
        @Secure(field = "homeAddressMask", secureType = SecureType.DISCRETE_RATE, maskRate = 50, sourceField = "homeAddress"),
        @Secure(field = "homeAddress", secureType = SecureType.ENCRYPT) })
public class SecureCaseVO implements Serializable {
    @Schema(name="staffId",description="工号",nullable=false)
    @Id(strategy="generator",generator="org.sagacity.sqltoy.plugins.id.impl.DefaultIdGenerator")
    @Column(name="STAFF_ID",comment="工号",length=22L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String staffId;

    @Schema(name="staffName",description="姓名",nullable=false)
    @Column(name="STAFF_NAME",comment="姓名",length=30L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String staffName;

    @Schema(name="telNo",description="移动电话",nullable=false)
    @Column(name="TEL_NO",comment="移动电话",length=500L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String telNo;

    @Schema(name="telNoMask",description="电话检索",nullable=false)
    @Column(name="TEL_NO_MASK",comment="电话检索",length=30L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String telNoMask;

    @Schema(name="homeAddress",description="家庭地址",nullable=false)
    @Column(name="HOME_ADDRESS",comment="家庭地址",length=500L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String homeAddress;

    @Schema(name="homeAddressMask",description="家庭地址检索",nullable=false)
    @Column(name="HOME_ADDRESS_MASK",comment="家庭地址检索",length=100L,type=java.sql.Types.VARCHAR,nativeType="VARCHAR",nullable=false)
    private String homeAddressMask;

    ......
}
```

- 调用对象save数据库字段中就会存储加密和脱敏的值
- 基于对象load则会自动将数值解密
- 基于sql语句查询解密

```
List result = lightDao.findByQuery(new QueryExecutor(sql).secureDecrypt("address")
                .values(MapKit.keys("staffCode").values("S0001")).resultType(StaffInfoVO.class)).getRows();
```

- xml中查询解密:secure-decrypt

```
<sql id="qstart_secure_decrypt">
    <!-- 解密配置 -->
    <secure-decrypt columns="tel_no,home_address" />
    <value>
        <![CDATA[
            select * from sqltoy_secure_case
        ]]>
    </value>
</sql>
```

分库分表

> sqltoy 支持分库分表，但随着 Doris、StarRocks 等分布式高性能数据库的普及，水平分库分表的意义在淡化。**真正长期有效的用法是"按日期把实时表和历史表分离"**（近期数据在实时表、历史数据归档到历史表），以及多库按 hash 水平拆分。本章给出完整的策略配置、注解/XML/代码三种声明方式与已知边界。

一、两种分片维度

维度	决定的元素	声明位置
分库（datasource）	同一条 SQL 落到哪个数据源执行	`<sharding-datasource>` / `QueryExecutor.dbSharding(...)` / `@Sharding(db=...)`
分表（table）	SQL 中出现的物理表名替换成哪个表	`<sharding-table tables="..." />` / `QueryExecutor.tableSharding(...)` / `@Sharding(table=...)`

两者可单独使用，也可同时使用（一个 SQL 中同时分库 + 多个分表，策略各自独立定义）。**查询**用 XML/代码声明，**增删改**用实体注解 `@Sharding` 声明。

二、分片策略

sqltoy 提供两个开箱即用的实现，也可以自行扩展。

2.1 策略接口

```
public interface ShardingStrategy {
    /** 根据条件决定当前 sql 中的表名；返回 null 表示使用原表 */
    String getShardingTable(SqlToyContext sqlToyContext, Class entityClass, String baseTableName,
            String decisionType, IgnoreCaseLinkedMap<String, Object> paramsMap);

    /** 根据条件决定最终执行的库；返回空模型表示使用当前默认数据源 */
    ShardingDBModel getShardingDB(SqlToyContext sqlToyContext, Class entityClass, String tableOrSql,
            String decisionType, IgnoreCaseLinkedMap<String, Object> paramsMap);

    /** bean 初始化（initMethod），用于校验配置、启动探活线程 */
    void initialize();
}
```

`paramsMap` 是策略 `fields` 所声明字段的值映射；`decisionType` 让策略区分场景（查询/更新等）。

2.2 HashShardingStrategy（按 hash 取模，core 内置）

属性	说明
`setDataSourceMap(Map<String,String>)`	取模结果 → 数据源名，key 为 `"0"`、`"1"`…
`setTableMap(Map<String,String>)`	取模结果 → 物理表名，key 同上

取模用 `Math.floorMod(hashCode, N)`（对负数安全，避免落回基准表），`N` 为映射条目数。

```
@Bean(name = "hashDataSource", initMethod = "initialize")
public HashShardingStrategy hashDataSource() {
    HashShardingStrategy strategy = new HashShardingStrategy();
    HashMap<String, String> dataSourceMap = new HashMap<>();
    dataSourceMap.put("0", "ds0");
    dataSourceMap.put("1", "ds1");
    strategy.setDataSourceMap(dataSourceMap);
    return strategy;
}
```

> `initialize()` 会按映射条目数确定模数；**分片值为 null 时抛异常并指明字段**，常见诱因是查询条件参数别名与 `@Strategy.fields` 不一致或分片字段本身为空。

2.3 DefaultShardingStrategy（按日期分表 + 按权重分库）

该类位于 **spring / spring-starter 模块**（包名不变：`org.sagacity.sqltoy.plugins.sharding.impl`），core 仅保留 `HashShardingStrategy`；纯 Java 环境需引入 spring 模块或自行实现。

属性	说明
`setDays("14")` / `setDays("1,15,90")`	**距今天数阈值**（升序），与 `setTableNamesMap` 的表名数组一一对应
`setTableNamesMap(Map)`	`基准表名 → "表1,表2,表3"`（逗号分隔，按与 `days` 相同的顺序排列）
`setDataSourceWeight(Map<String,Integer>)`	`数据源名 → 权重`，按权重概率路由（配置多个时才生效）
`setCheckSeconds(int)`	数据源探活间隔（秒）
`setDateParams("bizDate")`	参与计算间隔的业务日期参数字段名（多个时依次取第一个非空值）

按日期分表的匹配规则（本节最容易踩坑，务必对齐）：

- 计算 `intervalDays = |今天 - 业务日期|`；
- 按 `days` 的书写顺序，取**第一个** `intervalDays >= days[i]` 的 `i`；
- 命中则用 `tables[i]`（表名数组按同一顺序对齐）；**全部不中**（即间隔小于最小阈值）→ 返回 null，使用 SQL 中写的**原表**；
- 因此"原表"承担的是"最近一档"，`tables` 里**不要再放原表本身**，否则各档位会整体错位（顺序命中导致永远只取 `tables[0]`）。

```
// 正确示例：基准表(实时表) 15 天窗口，超过 14 天走历史表
@Bean(name = "realHisTable", initMethod = "initialize")
public DefaultShardingStrategy realHisTable() {
    DefaultShardingStrategy strategy = new DefaultShardingStrategy();
    strategy.setDays("14");
    HashMap<String, String> tableMap = new HashMap<>();
    tableMap.put("SQLTOY_TRANS_LOG_15D", "SQLTOY_TRANS_LOG_HIS");
    strategy.setTableNamesMap(tableMap);
    return strategy;
}

// 三档示例：原表=当天表：1~14 天 → _15 表：15~89 天 → _90 表：90 天以上 → _HIS
@Bean(name = "threeTierTable", initMethod = "initialize")
public DefaultShardingStrategy threeTierTable() {
    DefaultShardingStrategy strategy = new DefaultShardingStrategy();
    strategy.setDays("1,15,90");                        // 阈值升序
    HashMap<String, String> tableMap = new HashMap<>();
    tableMap.put("SQLTOY_TRANS_INFO",                   // 原表(当天)
            "SQLTOY_TRANS_INFO_15,SQLTOY_TRANS_INFO_90,SQLTOY_TRANS_INFO_HIS");
    strategy.setTableNamesMap(tableMap);
    return strategy;
}
```

按权重分库 + 自动探活：配置 `setDataSourceWeight` 后，`initialize()` 会启动 `IdleConnectionMonitor`（守护线程）：

- 启动延迟 60 秒后开始，按 `checkSeconds` 周期检查每个数据源的连通性；
- 某库连不上时**把它当前权重临时置 0 并打 error 日志**，后续请求不再按概率路由到它；恢复后权重复原；
- 只有一个数据源时不做概率选择，直接使用它。

```
@Bean(name = "weightDataSource", initMethod = "initialize")
public DefaultShardingStrategy weightDataSource() {
    DefaultShardingStrategy strategy = new DefaultShardingStrategy();
    HashMap<String, Integer> weight = new HashMap<>();
    weight.put("ds1", 70);   // 70% 概率落到 ds1
    weight.put("ds2", 30);
    strategy.setDataSourceWeight(weight);
    strategy.setCheckSeconds(120);   // 每 2 分钟探活一次
    return strategy;
}
```

> 权重是"按请求概率路由"，不具备跨库事务一致性；写多读少的强一致场景请评估是否真的需要按权重分库。

三、声明方式一：实体注解（增删改）

在 POJO 上加 `@Sharding`，之后 `lightDao.save/update/delete/load` 会自动路由：

```
@Data
@Accessors(chain = true)
@Entity(tableName = "SQLTOY_TRANS_INFO", pk_constraint = "PRIMARY")
@Sharding(db = @Strategy(name = "hashDataSource", fields = { "tenantId" }),
          table = @Strategy(name = "realHisTable", fields = { "bizDate" }))
public class SqlToyTransInfo implements Serializable {
    @Id
    @Column(name = "ID", type = java.sql.Types.VARCHAR, length = 50, nullable = false)
    private String id;

    @Column(name = "TENANT_ID", type = java.sql.Types.VARCHAR, length = 50, nullable = false)
    private String tenantId;

    @Column(name = "BIZ_DATE", type = java.sql.Types.DATE, nullable = false)
    private LocalDate bizDate;
}
```

只分表时可省略 `db`：

```
@Sharding(table = @Strategy(name = "realHisTable", fields = { "bizDate" }))
```

> `@Sharding` 的属性完整说明（含 `maxConcurrents`、`maxWaitSeconds`、`is_global_rollback`）见注解速查 · @Sharding。

四、声明方式二：XML（查询）

```
<sql id="sys_queryTransInfo">
    <!-- 按租户 id 取模决定数据源 -->
    <sharding-datasource strategy="hashDataSource" params="tenantId" />
    <!-- 按日期决定查询哪张表 -->
    <sharding-table tables="SQLTOY_TRANS_INFO" strategy="realHisTable" params="bizDate" />
    <value><![CDATA[
        select * from SQLTOY_TRANS_INFO
        where tenant_id=:tenantId and biz_date<=:bizDate
    ]]></value>
</sql>
```

- `params` 与策略 `@Strategy.fields` 对应，是**取值来源的参数名**；
- `tables` 是该策略可能替换的物理表（与原 SQL 中的表名一致）。

五、声明方式三：代码（QueryExecutor / EntityQuery）

```
lightDao.findByQuery(new QueryExecutor("""
        select * from SQLTOY_TRANS_INFO where tenant_id=:tenantId and biz_date<=:bizDate
        """)
        .values(MapKit.keys("tenantId", "bizDate").values("S001", LocalDate.now().minusDays(2)))
        .dbSharding("hashDataSource", "tenantId")
        .tableSharding("realHisTable", new String[] { "SQLTOY_TRANS_INFO" }, "bizDate"));
```

`EntityQuery` 同样提供 `dbSharding` / `tableSharding`，适用于 `findEntity` / `updateByQuery` / `deleteByQuery` 等单表链路。

六、并行执行、全局回滚与异常

- `@Sharding.maxConcurrents`（默认 10）控制多分片并行执行线程数：**0 表示不限制、1 表示串行**（6.0.3 修正了 1 不生效的缺陷）；
- `maxWaitSeconds` 控制并行等待上限；
- 开启 `is_global_rollback`（默认 true）时，任一分片失败即回滚全部；
- 6.0.3 起 `ShardingResult` 新增 `cause` 字段保留失败分片的原始异常，全局回滚抛出的 `DataAccessException` 携带根因堆栈，便于定位。

七、已知边界

- **不支持跨库的 join 查询与分页**：多库多表查询后的分页归并成本极高，属于设计问题而非实现问题；这类需求建议用 ClickHouse/StarRocks 等 MPP 数据库（sqltoy 均支持），详见 FAQ；
- **分库后的事务**：跨库写需要 JTA 或分布式事务框架配合，代码示例见 多数据源与事务；
- **分片表上的 `@DataVersion` /缓存翻译**：分表替换发生在 SQL 层，实体级增强照常生效；但**分库**后单条 SQL 只在其中一个库执行，聚合类统计需要自行汇总；
- **策略 bean 必须实现 `initialize()` 并配 `initMethod="initialize"`**：`DefaultShardingStrategy` 的权重探活、`HashShardingStrategy` 的模数都在该方法里初始化；native 镜像下 `@Bean` 请声明**具体返回类型**（接口声明会导致 initMethod 无法解析），参见 GraalVM Native Image 指引。

八、相关

- 多数据源配置与事务：多数据源与事务
- 注解属性：注解速查
- 演示项目：sqltoy-showcase（`DataSourceConfig` 含 hash 分库与日期分表的完整配置）

类型扩展处理typeHandler

设置针对json等特殊类型的处理类，实现sqltoy的抽象类: org.sagacity.sqltoy.plugins.TypeHandler

1、配置

```
# Spring Boot（前缀 spring.sqltoy）
spring:
  sqltoy:
    typeHandler: com.example.sqltoy_solon_demo.handler.MyTypeHandler

# Solon（前缀 solon.sqltoy）
solon:
  sqltoy:
    typeHandler: com.example.sqltoy_solon_demo.handler.MyTypeHandler
```

2、实现类

新建一个MyTypeHandler类继承org.sagacity.sqltoy.plugins.TypeHandler

```
public class MyTypeHandler extends TypeHandler {
    @Override
    public boolean setValue(Integer integer, PreparedStatement ps, int parameterIndex, int i1, Object o) throws SQLException {
        if (parameterIndex == 1) {
            ps.setString(parameterIndex, "localhost");
            System.out.printf("setValue: %s ==> %s%n", o, "localhost");
            return true;
        }
        return false;
    }
}
```

3、JSONTypeHandler示例

- 备注:5.6.82版本框架提供了默认的json/jsonb类型支持，quickvo升级1.0.22版本，jdbc-type=2016(json)、2017(jsonb)

3.1 quickvo 生成POJO配置类型映射关系

```
<type-mapping>
        <!-- 保留1个范例,一般无需配置 -->
        <!-- 增加雪花算法的演示 -->
        <sql-type native-types="BIGINT" jdbc-type="BIGINT" java-type="java.math.BigInteger" />

        <!-- 泛型注意xml转义符号，table-field指定具体表和字段; jdbc-type 可以直接填数字，这里java-type="List<StaffInfoVO>" -->
        <sql-type table-field="sqltoy_jsontype_showcae.staff_set" native-types="json" jdbc-type="2016" java-type="List&lt;StaffInfoVO&gt;"
        import-types="com.sqltoy.quickstart.vo.StaffInfoVO" />
</type-mapping>
```

3.2 定义JsonTypeHandler

```
public class JsonTypeHandler extends TypeHandler {
    // 根据情况对null进行设置,postgresql这个环节是必须要的
    @Override
    public boolean setNull(Integer dbType, PreparedStatement pst, int paramIndex, int jdbcType) throws SQLException {
        return false;
    }

    /**
     * <li>返回true表示类型匹配上，并完成了setValue赋值</li>
     * <li>返回false 表示常规类型,交回框架自行处理</li>
     */
    @Override
    public boolean setValue(Integer dbType, PreparedStatement pst, int paramIndex, int jdbcType, Object value)
            throws SQLException {
        // 通过quickvo.xml 中jdbc-type 里面直接设置int数字,来区分特定的类型
        if (jdbcType == 1021) {
            pst.setString(paramIndex, JSONObject.toJSONString(value));
            return true;
        }
        return false;
    }

    /*
     * <li>1、返回null表示属于常规类型，交回框架完成处理</li>
     * <li>2、返回非null,表示特殊类型，完成了类型转换可直接映射到VO属性</li>
     */
    @Override
    public Object toJavaType(String javaTypeName, Class genericType, Object jdbcValue) throws Exception {
        if (javaTypeName.equalsIgnoreCase("java.lang.string")) {
            return jdbcValue.toString();
        }
        // 是一个VO对象
        if (javaTypeName.contains("com.sqltoy")) {
            return JSON.parseObject(jdbcValue.toString(), Class.forName(javaTypeName));
        }
        if (javaTypeName.toLowerCase().contains("jsonobject")) {
            return JSON.parse(jdbcValue.toString());
        } else if (javaTypeName.toLowerCase().contains("jsonarray")) {
            return JSONArray.parseArray(jdbcValue.toString());
        }
        // List<VO>形式
        if (javaTypeName.equalsIgnoreCase("java.util.List") && genericType != null) {
            return JSONArray.parseArray(jdbcValue.toString(), genericType);
        }
        // 其他场景表示非json返回null交框架自行处理
        return null;
    }
}
```

慢 SQL 处理

sqltoy 内置慢 SQL（超时 SQL）收集与处理机制，便于定位性能瓶颈。

一、默认配置

框架默认提供 `DefaultOverTimeHandler`：

```
spring.sqltoy.overTimeSqlHandler=org.sagacity.sqltoy.plugins.overtime.DefaultOverTimeHandler
```

配合配置参数 `printSqlTimeoutMillis`（默认 8000 毫秒），当 SQL 执行超过该阈值时会输出告警日志。

二、代码中获取慢 SQL

通过 `SqlToyContext` 获取最慢的若干条 SQL：

```
// 获取最慢的前 10 条 SQL（第二个参数 hasSqlId：是否只取 xml 中定义的含 id 的 sql）
List slowest = lightDao.getSqlToyContext().getSlowestSql(10, true);
```

三、扩展慢 SQL 收集

如需自定义收集逻辑（如上报到监控系统、写入数据库、对接 APM），自行实现 `OverTimeSqlHandler` 接口，并配置：

```
spring.sqltoy.overTimeSqlHandler=com.yourpkg.YourOverTimeHandler
```

在实现类中可拿到超时 SQL 的语句、参数、耗时等信息，按需要落库、告警或上报。

> 相关：SQL 执行统计还可参考 `org.sagacity.sqltoy.SqlExecuteStat`、`SqlExecuteTrace`、`SqlExecuteLog`。常见 SQL 优化写法见常见 SQL 案例。

MongoDB 支持

说明：sqltoy 集成 MongoDB **只支持查询**，纯 CRUD（增删改）请直接使用 spring-data 的 `MongoTemplate`。sqltoy 集成 mongo 的出发点是：

1. 充分发挥 sqltoy **动态条件组织**的优势；
2. 结合 sqltoy 的**缓存翻译**，解决 mongo 关联配置表的复杂性（或 ETL 清洗的复杂性）；
3. 利用 sqltoy 提供的**结果集算法**（脱敏、格式化、旋转、分组汇总等），简化直接写查询/聚合的复杂度；
4. 便于与外部低代码框架（如报表平台）集成，实现配置化数据查询与处理。

> 演示项目：https://github.com/sagframe/sqltoy-showcase

一、配置

sqltoy 复用 spring-data-mongodb 的 `MongoTemplate`，按 Spring Boot 标准方式配置即可：

```
spring:
  data:
    mongodb:
      host: 192.168.56.101
      port: 27017
      database: sagframe
      username: sagframe
      password: sagframe
```

> **Spring Boot 4 迁移注意**：Boot 4 起连接属性从 `spring.data.mongodb.*` 迁移为 `spring.mongodb.*`（如 `spring.mongodb.uri`）；跨版本项目建议两者同时设置。
>
> 完整可运行示例：`sqltoy-showcase` 的 `NoSqlTest`（Testcontainers 起真实 MongoDB 容器，`<mql>` 动态 json 查询、`$in` 数组条件、`findPage` 分页实测）。

对象直接操作（`@Document(collection="")`、`@Id`、`@Field` 注解 + `MongoTemplate`）属于 spring-data 范畴，这里不展开；sqltoy 主要负责**查询 + 缓存翻译 + 结果集算法**。

二、编写 mql 查询

Mongo 查询用 `<mql>` 标签定义，`value` 中写 mongo 的 JSON 查询/聚合语句，动态条件用 `<#> ... </#>` 包裹、参数用 `@(:paramName)` 引用。

2.1 基本查询（字段别名 + 缓存翻译 + 脱敏）

```
<!-- fields 中 transType:transTypeName 表示把 transType 的值另起别名 transTypeName 存放，便于缓存翻译映射 -->
<mql id="sqltoy_mongo_find"
     fields="transId,merchantCode,transCode,transCode:transCodeName,transDate,agentOrg,transType,transType:transTypeName,transAmt"
     collection="fact_trans_details">
    <!-- 对别名列进行翻译，original-columns 指向作为代码来源的列 -->
    <!-- mongo 不能像 sql 那样 select trans_type as transTypeName，故用 fields 别名 + original-columns 实现 -->
    <translate cache="dictKeyName" cache-type="TRANS_CODE"
               columns="transCodeName" original-columns="transCode" />
    <translate cache="dictKeyName" cache-type="TRANS_TYPE"
               columns="transTypeName" original-columns="transType" />
    <!-- 安全脱敏 -->
    <secure-mask columns="merchantCode" type="bank-card" />
    <value><![CDATA[
        {
        <#>transType:@(:transType)</#>
        <#>@if(@(:flag)==1),transAmt:{$gt:@(:transAmt)}</#>
        <#>@else ,transAmt:{$gte:@(:transAmt)}</#>
        }
    ]]></value>
</mql>
```

要点：

- **字段别名**：mongo 无法像 SQL 那样 `select trans_type as transTypeName`，sqltoy 用 `fields` 中的 `源字段:别名` 语法把同一字段值复制到别名列，再用 `<translate original-columns="源字段" columns="别名">` 对别名列做缓存翻译，原代码列保持不变。
- **动态条件**：`<#>...</#>` 内参数为 null 时整段剔除；可配合 `@if()/@else` 做分支（见补充说明：标签与表达式参考）。
- **脱敏**：`<secure-mask>` 与 SQL 查询用法一致。

2.2 聚合查询

```
<!-- 聚合时 fields 可用 _id.xxx:别名 提取嵌套字段，如 group 多字段：fields="_id[transCode,colName:aliasName],count,totalAmt" -->
<mql id="sqltoy_mongo_agg"
     fields="_id.transCode:transCode,count,totalAmt"
     collection="fact_trans_details">
    <translate cache="dictKeyName" cache-type="TRANS_CODE" columns="transCode" />
    <value><![CDATA[
        [
         { $group: {
              _id: {transCode:"$transCode"},
              count: { $sum: 1 },
              totalAmt:{$sum:"$transAmt"}
           }
         },
         { $match: {
              <#>transType:@(:transType)</#>
              <#>,transAmt:{$gt:@(:transAmt)}</#>
           }
         }
        ]
    ]]></value>
</mql>
```

`value` 为聚合管道数组（`$group` / `$match` / `$sort` 等），`fields` 用 `_id.transCode:transCode` 把聚合结果中的嵌套 `_id.transCode` 提取为 `transCode` 列，再对其做缓存翻译。

2.3 mql 标签属性

属性	必填	说明
`id`	是	查询唯一标识
`collection`	是	mongo 集合名
`fields`	是	需要提取/映射的字段，支持 `源:别名`、`_id.xxx:别名`、`_id[a,b:别名]` 等写法
`aggregate`	否	是否聚合查询，默认 `false`
`blank-to-null`	否	空白参数转 null

`<mql>` 内同样支持 `filters`、`translate`、`secure-mask`、`pivot` / `unpivot` / `summary`、`page-optimize` 等子标签。

三、缓存翻译定义

上例中的 `dictKeyName` 缓存在 `sqltoy-translate.xml` 中定义（带内部分类 `cache-type`，即 `DICT_TYPE`）：

```
<sagacity xmlns="http://www.sagframe.com/schema/sqltoy-translate">
    <cache-translates>
        <!-- 基于 sql 查询获取缓存；查询结果第一列为分类(DICT_TYPE) -->
        <sql-translate cache="dictKeyName" datasource="dataSource">
            <sql><![CDATA[
                select t.DICT_KEY,t.DICT_NAME,t.STATUS
                from SQLTOY_DICT_DETAIL t
                where t.DICT_TYPE=:dictType
                order by t.SHOW_INDEX
            ]]></sql>
        </sql-translate>
    </cache-translates>
    <cache-update-checkers>
        <!-- 增量刷新检测，has-inside-group=true 表示结果第一列是分类 -->
        <sql-increment-checker cache="dictKeyName" check-frequency="15"
                               has-inside-group="true" datasource="dataSource">
            <sql><![CDATA[
                select t.DICT_TYPE,t.DICT_KEY,t.DICT_NAME,t.STATUS
                from SQLTOY_DICT_DETAIL t
                where t.UPDATE_TIME >=:lastUpdateTime
            ]]></sql>
        </sql-increment-checker>
    </cache-update-checkers>
</sagacity>
```

缓存翻译的完整用法见缓存翻译使用。

四、Java 调用

通过 `lightDao.mongo()` 链式调用（`sql` 传 mql 的 id）：

```
// 通过对象传参查询
PospTransDetailVO queryVO = new PospTransDetailVO();
queryVO.setTransType("N");
queryVO.setTransAmt(0d);
List<PospTransDetailVO> result = (List<PospTransDetailVO>) lightDao.mongo()
        .sql("sqltoy_mongo_find")
        .resultType(PospTransDetailVO.class)
        .entity(queryVO)
        .find();

// 通过 names/values 传参 + 分页
Page page = lightDao.mongo()
        .sql("sqltoy_mongo_find")
        .resultType(PospTransDetailVO.class)
        .names("transType", "transAmt", "flag").values("N", 13800, 2)
        .findPage(new Page<>());

// 聚合查询
List aggResult = lightDao.mongo()
        .sql("sqltoy_mongo_agg")
        .names("transType", "transAmt").values(null, null)
        .find();
```

`mongo()` 链式可设置：`sql`、`names`、`values`、`entity`、`resultType`、`humpMapLabel`；终止方法：`find()`、`getOne()`、`findTop(Float)`（大于 1 取固定条数，**小于等于 1 按总记录数比例提取**）、`findPage(Page)`。

五、注意事项

- sqltoy 对 mongo **只做查询**，增删改请用 `MongoTemplate`（对象注解 `@Document` / `@Id` / `@Field`）。
- 缓存翻译配合 `fields` 别名 + `original-columns`，可在不写关联查询的情况下完成码值翻译。
- 聚合结果通过 `fields` 的 `_id.xxx:别名` 提取嵌套字段后再翻译/处理。

Elasticsearch 支持

sqltoy 集成 Elasticsearch 的出发点（同样适用于 MongoDB）：

1. 充分发挥 sqltoy **动态条件组织**的优势；
2. 结合 sqltoy 的**缓存翻译**，解决 ES 关联配置表的复杂性（或 ETL 清洗的复杂性）；
3. 利用 sqltoy 提供的**结果集算法**（脱敏、格式化、旋转、分组汇总等），简化直接写查询/聚合的复杂度；
4. 便于与外部低代码框架（如报表平台）集成，实现配置化数据查询与处理。

> 演示项目：https://github.com/sagframe/sqltoy-showcase

一、配置 ES 节点

基于 spring-boot-starter，在 `application.yml` 中配置 `spring.sqltoy.elastic`（支持多节点/多集群）：

```yaml
spring:
  sqltoy:
    elastic:
      # 默认节点 id（不设置则取第一个）
      defaultId: default
      endpoints[0]:
        id: default
        # 集群多个节点用逗号分隔
        url: http://192.168.56.101:9200
        username: elastic
        password: skyline
        # sql 查询路径，见下方说明
        sqlPath: _nlpcn/sql
        # ===== 以下为 https/证书场景的可选项 =====
        #keyStore: 证书文件路径
        #keyStorePass: 证书密码
        #keyStoreType: jks          # 默认 jks，可不设置
        #keyStoreSelfSign: true     # 是否自签名证书，默认 true
        #authCaching: true          # 是否禁止抢占式身份认证，默认 true
```

`sqlPath`（SQL 模式的查询端点）可选值：

sqlPath	说明
`_sql`（即 `_xpack/sql`）	ES 原生 SQL，**已支持 cursor 游标模式分页**
`_nlpcn/sql`	elasticsearch-sql 插件（推荐，支持分页）
`_opendistro/_sql`	AWS opendistro SQL 插件

> ES 原生 SQL 早期不支持分页，**现已支持通过 cursor 游标模式分页**，可直接使用 `_sql`。elasticsearch-sql（`_nlpcn/sql`）、opendistro（`_opendistro/_sql`）插件仍可作为备选；其中 elasticsearch-sql 在 7.9.3 之后停止维护，可改用 AWS opendistro 插件。

二、编写 eql 查询

ES 查询用 `<eql>` 标签定义，支持两种模式：`mode="sql"`（SQL 模式）与原生 JSON REST 模式（不写 mode）。

2.1 SQL 模式

```
<eql id="es_find_company" fields="company_id,company_name,company_type" mode="sql">
    <value><![CDATA[
        select * from cc_company_info
        where 1=1
           #[@if(:flag==1) and company_name=:companyName]
           #[@else  and company_type<>:companyType ]
    ]]></value>
</eql>

<!-- 分页查询（ES 原生 SQL 已支持 cursor 游标分页，也可用 elasticsearch-sql 插件） -->
<eql id="es_find_company_page" fields="company_id,company_name,company_type" mode="sql">
    <value><![CDATA[
        select * from cc_company_info
        where company_name like :companyName and company_type<>:companyType
    ]]></value>
</eql>
```

2.2 原生 JSON REST 模式

不写 `mode` 即为原生 JSON 模式。此时动态条件用 `<#> ... </#>` 包裹（替代 SQL 模式的 `#[]`），参数用 `@(:paramName)` 引用：

```
<!-- fields 属性 与 查询体中的 _source 二选一:两者都没有时结果映射报错
     (collection or property name array is empty) -->
<eql id="sys_elastic_test_json" fields="company_id,company_name,company_type" index="cc_company_info">
    <!-- 依然可以使用 translate 缓存翻译，column 对应 _source 中定义的字段 -->
    <value><![CDATA[
        {
            "_source": ["company_id","company_name","company_type"],
            "query": {
                "bool": {
                    "filter": [
                        <#>{"terms":{"company_type":@(:companyTypes)}}</#>
                    ]
                }
            }
        }
    ]]></value>
</eql>
```

2.3 eql 标签属性

属性	必填	说明
`id`	是	查询唯一标识
`fields`	条件必填	最终 ES 返回 JSON 中需要提取的属性名。**与查询体中的 `_source` 二选一**（实测验证）：两者都不提供时触发 `collection or property name array is empty`（属性名数组为空）异常
`mode`	否	`sql`（SQL 模式）或 `original`（原生 JSON），缺省为原生 JSON
`index`	否	索引名（JSON 模式常用）
`end-point`	否	对应配置中的 endpoint id，缺省取第一个
`aggregate`	否	是否聚合查询，默认 `false`
`blank-to-null`	否	空白参数转 null，默认 `true`
`value-root` / `type`	否	JSON 结果根路径 / 类型

`<eql>` 内还可使用 `filters`、`translate`（缓存翻译）、`secure-mask`（脱敏）、`pivot` / `unpivot` / `summary`（结果集算法）、`page-optimize`（分页优化）等子标签，与 SQL 查询一致。

三、Java 调用

通过 `lightDao.elastic()` 链式调用（`sql` 传 eql 的 id）：

```
// 普通查询（SQL 模式）
List<CompanyInfoVO> result = (List<CompanyInfoVO>) lightDao.elastic()
        .sql("es_find_company")
        .values(MapKit.keys("companyName", "companyType", "flag").values("浙江华旭", "3", 2))
        .resultType(CompanyInfoVO.class)
        .find();

// 分页查询（ES 原生 SQL 已支持 cursor 游标分页）
Page page = (Page) lightDao.elastic()
        .sql("es_find_company_page")
        .values(MapKit.keys("companyName", "companyType", "flag").values("浙江华旭", "3", 2))
        .resultType(CompanyInfoVO.class)
        .findPage(new Page());

// 取 Top N
List top = lightDao.elastic().sql("es_find_company_page")
        .values(...).resultType(CompanyInfoVO.class).findTop(10);

// 原生 JSON 模式查询（参数为数组）
List<CompanyInfoVO> jsonResult = (List<CompanyInfoVO>) lightDao.elastic()
        .sql("sys_elastic_test_json")
        .names("companyTypes").values(new Object[]{ new Object[]{ "1", "2" } })
        .resultType(CompanyInfoVO.class)
        .find();
```

`elastic()` 链式可设置：`sql`、`names`、`values`、`entity`、`endPoint`、`resultType`、`humpMapLabel`；终止方法：`find()`、`getOne()`、`findTop(int)`（仅固定条数，不支持按比例提取）、`findPage(Page)`。

四、注意事项

- **分页**：ES 原生 SQL 已支持 cursor 游标模式分页；elasticsearch-sql / opendistro 插件亦支持，可按需选择 `sqlPath`。
- 多集群/多节点通过配置多个 `endpoints[n]` 实现，查询时可用 `end-point` 属性或 `.endPoint(id)` 指定。
- 纯 CRUD 建议直接用 ES 官方客户端；sqltoy 的价值在于**动态条件 + 缓存翻译 + 结果集算法**的组合。

五、接入注意事项（实测）

1. **endpoint 的 `id` 必填**：`endpoints[0].id` 缺省不写会在 `SqlToyContext` 初始化时报 `getId()` 为 null 的 NPE，务必为每个 endpoint 显式设置 id（或配置 `default-id`）。
2. **依赖**：sqltoy 的 ES 客户端（Rest5Client）基于 Apache HttpClient 5，项目需显式引入 `org.apache.httpcomponents.client5:httpclient5`（sqltoy 未传递该依赖）。
3. **`fields` 或 `_source` 必须提供其一**：见 2.3 属性表说明——两者都缺时结果映射报 `collection or property name array is empty`。
4. **JSON 库可插拔**：ES 插件栈与 JSON 列共用同一套 JsonProvider 抽象，按 classpath 自动探测 fastjson2 -> jackson3 -> jackson2；探测到的库还要通过一次冒烟解析才被采纳，不可用（如 jackson3 遇到过旧的 jackson-annotations）则告警并降级到下一个候选。也可用 `spring.sqltoy.jsonProvider` 显式指定，取值为内置别名 `fastjson2` / `jackson3` / `jackson2` 或自定义实现的全限定类名；皆不可用时启动仅告警、触及 JSON/ES 功能才报错。

> 完整可运行示例：`sqltoy-showcase` 的 `NoSqlTest`（Testcontainers 起真实 ES 容器，`<eql>` json 模式动态查询实测）。

六、安全连接（认证与 TLS）

生产环境的 ES 通常开启安全（认证 + TLS）。sqltoy 的 endpoint 支持完整的连接安全配置：

```yaml
spring:
  sqltoy:
    elastic:
      endpoints[0]:
        id: prod
        url: https://es.prod.internal:9200
        # Basic 认证
        username: elastic
        password: ${ES_PASSWORD}
        # TLS:信任的证书库(自签证书场景)
        key-store: classpath:certs/es-truststore.p12
        key-store-pass: ${ES_TRUSTSTORE_PASS}
        key-store-type: PKCS12
        key-store-self-sign: true   # 自签证书
        # 认证缓存(默认开启,避免每次请求重复认证)
        auth-caching: true
```

> 完整属性见 `ElasticEndpoint`（id/url/username/password/keyStore*/requestTimeout 等）。演示项目 `NoSqlTest` 出于零门槛用了无安全容器（`xpack.security.enabled=false`），生产配置以上述为准。

工具类：DTO 与 POJO 互转

在 DTO(VO) 与 POJO(entity) 严格分层的场景下，sqltoy 提供了便捷的对象/集合快速转换与赋值能力，避免手写大量 `get/set` 拷贝代码。

一、convertType（LightDao / SqlToyCRUDService）

`LightDao`、`SqlToyCRUDService`、`SqlToyDaoSupport` 中都提供了 `convertType` 快捷方法：

```java
// 单个对象转换
OrderInfo entity = lightDao.convertType(orderInfoVO, OrderInfo.class);

// 忽略部分属性
OrderInfo entity2 = lightDao.convertType(orderInfoVO, OrderInfo.class, "createBy", "createTime");

// 集合转换
List<OrderInfoVO> voList = lightDao.convertType(entityList, OrderInfoVO.class);

// 分页对象转换（连同分页信息一起转）
Page<OrderInfoVO> voPage = lightDao.convertType(entityPage, OrderInfoVO.class);
```

签名：

```java
<T extends Serializable> T convertType(Serializable source, Class<T> resultType, String... ignoreProperties);
<T extends Serializable> List<T> convertType(List sourceList, Class<T> resultType, String... ignoreProperties);
<T extends Serializable> Page<T> convertType(Page sourcePage, Class<T> resultType, String... ignoreProperties);
```

典型用法（保存时 VO→POJO）：

```java
@Transactional
public void createOrderInfo(OrderInfoVO orderInfoVO) {
    OrderInfo entity = lightDao.convertType(orderInfoVO, OrderInfo.class);
    lightDao.save(entity);
}
```

提示

千分位数值自动剥离（6.0.3 起）：查询结果经 `number-format` 格式化后的带千分位逗号数值（如 `"8,888.0000"`）再映射到 `BigDecimal` / `Integer` / `Long` / `Double` 等数值属性时，convertType 会自动剥离分组逗号。仅当整个字符串严格匹配千分位分组形态时才剥离（`"1,23"` 等非法形态不处理、保持原解析报错）。

二、MapperUtils（5.6.x 起）

5.6.x 版本开始也可直接使用 `MapperUtils` 工具类进行映射处理，适合在非 DAO 上下文中做对象映射。

三、字段名不一致：@SqlToyFieldAlias

当 VO 与 POJO 字段名称不一致时，用 `@SqlToyFieldAlias` 标注对应关系：

```
// VO 中的 postType 对应 POJO 中的 post
@SqlToyFieldAlias("post")
private String postType;
```

这样 `convertType` 即可正确映射名称不同的字段。

> 严格分层的完整示例参见项目 https://github.com/sagframe/sqltoy-strict ， POJO/VO 由 quickvo 生成到不同包。

四、常用工具类

sqltoy 的 `org.sagacity.sqltoy.utils` 下有一批"项目里总要自己写一遍"的工具，直接复用即可（`org.sagacity.sqltoy.plugins.utils.CalculateUtils` 提供按精度/舍入方式的计算）：

工具类	常用能力
`IdUtil`	主键/标识生成：`getUUID()`（UUIDv7）、`getNanoTimeId(workerId)` / `getShortNanoTimeId(...)`（26 位有序数字，返回 `BigDecimal`）、`getId(distributeIdGenerator, tableName, signature, ...)`（基于分布式生成器的业务主键）、`getDebugId()`
`SnowflakeIdWorker` / `StandardUUIDv7Generator`	雪花、UUIDv7 的独立实现，可自行持有实例
`DBTransUtils`	手动事务：`doTrans(dataSource, transactionHandler)` 把一段逻辑包在事务里、`getCurrentConnection()` 取当前线程绑定的连接
`StringUtil`	空白判断、驼峰/下划线互转、按符号切分（可排除括号内内容）、占位符替换、`splitExcludeSymMark`
`DateUtil`	日期格式化/解析、月末/季末、按间隔取日期、`getIntervalDays`（分片策略用到）
`NumberUtil`	数字格式化（千分位/百分比）、`toCapitalMoney`（中文大写金额）、解析、`getProbabilityIndex(int[])`（按权重取随机下标，权重分库用到）、`getRandomNum`
`CalculateUtils`	带精度与舍入方式的加减乘除（`RoundingMode` 可控）
`CollectionUtil`	集合切分、分组、判空、类型转换
`BeanUtil`	属性读写、对象与 Map 互转、枚举取值、类型转换（`convertType` 的底层）
`SqlUtil`	SQL 片段识别与处理（剥离 order by、判断 union/group 等），自定义拦截器常用
`ResultUtils`	结果集到 Map/VO 的映射与扩展类型归一（自行接管结果处理时参考）
`MapperUtils`	见第二节，VO↔POJO/Map 映射
`ReservedWordsUtil`	保留字按方言加引用符（`@joinFields` 内部使用）

```
// 不依赖 dao 生成标识
String uuid = IdUtil.getUUID();                                  // UUIDv7
BigDecimal nanoId = IdUtil.getNanoTimeId("0");                   // 26 位有序数字(workerId 用于多实例区分)

// 手动事务（纯 Java 场景）：把一段逻辑整体放进事务
Object result = DBTransUtils.doTrans(lightDao.getDataSource(), () -> {
    lightDao.updateByQuery(StaffInfo.class,
            EntityUpdate.create().set("sexType", "F").where("staffId=?").values("S0001"));
    return lightDao.find("select * from staff_info where status=:status",
            MapKit.map("status", "1"), StaffInfo.class);
});
```

> 这些工具类基本都是**无状态静态方法**（除 `SnowflakeIdWorker` 等需实例的以外），可多线程直接调用；`DBTransUtils` 的事务状态绑定在当前线程。更多工具入口见API 速查。

表元数据操作（TableApi）

`lightDao.tableApi()` 提供数据库表元数据查询与表级操作（truncate/drop），支持按表名或实体类操作，并可临时切换数据源。

一、API

```
TableApi tableApi = lightDao.tableApi();

// 获取表信息（catalog/schema/tableName 支持通配，可传 null）
List<TableMeta> tables = tableApi.getTables(null, null, "sqltoy%");

// 获取表的列信息
List<ColumnMeta> columns = tableApi.getTableColumns(null, null, "sqltoy_staff_info");

// 清空表（支持表名或实体类）
tableApi.truncate("sqltoy_staff_info");
tableApi.truncate(StaffInfo.class);

// 删除表（支持表名或实体类）
tableApi.drop("sqltoy_staff_info");
tableApi.drop(StaffInfo.class);

// 临时切换数据源
tableApi.dataSource(otherDataSource).truncate("sqltoy_staff_info");
```

方法	说明
`getTables(catalog, schema, tableName)`	返回 `List<TableMeta>`，表名支持通配匹配
`getTableColumns(catalog, schema, tableName)`	返回 `List<ColumnMeta>`，含列名、类型、长度、注释等
`truncate(Class entityClass)` / `truncate(String tableName)`	清空表数据
`drop(Class entityClass)` / `drop(String tableName)`	删除表
`dataSource(DataSource)`	指定数据源

二、说明

- `LightDao` 上也直接提供了 `truncate(Class entityClass)` 快捷方法。
- `TableMeta` / `ColumnMeta` 封装了数据库元数据，可用于动态表单、代码生成、数据字典等场景。
- 不同数据库获取表信息的方式不同，有的需要通过 SQL 查询（如 Oracle 的 `select * from user_tab_comments`），此时直接用 sqltoy 的 SQL 查询即可；如需自行处理 `ResultSet`，可自定义 Dao 继承 `SpringDaoSupport`，使用 `DataSourceCallbackHandler` 反调模式。

> 由 POJO 反向生成建表 DDL 见 POJO 生成表结构 DDL。

配置化服务底座（报表 / 配置化 API 服务）

> `XMLBinding` 与 `parseSqlSegment` 注册能力自 sqltoy 6.0.x 提供。

sqltoy 的 **sqlId + 命名参数** 模式天然适合把查询做成配置：SQL 片段以 `<sql>` 形式**存进数据库（或配置中心、报表模型）**，运行期由框架动态解析执行。这一能力正是页面配置化报表、配置化 API 服务、低代码平台的底座——开发侧只需维护"配置"，接口按 id 取用，无需反复发版。

一、整体思路

配置侧：每条 SQL 配置携带**唯一 id** 与**最后修改时间**（用于缓存失效判断）。

执行侧：两种姿势，按配置变化频率选择：

姿势	适用场景	说明
`XMLBinding` 动态执行	配置频繁变化的查询	每次请求带 id + lastUpdateTime，框架按时间判断命中解析缓存，变化才重新解析
`parseSqlSegment` + `putSqlToyConfig` 注册	相对稳定的查询	启动 / 变更时解析注册一次，后续 `lightDao.find(id, params)` 与静态 sqlId 完全一致

动态列：用 `@joinFields(:fields, default)` 让配置决定 `select` / `group by` / `order by` 的列（字段白名单校验 + 保留字按方言加引用符），详见标签与表达式。

二、配置存储示例（数据库）

```
create table report_sql_config (
    report_id   varchar(64)  primary key,
    sql_xml     text         not null,   -- 完整的 <sql>...</sql> 片段
    update_time datetime     not null
);
```

三、姿势一：XMLBinding 动态解析执行

```
@Service
public class ReportService {

    @Autowired
    private LightDao lightDao;

    /** 从数据库 / 配置中心加载 SQL 配置片段 */
    private String loadSqlXml(String reportId) {
        // 例如：select sql_xml from report_sql_config where report_id = ?
        // 或从配置中心、报表模型读取
        return lightDao.getValue("select sql_xml from report_sql_config where report_id = :reportId",
                MapKit.keys("reportId").values(reportId), String.class);
    }

    public List<Map> query(String reportId, LocalDateTime updateTime,
                           Map<String, Object> params) {
        String sqlXml = loadSqlXml(reportId);
        return lightDao.findByQuery(
                new QueryExecutor(new XMLBinding(sqlXml)
                        .id(reportId)
                        .lastUpdateTime(updateTime))  // 未变化时直接命中解析缓存
                .values(params)
                .resultType(Map.class))
            .getRows();
    }
}
```

缓存规则（框架语义）：

- 未带 `lastUpdateTime`：首次解析后一直走缓存；
- 带 `lastUpdateTime`：缓存为空、或时间比缓存新、或缓存本身无时间 → 重新解析；否则直接命中缓存，不重复解析。

四、姿势二：解析并注册，按 sqlId 复用

```
public void registerReport(String reportId, String sqlXml) throws Exception {
    SqlToyConfig config = lightDao.getSqlToyContext().parseSqlSegment(sqlXml);
    // 一张报表可能含多个 sql 片段，用 reportId + "_" + index 保证唯一
    config.setId(reportId);
    lightDao.getSqlToyContext().putSqlToyConfig(config);
}

public List<Map> queryById(String reportId, Map<String, Object> params) {
    // 注册后与静态 sqlId 用法完全一致
    return lightDao.find(reportId, params);
}
```

> 注销不再使用的注册： `lightDao.getSqlToyContext().removeSql(reportId)` 。

五、动态列：配置决定 select / group by / order by

```
<sql id="report_dynamic_columns">
    <value><![CDATA[
        select @joinFields(:fields, t.order_id)
        from sqltoy_order_info t
        #[and t.trans_date >= :startDate]
        #[and t.trans_date < :endDate]
        order by @joinFields(:orderFields, t.trans_date)
    ]]></value>
</sql>
```

```
List rows = lightDao.findByQuery(
        new QueryExecutor("report_dynamic_columns")
            .names("fields", "orderFields", "startDate", "endDate")
            .values(
                "t.order_id,t.customer_name,t.total_amt",  // 纯标识符，逗号分隔
                "t.total_amt,t.trans_date",
                "2026-01-01", "2026-01-31")
            .resultType(Map.class))
        .getRows();
```

注意：

- `@joinFields` 只受理**纯标识符**（字段白名单：字母含中文、数字、下划线，最多三段点号）； `sum(t.total_amt)` 、 `t.*` 这类表达式请继续用 `@value` （安全自负）。
- 字段名可能命中保留字（ `order` / `desc` / `level` / `group` 等）时，必须配置 `spring.sqltoy.reservedWords` ，框架才会按方言加引用符（mysql 反引号、oracle/pg 双引号、sqlserver 方括号）。
- 排序方向建议固定写在 SQL 文本里；确需由配置决定方向时，用 `@value` 拼接并自行保证安全。

六、与报表 / 低代码平台的对接

- **一张报表多个 SQL 片段**：各自解析注册， `reportId + "_" + index` 唯一化后按序取数；
- **叠加能力**：配置片段里照常写 `<translate>` 缓存翻译、 `<summary>` / `<pivot>` / `<link>` 等内存分析标签，无需额外代码；
- **分页**： `findPage` / `Page` 用法与静态 sqlId 完全一致。

相关

- 动态 SQL 编写规范（ `#[]` 动态段、 `@include` 、 `@joinFields` 所在体系）
- 标签与表达式（ `@joinFields` 、 `@value` 、 `@loop` / `@secure-loop` 等）
- 平台集成（报表 / 低代码）

扩展集成（报表 / 低代码平台）

sqltoy 打造的并不仅仅面向常规应用开发（管理系统后台、微服务接口实现等），还面向**报表平台**以及目前流行的**低代码平台**（基于页面配置化完成传统功能页面、接口服务开发部署的新开发业态）。

例如基于 sqltoy 开发的：

- **sagacity-rainbow**：快速接口开发框架；
- **sagacity-nebula**：报表框架。

一、集成方式

核心是把 SQL 的 xml 片段交由 `SqlToyContext` 托管。报表/低代码平台在运行时动态产生或加载 SQL 配置，由 sqltoy 负责解析、参数绑定、方言适配与执行。

二、参数绑定：getFullParamNames

集成时常见的疑问是：**怎么把（报表/页面的）条件参数跟 SQL 里的条件参数绑定?**

如果是报表，通常会把条件参数以 Map 形式组织起来。`SqlToyConfig` 提供了 `getFullParamNames` 方法，让你获取一条 SQL 中的**全部参数名称**，从而把外部条件 Map 与 SQL 参数对齐：

```
// 取得某个 sqlId 对应配置的全部参数名
SqlToyConfig config = lightDao.getSqlToyConfig("report_sql_id", SqlType.SELECT);
String[] paramNames = config.getFullParamNames();
// 据此从报表条件 Map 中挑选/组装实际传参，再交给 sqltoy 执行
```

三、设计理念

> 有时候适度灵活一下，中间加一个过程转化一下就可以了。sqltoy 作为基础框架，不会把接口提供得过于泛滥而对常规开发人员形成干扰（"左也行、右也行"反而会让开发蒙圈）。把 SQL 托管、参数对齐、方言适配这些"脏活"交给框架，平台层只需专注业务编排。

四、相关

- SQL 配置驱动（报表/API 服务）的实战写法见常见 SQL 案例。
- 配置参数总览见sqltoy 配置参数。

GraalVM Native Image 指引

sqltoy 的 spring-starter 与 solon-plugin 均内置 AOT/native 支持：构建期自动注册 ORM 所需的反射与资源元数据，运行期自动切换为镜像内资源加载。本页给出完整的支持形态、构建步骤、应用侧必要配合与已知限制。

> **版本说明**：AOT 支持自 5.6 引入；**6.0.8 起**大幅增强（按实际配置动态注册资源与扩展类，实体扫描不再依赖构建期 bean 快照），本文以 6.0.8 为准，6.0.7 及以下的差异在各节单独标注。

一、支持形态总览

模块	AOT 组件	注册方式	作用
`sagacity-sqltoy-spring-starter`	`SqltoyRuntimeHintsRegistrar`	`META-INF/spring/aot.factories` 自动注册	固定资源 pattern（`sqltoy-translate.xml`、`translates/**`、`sqltoy/**`）+ 配置属性类、6 个内置主键生成器（Default/UUID/ULID/NanoTime/Redis/Snowflake）反射
`sagacity-sqltoy-spring-starter`	`SqltoyBeanFactoryInitializationAotProcessor`	同上	实体反射、按配置动态注册的资源 pattern 与扩展类反射（6.0.8 起，见第三节）
`sagacity-sqltoy-solon-plugin`	`SqltoyRuntimeNativeRegistrar`	`SqlToyPluginImpl` 在 AOT 模式下自动挂载	与 spring 侧对称（solon 的 AOT 构建走真实启动流程，配置 bean 真实可用）

native 运行期自适配（所有版本）：sqltoy 检测到 `org.graalvm.nativeimage.imagecode` 系统属性后，把 `sqlResourcesDir` 目录配置转换为逐个 `.sql.xml` 的镜像内资源列表加载（`SqlScriptLoader` 按资源列表模式工作），solon 侧对 `translateConfig` 同样处理。

> **注意**
>
> 由此带来一个行为差异：native 下 sql 文件已打进镜像，**debug 模式的 sql 文件热更新自动失效**（文件变更检测只对文件系统上的 File 资源生效）。

二、构建步骤（Spring Boot）

前置条件（与 Spring Boot 官方要求一致，详见 Spring Boot Native Image 文档）：

- **GraalVM JDK 25+**：低版本与当前 GraalVM reachability metadata repository 不兼容（实测矩阵：**17 与 23** 不识别新元数据 schema，被 native-maven-plugin 直接拒绝；**21** 不支持 `typeReached` 条件元数据——条件被当作恒真，spring-aop 的 AspectJ 序列化条目被误注册，无 aspectjweaver 时构建失败）；25.0.4 已双框架全矩阵验证；CI 中常用 `graalvm-community`；
- Windows 需安装 Visual Studio Build Tools（MSVC），Linux 需 gcc 工具链，macOS 需 Xcode Command Line Tools。

> **注意**
>
> 项目使用 lombok 时注意：JDK 23+ 的 javac 不再隐式运行 classpath 上的注解处理器，需在 `maven-compiler-plugin` 的 `annotationProcessorPaths` 中显式声明 lombok，否则用 GraalVM 25 编译时实体 getter/setter 不会生成。

项目 parent 为 `spring-boot-starter-parent` 时自带 `native` profile，直接构建：

```
# 构建原生镜像（sqltoy 的 AOT 元数据自动生效）
mvn -Pnative native:compile -DskipTests

# 运行
./target/your-app
```

三、应用侧必要配合

1. packagesToScan 必配（所有版本）

sqltoy 实体是**即用即加载**的（首次使用时才解析注册），而 native 构建只能在构建期登记反射元数据。因此 native 场景必须配置 `packagesToScan`（或 `annotatedClasses`），让实体在启动期完成注册：

```
spring:
    sqltoy:
        # native 场景必配:构建期按此包扫描 @Entity 注册反射元数据
        packagesToScan: com.yourcorp.project.entity
        sqlResourcesDir: classpath:sqltoy
```

> 普通 JVM 场景该配置非必需（实体按需解析），配置后仅意味着启动期提前解析，行为无差异。
>
> `packagesToScan` 只覆盖 `@Entity`；查询返回的普通 DTO/VO 用 3.1 的 `native-reflection-packages` / `native-reflection-classes` 登记。

2. 资源与扩展类注册（6.0.8 起自动）

6.0.8 起，`SqltoyBeanFactoryInitializationAotProcessor` 直接从 Environment 绑定 `spring.sqltoy` 配置（AOT 构建期容器不实例化/绑定 bean，此前依赖 bean 快照的实现拿不到配置），自动注册：

注册内容	配置来源
实体反射	`packagesToScan` / `annotatedClasses` 扫描
主键生成器反射	6 个内置生成器固定注册（Default/UUID/ULID/NanoTime/Redis/Snowflake）；实体扫描到的 `@Id(generator/strategy)`、`@BusinessId(generator)` 自定义生成器类随实体自动注册
配置属性嵌套类	`Elastic`、`ElasticConfig`（solon 运行期 `cfg().toBean(...)` 绑定嵌套 endpoints 时需要）
sql 文件资源 pattern	`sqlResourcesDir`（多目录分隔）、`sqlResources`
翻译配置资源	`translateConfig`（`.xml` 按文件、目录按 `/**`）
密钥资源	`securePublicKey` / `securePrivateKey`
扩展类反射	`unifyFieldsHandler`、`typeHandler`、`sqlInterceptors`、`sqlFormater`（含 `default` 别名展开）、`translateCacheManager`、`dataSourceSelector`、`connectionFactory`、`fieldsSecureProvider`、`desensitizeProvider`、`customFilterHandler`、`overTimeSqlHandler`、`dialectDDLGenerator`、`firstBizCodeTrace`、`dynamicCacheFetch`、`dynamicFecthCacheManager`（约定同运行期：含 `.` 视为类名，否则为 bean 名称由 Spring AOT 覆盖）、`functionConverts`（含 `default` 展开的内置函数集，经 `FunctionUtils.resolveFunctionClassNames` 与运行期同语义解析）
集成类反射（条件）	注册跟随 classpath 而非构建期配置：caffeine 库在 → 翻译缓存管理器（ehcache 管理器为运行期直接 new，无需注册）；mongo 驱动在 → `SpringMongoQuery`；spring-data-redis 在 → `SpringRedisIdGenerator`
扩展类型驱动专属类（条件）	对应驱动在 classpath 时注册：PG 系 `PGobject` 同构类 5 个（postgresql 及 stardb 等兜底、opengauss/mogdb、vastbase、gaussdb、kingbase 的 `KBobject`——json/jsonb/vector/geometry 经反射构造与读写）、mssql-jdbc 的 `Geometry` / `Vector`、oracle 的 `OracleTypes`
JsonProvider 反射	classpath 上存在对应 JSON 库的**全部**内置实现（fastjson2/jackson3/jackson2 按各自 marker）+ 显式配置的自定义实现类名
lambda 序列化基础	`SerializedLambda` 与 sqltoy 镜像类的 java 序列化注册（固定；还需应用侧配合注册 lambda 定义类，见下节）

即：**实体、资源与扩展类按约定配置后，6.0.8 起 native 构建无需额外 AOT 代码**；非实体 DTO/VO 用下节的 `native-reflection-classes` / `native-reflection-packages` 登记。

> **警告**
>
> `sqlFormater` 的内置实现 `DefaultSqlFormater` 硬依赖 druid（core 中标记 optional）：配置 `sqlFormater` 的应用需自行引入 druid，否则 debug 统计路径（`SqlExecuteStat`）触发格式化时 `NoClassDefFoundError`（debug 关闭不触发）。

> **注意**
>
> **运行期动态切换**：native 镜像在构建期冻结，因此集成类与 JsonProvider 的注册跟随 **classpath**（而非构建期的 `cacheType` / `jsonProvider` 取值）——只要对应库在 classpath 上，运行期用启动参数切换 `cacheType`（ehcache↔caffeine）或 `jsonProvider`（fastjson2/jackson3/jackson2/自定义）均可用，sqltoy-aot-verify 已双框架实测切换轮。

3. 仍归应用侧的部分（任何版本）

以下五类框架无从推断，由应用注册——完整可运行示范见 sqltoy-aot-verify（`AotVerifyRuntimeHints` / solon `Config#aotVerifyNativeRegistrar`）与 showcase 的 `ShowcaseRuntimeHints`：

1. **应用代码自行加载的 classpath 资源**：如启动建表脚本（showcase 的 `mock/schema-*.sql` 由 `DataSourceConfig` 自己读取）；
2. **被 SQL/第三方机制按类名引用的类**：如 H2 的 `CREATE ALIAS ... FOR 'com.x.Procedures.listStaff'`（运行期由 H2 `Class.forName` 加载）；
3. **接口返回类型 + `initMethod` /事务代理的 `@Bean`**：Spring AOT 按声明类型预判 bean 类型，接口声明会让具体类不可见（native 运行期报 init 方法找不到或 "CGLIB runtime enhancement not supported"）——**改用具体返回类型声明即可**，无需 hints（showcase 的分片策略 bean 与 6.0.8 starter 的 DAO bean 均如此声明）；

4. **作为查询 resultType 的 DTO/VO**：非 `@Entity`（`packagesToScan` 扫不到），运行期由 sqltoy 反射实例化并逐列赋值——6.0.8 起优先用 `native-reflection-classes` / `native-reflection-packages` 配置登记（见 3.1），无需手写 `RuntimeHints`；
5. **使用 lambda 条件（lambdaQuery/lambdaUpdate/lambdaDelete）的类**：属性提取经 writeReplace/序列化镜像，native 下需按"lambda 定义类"注册——spring 侧在 `META-INF/native-image/**/serialization-config.json` 声明 `lambdaCapturingTypes`（或 `hints.reflection().registerLambda(定义类, b -> b.withInterfaces(LambdaFn.class))`），solon 侧 `metadata.registerLambdaSerialization(定义类)`；不注册则 lambda 体系在 native 下全部不可用（JVM 不受影响）。

```
public class ShowcaseRuntimeHints implements RuntimeHintsRegistrar {
    @Override
    public void registerHints(RuntimeHints hints, ClassLoader classLoader) {
        // 应用自行加载的启动建表脚本(RSA 密钥已由 starter 按 secure* 配置注册)
        hints.resources().registerPattern("mock/**");
        // H2 CREATE ALIAS 按类名引用,运行期 Class.forName 加载
        hints.reflection().registerType(Procedures.class, MemberCategory.values());
    }
}

// 启动类挂载
@SpringBootApplication
@ImportRuntimeHints(ShowcaseRuntimeHints.class)
public class ShowcaseApplication { ... }
```

提示

sql 文件放在 `classpath:sqltoy/` 下（solon 默认约定）可命中内置固定 pattern，任何版本都无需为 sql 资源写注册代码。

3.1 DTO/VO 反射登记（6.0.8 起）

`packagesToScan` 只识别 `@Entity`，普通 resultType DTO/VO 不在其中。6.0.8 起两个 starter 都提供两个构建期配置项，由 AOT 处理器自动注册反射元数据，应用不需要再写 `RuntimeHints`：

配置项	取值	适用场景
`native-reflection-classes`	全限定类名列表	少量特殊类、按名引用的类、单个 VO/DTO
`native-reflection-packages`	包路径列表	VO/DTO 较多，按包批量扫描

```
# Spring Boot
spring:
  sqltoy:
    # 包下全部类递归注册（VO/DTO 多时用这个）
    native-reflection-packages:
      - com.yourcorp.project.dto
      - com.yourcorp.project.vo
    # 个别特殊类再单独补
    native-reflection-classes:
      - com.yourcorp.project.report.TreeModel
```

```
# Solon
solon:
  sqltoy:
    native-reflection-packages:
      - com.yourcorp.project.dto
      - com.yourcorp.project.vo
    native-reflection-classes:
      - com.yourcorp.project.report.TreeModel
```

规则：

- `native-reflection-packages` 递归扫描包下所有 `.class`，注册 `MemberCategory.values()`；普通 DTO/VO、record、enum、实体类都会注册。
- 跳过接口、注解、匿名类、局部类、合成类、`package-info`；类加载失败只记日志跳过，不中断 AOT 构建。
- 两个配置可同时使用；包扫描和显式类重复注册无副作用。
- 包扫描会把包内所有类都打进元数据，镜像会略大；不要把包含大量无关类的包整个加进来，优先单独放 DTO/VO 的包。
- 这两个配置只在构建期生效，运行期 JVM 模式不受影响；它们不替代 `packagesToScan`，`@Entity` 仍需 `packagesToScan` / `annotatedClasses`。

注意

已在 `sqltoy-aot-verify` 的 Spring/Solon 真实 native 构建中验证：移除应用侧 DTO 反射代码、仅保留 `native-reflection-packages` 后，B1 DTO 翻译查询仍通过；该轮未配置 `native-entity-proxies`，双栈 native 冒烟为 **16 PASS + 0 FAIL + 1 KNOWN-LIMITATION**（配置代理后的结果见 3.2，为 17 PASS + 0 KNOWN）。生成的 `reachability-metadata.json` / `reflect-config.json` 中可看到 DTO 类型。

3.2 updateSaveFetch 实体代理（native-entity-proxies）

`updateSaveFetch(entity, EntityUpdateCallback, ...)` 的“行已存在”分支需要一个能拦截实体 getter/setter 的代理：getter 直接读 `ResultSet` 当前行，setter 直接写当前行。JVM 下由 ByteBuddy 在运行期生成子类；native image 是 closed-world，**不能运行期定义新类**，因此该分支默认不可用。

6.0.8 起增加 `native-entity-proxies`，在 AOT 构建期只为明确列出的实体预生成代理子类（含 ByteBuddy 生成的 auxiliary 类），写入镜像并注册反射；运行期 `EntityResultSetProxy` 先按 `<实体类名>$$SqltoyEntityProxy` 加载预生成类，加载不到才回退 JVM 动态生成：

```
# Spring Boot:只列出真正使用 EntityUpdateCallback 的实体
spring:
  sqltoy:
    native-entity-proxies:
      - com.yourcorp.project.entity.StockVO
      - com.yourcorp.project.entity.AccountVO
```

```
# Solon
solon:
  sqltoy:
    native-entity-proxies:
      - com.yourcorp.project.entity.StockVO
      - com.yourcorp.project.entity.AccountVO
```

约束：

- 实体必须可被子类化：不能是 `final`，且要有可访问的无参构造器；不满足时 AOT 记 warn，运行期该实体仍会报“not pre-generated”。
- `byte-buddy` 在 starter/plugin 中为 optional，不会传递；使用 `native-entity-proxies` 的应用需显式引入 `net.bytebuddy:byte-buddy`。未引入时 AOT 构建会输出明确警告并跳过代理生成，运行期该实体仍不可用。
- 只对列出的实体生效；未列出的实体仍保持“native 下不可用”的限制，不会静默退回运行期生成。
- 生成的代理类和 auxiliary 类都会进入镜像，实体越多镜像略增；不要无差别把所有实体都列进来。

注意

已在 `sqltoy-aot-verify` 双栈真实 native 构建中验证：配置 `native-entity-proxies: com.sqltoy.aotverify.entity.AotUsfVO` 后，C1（`updateSaveFetch` 的 `EntityUpdateCallback`）由 KNOWN-LIMITATION 变为 **PASS**，双栈冒烟均为 **17 PASS + 0 FAIL + 0 KNOWN-LIMITATION**。

历史注记（6.0.7 及以下）：starter 的 AOT 实体注册在 Spring AOT 下不生效（构建期配置 bean 未绑定、实体快照为空），且内置固定 pattern 只覆盖 `sqltoy/**`、`translates/**`、`sqltoy-translate.xml` 约定目录，应用需自行补齐五件事：① 自定义 sql 目录的资源 pattern；② 按类名配置的扩展类反射；③ 实体反射（扫描 `packagesToScan` 同款包下的 `@Entity`）；④ sqltoy XML 解析的 8 个模型类反射（缺失时 `<unpivot>` / `<tree-sort>` /环比等标签属性**静默丢失**并解析 NPE）；⑤ DAO bean 用具体返回类型覆盖声明。完整 6.0.7 时代配方可查 showcase 仓库历史。

四、已知限制

限制	原因	影响范围
`updateSaveFetch`（`EntityUpdateCallback`）默认不可用	`EntityResultSetProxy` 经 byte-buddy 在运行期生成实体子类，native image 不支持运行期类生成（实测异常形态：`IllegalArgumentException: Could not create type`）	仅该特性；**配置 `native-entity-proxies` 列出实体后可在构建期预生成，已验证 C1 PASS**；未配置的实体仍不可用；常规 CRUD/查询与 insert 分支不受影响
sql 文件热更新失效	sql 文件打进镜像，按资源列表加载，无 File 可监听	仅 debug 热更新；开发期用 JVM 模式即可
启动期出现 "no entity classes ... matched in the sqltoy pojo scan path" 告警属预期	`EntityScanner` 依赖 file/jar 协议的目录枚举，镜像内不可用，`packagesToScan` 的启动期扫描在 native 下为空	无功能影响：实体反射元数据已在构建期注册，运行期按首次使用懒解析（native 冒烟已验证 CRUD/翻译缓存可用）
第三方组件依赖各自元数据	mongo/ES/redis/ehcache 等客户端的 native 支持由其官方 reachability metadata 提供	使用对应组件时按其官方文档确认。已知例子：**fastjson2** 自带的 native-image.properties 漏掉嵌套类（`JSONFactory$CacheItem` 等），GraalVM 25 的 image heap 校验会构建失败，应用需追加构建参数 `--initialize-at-build-time=com.alibaba.fastjson2`（sqltoy-aot-verify 已内置）；补参后其 `JDKUtils` 的 Unsafe 字段偏移仍在构建期重算失败，运行期探测按设计**静默跳过 fastjson2 降级到 jackson3/jackson2**（功能不受影响）——native 下 JSON 列建议使用 jackson2/jackson3（均已实测）
JSON 序列化的业务类型	JsonProvider 实现类已自动注册，但被序列化业务类型是否可达取决于 JSON 库自身的 native 支持	JSON 列/ES 场景，Spring Boot 对 Jackson 已有内建支持

五、Solon 集成

solon-plugin 内置 `SqltoyRuntimeNativeRegistrar`（实现 solon 的 `RuntimeNativeRegistrar` 接口，`SqlToyPluginImpl` 在 AOT 模式下自动挂载），注册内容与 spring 侧对称：固定资源 pattern、配置属性类与 6 个内置主键生成器反射、实体反射，6.0.8 起同样按配置动态注册资源 pattern、扩展类反射以及 `native-reflection-classes` / `native-reflection-packages` 指定的 DTO/VO 与特殊类，并支持 `native-entity-proxies` 在构建期预生成 updateSaveFetch 代理。solon 默认 `sqlResourcesDir=classpath:sqltoy` 命中内置 pattern，零配置即可。构建方式参见 solon 官方 native 文档与 Solon 集成参考。

> **注意**
>
> solon 把 yml **标量**绑定到 `String[]` 时元素会带上引号（`packagesToScan: com.x.entity` 绑定为 `"com.x.entity"`），带引号包名会让实体扫描静默为空。6.0.8 起插件已自动剥离首尾成对引号，标量与 yaml 列表两种写法均可；6.0.7 及以下请使用 yaml 列表写法。

六、演示与验证

专项验证工程 sqltoy-aot-verify（spring + solon 双栈共用验证矩阵，PostgreSQL pgvector+postgis）：矩阵覆盖 CRUD×json/vector/geometry 三形态（String/float[]/JTS Geometry）、ehcache+caffeine 双实现缓存翻译、updateSaveFetch 代理路径、lambda 单表/join/setExpr/matchAll 守卫/删除；JVM 全矩阵双框架 17/17 PASS，配置 `native-entity-proxies` 后 native 冒烟双框架 **17 PASS + 0 KNOWN-LIMITATION**（C1 已由预生成代理解决）。

演示项目 sqltoy-showcase 提供两层验证：

1. **JVM 层（随 `mvn test` 运行，无需 GraalVM）**：`NativeAotTest` 运行与 `native:compile` 完全相同的 Spring Boot AOT 处理（`SpringApplicationAotProcessor`），解析生成的 `reachability-metadata.json`，断言实体反射、sql/mock 资源 pattern、扩展类反射全部就位；
2. **真实编译（CI）**：native-build workflow 在 ubuntu + GraalVM 上执行 `mvn -Pnative native:compile`，并以 `--smoke` 参数运行二进制做启动 + 查询路径冒烟（`NativeSmokeRunner`：实体加载、SQL 计数、缓存翻译；手动/每周触发，不占用常规 PR CI）。

> 完整示例源码：演示项目 sqltoy-showcase 的 `NativeAotTest.java`、`ShowcaseRuntimeHints.java`

相关文档

- 其他特性
- Solon 集成参考

- sqltoy 配置参数

其他特性

本页汇总几个使用频率较低、但特定场景下很有价值的能力。

一、GraalVM Native Image（AOT 原生支持）

`sagacity-sqltoy-spring-starter` 内置 Spring AOT 支持（经 `META-INF/spring/aot.factories` 自动注册 `SqltoyRuntimeHintsRegistrar` 与 `SqltoyBeanFactoryInitializationAotProcessor`），`sagacity-sqltoy-solon-plugin` 对称内置 `SqltoyRuntimeNativeRegistrar`，按 Spring Boot 官方方式构建原生镜像即可；6.0.8 起按实际配置动态注册资源 pattern 与扩展类反射，native 场景实体需配置 `packagesToScan`，非实体 DTO/VO 可用 `native-reflection-packages`（按包）或 `native-reflection-classes`（按类名）登记：

```
# 按 Spring Boot 标准方式构建 native image，sqltoy 的 hints 自动生效
mvn -Pnative native:compile
```

> 完整指引（构建前置条件、应用侧必要配合、已知限制、演示工程）见 GraalVM Native Image 指引。

二、geometry 空间类型支持（JTS）

sqltoy 内置对数据库 **geometry 空间列**的自动适配（GIS 场景），核心在 `GeometryTypeUtil`：

- 自动识别 geometry 列类型，处理 **MySQL internal bytes ↔ WKT**、**Oracle SDO ↔ WKT** 的转换；
- classpath 存在 **JTS**（`org.locationtech.jts`，`hasJts()` 自动探测）时，查询结果自动解析为 **JTS `Geometry` 对象**，写入时同样支持 `Geometry` 对象；
- 未引入 JTS 时，以 **WKT 字符串**（如 `POINT(121.4 31.2)`）交互。

```
<!-- 使用 Geometry 对象交互时引入 JTS -->
<dependency>
    <groupId>org.locationtech.jts</groupId>
    <artifactId>jts-core</artifactId>
    <version>1.20.0</version>
</dependency>
```

> POJO 的 geometry 属性声明为 JTS `Geometry`（或 WKT 字符串）即可，读写由框架自动编解码（`JtsGeometryCodec` / `GeometryTypeUtil`）。

三、公共字段赋值开关（UnifyUpdateFieldsController）

配置了统一字段处理器（unifyFieldsHandler）后，save/update 会自动补漏 `updateBy` / `updateTime` 等公共字段。个别场景（如系统级数据修复脚本，不希望改动"最后更新人"）可临时**暂停**当前线程的公共字段赋值：

```
try {
    // 暂停当前线程的公共字段自动赋值（ThreadLocal 开关，只影响当前线程）
    UnifyUpdateFieldsController.stop();
    lightDao.update(fixEntity);   // 本次更新不自动填充 updateBy/updateTime
} finally {
    // 必须恢复，否则当前线程后续操作都不会自动赋值
    UnifyUpdateFieldsController.resume();
}
```

方法	说明
`stop()`	暂停当前线程公共字段赋值
`resume()`	恢复当前线程公共字段赋值
`useUnifyFields()`	查询当前线程是否启用

四、自定义 Connection 操作（DataSourceCallbackHandler）

需要使用框架未封装的原生 JDBC 能力时，自定义 Dao 继承 `SpringDaoSupport`（或 `SqlToyDaoSupport`），通过 `DataSourceUtils.processDataSource` + 匿名 `DataSourceCallbackHandler` 拿到原生 `Connection` 自行处理（框架负责连接获取与释放）：

```
public class MySpecialDao extends SpringDaoSupport {

    // 参照框架内部的使用模式
    public Object doSpecial(final DataSource dataSource) {
        DataSourceUtils.processDataSource(getSqlToyContext(),
                getDataSource(dataSource),
                new DataSourceCallbackHandler() {
                    @Override
                    public void doConnection(Connection conn, DBProfile profile) throws Exception {
                        // profile 为数据库执行档案：getDbType()/getDialect()/getProductName()/getMajorVersion()
                        // 以及 isMysqlFamily()/isOracleFamily()/isOceanBase() 等便捷判断
                        try (PreparedStatement pst = conn.prepareStatement("select ...")) {
                            ResultSet rs = pst.executeQuery();
                            // ... 自行处理 rs
                            setResult(result);   // 处理结果放回，供外部 getResult() 获取
                        }
                    }
                });
        return getResult();
    }
}
```

成员	说明
`doConnection(conn, profile)`	抽象方法，拿到原生 `Connection` 与 `DBProfile` 数据库执行档案，自行处理
`DBProfile`	提供 `getDbType()` / `getDialect()` / `getUrl()` / `getProductName()` / `getMajorVersion()`，及 `isMysqlFamily()` / `isOracleFamily()` / `isOceanBase()` / `isBackslashEscape()` 等便捷判断
`setResult(...)` / `getResult()`	在回调中暂存结果 / 外部获取结果

> **版本说明**：6.0 起回调签名由 `doConnection(Connection conn, Integer dbType, String dialect)` 调整为 `doConnection(Connection conn, DBProfile profile)`（`org.sagacity.sqltoy.model.DBProfile`），5.6.x 版本请按旧签名使用。

补充说明：标签与表达式参考

除了 `#[]` 动态条件片段，sqltoy 还在 SQL 中预留了一组 `@` 宏指令，用于应对高度复杂的动态 SQL 场景。灵活搭配 `@if`、`@value`、`@blank`、`@loop` 可解决绝大多数问题；遇到更复杂的场景，推荐在代码中组织好 SQL 片段，再用 `@include(:sqlScript)` 动态嵌入（片段内可继续使用命名参数）；纯文本注入也可用 `@value(:sqlScript)`。

一、@if() / @elseif() / @else

为特殊业务场景预留的条件逻辑判断，可写在 SQL 中。

两种判断模式：

```
单逻辑判断：@if(:paramName>=value)
双逻辑判断：@if(:paramName1>=value1 && :paramName2<=value2)
```

- 比较符号：`>`、`>=`、`==`、`<`、`<=`、`!=`、`<>`
- 逻辑运算符：`&&`、`||`、`include`、`in`、`out`
- 配合 `@elseif()`、`@else` 构成完整分支。

时间/日期比较：

```
@if(:paramName>=now()+x)   或   @if(:paramName>=now()-x)
```

- 表示时间：`now()`、`nowtime()`、`${.now}`
- 表示日期：`day()`、`sysdate()`、`${.day}`

集合/包含判断：

写法	含义
`@if(:statusAry include 1)`	statusAry 为数组时判断是否包含 1；为字符串时等价于 `statusAry.contains("1")`
`@if(:statusAry exclude 1)`	与 include 相反，不包含
`@if(size(:statusAry)>1)`	通过 `size(:参数名)` 取集合/数组长度
`@if(:status in '1,2,3')`	status 属于 1/2/3 之一时返回 true
`@if(:status out '1,2,3')`	status 不属于 1/2/3 时返回 true

二、@fast / @fastPage

`@fast()` 写在 SQL 中，用于快速分页（先分页后关联）。**非分页查询时框架会自动剔除 `@fast`**，因此不影响 find/findTop 等其他查询。`@fast` 与 `@fastPage` 等效。

三、@blank()

为非条件部分的语句**虚构出一个参数**，使其符合 sqltoy 的 `#[]` 组织规则。

```
#[@blank(:sexType) t.SEX_TYPE]
```

- 当 `sexType` 为 null 时，按 `#[]` 逻辑整段被剔除；
- 当 `sexType` 不为 null 时，`@blank(:sexType)` 自动变为空白，剩下 `t.SEX_TYPE`，不影响 SQL 执行。

`@blank` 并非必须，常用于融合 `and`、括号等其他语句。复杂场景请结合 `@if` 和 `@value` 使用。

四、@value()

`@value(:paramName)` 类似 `@blank(:paramName)`，唯一区别是它会**直接显示 paramName 对应的值**。

典型用法：`@value(:sqlScript)` 直接把代码中组织好的 SQL 片段嵌入进来，应对高度复杂动态 SQL；片段内还需使用命名参数时，请改用第七节的 `@include(:scriptParam)`。

> **注意**：`@value` 注入的是未经校验的文本。若只是要拼接**字段名列表**（页面配置化的报表/接口服务按配置决定 select、group by、order by 的列），请改用第八节的 `@joinFields()` ——它带字段名白名单校验与按方言的保留字引用。

五、@loop() / @secure-loop() / @loop-full() / @secure-loop-full()

用于 SQL 动态循环拼接字段、条件，一般针对数组或集合，**不适用于 in 场景**（in 直接用 `in (:param)` 即可）。

- `@loop()`：参数直接拼接到 SQL 中（**不推荐**，有注入风险）。
- `@secure-loop()`：参数以 `?` 形式入参，防止 SQL 注入（**推荐**）。
- `-full` 版本：默认 `@secure-loop` 会跳过 null 和空值不参与循环；若不想跳过，用 `@secure-loop-full`。

三种格式：

```
@secure-loop(:loopParam, loopContent)
@secure-loop(:loopParam, loopContent, linkSign)
@secure-loop(:loopParam, loopContent, linkSign, startIndex, endIndex)
```

参数说明： `startIndex` / `endIndex` 是可选的循环下标区间（从 0 开始计数，`endIndex` 超出数组长度时自动截断到末尾），用于只循环集合中的一段——例如跳过首个元素、从第 2 项开始循环到结尾。**下标直接写数字即可，无需引号**：

```
@secure-loop(:staffIds, { t.STAFF_ID=:staffIds[i] }, { or }, 1, 100)
```

注意事项：

- 内部参数以逗号分隔，若 `loopContent` 中含逗号，请用 `{}`、`''` 或 `""` 包裹内容，确保切割准确。
- 支持子对象属性模式：`staffInfos[i].xxxx`，应对极端场景。
- `loopParam` 为空或未传时，宏返回 `@blank(:loopParam)`，配合外层 `#[]` 会把整段条件自动剔除，无需额外判空。
- in 条件参数数组超过 1000 时，框架（5.1.32 / 4.19.25 起）会**自动切分**。
- `@secure-loop` 自 5.1.44 版本支持。

范例：

```
-- 1) OR 条件链（值条件用 @secure-loop，参数以 ? 占位）：staffIds = ["S0001","S0003"] 时展开为：
--    and ( t.STAFF_ID=?  or  t.STAFF_ID=? )
#[and (@secure-loop(:staffIds, { t.STAFF_ID=:staffIds[i] }, { or }))]

-- 2) 动态列（列名属标识符，占位符机制不适用，只能用 @loop 拼接）：
--    fields = ["STAFF_NAME","EMAIL"] 时展开为 select t.STAFF_ID ,t.STAFF_NAME ,t.EMAIL from ...
select t.STAFF_ID @loop(:fields, { ,t.:fields[i] }) from sqltoy_staff_info t

-- 3) 下标切片（数字直接写，无需引号）：从第 2 个元素开始循环（跳过首元素），endIndex 超长自动截断
@secure-loop(:staffIds, { t.STAFF_ID=:staffIds[i] }, { or }, 1, 100)
```

> 涉及**值**的循环一律优先 `@secure-loop`（防注入）；`@loop` 仅用于列名等标识符拼接场景。

六、@include(sqlId)

支持 `@include(sqlId)` 进行 SQL 拼接：在 `@include` 处用对应 `sqlId` 的 SQL 替换。可多处 include、嵌入任意位置。

> 提供该功能但**不推荐**：可读性较差、存在关联影响。

七、@include(:scriptParam)

`@include(sqlId)` 的变种，通过**动态传参**形式传入 SQL 片段：

```
String sql = "select * from sqltoy_fruit_order where status=:status @include(:sqlScript)";
List result = lightDao.find(sql,
        MapKit.keys("status", "saleCount", "sqlScript")
              .values(1, 12, "and sale_count>:saleCount"));
```

八、@joinFields()（6.0.7 新增）

把运行期传入的**字段名列表**拼成 SQL 标识符串，用于页面配置化的报表与接口服务（由配置决定 `select` / `group by` / `order by` 的列）。

格式：

```
@joinFields(:fields)
@joinFields(:fields, default)
@joinFields(?, default)
```

- 与 `@value` 处于同一处理位置（命名参数已转成 `?` 之后按位置取值），因此 `@joinFields(:fields,...)` 与在 SQL 里直写 `@joinFields(?,...)` 完全等价；被消费的参数会从参数集合中剔除，不影响其余 `?` 的对位。
- `fields` 为 null、空集合或全空白时输出 `default`；未提供 `default` 则输出空串。
- `default` 取自 SQL 文本（开发者编写，可信），原样输出、不做标识符校验，因此可以写 `t.*`、`sum(t.amount)` 这类表达式；含逗号或括号时用 `{}`、`''` 或 `""` 包裹，如 `@joinFields(:fields,{a.id,a.code})`。

入参归一规则：

传入形态	处理
单个字符串 `"a,b,c"`	按逗号切割成 3 个字段
长度为 1 的数组 `["a,b,c"]`	同上（页面多选常序列化成这种形态）
长度 > 1 的数组 `["a","b"]`	元素即字段，**不做二次切割**
长度 > 1 且某元素含逗号 `["a,b","c"]`	报错，不静默拆分
空白段（如尾逗号 `"a,b,"`）	跳过

安全： 字段名走字符白名单——字母（含中文）、数字、下划线，最多三段点号限定（`schema.table.column`）；显式排除 `$` 与 `#`（与 `#[]` 动态段和 `${}` / `#{}` 参数占位冲突），以及空白、引号、括号、分号、逗号、注释符、反斜杠。非法值**直接抛异常并带上具体值**，不做静默清洗。合法字段再按方言对**保留字**加引用符（mysql 系反引号、oracle/pg 系双引号、sqlserver 方括号），非保留字保持裸标识符，避免 oracle/pg 上引号引入的大小写敏感问题。

只受理纯标识符：`sum(col)`、`col as alias` 这类表达式请继续用 `@value`（安全自负）。

前提：保留字引用依赖 `spring.sqltoy.reservedWords` 配置，而该配置没有默认值。 未配置时保留字集合为空，`ReservedWordsUtil.convertWord` 会原样返回、不加任何引用符，于是字段列表里含保留字（`order`、`desc`、`level`、`group` 等）时会产出裸标识符并被数据库拒绝——三库实测均报错：PostgreSQL `ERROR: syntax error at or near ","`（`select order,staff_code` 里的 `order` 被当成 `ORDER BY` 关键字解析）、MySQL `You have an error in your SQL syntax`、SQL Server `Incorrect syntax near the keyword 'order'`。因此**若字段名可能命中保留字，必须配置保留字词表**：

```
spring.sqltoy.reservedWords=order,desc,level,group,key,comment
```

配置后同一查询在三个库上分别产出 `select "order",staff_code`（PG）、`select `order`,staff_code`（MySQL）、`select [order],staff_code`（SQL Server），实测均正常执行并取回列值。

另外，字段名校验失败抛出的 `IllegalArgumentException` 经 DAO 层会被统一包装成 `DataAccessException`，捕获时请取根因（`getCause()` 链末端），异常信息里带有具体的非法值。

约束： 与 `@loop` 等宏一样，SQL 完全没有任何参数时会提前返回、宏不展开；实际报表场景总有 `:fields`，故不受影响。

示例：

```
<sql id="report_columns">
    select @joinFields(:cols,{t.*}),count(1) total
    from sqltoy_staff_info t
    #[where t.status=:status]
    group by @joinFields(:cols,{t.organ_id})
</sql>
```

```
// 由页面配置决定展示与分组哪些列
List result = lightDao.find("report_columns",
        MapKit.keys("cols", "status").values("organ_id,dept_id", 1));
```

九、@df() / @day()（日期格式化）

把日期格式化成字符串后拼进 SQL，**不需要**关心目标库的日期函数（这是**字符串拼接**，不是数据库函数替换）。

```
@df(格式)              → 取当前时间按格式输出
@df(参数, 格式)        → 把参数值按格式输出
@day(...)              → 与 @df 完全等价（历史别名）
```

- 默认格式 `yyMMdd`；
- 格式为空串或 `null` 时**输出空串**（`@df('')` 可实现"某段条件整体省略"的写法）；
- 参数可以是命名参数（`@df(beginDate, yyyy-MM-dd)`）或 `${beginDate}` 形式，也可直接用字面量；**不要写成 `:beginDate`**——宏的取值来自传入的参数 Map，键名不带冒号。

```
<sql id="query_by_month">
    select * from sqltoy_order_info t
    where t.trans_month = '@df(yyyyMM)'
    #[and t.trans_date >= '@df(beginDate, yyyy-MM-dd)']
</sql>
```

> 与 filter 的 `<to-date>` 区别：`@df` 是把**值格式化后拼进 SQL 文本**（会形成字面量），`<to-date>` 是把**参数转成日期类型后仍以 `?` 绑定**。涉及外部输入时优先用 `<to-date>`，`@df` 更适合报表拼接期间、分区表名等场景。

十、@case()（decode 式枚举判断）

类似 Oracle `decode` 的取值映射，把参数值按枚举翻译成对应的字符串或 SQL 片段：

```
@case(参数, 值1, 结果1, 值2, 结果2, ..., 默认结果)
```

- 参数可以是命名参数（`@case(orderType, ...)`）或 `${orderType}` 形式；
- 参数少于 3 个时输出空串；
- **最后一个参数是默认结果**（没有匹配项时输出）。

```
<!-- 按订单类型拼不同的排序规则 -->
select * from sqltoy_order_info t
where t.status=1
order by @case(orderType,'SALE','t.sale_date desc','BUY','t.buy_date desc','t.create_time desc')
```

> 业务主键场景同样使用该宏，见 主键策略 · @BusinessId。

十一、@substr() / @substring()（子串截取）

```
@substr(参数, 起始位置, 长度)
```

- 起始位置从 0 开始；**为负数表示从尾部截取**；
- 起始位置大于等于字符串长度时输出空串；
- 参数为空时输出空串；`@substring` 与 `@substr` 等价。

```
<!-- 用订单号前两位区分业务线 -->
select * from sqltoy_order_info t
where t.biz_line = '@substr(orderId,0,2)'
```

十二、@virtualTable()（跨库虚表）

"select 常量行 union 真实查询"类写法（字典下拉兜底行、报表固定合计行等）在 oracle/db2/hana 上必须挂伪表，而 mysql/pg/sqlserver 等写 `from dual` 反而报错。用该宏在解析期按方言自动展开：

```
select '0' id,'全局公共' name @virtualTable()
union
select id,name from sqltoy_dict_detail where status=1
```

- oracle 系（oracle/oracle11/dm/oceanbase）→ `from dual`；db2 → `from sysibm.sysdummy1`；hana → `from dummy`；其余方言直接剔除；
- 兼容别名 `@dual()` / `@dummyTable()`，三者等价、大小写不敏感、括号内空白容错；一条 SQL 可出现多次；
- 不含该宏的 SQL 零影响；详见方言自适配与函数扩展。

十三、@split()（参数拆分数组）

把传入的**一个字符串参数**按分隔符拆分成数组，交给 `in` 等集合展开机制使用——典型场景是前端只传 `"A;B;C"` 一个字符串，SQL 里直接 `field in (...)`，省去 Java 侧手工 split。在 SQL 解析期由 `SqlConfigParseUtils` 处理。

三种形态：

```
-- 1、默认逗号分隔（按正则 \s*,\s* 切分，容忍空白）
select * from sqltoy_order_info t where t.status in (@split(:statusStr))

-- 2、指定分隔符（可带单引号或双引号）
select * from sqltoy_order_info t where t.status in (@split(:statusStr,';'))

-- 3、指定分隔符并转数字数组（各段转 Integer，空白段跳过）
select * from sqltoy_order_info t where t.status in (@split(:statusStr,int,','))
```

要点：

- `@split(...)` 执行期被替换回 `?`，切分后的数组放入对应参数位，随后按集合展开机制自动拆成多个 `?` 占位符；
- 仅当参数值为 `String` 时才切分；每段自动 trim，参数自带首尾引号时先剥离；
- 不传第二个参数时默认分隔符为逗号（`\s*,\s*`）。

Java 调用（前端传一个字符串即可）：

```
List rows = lightDao.findByQuery(
        new QueryExecutor("sqltoy_order_info_find")
            .names("statusStr").values("1,2,3")   // 一个字符串，无需前端给数组
            .resultType(OrderVO.class))
        .getRows();
```

> 相关：`#[]` 与 filters 过滤器、常见 SQL 案例、分页优化、注解速查。

注解速查

sqltoy 的全部注解都位于 `org.sagacity.sqltoy.config.annotation`（共 23 个），按用途分为实体映射、主键与业务号、关联、企业级特性、缓存翻译、DDL 六组。本页给出每个注解的**目标位置、属性、默认值与用途**，细节用法请跳转到对应章节。

> 绝大多数 POJO 由 quickvo 按数据库表自动生成并带好注解，手写注解主要出现在：调整映射、补充 `@Index` / `@Partition` 等 DDL 信息、给已有实体加企业级特性。

一、总览

注解	目标	用途	详见
`@SqlToyEntity`	类	标记为 sqltoy 实体（`@Entity` 的语义化别名，扫描依据之一）	本页
`@Entity`	类	表映射：表名/模式/注释/主键约束名	对象化 CRUD
`@Column`	字段	列映射：列名、JDBC 类型、长度、精度、默认值等	对象化 CRUD
`@Id`	字段	主键：生成策略、sequence、生成器、辅助主键	主键策略
`@BusinessId`	字段	业务主键（订单号等规则号）与 `generateBizId` 配合	主键策略
`@SqlToyFieldAlias`	字段	VO/POJO 字段名不一致时的别名映射	工具类
`@OneToOne`	字段	一对一/一对多级联加载与保存	对象化 CRUD
`@OneToMany`	字段	一对多级联（可带过滤、排序）	对象化 CRUD
`@Foreign`	字段	外键约束（DDL 生成与删除/更新参照动作）	POJO 生成 DDL
`@Sharding`	类	分库分表策略（库、表、并发、全局回滚）	分库分表
`@Strategy`	类	分片策略引用（嵌在 `@Sharding` 内）	分库分表
`@Tenant`	类 / 字段	多租户字段标记	多租户
`@DataVersion`	类 / 字段	乐观锁版本号字段	数据版本控制
`@Secure`	类	字段加密或脱敏规则	数据脱敏与加解密
`@SecureConfig`	类	多条 `@Secure` 的容器	数据脱敏与加解密
`@Translate`	字段	查询结果缓存翻译（单条）	缓存翻译使用
`@Translates`	字段	多条 `@Translate` 的容器	缓存翻译使用
`@Index`	类	索引定义（进 DDL）	POJO 生成 DDL
`@Indexes`	类	多条 `@Index` 的容器	POJO 生成 DDL
`@Partition`	类	表分区策略（进 DDL）	POJO 生成 DDL
`@PartitionDef`	类	单个分区明细（嵌在 `@Partition` 内）	POJO 生成 DDL
`@PartitionKey`	字段	分区字段标记（MPP 库建表与 DML 提示）	POJO 生成 DDL
`@MppTable`	类	MPP 分析库表引擎元数据（ClickHouse/Doris/StarRocks）	POJO 生成 DDL

二、实体与字段映射

@SqlToyEntity / @Entity

`@SqlToyEntity` 无属性，是 `@Entity` 的语义化别名；二者都可用于实体扫描（`packagesToScan`）。

`@Entity` 属性	类型	默认	说明
`tableName`	String	必填	数据库表名（不含 schema）
`schema`	String	`""`	模式名，多 schema 场景使用
`comment`	String	`""`	表注释，建表时写入
`pk_constraint`	String	`""`	主键约束名，如 `PRIMARY`；建表时用于命名主键

@Column

属性	类型	默认	说明
`name`	String	必填	数据库列名
`type`	int	`java.sql.Types.OTHER`	JDBC 类型；`OTHER` 时由字段 Java 类型推断
`nativeType`	String	`""`	原生类型名（如 `varchar(50)`、`jsonb`），优先于 `type` 用于建表
`nullable`	boolean	`true`	是否可空
`length`	long	`0`	长度（字符/字节按字段类型）
`precision` / `scale`	int	`0`	数值精度与小数位
`generatedType`	`GeneratedType`	`DEFAULT`	生成列：`DEFAULT`（普通）/ `VIRTUAL`（虚拟列）/ `STORED`（存储生成列）
`keyword`	boolean	`false`	列名是否数据库保留字（影响对象操作为列名加引用符）
`comment`	String	`""`	列注释
`defaultValue`	String	`SqlToyConstants.DEFAULT_NULL`	建表默认值；未设置表示无默认值
`autoIncrement`	boolean	`false`	是否自增列（DDL 用）

> `keyword=true` 与全局 `spring.sqltoy.reservedWords` 是两条不同路径：前者解决**对象操作**中该列名的引用，后者解决**自定义 SQL** 中保留字的按方言适配，详见常见问题。

@SqlToyFieldAlias

属性	类型	说明
`value`	String	目标 POJO 的属性名；用于 VO 与 POJO 字段名不同时 `convertType` 的正确映射

三、主键与业务号

@Id

属性	类型	默认	说明
`strategy`	String	`assign`	主键策略：`assign`（手工赋值）、`sequence`、`identity`、`default`（22 位）、`nanotime`（26 位）、`snowflake`、`UUID`、`ULID`、`redis` 等，取值与主键策略一致
`sequence`	String	`""`	`sequence` 策略对应的序列名
`generator`	String	`""`	自定义生成器实现类（实现 `IdGenerator`），配置后优先于 `strategy` 的内置实现
`isAssist`	boolean	`false`	辅助主键：分区表的分区字段在有主键时必须同为主键，用它标记（mysql、pg 等）

@BusinessId

属性	类型	默认	说明
`signature`	String	必填	识别区分符表达式，支持 `@case(name,v1,then1,...)`、`@day(yyMMdd)`、`@substr(name,start,length)` 等宏
`generator`	String	必填	生成策略实现类，如 `org.sagacity.sqltoy.plugins.id.impl.RedisIdGenerator`
`length`	int	`-1`	业务主键总长度，-1 取全局配置
`sequenceSize`	int	`-1`	尾部流水位数，-1 取全局配置
`relatedColumns`	String[]	`{}`	参与生成的关联字段（取值来源）
`start`	long	`1`	流水初始值

> 一个表只支持配置**单个**业务主键；多个业务主键时用 `@BusinessId` + `lightDao.generateBizId(...)` 自行赋值，详见主键策略。

四、关联与级联

@OneToOne / @OneToMany

两者属性基本一致，`@OneToMany` 多一个 `orderBy`：

属性	类型	默认	说明
`fields`	String[]	`{}`	主表参与关联的字段（为空表示用主键）
`mappedFields`	String[]	必填	子表对应的字段
`delete`	boolean	`false`	删除主表时是否级联删除子表
`load`	String	`""`	级联加载时附带的过滤条件（如 `status=1`）
`update`	String	`""`	级联修改控制
`notNullField`	String	`""`	非空校验字段
`orderBy`	String	`""`	仅 `@OneToMany`：子集合排序

> sqltoy 只支持**一级级联**；`loadAllCascade` / `updateCascade` 等批量方法底层用 in 查询优化，避免 N+1。

@Foreign

属性	类型	默认	说明
`constraintName`	String	`""`	外键约束名
`table`	String	必填	被引用的表
`field`	String	必填	被引用的列
`deleteRestict`	int	`1`	删除参照动作（`ReferentialAction` 常量：CASCADE/RESTRICT/SET_NULL/NO_ACTION/SET_DEFAULT）
`updateRestict`	int	`1`	更新参照动作，同上

> 6.0.3 起按数据库兼容性自动裁剪不支持的参照动作；字段名拼写 `Restict` 为历史遗留，勿改。

五、企业级特性

@Sharding 与 @Strategy

`@Sharding` 属性	类型	默认	说明
`db`	`@Strategy`	空	分库策略
`table`	`@Strategy`	空	分表策略
`maxConcurrents`	int	`10`	分片并行线程数：0 不限制、1 串行
`maxWaitSeconds`	int	`-1`	分片并行等待上限（秒）
`is_global_rollback`	boolean	`true`	任一分片失败是否全局回滚

`@Strategy` 属性	类型	默认	说明
`name`	String	`""`	策略 bean 名称
`fields`	String[]	`{}`	参与决策的字段
`aliasNames`	String[]	`{}`	字段别名（与 `fields` 对应，用于参数别名场景）
`decisionType`	String	`""`	决策类型（传给策略实现区分场景，如查询/更新）

```
@Sharding(db = @Strategy(name = "hashDataSource", fields = { "tenantId" }),
          table = @Strategy(name = "realHisTable", fields = { "bizDate" }))
```

@Tenant

属性	类型	默认	说明
`field`	String	`""`	租户字段名（实体属性名，框架自动映射列名）；为空时按约定/拦截器处理

@DataVersion

属性	类型	默认	说明
`field`	String	`""`	版本号属性名
`startDate`	boolean	`false`	版本号是否以日期开头（如 `202209231`）

@Secure / @SecureConfig

`@Secure` 属性	类型	默认	说明
`field`	String	必填	被加密/脱敏的列名
`secureType`	`SecureType`	`ENCRYPT`	`encrypt`（RSA 加解密）或脱敏类型：`tel`、`name`、`email`、`address`、`bank-card`、`id-card`、`public-account`、`discrete-rate`、`other`
`sourceField`	String	`""`	脱敏取值的源字段（如姓名取 `staffName` 的某个字）
`maskCode`	String	`""`	脱敏替换字符，默认 `*`
`headSize` / `tailSize`	int	`0`	保留首/尾字符数
`maskRate`	int	`0`	离散脱敏比例（`discrete-rate` 用）

`@SecureConfig` 只有一个属性 `secures`（`Secure[]`），把多条 `@Secure` 装配到同一个实体上。

六、缓存翻译

@Translate / @Translates

`@Translate` 属性	类型	默认	说明
`cacheName`	String	必填	缓存名称（对应 `sqltoy-translate.xml` 中的 `cache`）
`cacheType`	String	`""`	缓存内的类型（如字典分类）
`cacheIndex`	int	`1`	用结果集的第几列去匹配缓存 key
`keyField`	String	必填	翻译结果写入的属性名
`split`	String	`""`	多个 key 的分隔符（一个字段含多个码值时）
`join`	String	`""`	翻译结果多值时的拼接符
`where`	String	`""`	条件翻译：仅当条件成立才翻译
`uncached`	String	`""`	未命中时的占位输出

`@Translates` 只有一个属性 `value`（`Translate[]`）。

七、DDL 相关

注解	必要属性	说明
`@Index`	`name`、`columns`	`isUnique`（默认 false）、`sortTypes`（各列 ASC/DESC）
`@Indexes`	`indexes`	多条 `@Index`
`@Partition`	`strategy`	分区策略（RANGE/LIST/HASH/RANGE COLUMNS/LIST COLUMNS）；`columns`、`expression`、`partitions`
`@PartitionDef`	`name`	分区明细：`value`、`tablespace`、`endValue`、`endInclusive`（默认 true）
`@PartitionKey`	无	标记分区字段，MPP 库建表与 DML 的分区键提示
`@MppTable`	无（按需）	`engine`、`engineArgs`、`keyModel`、`orderBy`、`distributedBy`、`buckets`、`properties`，对应 ClickHouse/Doris/StarRocks 的引擎元数据

```
@SqlToyEntity
@Entity(tableName = "sqltoy_order_info", pk_constraint = "PRIMARY", comment = "订单主表")
@Indexes({ @Index(name = "idx_order_staff", columns = { "STAFF_CODE" }),
           @Index(name = "uk_order_no", isUnique = true, columns = { "ORDER_NO" }) })
@Partition(strategy = "RANGE", columns = { "TRANS_DATE" }, partitions = {
        @PartitionDef(name = "p202601", endValue = "2026-02-01"),
        @PartitionDef(name = "p202602", endValue = "2026-03-01", endInclusive = false) })
public class OrderInfo implements Serializable {
    @Id(strategy = "default")
    @Column(name = "ID", type = java.sql.Types.VARCHAR, length = 22, nullable = false)
    private String id;
    // ...
}
```

八、常见组合与注意事项

- **实体扫描**：`@Entity` / `@SqlToyEntity` 二者任一即可；`packagesToScan` 未配置时实体在首次使用时按需加载（native 场景**必须**配置，见 GraalVM Native Image 指引）；
- **`@Id` 与 `@Column` 必须同时标注在主键字段上**，`@Id` 只描述生成策略、`@Column` 描述列；
- **`@Column.type` 留 `OTHER`** 让框架按 Java 类型推断是最省心的做法；跨库建表有差异时用 `nativeType` 精确控制；
- **复合主键**：多个字段标 `@Id`，join 的 lambda 更新/删除会按 `LPK1…LPKn` 关联；

- **DDL 相关注解（`@Index` / `@Partition` / `@MppTable` / `@Foreign`）只在 `autoDDL=true` 建表或由 `lightDao.tableApi()` 生成 DDL 时生效**，不影响运行时 CRUD；
- **企业级注解是"声明"不是"配置"**：`@Tenant` / `@Sharding` / `@Secure` 都需要对应的拦截器、策略 bean 或 provider 配合，单独标注不会生效。

API 速查

`LightDao` 是 sqltoy 的统一操作入口：**72 个方法名、109 个声明（含重载）**，覆盖对象操作、查询、执行、缓存翻译、扩展元数据与链式入口。本页按用途分组给出方法名与对应章节，便于"知道要做什么、找到方法名、再跳到详细页"。

> - 以 `LightDao` 接口为准（`org.sagacity.sqltoy.dao.LightDao`）；`SqlToyDao` / `SqlToyLazyDao` 为早期历史接口，新项目统一用 `LightDao`。
> - 所有方法都有基于 `QueryExecutor` / `EntityQuery` 的底层实现，理解两者即可灵活组合，见常规查询 API。
> - 完整的参数签名与 Javadoc 见 javadoc.io。

一、链式入口（13）

链式 API 适合需要"多参数 + 指定数据源 + 批量 + 锁模式"的场景，每个入口返回对应构建器，最终以 `one()/many()/find()/update()` 等终结。

方法	返回	用途	详见
`query()`	`Query`	链式查询	Link 链式操作
`save()`	`Save`	链式保存（含 `saveMode`、`batchSize`）	Link 链式操作
`update()`	`Update`	链式更新（含 `forceUpdateProps`）	Link 链式操作
`delete()`	`Delete`	链式删除	Link 链式操作
`load()`	`Load`	链式加载（含 `lock`、级联）	Link 链式操作
`unique()`	`Unique`	唯一性校验并取记录	Link 链式操作
`execute()`	`Execute`	执行任意 SQL/DML	Link 链式操作
`batch()`	`Batch`	批量执行	Link 链式操作
`store()`	`Store`	存储过程（含多结果集）	存储过程调用
`treeTable()`	`TreeTable`	树形表操作	树形表节点路径构造
`tableApi()`	`TableApi`	表/列元数据、truncate/drop	表元数据操作
`elastic()`	`Elastic`	Elasticsearch（sql/json 双模式）	Elasticsearch 支持
`mongo()`	`Mongo`	MongoDB 查询/聚合	MongoDB 支持

二、Lambda 条件（3）

方法	用途	详见
`lambdaQuery(Class)`	返回绑定当前 dao 的 `LambdaQuery`，终端 `find/getOne/findPage/findTop/findRandom/count/exists`	Lambda 条件查询与更新
`lambdaUpdate(Class)`	返回 `LambdaUpdate`，`set/setExpr` + 条件后 `update()`	Lambda 条件查询与更新
`lambdaDelete(Class)`	返回 `LambdaDelete`，条件后 `delete()`	Lambda 条件查询与更新

三、保存与插入（5）

方法	说明
`save(entity)`	保存单条，返回主键（有主键策略时回填）
`saveAll(List)`	批量保存，返回影响行数
`saveAllIgnoreExist(List)`	批量保存，已存在则跳过（对应 `insert ignore` / `on conflict do nothing` 语义）
`saveOrUpdate(entity, forceUpdateProps...)`	存在则改、不存在则插（merge 语义）
`saveOrUpdateAll(List, forceUpdateProps...)`	批量 merge

四、修改（8）

方法	说明	详见
`update(entity, forceUpdateProps...)`	弹性更新：非 null 字段参与，`forceUpdateProps` 指定强制更新字段	对象化 CRUD
`updateDeeply(entity)`	深度更新：全部字段参与（含 null）	对象化 CRUD
`updateAll(List, forceUpdateProps...)` / `updateAllDeeply(List)`	批量弹性/深度更新	对象化 CRUD
`updateByQuery(Class, EntityUpdate)`	按单表条件更新，支持 `set 字段=字段+?` 自更新	对象化 CRUD
`updateSaveFetch(...)`	锁查询、不存在则插入、存在则修改，并返回结果（多组唯一键重载；台账类高并发场景首选）	对象化 CRUD
`updateFetch(QueryExecutor, UpdateRowHandler)`	锁查询→校验→修改→返回，一次交互（记录必然存在、条件自由编写的场景）	对象化 CRUD
`updateCascade(entity, ...)`	级联修改（含子表强制更新字段 Map）	对象化 CRUD

五、删除（5）

方法	说明
`delete(entity)` / `deleteAll(List)`	按对象主键删除
`deleteByIds(Class, Object... ids)`	按主键集合删除
`deleteByQuery(Class, EntityQuery)`	按单表条件删除
`truncate(Class)`	清空表

六、加载（8）

方法	说明	详见
`load(entity)` / `load(entity, LockMode)`	按主键加载，可加锁	对象化 CRUD
`loadById(Class, id)` / `loadByIds(Class, Object... ids)`	按主键（集合）加载	对象化 CRUD
`loadAll(List)` / `loadAll(List, LockMode)`	批量加载（底层 in 优化）	对象化 CRUD
`loadCascade(entity, LockMode, Class... cascadeTypes)` / `loadAllCascade(List, Class...)`	级联加载（可选加载哪些子表）	对象化 CRUD
`loadEntity(Class, EntityQuery[, resultType])`	单表条件加载	常规查询 API
`loadByQuery(QueryExecutor)`	按 `QueryExecutor` 加载	常规查询 API

七、查询（15）

方法	说明	详见
`findOne(sqlOrSqlId, params, resultType)`	单条	常规查询 API
`find(sqlOrSqlId, params[, resultType])`	列表（多种参数形态重载）	常规查询 API
`findByQuery(QueryExecutor)`	`QueryExecutor` 统一入口	常规查询 API
`findPage(page, sqlOrSqlId, params[, resultType])`	分页	分页优化
`findPageByQuery(page, QueryExecutor)`	分页统一入口	分页优化
`findTop(sqlOrSqlId, params, topSize[, resultType])`	前 N 条 / 前 N%	常规查询 API
`findTopByQuery(QueryExecutor, topSize)`	同上，`QueryExecutor` 形态	常规查询 API
`findRandom(sqlOrSqlId, params, randomCount[, resultType])`	随机 N 条 / 比例	常规查询 API
`findRandomByQuery(QueryExecutor, randomCount)`	同上，`QueryExecutor` 形态	常规查询 API
`findEntity(Class, EntityQuery[, resultType])`	单表实体查询	常规查询 API
`findPageEntity(page, Class, EntityQuery[, resultType])`	单表实体分页	常规查询 API
`getValue(sqlOrSqlId, params[, resultType])`	取单值	常规查询 API
`getCount(sqlOrSqlId, Map)` / `getCount(Class, EntityQuery)`	取记录数	常规查询 API
`isUnique(entity, paramsNamed...)`	唯一性校验	常规查询 API
`fetchStream(QueryExecutor, StreamResultHandler)`	流式回调，超大数据量	常规查询 API

八、执行与其他（见下表）

方法	说明	详见
`executeSql(sqlOrSqlId[, params])`	执行变更类 SQL（4 个参数形态重载）	Link 链式操作
`batchUpdate(sqlOrSqlId, dataSet[, autoCommit])`	批量执行	Link 链式操作
`insertReturnPrimaryKey(sqlOrSqlId, entity, pkField)`	插入并返回主键	Link 链式操作
`executeStore(storeSqlOrKey, inParamValues)` / `executeMoreResultStore(...)`	存储过程（含多结果集）	存储过程调用
`parallelQuery(List<ParallelQuery>, paramsMap[, ParallelConfig])`	并行查询	并行查询
`wrapTreeTableRoute(TreeTableModel)`	构造树节点路径/层级/叶子标记	树形表节点路径构造
`generateBizId(signature, increment)` / `generateBizId(entity)` / `generateBizId(table, signature, params, ...)`	生成业务主键	主键策略
`convertType(source, resultType, ignoreProperties...)`	DTO↔POJO、List、Page 转换	工具类
`getTranslateCache(cacheName, cacheType[, resultType])` / `translate(...)` / `cacheMatchKeys(...)` / `existCache(...)` / `getCacheNames()`	缓存翻译相关 API（下拉框取值、反调翻译、反向匹配 key）	缓存翻译使用
`getSqlToyConfig(sqlKey, SqlType)` / `getEntityMeta(Class)`	取 SQL 配置 / 实体元数据（报表、低代码平台集成常用）	扩展集成
`getSqlToyContext()` / `getDataSource()` / `flush()`	上下文、数据源、刷新	技术架构

九、扩展点（SPI）速查

除 DAO 方法外，sqltoy 通过以下接口开放扩展能力；实现后按配置参数中对应属性挂载。

9.1 配置式扩展点（15 个）

配置项	接口	用途
`unifyFieldsHandler`	`IUnifyFieldsHandler`	公共字段统一赋值（创建人/时间、租户等）
`typeHandler`	`TypeHandler`	自定义列类型映射（JSON/vector/geometry 已内置）
`sqlInterceptors`	`SqlInterceptor`	SQL 集中加工（租户过滤、改写 schema）
`sqlFormater`	`SqlFormater`	SQL 日志格式化（默认依赖 druid）
`translateCacheManager`	`TranslateCacheManager`	缓存翻译容器（ehcache/caffeine 之外的实现）
`dynamicCacheFetch`	`DynamicCacheFetch`	FIFO 动态缓存的批量捞取（常基于 redis）
`dynamicFecthCacheManager`	`DynamicFetchCacheManager`	动态缓存管理器
`dataSourceSelector`	`DataSourceSelector`	动态数据源路由
`connectionFactory`	`ConnectionFactory`	连接获取/释放（纯 Java 场景）
`fieldsSecureProvider`	`FieldsSecureProvider`	字段加解密算法
`desensitizeProvider`	`DesensitizeProvider`	脱敏算法
`customFilterHandler`	`FilterHandler`	自定义参数过滤器（`<custom-handler>`）
`overTimeSqlHandler`	`OverTimeSqlHandler`	慢 SQL 收集与上报
`dialectDDLGenerator`	`DialectDDLGenerator`	自定义建表 DDL 生成
`firstBizCodeTrace`	`FirstBizCodeTrace`	记录 SQL 的业务调用来源

9.2 回调与集成接口

包	接口	用途
`callback`	`StreamResultHandler`	流式查询逐行回调（`fetchStream`）
`callback`	`UpdateRowHandler` / `UpdateRowCallback`	`updateFetch` 对锁记录的处理
`callback`	`EntityUpdateCallback`	`updateSaveFetch` 的实体更新回调
`callback`	`DataSourceCallbackHandler`	直接操作 `Connection`（`doConnection(conn, DBProfile)`）
`callback`	`TransactionHandler`	自定义事务内逻辑
`callback`	`LockSqlHandler` / `UniqueSqlHandler` / `GenerateSqlHandler`	方言级 SQL 生成接管（加锁、唯一性取 top、自定义 sql）
`callback`	`GenerateSavePKStrategy`	自定义主键回填策略
`callback`	`TreeIdAndPidGet`	树形集合排序取 id/pid
`callback`	`DbAdapterHandler`	跨库适配回调
`callback`	`DecryptHandler` / `CacheFilter`	查询解密、缓存 key 过滤（租户/状态维度）
`callback`	`RowCallbackHandler` / `PreparedStatementResultHandler` / `CallableStatementResultHandler` / `InsertRowCallbackHandler` / `XMLCallbackHandler`	行/语句/存储过程/批量插入/XML 解析级回调（极少使用，留给极端场景）
`integration`	`AppContext` / `ConnectionFactory` / `DistributeIdGenerator` / `MongoQuery`	框架无关 SPI，Spring/Solon/纯 Java 各自实现

> 自定义实现类在 **native 镜像**下需要注册反射，6.0.8 起上表 15 个配置式扩展点会按配置自动注册，见 GraalVM Native Image 指引。

版本更新日志（6.0 线）

6.0 线从 JDK 17 起（JDK 8 项目请留在 `5.6.96.jre8`）。本页是**索引**：每个版本的完整变更说明、注意事项与升级动作都在6.0 升级指南对应章节。

版本	要点	详见
6.0.8	**lambda 条件体系并入核心**（`org.sagacity.sqltoy.lambda`，单表/join 一套构建器，原 sqltoy-plus 能力内建）；**GraalVM AOT 重写**（动态注册资源/扩展类/JSON/驱动类反射，新增 `native-reflection-classes` / `native-reflection-packages` / `native-entity-proxies`，修复 5.6 以来实体 hints 恒为空）；`#[...]` 剔除后清理悬挂 and/or	十、6.0.8 · Lambda 条件 · GraalVM
6.0.7	JSON 库可插拔（`JsonProvider`：fastjson2/jackson3/jackson2）；新增 `@joinFields()` 字段名列表拼接宏；`#[]` 剔除后清理括号内悬挂 and/or	九、6.0.7 · 标签与表达式
6.0.6	新增虚表宏 `@virtualTable()`（兼容 `@dual()` / `@dummyTable()`），解决"常量行 union 查询"的跨库写法；quickvo-maven-plugin 2.0.3	八、6.0.6 · 方言与函数
6.0.5	新增瀚高 HighGo 方言（方言总数 24→25）；新增 11 个跨库函数类；PG 折叠家族（OpenGauss/GaussDB/MogDB/Vastbase/StarDB/Oscar/Kingbase/HighGo）插入返回主键列名统一转小写	七、6.0.5
6.0.4	缓存翻译 rest 模式认证增强（basic/jwt，支持 token-url 登录、本地自签、静态 token）；函数适配完善（`nvl2`、负起点 `substr`、`curdate` 等）；修复 SubStr 负起点与 HH 时间 token 的静默错值	六、6.0.4
6.0.3	`<sql dialect="xxx">` 元素属性；`spring.sqltoy.realDialectFirst`；POJO 生成 DDL 增强（`@Partition` / `@PartitionDef` / `@MppTable`、Doris/StarRocks/ClickHouse/DB2/HANA/SQLite 专用生成器、`@Foreign` 参照动作）；分库分表并行执行修复	五、6.0.3 · POJO 生成 DDL
6.0.0	JDK 17+；API 更名 `parallQuery`→`parallelQuery`（`ParallelQuery`）、`doConnection(conn, DBProfile)`；包结构调整（`PageOptimizeUtils` / `ParallelUtils` / `TranslateUtils` 等迁移）；新增 SAP HANA 方言	二、API 变更 · 三、包结构调整
5.6.96.jre8	JDK 8 的最终版本（仍支持 quickvo-maven-plugin）	环境要求

怎么看某个版本改了什么

1. 本页定位版本 → 点"详见"进升级指南对应章节；
2. 需要**逐行细节**（每个 bug 的根因、验证数据库、测试用例）看源码仓库的 `trunk/sqltoy-orm-core/changelog.md`；
3. 需要**是否要改代码**的结论：升级指南第二节（API 变更）、第三节（包结构调整），其余为增量能力。

版本选择建议

场景	建议
新项目	直接用最新 6.0.x
存量 5.6.x（JDK 8）	保持 `5.6.96.jre8`，不要为了升级而升级 JDK
存量 5.6.x（JDK 17）	按升级指南各节顺序评估，重点看 6.0.0 的 API/包变更
使用 sqltoy-plus	升到 6.0.8，按升级指南第十节第 2 条迁移到内置 lambda
要构建 native 镜像	6.0.8+，且 GraalVM 25+

> 历史版本（5.x 及更早）的变更请查阅源码仓库 changelog；本页只维护 6.0 线。

必杀集锦

增删改查、级联、跨库、分库分表等，sqltoy 与 hibernate/mybatis 并无太多差异。sqltoy 真正想分享的是**不同的思考方式**——下面这些"必杀技"是它在大量项目实践中沉淀出的高价值特性。

一、整洁的 SQL 写法

需求：查询订单表，条件包含交易日期范围、订单状态、机构、员工、订单号。

mybatis 写法（`<if>` 夹杂在 SQL 中，繁琐且易错）：

```
<select id="findOrder" resultType="OrderInfoVO">
    select * from sqltoy_order_info t
    <where>
        <if test="beginDate!=null"> and t.trans_date &gt;= #{beginDate} </if>
        <if test="endDate!=null"> and t.trans_date &lt;= #{endDate} </if>
        <if test="status!=null"> and t.status=#{status} </if>
        <if test="organId!=null"> and t.organ_id=#{organId} </if>
        <if test="staffCode!=null"> and t.staff_code=#{staffCode} </if>
        <if test="orderId!=null"> and t.order_id=#{orderId} </if>
    </where>
</select>
```

sqltoy 写法（SQL 整洁、可直接复制到客户端运行，参数为 null 自动剔除对应片段）：

```
<sql id="findOrder">
    <value><![CDATA[
        select * from sqltoy_order_info t
        where 1=1
        #[and t.trans_date>=:beginDate]
        #[and t.trans_date<=:endDate]
        #[and t.status=:status]
        #[and t.organ_id=:organId]
        #[and t.staff_code=:staffCode]
        #[and t.order_id=:orderId]
    ]]></value>
</sql>
```

一眼就能看明白业务含义，后期维护、优化、调整都更容易。详见动态 SQL 编写规范。

二、快速分页与 count 优化

sqltoy 自动提供智能分页，开发者无需了解各数据库分页方言。其 count 查询**不是**简单粗暴的 `select count(1) from (你的sql) as t`，而是：

- 剔除不必要的 `order by`；
- 剔除 `select` 与 `from` 之间不必要的函数运算与无关信息；
- 充分辨别 `union` 与 `group` 类型查询。

配合 `@fast()`（先分页后关联）与 `<page-optimize>`（缓存 count），可让原本 2 次的分页查询降到约 **1.35 次**。详见分页优化。

三、缓存翻译

把"码值→名称"的转换交给缓存，而不是写一堆 `left join`。没有缓存翻译时，多表关联的 SQL 又长又慢；用了缓存翻译，SQL 简洁清晰、效率几何级提升。详见缓存翻译使用。

四、函数替换（跨库）

对象操作容易跨库，但**手写的 SQL** 呢？用 MySQL 写的 SQL 能直接跑在 Oracle/SQL Server 上吗？sqltoy 给了答案：在首次加载或动态执行时，**根据数据库类型自动替换 SQL 中的函数**（如 `nvl` / `ifnull` / `isnull`、`concat`、`to_char`、`date_format` 等）。详见方言自适配与函数扩展。

五、防止 SQL 注入

很多人误解 SQL 注入原理，靠"过滤页面传来的参数值是否含 SQL 片段"来防护，南辕北辙。例如 `select * from table t where t.password=:password`，若把 password 变成 `(select password from table where id=xxx)` 就可能穿透这种检查。

sqltoy 从根本上杜绝：**SQL 参数最终全部转换成 `?` 形式，通过 `PreparedStatement` 设置参数值**，参数永远只会被当作"值"而非"SQL 片段"。

```
-- 你写的：
select * from sqltoy_staff_info t where t.staff_name=:staffName
-- 实际执行：
select * from sqltoy_staff_info t where t.staff_name=?
```

此外还可用 `<valid-sqlInjection>` 过滤器对参数做注入校验（见动态 SQL 编写规范）。

六、SQL 中可以加注释

sqltoy 坚持让开发者把在 DBeaver 等客户端写好的 SQL **直接 copy 到 xml**（修改时也可从 xml copy 回客户端调试），因此 SQL 可以保留注释：

- `--` 与 `/* */` 形式的注释在 SQL 加载时会被自动剔除；
- 但 `/*+ xxxx */` 形式的注释**不会被剔除**——这种通常作为数据库 hint 用于优化 SQL 性能。

> 更多实战写法见常见 SQL 案例与补充说明：标签与表达式参考。

常见问题解答（FAQ）

一、分库分表为什么不支持多库多表的查询和分页?

实现并不难，复杂的是多库多表查询后的分页归并。sqltoy 不做这个实现**不是做不了，而是没有必要**——出现这种需求本质是设计问题。目前一般做法是通过 MPP 数据库来解决，如 ClickHouse、StarRocks，这些 sqltoy 都支持。

详见分库分表。

二、sqltoy-orm 性能如何?

sqltoy 的出发点之一就是解决性能问题：

- **主键策略**：支持非中心式的程序自动产生模式（22 位、26 位有序纳秒、雪花算法等）。早期用过数据库锁模式，教训惨烈。
- **update 操作**：改变 hibernate"先 load 再 update"的模式，变成**一步数据库交互**（弹性更新）。
- 所有 SQL 查询都基于 `PreparedStatement`，减少数据库对 SQL 的预编译过程。
- 批量、级联加载都基于批量查询，用程序在后台组织数据，避免多次数据库交互（参见 `loadAllCascade` 底层实现）。
- 分页除快速分页、分页优化外，对 count 查询做了针对性优化，不是简单的 `select count(1) from (你的sql) as t`。

系统核心瓶颈大多在 IO（尤其数据库 IO），sqltoy 在数据库交互上做了大量优化。框架自身对 SQL 语句的组织处理消耗极微，通常在 **1 毫秒以内**。

三、数据库保留字如何处理?

在 `sqltoyContext` 中指定 `reservedWords` 属性，定义当前系统用到的保留字。

- **基于对象的操作**：sqltoy 自动完成保留字兼容。
- **自定义 SQL**：需按当前数据库对特定字段自行处理，如 SQL Server 中保留字要加 `[]`：

```
select [maxvalue],name from table
```

跨数据库时，sqltoy 会根据定义的 `reservedWords` 对自定义 SQL 中的保留字进行适配。例如上述 SQL 在 MySQL 项目下会自动变成：

```
select `maxvalue`,name from table
```

> 注意：是把 `[maxvalue]` 转成 `` `maxvalue` ``，而不是直接把 `maxvalue` 转成 `` `maxvalue` ``。

四、多数据源怎么弄?

完整的 dynamic-datasource 配置、`@DS` 切换与 `@DSTransactional` 跨库事务范例，见多数据源与事务。

sqltoy 可以配置默认数据源：

```
spring.sqltoy.defaultDataSource=primaryDB
```

多种实现方式：

1. **dynamic-datasource**（支持事务）：详见多数据源与事务
2. **定义多个** `LightDao`，给不同的 lightDao 注入不同的 dataSource，在 service 中按 bean 名称引入。
3. **查询级切换**：`findByQuery` / `findPageByQuery` 等的 `QueryExecutor` 中可直接 `.dataSource(xxx)`；链式 API 也支持 `.dataSource(...)`（见 Link 链式操作）。
4. 涉及**多数据库事务**时，可结合 JTA 实现（如 Atomikos/Narayana）。sqltoy 原生的命名多数据源演示见 sqltoy-showcase 的 `MultiDataSourceTest`。

五、可以跟其他 ORM 一起组合使用吗?

可以。你可以独立使用 sqltoy，也可以结合 JPA + sqltoy 的查询，根据实际情况灵活组合。

六、多字段 in 如何处理?

直接用元组 in 写法，参数传数组/集合即可：

```
#[and (t.TECH_GROUP,t.PROD_GROUP) in (:techGroups,:prodGroups)]
```

当 in 条件参数数组超过 1000（如 Oracle 限制）时，框架（5.1.32 / 4.19.25 起）会**自动切分**，无需手工处理。也可用 `@secure-loop` 拼接复杂条件，见补充说明：标签与表达式参考。

七、sqltoy 是否能在非 Spring 和 Solon 环境下使用?

支持。sqltoy 核心模块 `sagacity-sqltoy` 可纯 Java 使用，自行构建 `SqlToyContext` + 数据源 + `DefaultLightDaoImpl` 即可，事务用 `DBTransUtils`。详见纯 Java 项目如何使用。

八、启动就报 sqlId 重复 / 后加载的 SQL 覆盖前面的?

默认 `breakWhenSqlRepeat=true`：发现重复 `sqlId` **直接抛异常终止启动**，这是刻意设计——同名 sql 在两个文件里各写一份，线上很可能执行到错的那条。

排查：

```
# 临时定位用：改为 false，后面的覆盖前面的（仅用于排查，不要带到生产）
spring.sqltoy.breakWhenSqlRepeat=false
```

- 常见原因：模块拆分后两个 `*.sql.xml` 用了同一个 `id`；`@Sql` 注解里的 id 与 xml 冲突；
- **注意**：`sqlId` 是全局命名空间，不带模块前缀；建议命名加业务前缀（如 `crm_staff_findlist`）；
- 若确实需要按数据源区分同名 SQL，用 `<sql dataSource="...">` 或方言变体（`xxx_mysql`），而不是两个同名文件。

详见动态 SQL 编写规范。

九、缓存翻译不生效 / 数据改了页面还是旧值?

按下面顺序排查：

1. **缓存没加载**：`translateConfig` 指向的文件没被扫到（默认 `classpath:sqltoy-translate.xml;classpath:translates`），启动日志里能看到加载了几个 cache；
2. **参数没对上**：`<translate columns="POST_TYPE" cache="dictCache">` 的 `columns` 必须是**结果集中的列名**，不是属性名；
3. **码值类型不一致**：`1`（数字）与 `"1"`（字符串）匹配不到，可用 filter 的 `to-string` 统一；
4. **增量更新没开**：默认全量加载后不再变；要跟随库表变化需配 `cache-update-checker`（按时间戳/版本增量）或 `rest-checker`（外部接口）；
5. **FIFO 动态缓冲的预期行为**：`dynamic-cache` 模式只更新**本地已存在**的 key，没加载过的 key 下次动态获取时自然是新值——这不是 bug；
6. **多实例**：本地缓存（ehcache/caffeine）各实例独立，增量检测间隔内不同实例数据可能短暂不一致，对强一致要求高的码表请缩短检测间隔或改用 `rest-checker`。

详见缓存翻译使用与超大规模主数据缓存。

十、用了 `@fast` 之后 count 和实际数据对不上?

`@fast` 的语义是"**先对内层分页取 pageSize 条，再拿这几条去 join 外表**"，因此它的 count **默认用 @fast 内层 SQL**（简化形态，跳过 join 过滤）。如果 join 后的外表条件会改变结果行数，count 就会与数据不一致。

处理方式（按优先级）：

1. **把过滤条件下推到 @fast 内层**（内层能过滤的不要放到外层 join 条件里）；
2. 用 `<count-sql>` 显式给出 count（`QueryExecutor.countSql(...)` 同理），这是最可控的做法；
3. 确认不是 `union` 场景——`union` 的 count 有专门的 union-all-count 优化（拆分支求和），与 `@fast` 叠加时以 `count-sql` 为准。

详见分页优化。

十一、分页查不出数据 / 报"超过最大单页数据量"?

两个保护机制，都可配置：

参数	默认	行为
`pageFetchSizeLimit`	100000	单页最大记录数；`pageNo` 传 `-1` 做全量提取时会撞上它（`-1` 表示不限制）
`overPageToFirst`	未设置	页码超过最大页时是否回到第一页；不设置时按框架内置行为处理，建议显式配置

```
# 明确需要大页导出时调大（注意内存）
spring.sqltoy.pageFetchSizeLimit=500000
# 页码越界回到首页
spring.sqltoy.overPageToFirst=true
```

> 大结果集导出不建议用分页，用 `fetchStream`（流式回调）或 `findTop` 分批。详见分页优化。

十二、lambda 的 `getOne()` 为什么抛异常了?

这是 6.0.8 起的**严格语义**：0 条返回 `null`，**多于 1 条抛 `DataAccessException`（异常里带匹配条数）**。目的是避免"条件缺陷被静默掩盖成取第一条"的错值。

处理：

- 条件确实应唯一 → 检查是不是漏了租户/状态等条件；
- 确实可能多条但只需要一条 → 改用 `find()` 取第一条，或加 `orderBy` + `findTop(1)`；
- 同理，**不带任何 where 的 `update()/delete()` 默认抛异常**，需要全表操作时显式 `.matchAll()`。

详见Lambda 条件查询与更新。

十三、native 镜像里 lambda 条件不可用 / 启动报反射缺失?

lambda 依赖 `SerializedLambda` 与序列化镜像来提取属性名，closed-world 的 native 镜像必须显式注册：

- **应用侧**：按"lambda 定义类"注册（spring 用 `META-INF/native-image/**/serialization-config.json` 的 `lambdaCapturingTypes`，或 `registerLambda(定义类, b -> b.withInterfaces(LambdaFn.class))`；solon 用 `registerLambdaSerialization(定义类)`）；
- 不注册 = native 下 lambda 体系全部不可用（JVM 模式不受影响）；
- 另外 `updateSaveFetch` **在 native 下需要** `native-entity-proxies` 预生成代理，否则该分支不可用。

完整清单（实体扫描、DTO 反射、扩展类、函数类、JSON/vector/geometry 驱动类）见 GraalVM Native Image 指引。

十四、从 sqltoy-plus / MyBatis 迁过来要改什么?

- **sqltoy-plus**：6.0.8 已把 lambda 能力并入核心，移除插件依赖与 `@EnableSqlToyPlus`，按升级指南第十节第 2 条的对照表改包名/类名，并回归 `getOne()` 多匹配、无条件 update/delete 两类行为变化；
- **MyBatis / MyBatis-Plus**：`<if>` → `#[]`、`QueryWrapper` → `EntityQuery` 或 lambda、`ResultMap` → 结果类型/VO 映射、分页插件 → `findPage`。完整对照与落地顺序见从 MyBatis 迁移。